Natural Language Processing

Course Notes

Contents

1 The Birth of Human Language **8**
- 1.1 Human Language vs. Animal Communication . . . 8
  - 1.1.1 Hint 1: talking about what is not here and now . . . 8
  - 1.1.2 Hint 2: the bees, an exception . . . 9
- 1.2 When Did Language Appear? . . . 10
  - 1.2.1 The timeline of humankind . . . 10
  - 1.2.2 What recent research says . . . 11
  - 1.2.3 Did the Neanderthals speak? . . . 11
- 1.3 Why Did Language Evolve? . . . 12
  - 1.3.1 The obstetrical dilemma . . . 12
  - 1.3.2 Other key drivers . . . 13
  - 1.3.3 Three theories of how language emerged . . . 13
- 1.4 The Role of Gossip . . . 14
  - 1.4.1 Functions of early gossip . . . 15

2 What Is Natural Language Processing? **16**
- 2.1 About the Course . . . 16
- 2.2 Definition of NLP . . . 17
- 2.3 A Short History of NLP . . . 18
  - 2.3.1 Symbolic NLP (1950s – early 1990s) . . . 18
  - 2.3.2 Statistical NLP (1990s – 2010s) . . . 18
  - 2.3.3 Neural NLP (2010s) . . . 19
  - 2.3.4 Transformers (“it seems a long time ago...”) . . . 19
  - 2.3.5 Generative AI and LLMs (present) . . . 19
- 2.4 Why Traditional NLP Still Matters . . . 19
- 2.5 Case Study: ChatBDI . . . 20
  - 2.5.1 The idea: think BDI, talk LLM . . . 20
  - 2.5.2 Scenarios . . . 21
  - 2.5.3 Architecture (kept simple) . . . 21
  - 2.5.4 Experiments: it is a hard problem . . . 22

3 Levels of Linguistic Analysis **23**
3.1 The Three Levels . . . 23
3.2 Analogy with Programming Languages . . . 24
3.3 A Map of the Course . . . 25
3.3.1 Syntax . . . 25
3.3.2 Semantics . . . 26
3.3.3 Transformers and LLMs . . . 26
3.3.4 Features for representing a text . . . 26
3.3.5 Pragmatics . . . 27

4 Terminology **27**
4.1 Morphemes, Words and Compounds . . . 28
4.1.1 Morphemes . . . 28
4.1.2 Derivational and inflectional morphemes . . . 29
4.1.3 Words . . . 30
4.1.4 Compounds . . . 30
4.2 Parts of Speech: a Grammar Recap . . . 31
4.2.1 Overview . . . 31
4.2.2 Nouns . . . 32
4.2.3 Pronouns . . . 32
4.2.4 Verbs . . . 34
4.2.5 Adjectives . . . 35
4.2.6 Adverbs . . . 36
4.2.7 Determiners and articles . . . 36
4.2.8 Prepositions . . . 37
4.2.9 Conjunctions . . . 39
4.2.10 Interjections, numerals and particles . . . 39
4.2.11 Worked example: analisi grammaticale in English . . . 40
4.2.12 From words to sentences: analisi logica . . . 40
4.3 Corpora, Genre and Collocations . . . 41
4.3.1 Corpus . . . 41
4.3.2 Genre . . . 42
4.3.3 Collocations . . . 42
4.4 Precision and Recall . . . 43
4.4.1 The same thing with a confusion matrix . . . 44
4.4.2 Four worked examples . . . 44
4.4.3 Another example: five algorithms . . . 46
4.4.4 Exercises . . . 47

5 Regular Expressions **48**
5.1 The Syntax of Regular Expressions . . . 49
5.1.1 Disjunction: square brackets and the pipe . . . 49
5.1.2 Negation inside brackets . . . 49
5.1.3 Repetition: `?`, `*`, `+` and the wildcard `.` . . . 50
5.1.4 Anchors: `^` and `$` . . . 50

5.1.5 Summary of the operators . . . 50
5.2 Worked Example: Finding "the" . . . 51
5.3 ELIZA: Conversation by Substitution . . . 52
5.3.1 A tiny ELIZA in Python . . . 52
5.3.2 Exercises . . . 53

6 Syntax at the Word Level **54**
6.1 Word Segmentation . . . 54
6.1.1 Why it is not trivial . . . 54
6.1.2 The MaxMatch algorithm . . . 55
6.1.3 NLTK . . . 55
6.2 Stop Words and TF-IDF . . . 56
6.2.1 Stop words . . . 56
6.2.2 Frequent in the document or frequent in the language? . . . 56
6.2.3 Term frequency and inverse document frequency . . . 57
6.2.4 Worked example on C_2 . . . 58
6.3 Stemming and Lemmatisation . . . 59
6.3.1 The goal: one form for many . . . 59
6.3.2 Stemming vs. lemmatisation . . . 60
6.3.3 The Porter stemmer . . . 60
6.3.4 The seven steps . . . 61
6.3.5 Examples and problems . . . 62
6.3.6 Two texts to test a stemmer . . . 63
6.4 Minimum Edit Distance . . . 64
6.4.1 How similar are two strings? . . . 64
6.4.2 Definition . . . 65
6.4.3 Edit distance as search . . . 66
6.4.4 The algorithm . . . 66
6.4.5 The table and the backtrace . . . 67
6.4.6 Exercises . . . 68

7 Syntax at the Sentence Level **69**
7.1 Sentence Breaking . . . 70
7.2 Part-of-Speech Tagging . . . 71
7.2.1 The Penn Treebank tagset . . . 71
7.3 Grammars and Parsing . . . 72
7.3.1 Chomsky and generative grammar . . . 73
7.3.2 Grammars and the Chomsky hierarchy . . . 73
7.3.3 A grammar for the Wumpus world . . . 74
7.3.4 Definite Clause Grammars . . . 76
7.4 Statistical POS Tagging: Hidden Markov Models . . . 77
7.4.1 The model . . . 77
7.4.2 The weather example . . . 78
7.4.3 The Viterbi algorithm . . . 78
7.4.4 From weather to POS tags . . . 80

7.4.5 Maximum Entropy Markov Models . . . 80
7.4.6 Exercises . . . 81

8 Distributional Semantics **82**
8.1 The Distributional Hypothesis . . . 82
8.1.1 Meaning and distribution . . . 82
8.1.2 What is the meaning of "bardiwac"? . . . 83
8.2 The Co-occurrence Matrix . . . 83
8.2.1 A thought experiment: deciphering hieroglyphs . . . 83
8.2.2 English as seen by the computer . . . 84
8.3 Geometric Interpretation . . . 85
8.3.1 Words as points . . . 85
8.3.2 Direction, not location . . . 86
8.4 Distributional Semantic Models in Practice . . . 87
8.4.1 General definition of DSMs . . . 87
8.4.2 Building a distributional model . . . 88
8.4.3 What a DSM gives us . . . 90
8.4.4 Distributional vs. distributed . . . 95
8.4.5 Three famous examples . . . 96
8.4.6 Applications and software . . . 98
8.4.7 R as a toy laboratory . . . 99

9 Frame and Model-Theoretic Semantics **100**
9.1 Frame Semantics . . . 101
9.1.1 Word meaning revisited . . . 101
9.1.2 Two properties of frames . . . 101
9.1.3 From syntax to semantics . . . 102
9.1.4 Semantic role labelling . . . 102
9.2 Three Types of Semantics . . . 103
9.3 Model-Theoretic Semantics in Practice: Semantic Parsing . . . 104
9.3.1 Executable semantic parsing . . . 104
9.3.2 A brief history of semantic parsing . . . 105
9.3.3 Compositional semantics . . . 105
9.3.4 Language variation . . . 106

10 Lexical Semantics, WSD and NER **107**
10.1 The Lexicon and Lexical Relations . . . 107
10.1.1 Lexicon and lexical units . . . 107
10.1.2 Synonymy . . . 108
10.1.3 Antonymy . . . 108
10.1.4 Hyponymy and hypernymy . . . 109
10.1.5 Meronymy and holonymy . . . 109
10.1.6 Homonymy . . . 109
10.2 Thesauri and WordNet . . . 110
10.2.1 Structured lexicons and thesauri . . . 110

10.2.2 WordNet . . . 110
10.3 Word Sense Disambiguation . . . 112
10.3.1 The Lesk algorithm . . . 112
10.3.2 Extensions and limitations . . . 113
10.4 Named Entity Recognition . . . 114

11 The Four Approaches to Semantics Compared 115
11.1 Reading the table column by column . . . 116

12 Ontologies and the Semantic Web 119
12.1 What Is an Ontology . . . 119
12.1.1 Two definitions . . . 119
12.1.2 Motivation: semantic interoperability . . . 120
12.1.3 Gruber's definition, 2008 . . . 120
12.1.4 An example: a wine ontology . . . 121
12.1.5 Ontologies and databases . . . 122
12.1.6 Knowledge graphs . . . 122
12.2 Building Ontologies: Methodology and Tools . . . 123
12.2.1 Ontology Development 101 . . . 123
12.2.2 Tools: Protégé . . . 124
12.3 Ontology Languages: RDF, RDF Schema, OWL . . . 125
12.3.1 Evolution of the languages . . . 125
12.3.2 RDF: resources, properties, statements . . . 125
12.3.3 RDF Schema . . . 127
12.3.4 OWL: the Web Ontology Language . . . 128
12.4 The Semantic Web . . . 130
12.4.1 The vision . . . 130
12.4.2 The Semantic Web stack . . . 130
12.4.3 What is the Semantic Web really about? . . . 132
12.4.4 Web 4.0 and Web 5.0 . . . 132

13 Ontology Engineering, Learning and Matching 133
13.1 Ontology Engineering: the Design Process . . . 133
13.1.1 Many methodologies, one winner . . . 134
13.1.2 The ontology design process . . . 134
13.1.3 Steps 1–2: requirements and scope . . . 135
13.1.4 Step 3: consider reuse . . . 136
13.1.5 Step 4: enumerate terms . . . 137
13.1.6 Step 5: define classes and their taxonomy . . . 137
13.1.7 Step 6: define properties . . . 140
13.1.8 Step 7: define constraints . . . 141
13.1.9 Step 8: create instances . . . 142
13.2 N-ary Relations . . . 143
13.2.1 The problem . . . 143
13.2.2 Pattern 1: the relation becomes a class . . . 144

13.2.3 Pattern 2: lists for arguments in a relation . . . 146
13.3 Ontology Learning . . . 147
13.3.1 What, why and from where . . . 147
13.3.2 How to learn ontologies . . . 148
13.3.3 Term extraction . . . 149
13.3.4 Synonym extraction . . . 150
13.3.5 Concept extraction . . . 150
13.4 Ontology Matching . . . 151
13.4.1 The matching problem . . . 151
13.4.2 Correspondence, alignment, matching process . . . 153
13.4.3 Application domains . . . 154
13.4.4 Classification of matching techniques . . . 155
13.4.5 Basic techniques . . . 156
13.4.6 Composing matchers and selecting the alignment . . . 158
13.4.7 State-of-the-art systems . . . 158
13.4.8 Conclusions, tools and benchmarks . . . 159
13.5 Applications of Ontologies . . . 160
13.5.1 Some research from the course's group . . . 160
13.5.2 Application areas . . . 160

14 Text Features **162**
14.1 Families of Features . . . 162
14.1.1 Stylistic features . . . 162
14.1.2 Content-based features . . . 164
14.1.3 Content- and style-based features: LIWC . . . 165
14.1.4 Vocabulary and idiosyncrasies . . . 166
14.1.5 Graph-based features . . . 166
14.2 Which Features for Which Goal . . . 167
14.2.1 Content, FW, POS: when, why? . . . 167
14.2.2 So, what? . . . 168
14.2.3 The blackboard summary . . . 168

15 Pragmatics **171**
15.1 What Is Pragmatics . . . 171
15.2 Communication as Action . . . 172
15.2.1 From true/false sentences to actions . . . 172
15.2.2 Speech acts and the stages of communication . . . 173
15.3 Does Language Shape Thought? . . . 174
15.3.1 Newspeak and the Sapir–Whorf hypothesis . . . 175
15.3.2 How languages and cultures shape the ways we think . . . 175
15.3.3 Space and time . . . 177
15.3.4 Colour . . . 178
15.3.5 Grammatical gender . . . 178
15.3.6 Agency: who did it? . . . 179
15.4 The Language of Numbers and the Power of Names . . . 179

15.4.1 A few words on maths . . . 179
15.4.2 The importance of what a thing is called . . . 181

16 Language Models and RNNs **182**
16.1 What Is a Language Model? . . . 183
16.1.1 Predicting and evaluating language . . . 183
16.1.2 Next-word prediction . . . 183
16.1.3 Evaluating a text: the chain rule . . . 184
16.1.4 Why care about language models? . . . 185
16.2 Classical Language Models: n-grams . . . 185
16.2.1 n-grams and the Markov assumption . . . 186
16.2.2 Sparsity problems . . . 188
16.2.3 Storage problems . . . 190
16.2.4 Generating text with an n-gram model . . . 190
16.3 Neural Language Models . . . 192
16.3.1 Aside: word embeddings via word2vec . . . 192
16.3.2 A fixed-window neural language model . . . 193
16.3.3 How to train a neural language model . . . 195
16.3.4 Pros and cons of the fixed-window model . . . 196
16.4 Recurrent Neural Networks . . . 197
16.4.1 The idea: reuse the same weights at every step . . . 197
16.4.2 Pros and cons . . . 199
16.4.3 Training an RNN language model . . . 199
16.4.4 Backpropagation through time . . . 200
16.4.5 Generating text with an RNN . . . 202
16.4.6 Evaluating language models: perplexity . . . 203
16.5 Problems with RNNs: Vanishing and Exploding Gradients . . . 205
16.5.1 Vanishing gradients: intuition . . . 206
16.5.2 Vanishing gradients: proof sketch . . . 206
16.5.3 Why it matters: long-term dependencies . . . 208
16.5.4 Exploding gradients and gradient clipping . . . 208
16.5.5 Solutions for vanishing gradients . . . 209

List of Blackboards

Tables the lecturer wrote on the blackboard in class, rewritten from the photos. They summarise exam material.

Blackboard 1 The four kinds of semantics compared . . . 116
Blackboard 2 Which feature is useful for which task . . . 169

1 The Birth of Human Language

[Slides · Intro: 4–21]

Before teaching computers to process language, it is worth asking what language *is* and where it comes from. This section is not technical: it frames the object of study of the whole course. The questions it raises (what makes human language special? why did it appear?) come back later, for example when we see that language is used not only to describe the world but also to *act* on other people (pragmatics, Section 3).

This section covers:

1. what distinguishes human language from animal communication (Section 1.1);
2. when language appeared, on the timeline of human evolution (Section 1.2);
3. the evolutionary pressures that may have pushed our species towards language, and the main theories about how it emerged (Section 1.3);
4. the "gossip" hypothesis: language as a tool for managing social relationships (Section 1.4).

1.1 Human Language vs. Animal Communication

[Slides · Intro: 5–8]

Many animals communicate: birds sing to mark territory, vervet monkeys have different alarm calls for eagles, leopards and snakes, dogs growl and wag their tails. So what is special about *human* language? The course poses this as an open question and gives two hints.

1.1.1 Hint 1: talking about what is not here and now

[Slides · Intro: 5–7]

The first hint is given only through two pictures: a portrait of the Latin poet **Horace** and the poster of the film ***Dead Poets Society***. What links them is the phrase *carpe diem* ("seize the day"): it comes from one of Horace's odes, and it is the motto that the teacher in the film gives his students.

The most likely reading of the hint is this: *carpe diem* is a sentence about *time*. It urges you to act now because the future is uncertain and the past is gone. To say it at all, a language must be able to talk about things that are not in front of the speaker: the future, the past, possibilities, abstract ideas. Animal signals almost never do this: an alarm call means "danger *here*, *now*".

Definition — Displacement

A communication system has **displacement** if its messages can refer to things that are *removed in time or space* from the moment and place of communication:

past and future events, distant places, hypothetical or imaginary entities, abstract concepts.

Displacement is one of the "design features" of language listed by the linguist Charles Hockett (1960). Others worth knowing, because they will come back in this course, are:

- **productivity**: a finite set of words and rules generates an unbounded number of new sentences, never heard before, that are still understood;
- **duality of patterning**: meaningless units (sounds, letters) combine into meaningful units (morphemes, words; see Section 4.1), which combine into sentences;
- **arbitrariness**: there is no natural link between a word and what it means (*dog*, *cane*, *Hund* all name the same animal);
- **cultural transmission**: a language is learnt from the community, not inherited genetically.

1.1.2 Hint 2: the bees, an exception

[Slides · Intro: 8]

The second hint says that the language of bees is an *exception* to the most evident difference between human language and animal communication: bees convey a specific piece of information that no other animal conveys in the wild.

That piece of information is *the location of a distant food source*. When a forager bee comes back to the hive, it performs the **waggle dance** (decoded by Karl von Frisch, Nobel Prize 1973):

- the **angle** α of the straight "waggle run" with respect to the vertical of the honeycomb equals the angle between the direction of the food and the direction of the sun;
- the **duration** of the waggle run encodes the **distance** to the food (longer run, farther food).

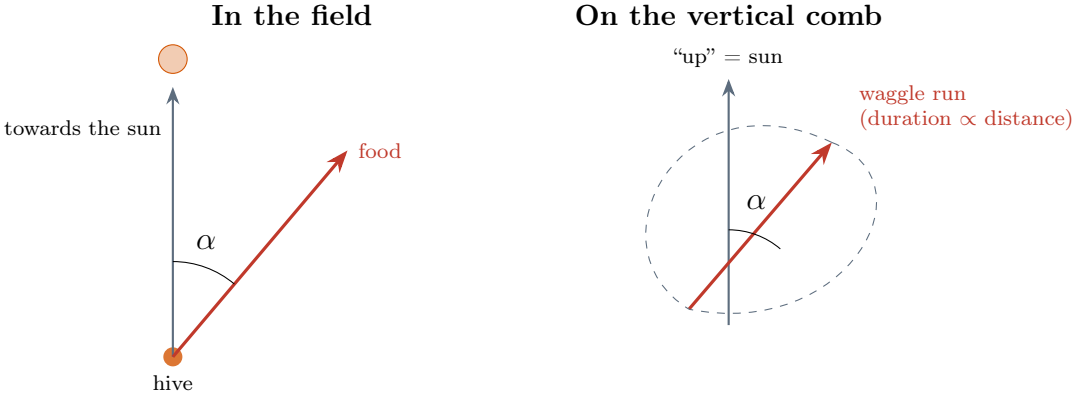


Figure 1: The waggle dance. The same angle α appears twice: in the field it separates the sun from the food, on the comb it separates "up" from the waggle run. The dashed loops are the return paths the bee walks between two waggle runs.

So bees can talk about something *far away*: their communication has a (limited) form of spatial displacement. It is still very far from human language: it covers only one topic (food and nest sites), it is not productive (bees cannot combine dances into new messages about new topics) and it is innate, not learnt.

Intuition — the answer in one line

Human language is (among other things) *displaced*: it lets us talk about the past, the future, the absent and the imaginary. Animal communication is almost always about the here and now; the bees' dance is the classic exception, because it describes a place the bees are not currently in.

1.2 When Did Language Appear?

[Slides · Intro: 9–12]

1.2.1 The timeline of humankind

[Slides · Intro: 9–10]

Table 1 places our species on the timeline of human evolution. It is adapted from Yuval Noah Harari's *Sapiens: A Brief History of Humankind* (2014). The abbreviations are MYA = million years ago, YA = years ago.

When	What happened
6 MYA	Last common "grandmother" of humans and chimpanzees
2.5 MYA	Evolution of the genus *Homo* in Africa; first stone tools
2 MYA	**Humans spread from Africa to Eurasia**; evolution of different human species (*Homo rudolfensis* and *Homo ergaster* in Africa, *Homo erectus* in East Asia)
500,000 YA	Neanderthals evolve in Europe and the Middle East
200,000 YA	*Homo sapiens* evolves in East Africa
70,000 YA	**Sapiens spreads out of Africa**
70,000–30,000 YA	**Cognitive revolution**: invention of boats, arrows, needles; beginnings of art, religion, commerce, social stratification, **and language**
45,000 YA	Sapiens settles Australia
30,000 YA	Extinction of Neanderthals
16,000 YA	Sapiens settles America
13,000 YA	Extinction of *Homo floresiensis*: **Homo sapiens is the only surviving human species**
12,000 YA	Agricultural revolution, permanent settlements

Table 1: Milestones of human evolution (adapted from Harari). Note how recent everything after the cognitive revolution is compared with the 6 million years of the whole story.

The course also shows a map of the movement of *Homo sapiens* out of Africa: from East Africa into the Middle East, then along the coasts of southern Asia to Australia (about 50,000 years ago), into Europe and northern Asia, and finally across the Bering region into America (about 15,000 years ago), while Neanderthals already lived in Europe and *Homo erectus* in Asia.

1.2.2 What recent research says

[Slides · Intro: 11]

Harari places the birth of language in the cognitive revolution, about 70,000–30,000 years ago. A 2025 study reported by MIT News ("When did human language emerge?", 14 March 2025) argues for an earlier date:

- **135,000 years ago**: genomic and geographic data suggest that the biological and cognitive *capacity* for language was already fundamentally present. The argument is that the earliest split between human populations happened about 135,000 years ago, and since *all* human populations have language, the capacity must predate the split.
- **100,000 years ago**: archaeological records show an explosion of symbolic behaviours, like art and complex tools, indicating active social communication. Language was being *used* socially.
- **Gradual origins**: scientists widely agree that language developed from a complex mix of hand gestures, facial expressions and vocalisations over time, not in a single step.

Watch out — capacity vs. use

The two dates answer different questions. "The capacity for language existed 135,000 years ago" is about biology (the brain and vocal tract could support language). "Symbolic behaviour exploded 100,000 years ago" is about culture (people were actually communicating with symbols). Do not mix them up when asked "when did language appear?".

1.2.3 Did the Neanderthals speak?

[Slides · Intro: 12]

The course leaves this as a question for discussion. There is no definitive answer, but these are the main pieces of evidence usually brought up:

- **for**: Neanderthals had the same variant as ours of the *FOXP2* gene, involved in speech and language; a Neanderthal hyoid bone (the bone that supports the

tongue) found at Kebara, Israel, is almost identical to ours; they made complex tools, used fire, buried their dead and probably made simple symbolic objects;
- **against / uncertain**: their symbolic record is much poorer than that of *Sapiens* of the same period, and soft tissue (the vocal tract, the brain itself) does not fossilise, so the anatomy of speech can only be inferred.

Many researchers today think Neanderthals had *some* form of language, though possibly less complex than ours. Note that if language capacity predates 135,000 years ago in *Sapiens*, the question of whether it also predates the split between *Sapiens* and Neanderthals (several hundred thousand years ago) remains open.

1.3 Why Did Language Evolve?

[Slides · Intro: 13–18]

Knowing *when* language appeared does not tell us *why*. This part of the course follows the linguist Sverker Johansson (*The Dawn of Language: How We Came to Talk*), for whom one major driver was the **obstetrical dilemma**.

1.3.1 The obstetrical dilemma

[Slides · Intro: 13–16]

Two evolutionary trends pulled the human body in opposite directions:

- **Upright walking (bipedalism)** restructured the human pelvis into a basket-like shape that supports the internal organs and makes walking efficient, but *narrows the birth canal.*
- **Large brains**: hominid brain size increased dramatically over millions of years, so the baby's head became large compared with the mother's pelvis.

Consequence: labour and delivery are difficult and dangerous in the human species (not only in humans, but especially in humans).

Evolution's "compromise" was to have babies born **immature**: their brain keeps growing a lot after birth. Human newborns cannot even cling to their mother, as the babies of all other higher primates do, and they need the protection of their family or social group for a very long time.

Intuition — how this leads to language

The course asks: how might these facts relate to the birth of language? Follow the chain:

1. A difficult birth means a mother needs help: **midwives** and social support. Giving birth becomes a social, cooperative event.
2. A baby that cannot cling must be carried or **put down**. A mother who puts the baby down to gather food can still soothe and monitor it from a distance with her **voice** ("motherese").

3. A long childhood is a long **window for learning**: there is time to absorb something as complex as a language from the community.

Table 2 summarises the argument.

Evolutionary pressure	**Physical consequence**	**Behavioural and language outcome**
Bipedalism and big brains	Narrow birth canal; difficult, dangerous labour	Need for **midwives and social support**, forcing cooperative social structures
Neurological immaturity	Babies are born completely helpless and cannot cling	Mothers must **"put the baby down"**, using vocalisations to soothe and monitor them from afar
Extended childhood	Years of dependency requiring group protection	Long childhoods provide a **massive window for learning** complex language from the community

Table 2: From the obstetrical dilemma to language.

1.3.2 Other key drivers

[Slides · Intro: 17]

Besides the obstetrical dilemma, three more drivers are usually cited:

- **Survival and cooperation**: early humans needed to coordinate group hunts, plan defences against predators and share crucial resources.
- **Social bonding**: communication grew from basic physical grooming into storytelling around fires, building community trust and deep empathy (this is developed in Section 1.4).
- **Abstract thought**: language enabled humans to refer to things not immediately present, like the past, the future or imaginary cultural concepts. This is displacement again (Section 1.1).

1.3.3 Three theories of how language emerged

[Slides · Intro: 18]

There are three main families of theories. They differ on *speed* (gradual or sudden) and on the *starting medium* (voice or gestures).

	Continuity theory	Discontinuity theory	Gesture-first theory
Core idea	Language evolved **gradually** over millions of years out of animal communication systems	Language appeared **suddenly** as a unique, exclusively human genetic mutation	Language originated from **physical gestures** and manual signs before shifting to speech
Key mechanism	Natural selection slowly shaped the vocal tract and brain tissue to handle complex communication	A sudden cognitive "leap" or single genetic mutation re-wired the brain for universal grammar	Bipedalism freed the hands, allowing early humans to use detailed hand and facial signs to communicate
Primary driver	The gradual need for social bonding and survival coordination in expanding groups	An internal computational upgrade in the brain, prioritising thought organisation over communication	The mechanical advantage of visual "miming" before vocalisation evolved to take over
Famous proponents	Charles Darwin	Noam Chomsky	Michael Corballis

Table 3: The three main theories on the emergence of language.

Intuition — telling them apart

- **Darwin**: language is like any other trait, shaped slowly by natural selection; "more of" what animals already have.
- **Chomsky**: language is primarily a tool for *thinking*, and its core (the ability to combine units recursively, "universal grammar") appeared in one jump; communication came second.
- **Corballis**: language started in the *hands*; the voice took over later because it works in the dark, at a distance and with the hands busy.

Chomsky's name will come back in this course for a very different reason: his work on formal grammars underlies the syntactic parsing we will study later.

1.4 The Role of Gossip

[Slides · Intro: 19–21]

One of the most popular explanations of *why* language evolved is surprisingly down-to-earth: **gossip**. The idea is due to the anthropologist Robin Dunbar (*Grooming, Gossip and the Evolution of Language*, 1996), and Harari's *Sapiens* makes it widely known.

Key result — Gossip as vocal grooming

Gossip acted as a *vocal substitute for physical grooming* in primates, allowing early humans to bond and manage social relationships in groups too large for direct touch alone.

The argument has three steps:

1. **Primate roots.** Primates maintain social bonds and group cohesion through physical grooming (picking ticks and fleas from each other).
2. **The group size limit.** Grooming is one-to-one and takes time. As early human groups grew larger, physical grooming became too time-consuming to bond with every member.
3. **Vocal grooming.** Language evolved as a form of "vocal grooming" that lets one person bond with *several* others at the same time, instead of one at a time.

Intuition — why the numbers matter

If grooming one partner takes a fixed amount of time per day, the time a primate must spend grooming grows with the size of its group, and there are only so many hours in a day. Dunbar estimated that the natural human group size ("Dunbar's number", about 150 people) would require spending around 40% of the day grooming. Talking to three or four listeners at once removes that bottleneck.

1.4.1 Functions of early gossip

[Slides · Intro: 21]

Gossip was not idle chatter. It did three useful jobs for the group:

- **Information sharing**: it transmitted crucial *reputational data*: who could be trusted, and who was a free rider (someone who takes advantage of the group without contributing).
- **Social cohesion**: it strengthened group identity, enforced social norms and managed complex social networks without written records.
- **Conflict reduction**: it reduced friction inside the group by aligning moral judgements and expectations.

Why this matters for NLP

Two ideas from this section shape how we look at language in NLP:

- language is mostly about *people* and their relationships, not just a code for describing facts. A lot of real text (social media, chats, reviews) is exactly "gossip" in this sense, and analysing it means dealing with opinions, emotions and social intentions;
- language is used to *do* things to others (to bond, to warn, to persuade), which is the subject of **pragmatics** (Section 3).

Cheat sheet — the birth of human language

- Key difference from animal communication: **displacement** (talking about the past, the future, the absent, the imaginary). The bees' waggle dance is the classic exception: it describes the direction and distance of distant food.
- Harari: language arises with the **cognitive revolution**, 70,000–30,000 years

ago. Recent studies: the *capacity* was present by **135,000** years ago, symbolic *use* exploded around **100,000** years ago; origins were gradual (gestures, faces, voice).
- **Obstetrical dilemma** (Johansson): bipedalism + big brains ⇒ difficult births and immature babies ⇒ need for social support, vocal soothing at a distance and a long learning window.
- Other drivers: survival and cooperation, social bonding, abstract thought.
- Theories: **continuity** (gradual; Darwin), **discontinuity** (sudden mutation; Chomsky), **gesture-first** (hands before voice; Corballis).
- **Gossip** (Dunbar): language as vocal grooming for groups too large for physical grooming; it shares reputational data, builds cohesion and reduces conflict.

2 What Is Natural Language Processing?

[Slides · Intro: 1–3; Terminology introduction: 1–22]

This section introduces the field: what the course covers, what NLP is, how it evolved from hand-written rules to large language models, why the "old" methods are still needed, and a real research example showing how hard the problems are even today.

This section covers:

1. how the course is organised (Section 2.1);
2. the definition of NLP and its two directions, understanding and generation (Section 2.2);
3. the five eras of NLP (Section 2.3);
4. when traditional NLP is still required (Section 2.4);
5. a case study: ChatBDI, letting software agents talk in natural language (Section 2.5).

2.1 About the Course

[Slides · Intro: 1–3; Terminology introduction: 2]

The course (taught by Viviana Mascardi, University of Genoa) loosely follows the textbook *Speech and Language Processing* (3rd edition draft) by Dan Jurafsky and James H. Martin, freely available at `https://web.stanford.edu/~jurafsky/slp3/`. Some slides integrate those associated with the book. There is no single reference book, however: the slides, plus the pointers to material on the web and on the course's AulaWeb page, are enough to understand the subject, even without attending the lessons.

- A large share of the lessons covers **"Annotating Linguistic Structure"** (Volume III of the book), extended with lessons on the birth of human language

(Section 1), semantics and pragmatics.
- The remaining lessons cover **Large Language Models** (Volume I).
- Topics of Volume I already covered in other Machine Learning courses (logistic regression, neural networks, RNNs, ...) are *not* repeated.

Key result — Exam

The exam has two parts: a **written exam** and an **individual project**. Students can propose the project themselves, so they are free to experiment in a practical way with the topics they liked most.

The (preliminary) programme for 2026–2027 is, in order: introduction and regular expressions; syntax at the word level (segmentation, stop words, TF-IDF, stemming, lemmatisation); minimum edit distance, the sentence level and rule-based POS tagging; grammars and statistical (Hidden Markov Model) POS tagging; semantics (distributional, frame, model-theoretic and lexical semantics, a comparison among types of semantics, ontologies and the Semantic Web); practical labs on NLP libraries, text summarisation, chatbots and agentic AI; pragmatics; a wrap-up on classical NLP; and finally BERT, GPT and LLMs. Section 3.3 shows how these topics fit together.

2.2 Definition of NLP

[Slides · Terminology introduction: 3]

Definition — Natural Language Processing (NLP)

Natural language processing is a subfield of linguistics, computer science and artificial intelligence concerned with the interactions between computers and human language. In particular, it studies how to program computers to
- process and analyse large amounts of natural language data;
- **understand** text: **Natural Language Understanding (NLU)**;
- **produce** understandable text in human languages from some underlying non-linguistic representation of information: **Natural Language Generation (NLG)**.

A "natural" language is one that evolved naturally among humans (English, Italian, Chinese, sign languages), as opposed to *formal* languages designed on purpose (Python, logic, arithmetic). Natural languages are ambiguous, redundant, constantly changing and full of exceptions; formal languages are designed to avoid all of that. This is exactly why processing them with a computer is hard.

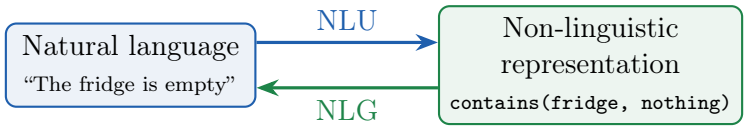


Figure 2: NLU and NLG go in opposite directions between text and an internal representation (here a logical fact a program can use).

2.3 A Short History of NLP

[Slides · Terminology introduction: 4–8]

NLP has gone through five main eras. Each changed *where the knowledge about language comes from*: first from people writing rules, then from statistics over corpora, then from neural networks learning their own representations.

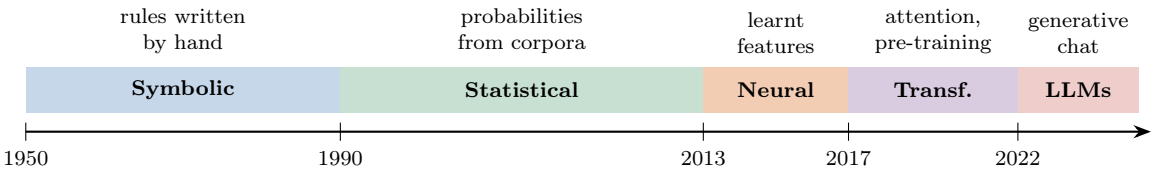


Figure 3: The eras of NLP. Boundaries are approximate and the eras overlap: the neural era is usually dated from word embeddings (around 2013), the transformer era from the 2017 transformer paper, the LLM era from the release of ChatGPT (2022).

2.3.1 Symbolic NLP (1950s – early 1990s)

[Slides · Terminology introduction: 4]

Programs based on **hand-written rules** and knowledge bases. Classic systems:

- **ELIZA** (1966, Joseph Weizenbaum, MIT): a chatbot imitating a psychotherapist, using simple pattern matching and substitution ("I am sad" → "Why are you sad?"). It understood nothing, yet many users attributed real understanding to it.
- **SHRDLU** (1968–1970, Terry Winograd, MIT): a program that understood commands in English about a small "blocks world" ("put the red pyramid on the blue block") and could answer questions about it.
- **PARRY** (1972, Kenneth Colby, Stanford): a chatbot simulating a person with paranoid schizophrenia; famously, it was made to "talk" with ELIZA.
- **Ontologies** (e.g. MARGIE, 1973, Roger Schank): representing the meaning of sentences with structured conceptual knowledge.

Links: https://en.wikipedia.org/wiki/ELIZA, https://en.wikipedia.org/wiki/SHRDLU, https://en.wikipedia.org/wiki/PARRY, https://www.ijcai.org/Proceedings/73.

2.3.2 Statistical NLP (1990s – 2010s)

[Slides · Terminology introduction: 5]

- **Large corpora** became available, also multilingual (e.g. parliament proceedings translated into many languages).
- Statistical models make **soft, probabilistic decisions** by attaching real-valued weights to each input feature, instead of applying hard rules.

- Example: **part-of-speech tagging with Hidden Markov Models** (covered later in the course).

2.3.3 Neural NLP (2010s)

[Slides · Terminology introduction: 6]

- **End-to-end learning** of a higher-level task (e.g. question answering), instead of relying on a pipeline of separate intermediate tasks (tokenise, tag, parse, . . .).
- No need to **engineer features** by hand: the network learns its own representation of the language.
- A new paradigm, distinct from statistical NLP (example: **neural machine translation**).
- Example: **word embeddings**, vectors representing words so that words with similar meaning are close in space.

2.3.4 Transformers ("it seems a long time ago. . . ")

[Slides · Terminology introduction: 7]

BERT (Bidirectional Encoder Representations from Transformers, Google) is a model pre-trained on huge amounts of text and then fine-tuned for specific tasks. It is built on the **transformer** architecture, introduced in the paper *Attention Is All You Need* (Vaswani et al., NeurIPS 2017).

Watch out — dates and the link on the slide

The slide says "BERT, Google (2019)" and links to a NeurIPS 2017 paper. That link is the *transformer* paper (*Attention Is All You Need*), not the BERT paper. BERT (Devlin et al.) was released in October 2018 and published at NAACL in 2019, so both 2018 and 2019 are found in the literature.

2.3.5 Generative AI and LLMs (present)

[Slides · Terminology introduction: 8]

- **GPT**: Generative Pre-trained Transformer. **ChatGPT** was released in November 2022.
- Other large language models and tools: Llama, Gemini, . . . ; and Ollama, a tool to run open models locally.

2.4 Why Traditional NLP Still Matters

[Slides · Terminology introduction: 9–10]

If LLMs can do almost everything, why study tokenisers, taggers and grammars? Because traditional NLP methods and approaches are still required in three situations.

Key result — When traditional NLP is required

1. **Not enough data**: when the amount of training data is insufficient to apply machine learning or deep learning methods successfully, or to train LLMs (e.g. low-resource languages, very specialised domains).
2. **Pre- and post-processing**: when NLP pipelines need preprocessing (e.g. tokenisation) or postprocessing and transformation of their output (e.g. extracting knowledge from syntactic parses). This includes *refining and validating the results* of ML, DL and LLM models.
3. **LLMs cannot be used**, for technical reasons (cost, latency, hardware), legal reasons (privacy, data that cannot leave the organisation) or ethical reasons (lack of transparency, bias).

Intuition

Even an LLM starts with a tokeniser, a very traditional NLP component. And when an LLM produces an answer that a program must use (a database query, a logical fact), checking that the answer is well formed and consistent is again a job for traditional methods. The case study below shows exactly this.

The course will therefore discuss (some of) the common problems of NLP following a *standard linguistic analysis approach* (Section 3), with examples of (some of) the common techniques and their common implementations.

2.5 Case Study: ChatBDI

[Slides · Terminology introduction: 11–22]

The course shows a real example from the lecturer's recent research, at the intersection of multi-agent systems (S&DAI) and NLP. It is useful early on because it shows, on a concrete problem, almost every topic of the course: syntax, semantics, pragmatics, edit distance and LLMs.

2.5.1 The idea: think BDI, talk LLM

[Slides · Terminology introduction: 12–13]

A **BDI agent** (Belief–Desire–Intention) is a software agent that reasons symbolically: it has *beliefs* about the world (facts such as `has(owner, beer)`), *desires* (goals) and *intentions* (plans it has committed to). BDI agents in a multi-agent system (MAS) talk to each other with an **Agent Communication Language (ACL)**: each message has a **performative** (the type of communicative act, e.g. `tell` to inform, `askOne`/`askAll` to ask a question, `achieve` to ask for a goal to be pursued) and a **content**, a logical term.

ChatBDI keeps the BDI agent as the "brain" and uses an LLM only as its **language sensor–actuator**: the LLM translates a human's natural-language sentence into an ACL message the agent understands (and the agent's answers back into natural

language). So humans can chat with agents that were never designed to understand natural language.

2.5.2 Scenarios

[Slides · Terminology introduction: 14–15]

- **Closed MAS**: a known set of agents, e.g. the classic domestic-robot example with a *robot*, an *owner* and a *supermarket* (the robot brings beer from the fridge to the owner and orders more from the supermarket). A user can ask "Which are the available agents?", ask the robot which plans it has, or ask it to describe a plan, and receives a natural-language answer.
- **Open MAS**: agents can join and leave, e.g. an auction of flight tickets where agents (Bob, Giacomo, Maria, . . .) post bids such as "I have a bid for the flight from Athens to Paris on 18/12/2015 for 50 euros" and the winner is announced in natural language.

2.5.3 Architecture (kept simple)

[Slides · Terminology introduction: 16–17]

A human writes a sentence addressed to an agent, say "@Bob, . . . ". Before that, ChatBDI has asked Bob for its **lexicon**: the list of terms (predicates) Bob understands.

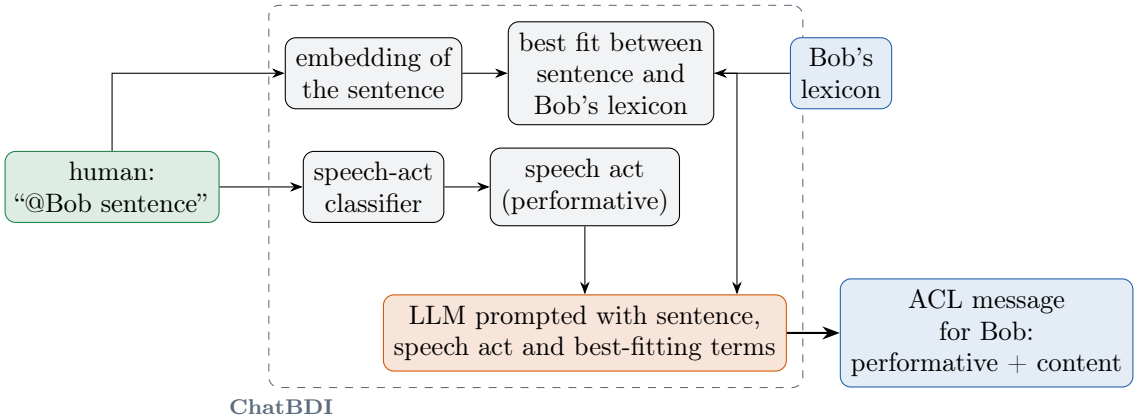


Figure 4: ChatBDI translating a human sentence into an ACL message. The performative comes from a classifier, the content from an LLM guided by the terms of the recipient's lexicon that best fit the sentence.

1. A **speech-act classifier** decides the performative: is the human informing (`tell`), asking (`askOne`, `askAll`), requesting an action (`achieve`)?
2. The sentence is turned into an **embedding** (a vector) and compared with Bob's lexicon to find the terms that **best fit** it.
3. An **LLM is prompted** with the sentence, the speech act and the best-fitting terms, and produces the content of the message in ACL.

2.5.4 Experiments: it is a hard problem

[Slides · Terminology introduction: 18–22]

Many LLMs were compared on the two sub-tasks: getting the **performative** right and getting the **message content** right (the latter scored lower overall). To measure how close a generated term is to the expected one, the experiments use the **Levenshtein distance**: the minimum number of character insertions, deletions and substitutions needed to turn one string into the other (the *minimum edit distance*, a topic of the course). Three examples show what can go wrong:

Worked example — a partially correct translation

Sentence: "Employer 7 attend JS course till 1/11/2025".

- Performative: `tell`, expected `tell`: correct.
- Term: `training_attended(7, JS, 1/11/2025)`; expected `training_attended(7, js, "1/11/2025")`.
- Levenshtein distance = 4: `J`→`j` and `S`→`s` (2 substitutions), plus the two quotation marks around the date (2 insertions).

Almost right, but for a symbolic agent `JS` and `js` are different constants and `1/11/2025` (an arithmetic expression!) is not the string `"1/11/2025"`: the agent's plan will not match. Generating the content took about 15 seconds.

Worked example — right form, wrong meaning

Sentence: "beans are very good for you, they contain 24 grams of protein per 100 grams".

- Performative: `tell`, correct.
- Term: `calories(beans, unit(24, gr), 100)`; expected `protein(beans, unit(100, gr), 24)`. Levenshtein distance = 13.

The term is syntactically perfect, but it says something different: the predicate is wrong (*calories* instead of *protein*) and the two quantities are swapped. This is a *semantic* error.

Worked example — the wrong speech act

Sentence: "Tell me the pressure of both front tires".

- Performative: `tell`; expected `askAll`: **incorrect**.
- Term: `tire_pressure(_, 28)`; expected `tire_pressure(front, _)`. Levenshtein distance = 7.

The sentence starts with the verb "tell", but the human is not *informing* the agent of anything: they are *asking* for information, about all ("both") front tyres, hence `askAll`. The surface form of a sentence and the act it performs can differ: this is a *pragmatic* error.

Why this matters for NLP — the three errors

The three examples are errors at the three levels of linguistic analysis of Section 3.1: wrong *form* of a constant (syntax), wrong *meaning* of the fact (semantics), wrong *act* performed by the sentence (pragmatics). They also show why traditional NLP is still needed next to LLMs: to check and repair the LLM's output (Section 2.4).

Cheat sheet — what is NLP

- NLP: computers processing human language; **NLU** (text → representation) and **NLG** (representation → text).
- Eras: **symbolic** (1950s–early 1990s; ELIZA, SHRDLU, PARRY, MARGIE), **statistical** (1990s–2010s; corpora, probabilities, HMM POS tagging), **neural** (end-to-end, learnt features, word embeddings), **transformers** (BERT), **generative LLMs** (GPT, ChatGPT 2022).
- Traditional NLP is still needed when data are scarce, for pre/post-processing and validation of model output, and when LLMs cannot be used (technical, legal, ethical reasons).
- ChatBDI: BDI agents "think", an LLM "talks"; a sentence becomes performative + content; errors appear at the syntactic, semantic and pragmatic level; Levenshtein distance measures how far the generated term is from the expected one.

3 Levels of Linguistic Analysis

[Slides · Terminology introduction: 23–33]

The course organises NLP problems following a standard linguistic analysis approach. This short section introduces the three levels of that analysis, explains them with an analogy from programming, and uses them as a map of the whole course.

This section covers:

1. the three levels: syntax, semantics, pragmatics (Section 3.1);
2. the analogy with programming languages (Section 3.2);
3. the map of the course topics, level by level (Section 3.3).

3.1 The Three Levels

[Slides · Terminology introduction: 23–24]

Watch out — exam

The lecturer is explicit: when the exam asks for "the three levels of linguistic analysis", it refers to **these three**: syntax, semantics and pragmatics.

Definition — The three levels of linguistic analysis

Given a natural-language utterance, we can ask three questions, each building on the previous one:

- **Syntax**: *what is grammatical?* The form and structure of the utterance: which sequences of words are well formed, and how words group into phrases and sentences.
- **Semantics**: *what does it mean?* The literal meaning of the words and of the way they are combined.
- **Pragmatics**: *what does it do?* The purpose of the utterance in its context: what the speaker wants to achieve by saying it.

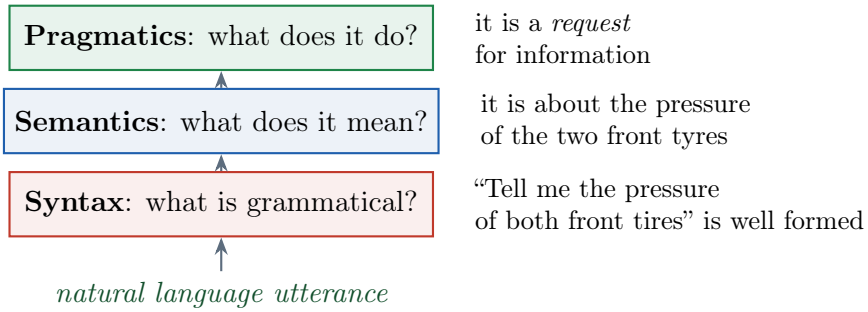


Figure 5: The three levels, from the bottom up, applied to a sentence from the ChatBDI case study (Section 2.5).

Worked example — the same idea with everyday sentences

- "Colorless green ideas sleep furiously" (Chomsky): syntactically correct, semantically nonsense.
- "Me want cookie": syntactically wrong, but the meaning is perfectly clear.
- "Can you pass the salt?": syntax and semantics describe a question about your *ability*; pragmatically it is a *request*, and answering "yes" without passing the salt misses the point.

3.2 Analogy with Programming Languages

[Slides · Terminology introduction: 25–26]

The three levels have a natural counterpart in programs:

Level	A program is correct at this level if...
Syntax	there are no compiler errors
Semantics	there are no implementation bugs
Pragmatics	it implements the right algorithm (for the purpose)

Three examples show that the levels are independent of each other:

- **Different syntax, same semantics** (both mean 5):

$$2+3 \quad \Longleftrightarrow \quad 3+2$$

- **Same syntax, different semantics** (1 and 1.5): the expression `3 / 2` is `1` in Python 2.7 (integer division between integers) but `1.5` in Python 3 (true division):

`3 / 2` (Python 2.7) $\quad \not\Longleftrightarrow \quad$ `3 / 2` (Python 3)

- **Good semantics, bad pragmatics**: a correct, bug-free implementation of a deep neural network for estimating the probability of a coin flip. It works, but it is absurd for the purpose: counting heads and dividing by the number of flips does the job.

Intuition

Syntax is about the *form*, semantics about the *content*, pragmatics about the *use*. Each level can be right while the one above is wrong: a sentence (or a program) can be well formed and meaningless, or meaningful and pointless.

3.3 A Map of the Course

[Slides · Terminology introduction: 27–33]

The topics of the course can be placed on the three levels. This list is a roadmap: each item will get its own section later in these notes.

3.3.1 Syntax

[Slides · Terminology introduction: 27–29]

The course starts with **regular expressions**, the basic tool for finding patterns in text (Section 5). Then the word level (Section 6) and the sentence level (Section 7):

Word level	**What it is about**
Word segmentation	splitting a text into words (tokens)
Identification of stop words	finding very frequent words with little content (*the*, *of*, *and*) that are often filtered out
Stemming and lemmatisation	reducing *running*, *ran*, *runs* to a common form (a crude stem or the dictionary lemma *run*)
Minimum edit distance	how many edits turn one string into another (the Levenshtein distance used in Section 2.5)
Sentence level	
Sentence breaking	splitting a text into sentences (harder than it looks: think of "Dr. Smith" or "3.14")
Part-of-speech tagging	labelling each word as noun, verb, adjective, …
Parsing	building the grammatical structure (tree) of a sentence

3.3.2 Semantics

[Slides · Terminology introduction: 30]

Covered in Sections 8 (distributional), 9 (frame and model-theoretic) and 10 (lexical semantics, WSD, NER), compared in Section 11; ontologies in Sections 12 and 13.

- **Distributional semantics**: the meaning of a word is learnt from the contexts it appears in ("you shall know a word by the company it keeps").
- **Lexical semantics**: the meaning of words and the relations among them (synonyms, antonyms, hypernyms such as *animal* for *dog*).
- **Ontologies**: formal, structured representations of the concepts of a domain and their relations.
- **Word sense disambiguation**: choosing the right meaning of an ambiguous word in context (*bank* of a river or a bank that lends money).
- **Named entity recognition (NER)**: finding names of people, places, organisations, dates, … in text.

3.3.3 Transformers and LLMs

[Slides · Terminology introduction: 31]

BERT and **GPT**, introduced in Section 2.3. Language models, from which LLMs descend, are covered in Section 16.

3.3.4 Features for representing a text

[Slides · Terminology introduction: 32]

Many NLP tasks (classification, sentiment analysis, authorship detection, …) need to turn a text into numbers (Section 14). The most common non-trivial features are:

- **Bag of Words**: a text is represented by the counts of its words, ignoring their order.
- **N-grams**, at word and character level: counts of sequences of n consecutive words ("natural language", $n = 2$) or characters ("nat", $n = 3$), which keep some local order.
- **TF-IDF** (term frequency – inverse document frequency): word counts reweighted so that words frequent in *this* document but rare in the collection count more.
- **Flesch–Kincaid readability**: how hard a text is to read, from the average sentence length and the average number of syllables per word. The grade-level version is
$$\text{FKGL} = 0.39\,\frac{\text{words}}{\text{sentences}} + 11.8\,\frac{\text{syllables}}{\text{words}} - 15.59,$$
roughly the US school grade needed to understand the text.

- **LIWC** (Linguistic Inquiry and Word Count): counts of words in psychologically meaningful categories (emotions, pronouns, cognitive processes, ...), from a hand-built dictionary.
- **Emotion words**, based on models of basic emotions: **Ekman**'s six (anger, disgust, fear, happiness, sadness, surprise) or **Plutchik**'s eight (joy, trust, fear, surprise, sadness, disgust, anger, anticipation).
- ...

3.3.5 Pragmatics

[Slides · Terminology introduction: 33]

Covered in Section 15. "**How to Do Things with Words**" is the title of the book (1962) by the philosopher J. L. Austin that founded the study of *speech acts*. Austin noticed that some sentences do not describe the world but *perform* an action: "I name this ship...", "I promise...", "Shoot her!". Saying them, in the right context, *is* doing the act. Compare them with a statement like "France is hexagonal", which describes the world and can be (roughly) true or false.

Why this matters for NLP

The *performatives* of agent communication languages in the ChatBDI case study (`tell`, `askAll`, `achieve`, ...; Section 2.5) come directly from speech-act theory: every message is classified by the act it performs.

Cheat sheet — levels of linguistic analysis

- The three levels (exam!): **syntax** (what is grammatical?), **semantics** (what does it mean?), **pragmatics** (what does it do?).
- Programming analogy: no compiler errors / no implementation bugs / the right algorithm. $2 + 3$ vs. $3 + 2$: different syntax, same semantics; `3/2` in Python 2.7 vs. 3: same syntax, different semantics; a neural network for a coin flip: good semantics, bad pragmatics.
- Syntax: regular expressions; word level (segmentation, stop words, stemming/lemmatisation, edit distance); sentence level (sentence breaking, POS tagging, parsing).
- Semantics: distributional and lexical semantics, ontologies, WSD, NER.
- Text features: BoW, n-grams, TF-IDF, Flesch–Kincaid, LIWC, emotion words.
- Pragmatics: Austin, *How to Do Things with Words*, speech acts.

4 Terminology

[Slides · Terminology introduction: 34–51; Semantics (long version): 44–50]

Every field has its vocabulary, and NLP borrows a lot of it from linguistics (morpheme, corpus, collocation) and from information retrieval (precision, recall). These terms are

used throughout the course without being redefined, so it pays to fix them now.

This section covers:

1. the units words are made of: morphemes, words and compounds (Section 4.1);
2. a recap of the parts of speech (noun, verb, adjective, ...) with their Italian names and Penn tags (Section 4.2);
3. collections of texts and what we observe in them: corpora, genre, collocations (Section 4.3);
4. how to measure the quality of a system that retrieves documents: precision and recall (Section 4.4).

4.1 Morphemes, Words and Compounds

[Slides · Terminology introduction: 35–38, 42]

4.1.1 Morphemes

[Slides · Terminology introduction: 35–36]

Definition — Morpheme

A **morpheme** is the smallest grammatical unit in a language; in other words, the smallest *meaningful* unit of a language. The field of linguistics that studies morphemes is called **morphology**.

A morpheme is not the same as a word. The principal difference is that a morpheme may or may not stand alone, whereas a word, by definition, is freestanding. In *dogs* there is one word but two morphemes: *dog* (the animal) and *-s* ("more than one"). Neither *d* nor *og* carries meaning, so they are not morphemes.

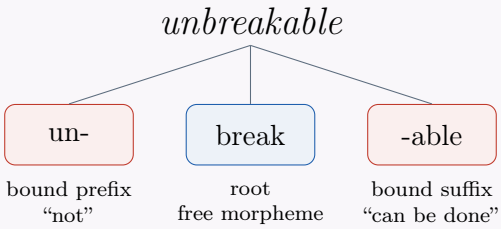
Definition — Free and bound morphemes

Every morpheme is either free or bound.

- A **free morpheme** can function independently as a word (e.g. *town*, *dog*).
- A **bound morpheme** appears only as part of a word, always together with a **root** and sometimes with other bound morphemes. For example, *un-* never appears alone. Most bound morphemes in English are **affixes**: **prefixes** (before the root, *un-*) and **suffixes** (after the root, *-ness*).

Worked example — unbreakable

Unbreakable is made of three morphemes:



Watch out — *-able* is bound

The slide calls *-able* “a free morpheme signifying *can be done*”. As a suffix it is **bound**: it cannot stand alone with the meaning “that can be done” and needs a root (*break-able*, *read-able*). The English word *able* exists, but it is a separate free morpheme with a different use (“she is able to swim”). So *unbreakable* = bound prefix + free root + bound suffix.

4.1.2 Derivational and inflectional morphemes

[Slides · Terminology introduction: 37]

Bound morphemes are further divided according to *what they change*.

Definition — Derivational and inflectional morphemes

- A **derivational morpheme**, combined with a root, changes the **meaning** or the **part of speech** of the word: it creates a *new word*.
- An **inflectional morpheme** modifies a verb’s tense, aspect, mood, person or number, or a noun’s, pronoun’s or adjective’s number, gender or case, *without* changing the word’s meaning or class (part of speech): it creates a different *form of the same word*.

Root	Morpheme	Result	What changed
Derivational			
happy (adjective)	-ness	happiness (noun)	part of speech: adjective → noun
kind	un-	unkind	meaning is inverted
Inflectional			
dog	-s	dogs	number (singular → plural)
wait	-ed	waited	tense (present → past)

Table 4: Derivation makes a new word, inflection a new form of the same word.

Intuition — a dictionary test

Ask: “would a dictionary give the result its own entry?” *Happiness* and *unkind* have their own entries (derivation), *dogs* and *waited* are found under *dog* and *wait* (inflection). This is exactly the difference between *lemmatisation* (going back to the dictionary form, removing inflection) and the analysis of word formation,

which we will meet later in the course.

4.1.3 Words

[Slides · Terminology introduction: 38]

Definition — Word

A **word** is the smallest element that can be uttered in isolation with objective or practical meaning.

This contrasts with a morpheme, which is the smallest unit of meaning but does not necessarily stand on its own:

- a word may consist of a **single morpheme**: *oh!*, *rock*, *red*, *quick*, *run*, *expect*;
- or of **several**: *rock-s*, *red-ness*, *quick-ly*, *runn-ing*, *un-expect-ed*;
- and a morpheme may be unable to stand on its own as a word: in the examples, *-s*, *-ness*, *-ly*, *-ing*, *un-*, *-ed*.

4.1.4 Compounds

[Slides · Terminology introduction: 42]

Definition — Compound

A **compound** is a word composed of more than one *free* morpheme.

The last part is usually the **head** (it gives the category of the whole word), and the first part is the **modifier**. Compounds can be written as one word (*football*), with a hyphen (*blue-green*) or even as two words (*snow white*).

Modifier	Head	Compound	Modifier	Head	Compound
noun	noun	football	noun	verb	browbeat
adjective	noun	blackboard	adjective	verb	highlight
verb	noun	breakwater	verb	verb	freeze-dry
preposition	noun	underworld	preposition	verb	undercut
noun	adjective	snow white	noun	preposition	love-in
adjective	adjective	blue-green	adverb	preposition	forthwith
verb	adjective	tumbledown	verb	preposition	takeout
preposition	adjective	over-ripe	preposition	preposition	without

Table 5: English compounds by the category of their two parts.

Watch out — compound vs. derived word

Blackboard is a compound (*black* and *board* are both free). *Unkind* is *not* a compound: *un-* is bound, so it is a derived word. Also note that the meaning of a compound is often not the sum of its parts: a *blackboard* can be green.

4.2 Parts of Speech: a Grammar Recap

[Not in the slides: added material]

The course often talks about nouns, pronouns, adjectives, determiners… without defining them: they are assumed from school grammar. This subsection is a quick recap to come back to whenever needed, with the link to the Italian grammar you already know and the Penn Treebank tags used in POS tagging (Section 7.2). Italian translations are always in *green*; most tables pair each English word with its Italian counterpart, one per row.

Intuition — English and Italian names

The English **parts of speech** (also *word classes* or *lexical categories*) are the Italian **parti del discorso**. Assigning one to each word of a sentence is what Italian school calls **analisi grammaticale**, and what NLP calls **POS tagging**. The sentence-level analysis (subject, object, …) is the **analisi logica**, recalled at the end of this subsection.

4.2.1 Overview

[Not in the slides: added material]

English	Italiano	Class	Penn tags	Examples
Noun	nome (sostantivo)	open	NN, NNS, NNP, NNPS	cat *gatto*, ideas *idee*, Rome *Roma*
Verb	verbo	open	VB, VBD, VBG, VBN, VBP, VBZ, MD	run *correre*, ran *corse*, is *è*, can *può*
Adjective	aggettivo (qualificativo)	open	JJ, JJR, JJS	red *rosso*, bigger *più grande*, best *il migliore*
Adverb	avverbio	open	RB, RBR, RBS, WRB	quickly *velocemente*, very *molto*, where *dove*
Pronoun	pronome	closed	PRP, PRP$, WP, WP$, EX	I *io*, me *me/mi*, mine *il mio*, who *chi*
Determiner	articolo, agg. determinativo	closed	DT, PDT, WDT	the *il/la*, a *un/una*, this *questo*, every *ogni*
Preposition	preposizione	closed	IN, TO	of *di*, to *a*, from *da*, in *in*
Conjunction	congiunzione	closed	CC, IN	and *e*, but *ma*, because *perché*
Interjection	interiezione	closed	UH	oh *oh*, wow *caspita*, ouch *ahi*
Numeral	aggettivo numerale	closed	CD	three *tre*, 1908
Particle	(no equivalent)	closed	RP	give *up* *rinunciare*, turn *off* *spegnere*

Table 6: Parts of speech at a glance. *Open* classes accept new words all the time (*to google* *googlare*, *selfie*); *closed* classes are small fixed lists (Section 7.3).

Watch out — variable and invariable

In Italian, five parts of speech are *variabili* (articolo, nome, aggettivo, pronome, verbo: they change with gender, number, person, tense) and four are *invariabili* (avverbio, preposizione, congiunzione, interiezione). English inflects much less: **articles and adjectives never change** (*the red car la macchina rossa*, *the red cars le macchine rosse*: in English only *car* changes), and nouns have number but **no grammatical gender**.

4.2.2 Nouns

[Not in the slides: added material]

A **noun** names a person, animal, thing, place or idea. **Italiano**: *nome* / *sostantivo*.

Type	Italiano	Examples	Penn
common	nome comune	city *città*, dog *cane*	NN
proper	nome proprio	Genoa *Genova*, Mary *Maria*	NNP
plural	plurale	dogs *cani*, children *bambini*, the Alps *le Alpi*	NNS, NNPS
countable	numerabile	a chair *una sedia*, two chairs *due sedie*	NN, NNS
uncountable (mass)	non numerabile	water *acqua*, information *informazioni*, advice *consigli*	NN
concrete / abstract	concreto / astratto	table *tavolo* / happiness *felicità*	NN
compound	nome composto	football *calcio*, bus stop *fermata dell'autobus*	NN
possessive *'s*	complemento di specificazione	Mary's book *il libro di Maria*	NNP + POS

Differences with Italian: no gender (*the table il tavolo*, *the chair la sedia*: the same *the*); uncountable nouns have no plural and no *a* even when Italian uses a plural (*some information delle informazioni*, *a piece of advice un consiglio*).

4.2.3 Pronouns

[Not in the slides: added material]

A **pronoun** stands for a noun (or a whole noun phrase). **Italiano**: *pronome*.

Personal, possessive and reflexive pronouns. One row per person:

Person	**Subject** *pers. soggetto*	**Object** *pers. complemento*	**Possessive adj.** *agg. possessivo*	**Possessive pron.** *pron. possessivo*	**Reflexive** *riflessivo*
1s	I *io*	me *me, mi*	my *mio*	mine *il mio*	myself *mi, me stesso*
2s	you *tu*	you *te, ti*	your *tuo*	yours *il tuo*	yourself *ti, te stesso*
3s m.	he *lui*	him *lui, lo, gli*	his *suo*	his *il suo*	himself *si, sé stesso*
3s f.	she *lei*	her *lei, la, le*	her *suo*	hers *il suo*	herself *si, sé stessa*
3s thing	it *esso, –*	it *lo, la*	its *suo*	–	itself *si*
1p	we *noi*	us *noi, ci*	our *nostro*	ours *il nostro*	ourselves *ci, noi stessi*
2p	you *voi*	you *voi, vi*	your *vostro*	yours *il vostro*	yourselves *vi, voi stessi*
3p	they *loro*	them *loro, li, le*	their *loro*	theirs *il loro*	themselves *si, sé stessi*
Penn tag	PRP	PRP	PRP$	PRP	PRP

The other pronouns.

Type	**English**	**Italiano**	**Penn**
Demonstrative	this / these	*questo, questa / questi, queste*	DT
	that / those	*quello, quella / quelli, quelle*	DT
Relative	who (people)	*che, il quale* *the man who called l'uomo che ha chiamato*	WP
	which (things)	*che, il quale* *the book which I read il libro che ho letto*	WDT
	that (people, things)	*che*	WDT
	whom	*cui* *the girl to whom I wrote la ragazza a cui ho scritto*	WP
	whose	*il cui* *the man whose car... l'uomo la cui macchina...*	WP$
Interrogative	who?	*chi?*	WP
	what?	*che cosa?*	WP
	which?	*quale?*	WDT
	whose?	*di chi?*	WP$
Indefinite	someone / somebody	*qualcuno*	NN
	something	*qualcosa*	NN
	nobody / no one	*nessuno*	NN
	nothing	*niente, nulla*	NN
	everybody / everyone	*tutti, ognuno*	NN
	everything	*tutto*	NN
	anybody / anything	*chiunque / qualsiasi cosa* (in questions: *qualcuno / qualcosa*)	NN

Differences with Italian:

- Italian is *pro-drop*: the subject pronoun can be omitted (*mangio*). English always needs it (*I eat*), even a dummy one: *it rains piove*, *there is a cat c'è un gatto* (*there* here is "existential", tag EX).
- The possessive adjective has *no article* in English: *my book il mio libro*; and it agrees with the *owner*: *her brother suo fratello* (of a woman), *his sister sua sorella* (of a man).

- Italian unstressed object pronouns (*mi, ti, lo, la, ci, vi, li, le, gli*) go before the verb, English ones after it: *lo vedo I see him.*

4.2.4 Verbs

[Not in the slides: added material]

A **verb** expresses an action, an event or a state. **Italiano**: *verbo.*

The forms of an English verb. English has only a few verb forms, and they are exactly what the Penn tags list:

Tag	Form	Example	Italian equivalent
VB	base form	*to eat mangiare, I can eat posso mangiare*	infinito
VBP	present, not 3rd sing.	*I/we/they eat mangio / mangiamo / mangiano*	presente indicativo
VBZ	present, 3rd sing.	*she eats (lei) mangia*	presente, 3ª sing.
VBD	past tense	*she ate mangiò / mangiava / ha mangiato*	passato remoto / imperfetto / pass. prossimo
VBN	past participle	*she has eaten ha mangiato*	participio passato
VBG	gerund / present participle	*she is eating sta mangiando, eating is fun mangiare è divertente*	gerundio, infinito sostantivato

English tenses and Italian tempi. Everything beyond those forms is built with auxiliaries (*be, have, do*) and modals:

English tense	Example	Italiano	Tempo / modo
present simple	I eat	*mangio*	presente
present continuous	I am eating	*sto mangiando / mangio*	presente (stare + gerundio)
past simple	I ate	*mangiai / ho mangiato*	passato remoto / prossimo
past continuous	I was eating	*mangiavo / stavo mangiando*	imperfetto
present perfect	I have eaten	*ho mangiato*	passato prossimo
past perfect	I had eaten	*avevo mangiato*	trapassato prossimo
future	I will eat	*mangerò*	futuro semplice
future perfect	I will have eaten	*avrò mangiato*	futuro anteriore
conditional	I would eat	*mangerei*	condizionale presente
past conditional	I would have eaten	*avrei mangiato*	condizionale passato
imperative	eat!	*mangia!*	imperativo
passive	it is / was eaten	*è / fu mangiato*	forma passiva

The Italian *congiuntivo* has (almost) no English form: *I think he is right penso che abbia ragione, if I were rich se fossi ricco.*

Auxiliaries and modals.

English	Italiano	Example	Penn
be	*essere* (also *stare*)	*she is tired è stanca*, *she is eating sta mangiando*	VB...
have	*avere*	*she has eaten ha mangiato*	VB...
do	– (no equivalent)	*do you know? sai?*, *I don't know non so*	VB...
can	*potere, sapere*	*I can swim so nuotare*, *can I go? posso andare?*	MD
must	*dovere*	*you must go devi andare*	MD
should	*dovrei* (condizionale)	*you should rest dovresti riposare*	MD
may / might	*potrei, può darsi che*	*it might rain potrebbe piovere*	MD
will	(futuro)	*she will come verrà*	MD
would	(condizionale)	*she would come verrebbe*	MD

Modals never change form (*she can*, not *she cans*) and are followed by the base form (VB). They correspond to the Italian *verbi servili* (*potere, dovere, volere*) or to a tense of the main verb.

Other verb distinctions.

- **Transitive** *transitivo*: *she reads a book lei legge un libro*; **intransitive** *intransitivo*: *she sleeps lei dorme*.
- **Phrasal verbs**: verb + particle (RP) with a new meaning: *give up rinunciare*, *look after badare a*, *turn off spegnere*, *find out scoprire*.

4.2.5 Adjectives

[Not in the slides: added material]

An **adjective** describes or qualifies a noun. **Italiano**: *aggettivo qualificativo*.

Degree *(grado)*	**Short adjective**	**Long adjective**	**Penn**
positive *(positivo)*	big *grande*	expensive *costoso*	JJ
comparative of majority *(comp. di maggioranza)*	bigger than *più grande di*	more expensive than *più costoso di*	JJR / RBR + JJ
comparative of minority *(comp. di minoranza)*	less big than *meno grande di*	less expensive than *meno costoso di*	RBR + JJ
comparative of equality *(comp. di uguaglianza)*	as big as *grande come*	as expensive as *costoso come*	RB + JJ
relative superlative *(superlativo relativo)*	the biggest *il più grande*	the most expensive *il più costoso*	JJS / RBS + JJ
absolute superlative *(superlativo assoluto)*	very big *grandissimo*	very expensive *costosissimo*	RB + JJ

Irregular: *good, better, best buono, migliore, il migliore*; *bad, worse, worst cattivo, peggiore, il peggiore*; *far, further, furthest lontano, più lontano, il più lontano*.

Position: **attributive** *a red car una macchina rossa* vs. **predicative** *the car is red la macchina è rossa* (*funzione attributiva / predicativa*).

Differences with Italian: English adjectives are invariable and come *before* the noun: *a red car, two red cars una macchina rossa, due macchine rosse*. What Italian calls *aggettivi determinativi* (possessivi, dimostrativi, indefiniti, interrogativi, numerali) are **not** adjectives in English grammar: *my mio* is PRP$, *this questo* and *every ogni* are DT, *three tre* is CD.

4.2.6 Adverbs

[Not in the slides: added material]

An **adverb** modifies a verb, an adjective, another adverb or a whole sentence. **Italiano**: *avverbio*. Manner adverbs are formed with *-ly*, the Italian *-mente*: *quick-ly veloce-mente*.

Type	**English → Italiano**	**Penn**
manner *(di modo)*	quickly *velocemente*, slowly *lentamente*, well *bene*, badly *male*	RB
time *(di tempo)*	now *ora*, today *oggi*, yesterday *ieri*, tomorrow *domani*, soon *presto*, already *già*, still *ancora*	RB, NN
place *(di luogo)*	here *qui*, there *lì*, everywhere *ovunque*, inside *dentro*, outside *fuori*	RB
frequency *(di frequenza)*	always *sempre*, often *spesso*, sometimes *a volte*, never *mai*	RB
quantity *(di quantità)*	very *molto*, too *troppo*, quite *abbastanza*, a little *un po'*, almost *quasi*	RB
affirmation / negation	yes *sì*, not *non*, also *anche*, certainly *certamente*	RB, UH
interrogative *(interrogativo)*	when? *quando?*, where? *dove?*, why? *perché?*, how? *come?*, how much? *quanto?*	WRB

Comparison works as for adjectives: *fast velocemente*, *faster più velocemente* (RBR), *fastest il più velocemente* (RBS).

4.2.7 Determiners and articles

[Not in the slides: added material]

A **determiner** comes before a noun and specifies *which* or *how many*. **Italiano**: *articolo* and most *aggettivi determinativi*.

Type	English	Italiano	Penn
definite article *(determinativo)*	the	*il, lo, la, l', i, gli, le*: *the boy, the girl, the books* *il ragazzo, la ragazza, i libri*	DT
indefinite article *(indeterminativo)*	a / an	*un, uno, una, un'*: *a dog, an apple, an hour* *un cane, una mela, un'ora*	DT
partitive *(partitivo)*	some / –	*del, dello, della, dei, degli, delle*: *some bread* *del pane*	DT
demonstrative *(dimostrativo)*	this / these	*questo, questa / questi, queste*	DT
	that / those	*quel, quello, quella / quei, quegli, quelle*	DT
quantifier *(indefinito)*	every / each	*ogni / ciascuno*: *every day* *ogni giorno*	DT
	some / any	*qualche, alcuni / qualsiasi*	DT
	no	*nessuno*: *no money* *nessun soldo*	DT
	all / both	*tutti / entrambi*	DT, PDT
	many / much	*molti / molto*: *many books, much time* *molti libri, molto tempo*	JJ
	few / little	*pochi / poco*	JJ
interrogative *(interrogativo)*	which? / what?	*quale? / che?*: *which book?* *quale libro?*	WDT
	whose?	*di chi?*: *whose car?* *di chi è la macchina?*	WP$

The English articles never change: one *the* for all seven Italian forms, and the choice between *a* and *an* depends only on the *sound* that follows (*an hour*, *a university*). Predeterminers (PDT) come before another determiner: *all the cats* *tutti i gatti*, *both the girls* *entrambe le ragazze*.

4.2.8 Prepositions

[Not in the slides: added material]

A **preposition** links a noun phrase to the rest of the sentence, expressing place, time, means, ... **Italiano**: *preposizione*.

The Italian preposizioni semplici and their English translations. There is no one-to-one mapping: each Italian preposition has several English translations, depending on the meaning.

Italiano	English	Examples
di	of	*a glass of wine un bicchiere di vino*
	from	*I am from Genoa sono di Genova*
	about	*let's talk about it parliamone (parliamo di questo)*
	than	*taller than me più alto di me*
	's	*Mary's book il libro di Maria*
a	to	*I go to Rome vado a Roma*, *I give it to Bob lo do a Bob*
	at	*at home a casa*, *at 5 alle 5*
	in	*I live in Genoa abito a Genova*
da	from	*from Rome to Milan da Roma a Milano*
	by	*written by Dante scritto da Dante*
	for / since	*for two years da due anni*, *since 2020 dal 2020*
	at / to someone's	*at Mario's da Mario*, *to the doctor dal dottore*
in	in / into	*in Italy in Italia*, *into the room nella stanza*
	by	*by train in treno*
con	with	*with Bob con Bob*, *with a knife con un coltello*
su	on	*on the table sul tavolo*
	about	*a book about Rome un libro su Roma*
	out of	*one out of ten uno su dieci*
per	for	*for you per te*, *for two hours per due ore*
	through	*through the city per la città*
	to (purpose)	*I study to learn studio per imparare*
tra / fra	between	*between Rome and Milan tra Roma e Milano*
	among	*among friends tra amici*
	in (time)	*in two hours tra due ore*

Other common English prepositions. *under sotto*, *over / above sopra*, *near vicino a*, *behind dietro*, *in front of davanti a*, *before prima di*, *after dopo*, *during durante*, *until fino a*, *without senza*, *against contro*, *towards verso*.

Preposizioni articolate. English has none: the preposition and the article stay two separate words (two tokens).

Italiano	*del* (di + il)	*al* (a + il)	*dal* (da + il)	*nel* (in + il)
English	of the / from the	to the / at the	from the / by the	in the
Italiano	*sul* (su + il)	*della, alla, dalla, nella, sulla*	*dei, ai, dai, nei, sui*	...
English	on the	of/to/from/in/on the	of/to/from/in/on the	

In Penn, all prepositions are IN except *to*, which always gets its own tag TO (also in *to run correre*).

4.2.9 Conjunctions

[Not in the slides: added material]

A **conjunction** connects words, phrases or clauses. **Italiano**: *congiunzione*.

Type	**English**	**Italiano**	**Example**
coordinating *(coordinanti)*, CC	and	*e*	*bread and butter pane e burro*
	or	*o, oppure*	*tea or coffee? tè o caffè?*
	but	*ma, però*	*tired but happy stanco ma felice*
	nor	*né*	*neither Bob nor Mary né Bob né Maria*
	so	*quindi, perciò*	*it rained, so I stayed pioveva, quindi sono rimasto*
subordinating *(subordinanti)*, IN	because	*perché, poiché*	*I stayed because it rained sono rimasto perché pioveva*
	although	*sebbene, anche se*	*although it was late sebbene fosse tardi*
	if	*se*	*if you want se vuoi*
	when	*quando*	*when I arrived quando sono arrivato*
	while	*mentre*	*while she slept mentre dormiva*
	that	*che*	*I know that you are right so che hai ragione*
	until	*finché*	*wait until I call aspetta finché non chiamo*
	so that	*affinché*	*so that you understand affinché tu capisca*
	as / since	*siccome*	*as it was late, ... siccome era tardi, ...*

Penn uses IN for both prepositions and subordinating conjunctions: *before lunch prima di pranzo* and *before I left prima che partissi* get the same tag.

4.2.10 Interjections, numerals and particles

[Not in the slides: added material]

Type	English → Italiano	Penn
interjection *(interiezione)*	oh *oh*, wow *caspita*, ouch *ahi*, hey *ehi*, yes *sì*, no *no*, hello *ciao*	UH
cardinal *(numerale cardinale)*	one *uno*, three *tre*, twenty *venti*, 1908, 3.14 *3,14*	CD
ordinal *(numerale ordinale)*	first *primo*, second *secondo*, third *terzo*	JJ
particle (phrasal verbs)	give up *rinunciare*, turn off *spegnere*, go away *andare via*	RP

Particles look like prepositions but belong to the verb, changing its meaning. No Italian part of speech corresponds; the closest is an adverb (*andare via*, *tirare su*). Note the decimal separator: *3.14* in English, *3,14* in Italian, which matters for tokenisers (Section 7.1).

4.2.11 Worked example: analisi grammaticale in English

[Not in the slides: added material]

Worked example — tagging a sentence

Wow, there are three very big dogs in your garden, but my sister did not see them because she was running.
Caspita, ci sono tre cani molto grandi nel tuo giardino, ma mia sorella non li ha visti perché stava correndo.

Word	it.	Tag	Parte del discorso	Word	it.	Tag	Parte del discorso
Wow	*caspita*	UH	interiezione	my	*mia*	PRP$	agg. possessivo
there	*ci*	EX	(sogg. fittizio)	sister	*sorella*	NN	nome comune
are	*sono*	VBP	verbo (essere)	did	–	VBD	verbo ausiliare
three	*tre*	CD	agg. numerale	not	*non*	RB	avv. di negazione
very	*molto*	RB	avv. di quantità	see	*visti*	VB	verbo
big	*grandi*	JJ	agg. qualificativo	them	*li*	PRP	pron. pers. compl.
dogs	*cani*	NNS	nome comune, plur.	because	*perché*	IN	cong. subordinante
in	*nel*	IN	preposizione	she	–	PRP	pron. pers. sogg.
your	*tuo*	PRP$	agg. possessivo	was	*stava*	VBD	verbo ausiliare
garden	*giardino*	NN	nome comune	running	*correndo*	VBG	verbo (gerundio)
but	*ma*	CC	cong. coordinante				

The "–" cells show two differences from Italian: the auxiliary *did* has no Italian counterpart (Italian negates with *non* alone), and the subject *she* is dropped in Italian (*stava correndo*). Note also *in your* → *nel tuo* (preposizione articolata). A statistical tagger (NLTK's default one) gets most of this right but tags *Wow* as NNP (a proper noun): a capitalised unknown word at the start of a sentence looks like a name, and interjections are rare in the newspaper text taggers are trained on.

4.2.12 From words to sentences: analisi logica

[Not in the slides: added material]

Parts of speech classify *words*. The **functions** words and phrases play in a sentence are the subject of *analisi logica*; in the parse trees of Section 7.3 they correspond to positions in the tree. Running example: *Cynthia sold the bike to Bob for $200 yesterday ieri Cynthia ha venduto la bici a Bob per 200 dollari*.

English	**Italiano**	**In the parse tree / example**
Subject	soggetto	the NP before the verb: *Cynthia Cynthia*
Predicate (verb)	predicato verbale	the verb of the VP: *sold ha venduto*
Predicate (*be* + noun/adj.)	predicato nominale	*she is happy lei è felice* (*happy* = nome del predicato)
Direct object	complemento oggetto	the NP after the verb: *the bike la bici*
Indirect object	complemento di termine	*to Bob a Bob*; also without *to*: *gave Bob the bike diede la bici a Bob*
Adverbials	complementi indiretti (prezzo, tempo, luogo, . . .)	PPs and adverbs: *for $200 per 200 dollari*, *yesterday ieri*, *in Genoa a Genova*

Why this matters for NLP

These functions are what semantic role labelling turns into roles (Section 9.1): in the running example the subject is the SELLER, the direct object the GOODS, the indirect object the BUYER, the *for*-phrase (*complemento di prezzo*) the PRICE.

Cheat sheet — parts of speech

- Open classes: **noun** (NN. . . = nome), **verb** (VB. . . , MD = verbo), **adjective** (JJ. . . = agg. qualificativo), **adverb** (RB. . . = avverbio).
- Closed classes: **pronoun** (PRP, WP = pronome), **determiner** (DT = articolo, agg. determinativi), **preposition** (IN, TO = preposizione), **conjunction** (CC coordinating, IN subordinating = congiunzione), **interjection** (UH = interiezione); plus numerals (CD) and particles (RP).
- English vs. Italian: no gender, invariable articles and adjectives, adjectives before the noun, subject pronoun always required, no preposizioni articolate, prepositions do not map one-to-one, few verb forms (the rest via auxiliaries and modals), *do* in questions and negations.
- Analisi grammaticale = POS tagging; analisi logica = functions in the sentence (subject, predicate, objects, adverbials).

4.3 Corpora, Genre and Collocations

[Slides · Terminology introduction: 39–41]

4.3.1 Corpus

[Slides · Terminology introduction: 39]

Definition — Corpus (plural: corpora)

A **text corpus** is a large and structured set of texts (nowadays usually stored and processed electronically). Corpora are used for statistical analysis and hypothesis testing, for checking occurrences, and for validating linguistic rules within a specific language territory.

Example: the corpora by Mark Davies, originally at `https://corpus.byu.edu/` and now at `https://www.english-corpora.org/`: COCA (Corpus of Contemporary American English), NOW (News on the Web), and others. You can search them to see how often and in which contexts a word or phrase is used.

Why this matters for NLP

Corpora are the raw material of statistical and neural NLP (Section 2.3): probabilities, word embeddings and LLMs are all learnt from corpora. What the corpus contains (its size, languages, genres, time period) determines what the model learns.

4.3.2 Genre

[Slides · Terminology introduction: 40]

Definition — Genre

- **Genre**: a style or category of art, music or literature.
- **Genre of a corpus**: usually defined on the basis of *text-external* features, such as medium, function and format (e.g. newspaper articles, academic papers, fiction, spoken conversation transcripts, social media posts).

"Text-external" means that the genre is decided by *where the text comes from and what it is for*, not by counting words inside it. Texts of different genres differ a lot (sentence length, vocabulary, formality), so a model trained on newspapers often performs worse on tweets.

4.3.3 Collocations

[Slides · Terminology introduction: 41]

Definition — Collocation

A **collocation** is a sequence of words or terms that co-occur *more often than would be expected by chance*. Examples: "pay attention", "fast food", "make an effort", "powerful engine".

Fast and *quick* mean roughly the same thing, yet native speakers combine them with different nouns (example from the NOW corpus):

Natural English	Unnatural English
the fast train	the ~~quick~~ train
fast food	~~quick~~ food
a quick shower	a ~~fast~~ shower
a quick meal	a ~~fast~~ meal

Intuition — "more often than by chance"

If words were combined at random, *fast food* and *quick food* would appear about as often as the frequencies of *fast* and *quick* predict. In a corpus, *fast food* appears far more often than that, and *quick food* hardly ever. That excess is what makes it a collocation. Nothing in grammar or meaning forbids "quick food": collocations are a matter of *usage*, which is why they are learnt from corpora.

4.4 Precision and Recall

[Slides · Terminology introduction: 43–51; Semantics (long version): 44–50]

Suppose a program searches a collection of documents and returns those it thinks are relevant to a query. How good is it? Two questions matter, and they pull in opposite directions:

- of what it *returned*, how much is actually relevant? (**precision**)
- of what is actually relevant, how much did it *find*? (**recall**)

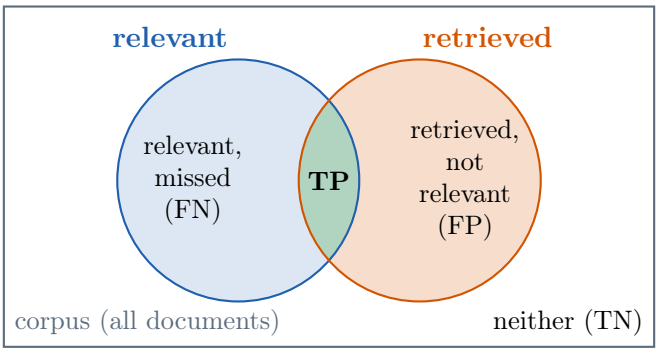


Figure 6: The green overlap (TP) holds the documents that are both relevant and retrieved. Precision looks at the orange circle (what fraction of it is green), recall looks at the blue circle (what fraction of it is green).

Definition — Precision and recall

$$\text{precision} = \frac{|\{\text{relevant documents}\} \cap \{\text{retrieved documents}\}|}{|\{\text{retrieved documents}\}|}$$

$$\text{recall} = \frac{|\{\text{relevant documents}\} \cap \{\text{retrieved documents}\}|}{|\{\text{relevant documents}\}|}$$

Precision (also called *positive predictive value*) is the fraction of retrieved doc-

uments that are relevant to the query. **Recall** (also called *sensitivity*) is the fraction of relevant documents that are successfully retrieved. Both are between 0 and 1; $|S|$ denotes the number of elements of the set S.

The numerator is the *same* in both formulas: the documents that are both relevant and retrieved. Only the denominator changes: precision divides by what the system *returned*, recall by what it *should have returned*.

4.4.1 The same thing with a confusion matrix

[Not in the slides: added material]

Each document falls in one of four cases:

	relevant	**not relevant**
retrieved	true positive (TP)	false positive (FP)
not retrieved	false negative (FN)	true negative (TN)

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}.$$

Neither formula uses TN: the (usually huge) number of irrelevant documents that were correctly left out does not count.

4.4.2 Four worked examples

[Slides · Terminology introduction: 44–51]

The setting is the same for all four examples. You implement an application that retrieves the documents containing one specific word ("cat") and its inflectional forms ("cats", see Section 4.1) in a corpus of **1000** documents. Exactly **24** documents contain "cat": these are the **relevant** documents.

Worked example — 1: a typical system

The application retrieves **36** documents; **16** of them contain "cat" and 20 do not. So $TP = 16$, $FP = 20$, $FN = 24 - 16 = 8$, $TN = 1000 - 16 - 20 - 8 = 956$.

$$\text{precision} = \frac{16}{36} = \frac{4}{9} \approx 0.444 \qquad \triangleleft\ TP/(TP + FP) = 16/(16 + 20)$$

$$\text{recall} = \frac{16}{24} = \frac{2}{3} \approx 0.667 \qquad \triangleleft\ TP/(TP + FN) = 16/(16 + 8)$$

Fewer than half of the results are right, and a third of the cat documents are missed.

Worked example — 2: return everything

The application retrieves **all 1000** documents; **24** of them contain "cat" and 976 do not. So $TP = 24$, $FP = 976$, $FN = 0$, $TN = 0$.

$$\text{precision} = \frac{24}{1000} = 0.024$$
$$\text{recall} = \frac{24}{24} = 1$$

Perfect recall, for free: a system that returns everything never misses anything. Its precision is terrible.

Watch out — a typo in the slides

Example 2 on the slides says the retrieved documents "out of which 24 contain the word cat, and 1986 do not contain it". Since only 1000 documents are retrieved, the correct number is $1000 - 24 = 976$. The precision and recall values on the slides are correct.

Worked example — 3: play it safe

The application retrieves only **2** documents, and both contain "cat". So $TP = 2$, $FP = 0$, $FN = 22$, $TN = 976$.

$$\text{precision} = \frac{2}{2} = 1$$
$$\text{recall} = \frac{2}{24} = \frac{1}{12} \approx 0.083$$

Perfect precision, almost for free: return only the one or two results you are completely sure about. Its recall is terrible.

Worked example — 4: the perfect system

The application retrieves **24** documents, all containing "cat". So $TP = 24$, $FP = 0$, $FN = 0$, $TN = 976$.

$$\text{precision} = \frac{24}{24} = 1, \qquad \text{recall} = \frac{24}{24} = 1.$$

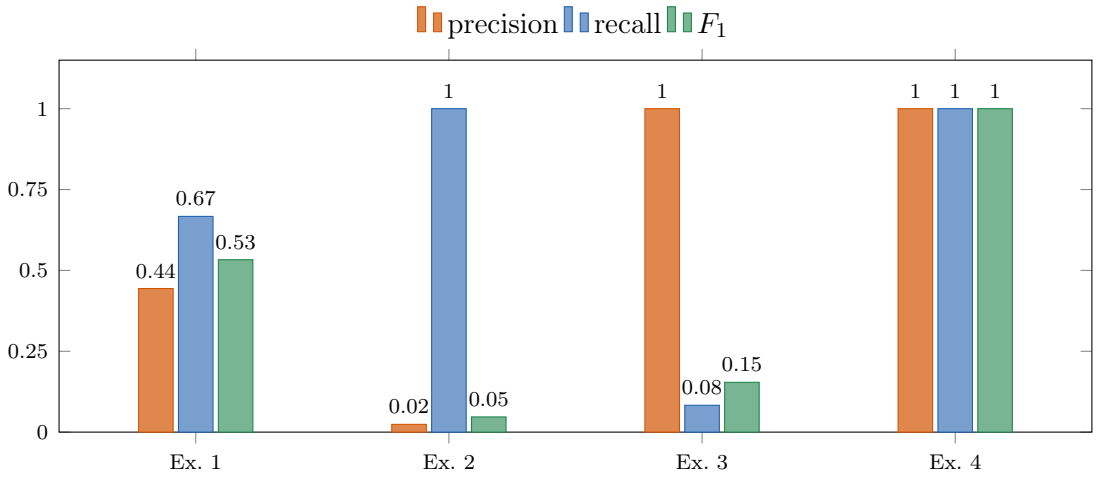


Figure 7: Precision, recall and F_1 for the four examples. Examples 2 and 3 each score perfectly on one measure with a trivial strategy; only F_1 exposes them.

Intuition — the trade-off

Precision and recall must always be read *together*. Returning more documents tends to raise recall and lower precision (Example 2 is the extreme); returning fewer tends to raise precision and lower recall (Example 3). A single number that is high only when *both* are high is the F_1 score below.

Definition — F_1 score (not on the slides)

The F_1 **score** is the harmonic mean of precision P and recall R:

$$F_1 = \frac{2PR}{P+R} = \frac{2\,TP}{2\,TP + FP + FN}.$$

The harmonic mean is dominated by the smaller of the two values, so a system cannot get a good F_1 by maximising only one of them.

For the four examples:

$$\begin{aligned}
\text{Ex. 1:}\quad F_1 &= \frac{2 \cdot 16}{2 \cdot 16 + 20 + 8} = \frac{32}{60} \approx 0.533 \\
\text{Ex. 2:}\quad F_1 &= \frac{2 \cdot 24}{2 \cdot 24 + 976 + 0} = \frac{48}{1024} \approx 0.047 \\
\text{Ex. 3:}\quad F_1 &= \frac{2 \cdot 2}{2 \cdot 2 + 0 + 22} = \frac{4}{26} \approx 0.154 \\
\text{Ex. 4:}\quad F_1 &= \frac{2 \cdot 24}{2 \cdot 24 + 0 + 0} = 1
\end{aligned}$$

4.4.3 Another example: five algorithms

[Slides · Semantics (long version): 44–50]

The long version of the semantics slides repeats the definitions with a second example.

A query asks for the documents about topic T in a corpus C. The correct answer is known: $D_1, D_2, D_4, D_5, D_6, D_7, D_8$ deal with T (7 relevant documents) and no other document does.

Algorithm	**Returns**	$\lvert\text{rel} \cap \text{ret}\rvert$	**Precision**	**Recall**
A1	D_1, D_2, D_3, D_4	3	$3/4 = 75\%$	$3/7 \approx 43\%$
A2	$D_1, \ldots, D_8$	7	$7/8 = 87.5\%$	$7/7 = 100\%$
A3	D_1 only	1	$1/1 = 100\%$	$1/7 \approx 14\%$
A4	all and only the relevant	7	$7/7 = 100\%$	$7/7 = 100\%$
A5	1000 docs, incl. the 7 relevant	7	$7/1000 = 0.7\%$	$7/7 = 100\%$

The same lessons as before: A3 buys perfect precision by returning almost nothing, A5 buys perfect recall by returning almost everything; only A4 is perfect on both. (On the slide for A2 the formulas are labelled "precision(A1)" and "recall(A1)": a copy-paste slip, the values are those of A2.)

4.4.4 Exercises

[Not in the slides: added material]

Exercise 1 — morphemes

Split each word into morphemes and classify each morpheme as free or bound, and each bound morpheme as derivational or inflectional: *unhappiness*, *walked*, *teachers*, *rewritten*. Which of *bookshelf*, *bookish*, *books* is a compound?

Solution

- *un-happi-ness*: *un-* bound, derivational (negates the meaning); *happy* free root; *-ness* bound, derivational (adjective → noun).
- *walk-ed*: *walk* free root; *-ed* bound, inflectional (past tense).
- *teach-er-s*: *teach* free root; *-er* bound, derivational (verb → noun, "one who teaches"); *-s* bound, inflectional (plural).
- *re-writ-ten*: *re-* bound, derivational ("again"); *write* free root; *-en* bound, inflectional (past participle).

Only *bookshelf* is a compound (*book* + *shelf*, both free). *Bookish* is derived (*-ish* is bound), *books* is inflected.

Lesson: derivational morphemes sit closer to the root than inflectional ones (*teach-er-s*, not *teach-s-er*).

Exercise 2 — precision and recall

A corpus has 500 documents, 40 of which are relevant to a query. A search engine returns 50 documents, 30 of which are relevant. Compute TP, FP, FN, TN, precision, recall and F_1. What happens to precision and recall if the engine also returns 50 more documents, none of them relevant?

Solution

$TP = 30$, $FP = 50-30 = 20$, $FN = 40-30 = 10$, $TN = 500-30-20-10 = 440$.

$$\text{precision} = \frac{30}{50} = 0.6, \qquad \text{recall} = \frac{30}{40} = 0.75,$$
$$F_1 = \frac{2 \cdot 30}{2 \cdot 30 + 20 + 10} = \frac{60}{90} \approx 0.667.$$

With 50 more irrelevant documents: $TP = 30$, $FP = 70$, $FN = 10$, so precision $= 30/100 = 0.3$ (halved) while recall stays $30/40 = 0.75$, and $F_1 = 60/(60 + 70 + 10) = 60/140 \approx 0.429$.
Lesson: adding wrong results never changes recall but always lowers precision.

Cheat sheet — terminology

- **Morpheme**: smallest meaningful unit; **free** (stands alone: *dog*) or **bound** (*un-*, *-ness*, *-s*).
- Bound morphemes: **derivational** (new word: meaning or part of speech changes, *happy* → *happiness*) or **inflectional** (new form of the same word: tense, number, …; *dog* → *dogs*).
- **Word**: smallest element uttered in isolation with meaning; one or more morphemes. **Compound**: word made of ≥ 2 *free* morphemes.
- **Corpus**: large structured set of texts. **Genre of a corpus**: from text-external features (medium, function, format). **Collocation**: words co-occurring more often than by chance (*fast food*, not *quick food*).
- precision $= TP/(TP+FP)$: of what I returned, how much is right; recall $= TP/(TP+FN)$: of what is right, how much I returned; $F_1 = 2PR/(P+R)$. Returning everything gives recall 1; returning very few sure hits gives precision 1.

5 Regular Expressions

[Slides · Syntax: RE & word: 1–15]

Regular expressions are the first tool of the syntactic level (Section 3.3). They are the simplest way to describe a *set of strings* by a pattern, and they are everywhere in NLP: tokenisers, sentence splitters, data cleaning, rule-based chatbots, information extraction. Formally they describe the *regular languages*, the bottom level of the Chomsky hierarchy that we will meet in Section 7.3.

This section covers:

1. the syntax of regular expressions: disjunction, negation, repetition, anchors (Section 5.1);
2. a worked example that shows the two kinds of errors a pattern can make (Section 5.2);
3. ELIZA: a whole "conversation" built from cascades of substitutions (Section 5.3).

In these notes a pattern is written in colour, e.g. `[wW]oodchuck`, and the text it matches is underlined.

5.1 The Syntax of Regular Expressions

[Slides · Syntax: RE & word: 3–8, 10]

Definition — Regular expression

A **regular expression** (RE, regex) is a formal language for specifying text strings. A regex is a *pattern*; searching a text with it returns the substrings that the pattern *matches*.

The motivating question: how can we search for any of *woodchuck*, *woodchucks*, *Woodchuck*, *Woodchucks* with a single query? A plain string search for "woodchuck" misses the capitalised forms; we need a way to say "either `w` or `W`" and "an optional `s`".

5.1.1 Disjunction: square brackets and the pipe

[Slides · Syntax: RE & word: 4, 6]

Letters inside square brackets mean **one character among these**:

Pattern	Matches	Example
`[wW]oodchuck`	*Woodchuck* or *woodchuck*	
`[1234567890]`	any digit	
`[A-Z]`	an upper-case letter (a **range**)	Drenched Blossoms
`[a-z]`	a lower-case letter	my beans were impatient
`[0-9]`	a single digit	Chapter 1: Down the Rabbit Hole

Brackets choose among single *characters*. To choose among longer strings use the **pipe** `|`:

Pattern	Matches
`groundhog|woodchuck`	*groundhog* or *woodchuck* (they are the same animal!)
`yours|mine`	*yours* or *mine*
`a|b|c`	the same as `[abc]`
`[gG]roundhog|[Ww]oodchuck`	both animals, capitalised or not

5.1.2 Negation inside brackets

[Slides · Syntax: RE & word: 5]

A caret `^` right after the opening bracket negates the set: `[^Ss]` means "any character that is *neither* `S` nor `s`".

Pattern	Matches	Example
`[^A-Z]`	not an upper-case letter	Oyfn pripetchik
`[^Ss]`	neither `S` nor `s`	I have no exquisite reason
`[^e^]`	neither `e` nor `^`	Look here
`a^b`	the literal string `a^b`	Look up a^b now

Watch out — the caret has three meanings

`^` means *negation* only when it is the *first* character inside `[]`. In `[e^]` it is an ordinary character, in `a^b` it is an ordinary character, and at the start of a pattern (`^[A-Z]`) it is an *anchor* (see below).

5.1.3 Repetition: `?`, `*`, `+` and the wildcard `.`

[Slides · Syntax: RE & word: 7]

These operators apply to the character (or group) right before them:

Pattern	Meaning	Matches
`colou?r`	previous character optional (0 or 1)	*color*, *colour*
`oo*h!`	0 or more of the previous character	*oh!*, *ooh!*, *oooh!*, ...
`o+h!`	1 or more of the previous character	*oh!*, *ooh!*, *oooh!*, ...
`baa+`	`ba` followed by one or more `a`	*baa*, *baaa*, *baaaa*, ...
`beg.n`	`.` is any single character	*begin*, *begun*, *beg3n*

The star `*` and the plus `+` are called the **Kleene star** and **Kleene plus**, after the mathematician Stephen C. Kleene who introduced regular expressions. Note that `oo*h!` and `o+h!` describe exactly the same set of strings: x^+ is just xx^*.

5.1.4 Anchors: `^` and `$`

[Slides · Syntax: RE & word: 8]

Anchors do not match a character but a *position*: `^` the start of the line, `$` the end.

Pattern	Matches	Example
`^[A-Z]`	an upper-case letter at the start of a line	Palo Alto
`^[^A-Za-z]`	a non-letter at the start of a line	1 "Hello"
`\.$`	a full stop at the end of a line	The end.
`.$`	any character at the end of a line	The end? The end!

In `\.$` the backslash **escapes** the dot: `\.` is a literal full stop, while `.` alone is the wildcard.

5.1.5 Summary of the operators

[Slides · Syntax: RE & word: 10]

Operator	Meaning	Example	Example meaning
`+`	one or more	`a+`	1 or more `a` characters
`*`	zero or more	`a*`	0 or more `a` characters
`?`	optional	`a?`	0 or 1 `a` characters
`[]`	choose 1	`[abc]`	`a` or `b` or `c`
`[-]`	range	`[a-z]`	any character between `a` and `z`
`[^]`	not	`[^a]`	any character that is not `a`
`()`	grouping	`([a-z])+`	one or more characters between `a` and `z`
`(|)`	or	`(ey|ax)`	the string `ey` or the string `ax`
`ab`	follow	`ab`	`a` followed by `b`
`^`	start	`^a`	`a` at the start of the string/line
`$`	end	`a$`	`a` at the end of the string/line
`\s`	whitespace	`\sa`	a whitespace followed by `a`
`.`	any character	`a.b`	`a`, any character, `b`

Watch out — a typo in the summary slide

The slide gives `(a-z)+` as the grouping example. Inside round brackets the hyphen is *not* a range: `(a-z)+` matches the literal three-character string `a-z` (repeated). The range needs square brackets: `([a-z])+` or simply `[a-z]+`.

Tools to experiment: `https://regex101.com/` shows step by step what a pattern matches and explains each part of it.

5.2 Worked Example: Finding "the"

[Slides · Syntax: RE & word: 9]

Task: find all instances of the word "the" in a text. Writing the right pattern takes a few attempts, and each attempt fixes one kind of error. We test them on

The other day the theology student said: the end.

Worked example — three attempts

1. `the`: matches *the* three times, but also the *the* inside *other* and *theology*, and it **misses** the capitalised *The* at the start.
2. `[tT]he`: now *The* is found, but it still **incorrectly returns** the *the* inside *other* and *theology*.
3. `[^a-zA-Z][tT]he[^a-zA-Z]`: "the" must be surrounded by non-letters, so *other* and *theology* are excluded.

Two kinds of errors appeared, and they are exactly the two denominators of precision and recall (Section 4.4):

- **false positives**: matching strings we should not have matched (*other*, *theology*). Reducing them increases **precision**;
- **false negatives**: not matching strings we should have matched (*The*). Reducing them increases **recall**.

Fixing one kind of error often introduces the other, so building a pattern is a continuous trade-off between the two.

Watch out — attempt 3 is still not perfect

`[^a-zA-Z][tT]he[^a-zA-Z]` requires a character *before* and *after* the word, so it misses "The" at the very beginning of a line (there is no character before it) and a "the" at the very end. It also includes the surrounding spaces in the match. Two common fixes:

- allow the start/end of the line as alternatives: `(^|[^a-zA-Z])[tT]he([^a-zA-Z]|$)`;
- use the *word boundary* anchor `\b` (available in Python and most regex engines): `\b[tT]he\b` matches *The*, *the*, *the* and nothing else in the test sentence.

5.3 ELIZA: Conversation by Substitution

[Slides · Syntax: RE & word: 11–15]

ELIZA (Section 2.3) is the classic proof of how far regular expressions alone can go. It understands nothing: it applies a **cascade** of regex substitutions to the user's sentence and returns the result. A conversation with an online version (https://www.masswerk.at/elizabot/) looks like this:

```
ELIZA: How do you do.  Please tell me your problem.
YOU:   I had an unpleasant discussion with a friend of mine
ELIZA: Can you elaborate on that ?
YOU:   She told me that I am stupid
ELIZA: Is it because you are stupid that you came to me ?
YOU:   yes, it is exactly because I am stupid...
ELIZA: I see.
```

"Is it because you are stupid that you came to me?" is just the user's *"I am stupid"* with *I am* rewritten to *you are* and pasted into a template.

5.3.1 A tiny ELIZA in Python

[Slides · Syntax: RE & word: 14–15]

Three substitutions applied in cascade with Python's `re.sub`, which takes a pattern, a replacement and a string:

```
1 import re
2 str = "I feel sad, my heart is broken"
3 str = re.sub("I", "you", str)        # you feel sad, my heart is broken
4 str = re.sub("my", "your", str)      # you feel sad, your heart is broken
5 str = re.sub(r"you (feel|are) (sad|sorry|desperate)(.*)",
6              r"Why do you say that you \1 \2\3?", str)
7 print(str)   # Why do you say that you feel sad, your heart is broken?
```

- Steps 1 and 2 swap the point of view (*I* → *you*, *my* → *your*).
- Step 3 uses **capture groups**: each pair of round brackets in the pattern remembers what it matched, and `\1`, `\2`, `\3` in the replacement paste it back. Here `\1` = *feel*, `\2` = *sad*, `\3` = *, your heart is broken.*
- The `r"..."` prefix makes a *raw string*, so that Python does not interpret the backslashes itself.

Watch out — order and context matter

`re.sub("I", "you", str)` replaces *every* capital `I`, also inside words ("*It*" becomes "*yout*"), and the order of the cascade matters: if we swapped *you* → *I* as well, a later rule could undo an earlier one. Real ELIZA implementations use word boundaries (`\bI\b`) and swap pronouns in a single pass.

5.3.2 Exercises

[Not in the slides: added material]

Exercise 1 — prices

Write a regex matching prices in dollars such as *$5* and *$19.99* (an optional decimal part with exactly two digits). What does it return on "not $3.5"?

Solution

`\$[0-9]+(\.[0-9][0-9])?`: an escaped dollar sign (`$` alone is the end anchor), one or more digits, then an optional group made of a literal dot and two digits. On "$3.5" the optional group cannot match (only one digit after the dot), so the regex returns just *$3*: a false positive of a wrong length. *Lesson*: a pattern can match only part of what you intended; add `\b` or a negative condition after the price if partial matches matter.

Exercise 2 — reading patterns

Describe in words what each pattern matches: (a) `^[A-Z][a-z]*\.$`; (b) `[0-9]+(st|nd|rd|th)`; (c) `\bcolou?rs?\b`.

Solution

(a) A whole line made of one capitalised word followed by a full stop ("Stop." but not "Stop now."). (b) An ordinal written with digits: *1st*, *22nd*, *3rd*, *100th* (and also the wrong *1th*: a regex checks form, not meaning). (c) The whole words *color*, *colour*, *colors*, *colours*, but not *discolor*.

Cheat sheet — regular expressions

- `[abc]` one of; `[a-z]` range; `[^a]` not (caret first in brackets); `x|y` one of two strings; `()` grouping and capture.
- `?` 0 or 1; `*` 0 or more (Kleene star); `+` 1 or more (Kleene plus); `.` any

character; `\.` a literal dot.
- Anchors `^` (start), `$` (end), `\b` (word boundary).
- Writing a pattern trades off **false positives** (lower precision) and **false negatives** (lower recall).
- ELIZA = a cascade of regex substitutions with capture groups (`\1`, `\2`, ...).

6 Syntax at the Word Level

[Slides · Syntax: RE & word: 16–82]

Before a sentence can be tagged or parsed, the text must be cut into words, and the words must often be cleaned and normalised. This section covers the word-level tasks of the syntactic level (Section 3.3):

1. **word segmentation**: where does a word start and end? (Section 6.1);
2. **stop words** and **TF-IDF**: which words carry little information, and how to measure how important a word is for a document (Section 6.2);
3. **stemming and lemmatisation**: reducing *organizes*, *organizing* to a common form, with the Porter stemmer in detail (Section 6.3);
4. **minimum edit distance**: how similar are two strings, computed with dynamic programming (Section 6.4).

6.1 Word Segmentation

[Slides · Syntax: RE & word: 16–20]

Definition — Word segmentation

Word segmentation (also *tokenisation*) is the problem of dividing a string of written language into its component words.

6.1.1 Why it is not trivial

[Slides · Syntax: RE & word: 17]

In English and in many other languages written with the Latin alphabet, the **space** is a good approximation of a word divider (word delimiter). It is only an approximation, because languages vary in how they treat collocations and compounds (Section 4.1): many English compound nouns are written in several ways, e.g. *multi-agent* = *multi-agent* = *multi agent*, and speakers differ on whether they think of them as one noun or as a noun phrase. Punctuation adds more cases (*don't*, *U.S.A.*, *3.14*, *e-mail*).

Many scripts have no word space at all, and there word segmentation is a hard problem:

- **Chinese** and **Japanese**: sentences are delimited, words are not;
- **Thai** and **Lao**: phrases and sentences are delimited, words are not;

- **Vietnamese**: syllables are delimited, words are not.

6.1.2 The MaxMatch algorithm

[Slides · Syntax: RE & word: 18]

A simple baseline for Chinese is the greedy **maximum matching** algorithm. It needs a **dictionary** (word list) of the language.

Definition — MaxMatch

1. Put a pointer at the beginning of the string.
2. Choose the **longest** dictionary word that matches the input starting at the pointer, and move the pointer to the end of that word.
3. If no word matches, move the pointer one character forward instead (creating a one-character "word").
4. Repeat from the new pointer position until the end of the string.

Worked example — MaxMatch on English without spaces

Input: `wecanonlyseeashortdistanceahead` ("we can only see a short distance ahead").

Remaining input	Longest word	Why
`wecanonly...`	*we*	
`canonlysee...`	*canon*	longer than *can*
`lyseea...`	*l*	no word starts here: one character
`yseea...`	*y*	same
`seeashort...`	*see*	
`ashort...`	*ash*	longer than *a* and *as*
`ort...`	*o*, *r*, *t*	three single characters
`distanceahead`	*distance*, *ahead*	

Output: *we canon l y see ash o r t distance ahead.*

Intuition — why greedy fails here and works for Chinese

Greedy choices are never undone: once *canon* is taken, the correct *can only* is impossible. In English, words are long and many short words hide inside longer ones, so this happens constantly. Chinese words are mostly one or two characters long, so the longest match is usually right, and MaxMatch is a surprisingly strong baseline there. Better segmenters score *whole* segmentations (e.g. with word probabilities) instead of committing greedily.

6.1.3 NLTK

[Slides · Syntax: RE & word: 19–20]

The course uses **NLTK** (Natural Language Toolkit, `https://www.nltk.org/`) to show some problems and some of their possible solutions. NLTK is a Python platform

for working with human language data: it provides interfaces to over 50 corpora and lexical resources such as WordNet (Section 10.2), and libraries for classification, tokenisation, stemming, tagging, parsing and semantic reasoning. The course also uses less known, but elegant, tools.

```
import nltk
# nltk.download()   # only once, to download corpora and models
from nltk.tokenize import word_tokenize
print(word_tokenize("Don't segment U.S.A. badly, Mr. Smith!"))
# ['Do', "n't", 'segment', 'U.S.A.', 'badly', ',', 'Mr.', 'Smith', '!']
```

Note how the tokeniser splits *don't* into *Do* + *n't* (two morphemes, Section 4.1) but keeps *U.S.A.* and *Mr.* together: the full stop is not always a word boundary.

6.2 Stop Words and TF-IDF

[Slides · Syntax: RE & word: 21–47]

6.2.1 Stop words

[Slides · Syntax: RE & word: 21–23]

Definition — Stop words

Stop words are words that are filtered out before or after processing natural language text. The term usually refers to the *most common* words of a language (*the*, *of*, *and*, *a*, *to*, ...), but there is no single universal list used by all NLP tools, and some tools do not use one at all.

- **How a stop list is built**: sort the terms by *collection frequency* (the total number of times each term appears in the document collection), take the most frequent ones, and often filter them by hand according to their meaning in the domain of the documents. The words of the stop list are then discarded during indexing. Example lists: `https://www.ranks.nl/stopwords`.
- **When not to remove them**: some tools avoid removing stop words to support *phrase search* (exact, not keyword-based): "to be or not to be" consists almost only of stop words.

A word cloud of a novel (the slides show one for a Thomas Hardy novel) is dominated by *the*, *and*, *of*, *to*, *a*, *was*, *her*: they are frequent in *any* English text and say nothing about this one.

6.2.2 Frequent in the document or frequent in the language?

[Slides · Syntax: RE & word: 24–29]

Intuition — the key question

Is a word that appears often in a document always a stop word? No. A word can be frequent in a document d for two very different reasons:

- it is related to the **topic** of d ("cat" in a document about cats);
- it is frequent in the **language** ("the" in every document).

To tell them apart, we must look not only at d but at a whole **corpus**: a topical word is frequent in *some* documents, a stop word in *all* of them.

The running example uses two short documents:

Worked example — the corpus

Doc1 (54 words): *The cat (Felis catus) is a small carnivorous mammal. It is the only domesticated species in the family Felidae and often referred to as the domestic cat to distinguish it from wild members of the family. The cat is either a house cat or a farm cat, which are pets, or a feral cat.*

Doc2 (42 words): *A car (or automobile) is a wheeled motor vehicle used for transportation. Cars became widely available in the early 20th century. One of the first cars accessible to the masses was the 1908 Model T. Cars were rapidly adopted in the US.*

	"the"	**"cat"**	**"car(s)"**
Doc1	6	6	0
Doc2	5	0	4
corpus $C_2 = \{\text{Doc1}, \text{Doc2}\}$	11	6	4

Corpus C_1 contains only Doc1; corpus C_2 contains Doc1 and Doc2.

Watch out — a counting slip in the slides

The slides count **4** occurrences of "the" in Doc2 (and 10 in C_2). Doc2 actually contains it **5** times: *the early*, *the first*, *the masses*, *the 1908*, *the US*. This changes tf("the", Doc2) from 4/42 to 5/42, but not the final TF-IDF value, which is 0 either way (see below).

In C_1 alone, "the" and "cat" are equally frequent (6 times each): with a single document there is no way to tell a topical word from a stop word. In C_2 the difference appears: "the" is in *both* documents, "cat" only in one.

6.2.3 Term frequency and inverse document frequency

[Slides · Syntax: RE & word: 30–34]

Definition — TF-IDF

Term frequency: the number of times term t occurs in document d, usually normalised by the length of the document:

$$\mathrm{tf}(t, d) = \frac{\text{number of occurrences of } t \text{ in } d}{\text{length}(d)}.$$

Inverse document frequency: the logarithm of the inverse fraction of documents of the corpus C that contain t:

$$\mathrm{idf}(t, C) = \log \frac{|C|}{|\{d \in C : t \in d\}|},$$

where $|C|$ is the number of documents in the corpus.
TF-IDF: the product of the two,

$$\text{tf-idf}(t, d, C) = \mathrm{tf}(t, d) \cdot \mathrm{idf}(t, C).$$

- **tf** is high when t is frequent *in this document*.
- **idf** is high when t is *rare in the corpus*. If t occurs in every document, the fraction is $|C|/|C| = 1$ and idf $= \log 1 = 0$: the word cannot distinguish documents at all, exactly like a stop word.
- The **logarithm** dampens the effect: a word in 1 document out of 1000 is more informative than one in 10 out of 1000, but not 10 times more.
- If t appears in *no* document the denominator is 0. A common fix is to add 1 to the denominator; it makes sense when the corpus is large enough, since it slightly distorts the values.

Watch out — base of the logarithm

The slides' numbers use the base-10 logarithm ($\log 2 \approx 0.301$). Other sources use the natural logarithm ($\ln 2 \approx 0.693$). The base only rescales all idf values by the same factor, so the *ranking* of words does not change; just be consistent.

6.2.4 Worked example on C_2

[Slides · Syntax: RE & word: 35–46]

Worked example — TF-IDF of "the", "cat", "car"

First the idf values ($|C_2| = 2$):

$$\begin{aligned}
\mathrm{idf}(\text{the}, C_2) &= \log \tfrac{2}{2} = \log 1 = 0 && \triangleleft \text{ "the" is in both documents} \\
\mathrm{idf}(\text{cat}, C_2) &= \log \tfrac{2}{1} = \log 2 \approx 0.301 && \triangleleft \text{ only in Doc1} \\
\mathrm{idf}(\text{car}, C_2) &= \log \tfrac{2}{1} = \log 2 \approx 0.301 && \triangleleft \text{ only in Doc2}
\end{aligned}$$

Then the products:

$$\begin{aligned}
&\text{tf-idf}(\text{the}, \text{Doc1}, C_2) = \tfrac{6}{54} \cdot 0 = 0 \\
&\text{tf-idf}(\text{the}, \text{Doc2}, C_2) = \tfrac{5}{42} \cdot 0 = 0 \\
&\text{tf-idf}(\text{cat}, \text{Doc1}, C_2) = \tfrac{6}{54} \cdot 0.301 \approx 0.111 \cdot 0.301 \approx 0.033 \\
&\text{tf-idf}(\text{cat}, \text{Doc2}, C_2) = 0 \cdot 0.301 = 0 \qquad \triangleleft\ \text{tf} = 0\text{, idf irrelevant} \\
&\text{tf-idf}(\text{car}, \text{Doc1}, C_2) = 0 \cdot 0.301 = 0 \\
&\text{tf-idf}(\text{car}, \text{Doc2}, C_2) = \tfrac{4}{42} \cdot 0.301 \approx 0.095 \cdot 0.301 \approx 0.029
\end{aligned}$$

tf-idf	"the"	"cat"	"car"
Doc1	0	**0.033**	0
Doc2	0	0	**0.029**

Table 7: TF-IDF in C_2. Each document's only non-zero word is its topic.

"The" is as frequent as "cat" in Doc1, but its TF-IDF is 0 because it appears everywhere; "cat" and "car" come out as the words that characterise their documents. This is how TF-IDF finds *keywords* and, symmetrically, *stop words* (terms with idf close to 0) automatically. Online analysers such as `https://www.online-utility.org/text/analyzer.jsp` compute word frequencies of a text you paste.

Why this matters for NLP

TF-IDF is one of the text features of Section 3.3: representing each document by the TF-IDF values of its words gives a vector that search engines use to rank documents for a query and classifiers use to decide the topic of a text.

6.3 Stemming and Lemmatisation

[Slides · Syntax: RE & word: 48–65]

6.3.1 The goal: one form for many

[Slides · Syntax: RE & word: 48]

Documents use different forms of a word: *organize*, *organizes*, *organizing* (inflectional morphemes, Section 4.1); and there are families of derivationally related words with similar meanings: *democratic*, *democratization* (derivational morphemes). For a search, it is often useful that a query for one of these words also returns documents containing another word of the same set.

The goal of both stemming and lemmatisation is to **reduce inflectional forms, and sometimes derivationally related forms, of a word to a common base**

form:

$$\begin{aligned} \text{am, are, is} &\rightarrow \text{be} \\ \text{car, cars, car's, cars'} &\rightarrow \text{car} \\ \text{the boy's cars are different colors} &\rightarrow \text{the boy car be differ color} \end{aligned}$$

6.3.2 Stemming vs. lemmatisation

[Slides · Syntax: RE & word: 49]

Definition — Stemming and lemmatisation

- **Stemming** is a *crude heuristic* process that chops off the ends of words, hoping to get it right most of the time. It often removes derivational affixes too. The result, the *stem*, need not be a real word.
- **Lemmatisation** does things properly, with a *vocabulary* and a *morphological analysis* of words. It normally removes inflectional endings only and returns the base or dictionary form of the word, the **lemma**.

Stemming most commonly collapses derivationally related words; lemmatisation commonly collapses only the different inflectional forms of a lemma.

Word	Porter stem	Lemma
are	ar	be
theses	these	thesis
universal	univers	universal
university	univers	university
generalizations	gener	generalization

Table 8: A stemmer chops, a lemmatiser looks words up. Stems are not words, and unrelated words can end up with the same stem.

6.3.3 The Porter stemmer

[Slides · Syntax: RE & word: 50–52]

The **Porter stemmer** (Martin Porter, 1980) is the classic stemming algorithm for English. It is a simple **rule-based** and **heuristic** method: a cascade of suffix-rewriting rules such as ATIONAL → ATE (*relational* → *relate*), applied in sets, in order. The slides present it in seven steps.

Consonants, vowels and the measure.

- A **consonant** is a letter other than `a`, `e`, `i`, `o`, `u`, and other than `y` when preceded by a consonant. A **vowel** is any other letter. (So the `y` of *toy* is a consonant, the `y` of *happy* is a vowel.)

- Write C for a sequence of one or more consonants and V for a sequence of one or more vowels. Then *every* word has the form

$$[C]\,(VC)^m\,[V]$$

where the square brackets mean "optional". The number m is the **measure** of the word (or stem): roughly, its number of syllables.

Worked example — measures

Word	Letters as C/V	Form	m
tree	tr·ee	$C\,V$	0
trouble	tr·ou·bl·e	$C\,(VC)\,V$	1
troubles	tr·ou·bl·e·s	$C\,(VC)(VC)$	2
oaten	oa·t·e·n	$(VC)(VC)$	2

The slides write *troubles* as "C V CVC": grouping it as $C\,(VC)(VC)$ gives $m = 2$.

Rule format. Each rule has the form (*condition*) $S_1 \to S_2$: if the word ends with suffix S_1 and the *stem* (what is left after removing S_1) satisfies the condition, S_1 is replaced by S_2. The conditions are:

Condition	Meaning
$m > k$, $m = k$	the measure of the stem
`*S`	the stem ends with the letter `S` (similarly `*L`, `*Z`)
`*v*`	the stem contains a vowel
`*d`	the stem ends with a double consonant (e.g. `-tt`, `-ss`)
`*o`	the stem ends with *cvc*, where the second *c* is not `w`, `x`, `y`

The measure condition is what makes the stemmer safe on short words: (m>0) NESS $\to \varepsilon$ turns *goodness* into *good*, but it would not touch a hypothetical word whose stem would be empty.

6.3.4 The seven steps

[Slides · Syntax: RE & word: 53–58]

In the rules, ε is the empty string (the suffix is deleted).

Step	Rules	Examples
1 (plurals)	SSES→SS; IES→I; SS→SS; S→ ε	caresses→caress, ponies→poni, ties→ti, caress→caress, cats→cat
2a (*-ed*, *-ing*)	(m>0) EED→EE; (`*v*`) ED→ ε; (`*v*`) ING→ ε	agreed→agree, feed→feed; plastered→plaster, bled→bled; motoring→motor, sing→sing
2b (clean-up, only if the 2nd or 3rd rule of 2a fired)	AT→ATE; BL→BLE; (`*d` and not `*L`, `*S`, `*Z`) → single letter; (m=1 and `*o`) → E	conflat(ed)→conflate, troubl(ing)→trouble, hopp(ing)→hop, tann(ed)→tan, fall(ing)→fall, fil(ing)→file, fail→fail
3 (y)	(`*v*`) Y→I	happy→happi, sky→sky
4 (derivational I)	(m>0) ATIONAL→ATE; IZATION→IZE; BILITI→BLE; ...	relational→relate, generalization→generalize, sensibiliti→sensible
5 (derivational II)	(m>0) ICATE→IC; FUL→ ε; NESS→ ε; ...	triplicate→triplic, hopeful→hope, goodness→good
6 (derivational III)	(m>1) ANCE, ENT, IVE, ER, AL, ANT, ... → ε	allowance→allow, dependent→depend, effective→effect
7a (final *e*)	(m>1) E→ ε; (m=1 and not `*o`) E→ ε	probate→probat, cease→ceas
7b (double *l*)	(m>1 and `*d` and `*L`) → single letter	controll→control, roll→roll

Table 9: The Porter stemmer as presented in the slides (in Porter's paper the same rules are numbered 1a, 1b, 1c, 2, 3, 4, 5a, 5b).

Watch out — corrections to the Porter slides

- **Consonant definition**: the slide reads "a letter other than A, E, I, O, U, and Y preceded by consonant". Porter's definition is "other than A, E, I, O, U, *and other than* Y preceded by a consonant": a Y after a consonant is a vowel (*happy*), otherwise a consonant (*toy*).
- **Step 2a**: the slide writes the condition of EED as $m > 1$, then uses *agreed* → *agree* as an example, but *agr* has $m = 1$. Porter's rule is ($m > 0$) EED → EE, as in the table above.
- **Step 7a**: the slide's second rule is "(m=1 and not `*o`) NESS → ε" with *goodness* as the example (copied from step 5). In Porter's algorithm this rule removes a final E: *cease* → *ceas*.

6.3.5 Examples and problems

[Slides · Syntax: RE & word: 59–62]

Worked example — running the steps

- *controlling* $\xrightarrow{\text{2a: }-\text{ING}}$ *controll* $\xrightarrow{\text{7b}}$ *control*.
- *singing* $\xrightarrow{\text{2a: }-\text{ING}}$ *sing* (no further rule applies).
- *computers* $\xrightarrow{\text{1: }-\text{S}}$ *computer* $\xrightarrow{\text{6: }-\text{ER}}$ *comput*.
- *generalizations* $\xrightarrow{1}$ *generalization* $\xrightarrow{\text{4: IZATION}\to\text{IZE}}$ *generalize* $\xrightarrow{\text{5: ALIZE}\to\text{AL}}$ *general* $\xrightarrow{\text{6: }-\text{AL}}$ *gener*.

Watch out — the examples slide

On the slide, *singing* ends with "Step 6, Rule 4 → compute", a line copied from the *computers* example: *singing* simply becomes *sing*. The slide also stops *computers* at *compute* and *generalizations* at *general*. Running the complete algorithm (e.g. NLTK's implementation) gives *comput* and *gener*: stems are not required to be words, and Porter's algorithm removes ER and AL in its last derivational step.

Problems. Being heuristic, the stemmer makes two kinds of errors:

- **over-stemming**: unrelated words get the same stem. *elephants* → *elephant* (step 1) → *eleph* (step 6 removes ANT as if it were a suffix); *university* and *universal* both become *univers*;
- **under-stemming**: forms of the same word get different stems. *thesis* → *thesi* but *theses* → *these*.

The slides also list *doing* → *doe* as a problem; the standard algorithm actually returns *do* for *doing* (the E of step 2b is added only when $m = 1$, and *do* has $m = 0$).

References: Porter, M. F. (1980), *An algorithm for suffix stripping*, Program 14(3):130–137; the Porter stemmer home page https://tartarus.org/martin/PorterStemmer/; Jurafsky and Martin, Section 3.4. Online demos: https://9ol.es/porter_js_demo.html; stemmers for other languages: http://text-processing.com/demo/stem/.

6.3.6 Two texts to test a stemmer

[Slides · Syntax: RE & word: 63–65]

The slides propose two short texts full of traps. Here is what a real stemmer does with them.

Worked example — Italian: a Snowball stemmer

Sono stato in uno stato il cui statuto prevede che la biscia non possa entrare in una bisca. Una messe di polemiche è piovuta sullo statuto, per le limitazioni messe alla biscia, che è andata a messa insieme al console per consolarsi.

Words	Meanings	Stem
stato, stato, statuto	"been", "state", "statute"	*stat*
biscia, bisca	"grass snake", "gambling den"	*bisc*
messe, messe, messa	"harvest", "put" (participle), "mass"	*mess*
console, consolarsi	"consul", "to console oneself"	*consol*

Every group is over-stemmed: completely unrelated words collapse. Even a lemmatiser needs context here: the two *stato* have different lemmas (*essere*, the verb; *stato*, the noun), and only the part of speech tells them apart (Section 7.2).

Worked example — English: Miss Marple

Once Miss Marple had a cold and she went to the doctor: "I go!", she said, and she read the doctor's doctoral thesis. "These are good theses!", and – although she missed her train – Miss Marple understood that University has a universal mission.

Porter gives: *Miss*, *missed* → *miss* (a title and a verb merged); *doctor*, *doctoral* → *doctor*; *University*, *universal* → *univers*; *theses* → *these*, the same as the pronoun *These*, while *thesis* → *thesi*; *has* → *ha*; *are* → *ar*. A lemmatiser instead returns *thesis* for *theses*, *be* for *are*, *have* for *has*, and keeps *university* and *universal* apart.

Intuition — which one to use

Stemming is fast, needs no dictionary and increases *recall* in search (more forms of a word are found), at the price of *precision* (unrelated words are merged). Lemmatisation is slower and needs a vocabulary and often the part of speech, but its output is made of real words. Choose according to which kind of error hurts your application more.

6.4 Minimum Edit Distance

[Slides · Syntax: RE & word: 66–82]

6.4.1 How similar are two strings?

[Slides · Syntax: RE & word: 66–68, 72–73]

Many tasks need a measure of how close two strings are:

- **Spell correction**: the user typed *graffe*. Which is closest: *graf*, *graft*, *grail* or *giraffe*?
- **Computational biology**: align two sequences of nucleotides (the bases Adenine, Guanine, Thymine/Uracil and Cytosine of DNA/RNA), giving each letter a partner letter or a gap -:

```
AGGCTATCACCTGACCTCCAGGCCGATGCCC
TAGCTATCACGACCGCGGTCGATTTGCCCGAC
```
aligned:
```
-AGGCTATCACCTGACCTCCAGGCCGA-TGCCC--
TAG-CTATCAC-GACCGC-GGTCGATTTGCCCGAC
```

Alignments are used in molecular phylogenetics (evolutionary relationships from hereditary differences in DNA), in comparing the genomes of ill and healthy individuals, and to understand the function of unknown proteins.

- **Evaluating machine translation and speech recognition**: compare the system's output (hypothesis H) with a reference (R), counting substitutions (S), insertions (I) and deletions (D):

R:	Spokesman	confirms	*	senior	government	adviser	was	shot	*
H:	Spokesman	said	the	senior	*	adviser	was	shot	dead
		S	I		D				I

- **Named entity extraction and coreference**: deciding that *IBM Inc.* and *IBM*, or *Stanford President John Hennessy* and *Stanford University President John Hennessy*, refer to the same entity.

6.4.2 Definition

[Slides · Syntax: RE & word: 69–71]

Definition — Minimum edit distance

The **minimum edit distance** between two strings is the minimum number of editing operations

- **insertion** of a character,
- **deletion** of a character,
- **substitution** of a character with another,

needed to transform one string into the other. If operations have *costs*, it is the minimum total cost. With cost 1 for insertions and deletions and **cost 2 for substitutions** it is called the **Levenshtein distance** (the version used in these notes unless stated otherwise).

An **alignment** shows the operations explicitly. For *intention* and *execution* (* = gap):

I	N	T	E	*	N	T	I	O	N
|	|	|	|	|	|	|	|	|	|
*	E	X	E	C	U	T	I	O	N
d	s	s		i	s				

One deletion (d), three substitutions (s) and one insertion (i):

- if every operation costs 1, the distance is $1 + 3 + 1 = 5$;
- with Levenshtein costs (substitution = 2), it is $1 + 3 \cdot 2 + 1 = 8$.

Worked example — the spelling question

Levenshtein distances from *graffe*: *giraffe* $= 1$ (insert `i`), *graf* $= 2$ (delete `f` and `e`), *graft* $= 3$ (delete `f`, substitute `e`→`t`), *grail* $= 5$. So *giraffe* is the best correction. (With all costs equal to 1: $1, 2, 2, 3$.)

6.4.3 Edit distance as search

[Slides · Syntax: RE & word: 74–78]

Finding the minimum edit distance is a search for a path (a sequence of edits) from the start string to the final string:

- **initial state**: the word we are transforming;
- **operators**: insert, delete, substitute;
- **goal state**: the word we are trying to reach;
- **path cost**: what we minimise, the number (or cost) of edits.

The space of all edit sequences is huge, so we cannot explore it naively. But many distinct paths end up in the *same* state (the same prefix of X turned into the same prefix of Y), and for each such state we only need to remember the *cheapest* way to reach it. That is exactly the idea of dynamic programming.

Definition — The subproblems

For strings X of length n and Y of length m, let

$$D(i, j) = \text{edit distance between } X[1..i] \text{ and } Y[1..j],$$

i.e. between the first i characters of X and the first j characters of Y. The edit distance between X and Y is $D(n, m)$.

Dynamic programming computes $D(n, m)$ *bottom-up* in a table, solving the problem by combining solutions to subproblems: first $D(i, j)$ for small i, j, then larger ones from the smaller values already computed.

6.4.4 The algorithm

[Slides · Syntax: RE & word: 79–80]

Key result — Levenshtein distance by dynamic programming

Initialisation (distance from the empty string):

$$D(i, 0) = i, \qquad D(0, j) = j.$$

Recurrence: for each $i = 1, \ldots, n$ and each $j = 1, \ldots, m$,

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 & \text{deletion of } X[i] \\ D(i,j-1) + 1 & \text{insertion of } Y[j] \\ D(i-1,j-1) + \begin{cases} 2 & \text{if } X[i] \neq Y[j] \\ 0 & \text{if } X[i] = Y[j] \end{cases} & \text{substitution or copy} \end{cases}$$

Termination: $D(n,m)$ is the distance.

Intuition — reading the recurrence

To turn $X[1..i]$ into $Y[1..j]$, look at the *last* column of an optimal alignment. Either the last character of X was deleted (and the rest is an optimal alignment of $X[1..i-1]$ with $Y[1..j]$), or the last character of Y was inserted, or the two last characters are aligned with each other (free if equal, a substitution otherwise). The minimum over the three cases is the answer.

The general version (Jurafsky and Martin) uses arbitrary costs `del-cost`, `ins-cost`, `sub-cost`, which can depend on the letters, to model that some letters are more likely to be inserted or confused than others; copying a letter costs `sub-cost`$(x, x) = 0$.

Watch out — indices on the slide

The slide defines X of length n and Y of length m, but the recurrence loops over $i = 1 \ldots M$, $j = 1 \ldots N$ and terminates with $D(N, M)$. The letters are swapped: i indexes X and runs to n, j indexes Y and runs to m, and the result is $D(n, m)$. The bottom-up loop also covers $1 \leq i \leq n$, $1 \leq j \leq m$ (the slide writes strict inequalities).

6.4.5 The table and the backtrace

[Slides · Syntax: RE & word: 81–82]

Worked example — intention → execution

Rows: the source *intention*; columns: the target *execution*; `#` is the empty prefix. Each cell is computed from its left, upper and upper-left neighbours.

	#	e	x	e	c	u	t	i	o	n
#	**0**	1	2	3	4	5	6	7	8	9
i	**1**	2	3	4	5	6	7	6	7	8
n	2	**3**	4	5	6	7	8	7	8	7
t	3	4	**5**	6	7	8	7	8	9	8
e	4	3	4	**5**	**6**	7	8	9	10	9
n	5	4	5	6	7	**8**	9	10	11	10
t	6	5	6	7	8	9	**8**	9	10	11
i	7	6	7	8	9	10	9	**8**	9	10
o	8	7	8	9	10	11	10	9	**8**	9
n	9	8	9	10	11	12	11	10	9	**8**

Table 10: Levenshtein table. The bottom-right cell is the distance (8). Shaded cells: one minimum-cost path found by the backtrace.
Some cells, step by step:

$$D(1,1) = \min\big(D(0,1)+1,\ D(1,0)+1,\ D(0,0)+2\big) = \min(2,2,2) = 2$$
$$D(4,1) = \min\big(D(3,1)+1,\ D(4,0)+1,\ D(3,0)+0\big) = \min(5,5,3) = 3$$
$$D(9,9) = \min\big(D(8,9)+1,\ D(9,8)+1,\ D(8,8)+0\big) = \min(10,10,8) = 8$$

In $D(1,1)$ the letters differ (`i`, `e`), so the diagonal costs 2; in $D(4,1)$ and $D(9,9)$ they are equal, so the diagonal is a free copy.

Backtrace. The table gives the distance, not the alignment. To get the alignment, when filling each cell we remember *which* neighbour(s) gave the minimum (an arrow $\leftarrow$, $\uparrow$ or $\nwarrow$). After the table is full, we start from the bottom-right cell and follow the arrows back to $(0,0)$: $\uparrow$ is a deletion, $\leftarrow$ an insertion, $\nwarrow$ a substitution or a copy. The shaded path, read from the top-left, is: delete `i`; `n`→`e`; `t`→`x`; copy `e`; insert `c`; `n`→`u`; copy `t`, `i`, `o`, `n`. That is exactly the alignment shown above, with cost $1+2+2+0+1+2 = 8$. When several neighbours tie, there are several optimal alignments.

Intuition — cost of the algorithm

The table has $(n+1)(m+1)$ cells, each computed in constant time from three neighbours, so the algorithm takes $O(nm)$ time and space; the backtrace adds $O(n+m)$. The naive search over all edit sequences is exponential.

6.4.6 Exercises

[Not in the slides: added material]

Exercise 1 — small tables

Compute the Levenshtein table for *cat* → *cut* and for *cat* → *act*. Give the distance and an optimal alignment for each.

Solution

cat → *cut* (rows c, a, t; columns c, u, t):

	#	*c*	*u*	*t*
#	0	1	2	3
c	1	0	1	2
a	2	1	2	3
t	3	2	3	2

Distance 2: copy `c`, substitute `a`→`u` (cost 2), copy `t`. Deleting `a` and inserting `u` also costs 2: there are two optimal alignments.
cat → *act* (columns a, c, t):

	#	*a*	*c*	*t*
#	0	1	2	3
c	1	2	1	2
a	2	1	2	3
t	3	2	3	2

Distance 2, e.g. delete `c`, copy `a`, insert `c`, copy `t`.
Lesson: swapping two adjacent letters costs 2. The Damerau–Levenshtein variant adds *transposition* as a fourth operation, because swaps are a very common typing error.

Cheat sheet — syntax at the word level

- Word segmentation: spaces are only an approximation; Chinese, Japanese, Thai have no word spaces. MaxMatch = greedy longest dictionary word (good for Chinese, bad for English).
- Stop words: very frequent, little content; no universal list. A word frequent in a document may be topical: compare with the corpus.
- $\mathrm{tf}(t,d) = \#t/\mathrm{len}(d)$, $\mathrm{idf}(t,C) = \log\big(|C|/|\{d : t \in d\}|\big)$, tf-idf = tf · idf; a word in every document has idf = 0.
- Stemming = heuristic suffix chopping (Porter: measure m in $[C](VC)^m[V]$, conditional rules in 7 steps; over- and under-stemming). Lemmatisation = dictionary + morphology, returns the lemma.
- Minimum edit distance: insertions, deletions, substitutions; Levenshtein: substitution costs 2. Dynamic programming: $D(i,0) = i$, $D(0,j) = j$, $D(i,j)$ = minimum over the upper neighbour +1 (deletion), the left neighbour +1 (insertion), the diagonal neighbour +2 (substitution) or +0 (copy); the backtrace gives the alignment; time and space $O(nm)$.

7 Syntax at the Sentence Level

[Slides · Syntax: sentence, rules: 1–17; Syntax: sentence, HMM: 1–20]

Once a text is split into words, the next questions concern whole sentences: where does a sentence end, what grammatical category does each word have in context, and how do the words group into a structure? This section covers the sentence-level tasks of the syntactic level (Section 3.3):

1. **sentence breaking** (Section 7.1);
2. **part-of-speech tagging** and the Penn Treebank tagset (Section 7.2);
3. **grammars and parsing**: Chomsky's generative grammar, the Chomsky hierarchy, a small grammar for English, Definite Clause Grammars (Section 7.3);
4. **statistical POS tagging** with Hidden Markov Models and the Viterbi algorithm (Section 7.4).

The first three are *rule-based* (deductive) approaches, the last one is *statistical* (inductive): two eras of NLP (Section 2.3) side by side.

7.1 Sentence Breaking

[Slides · Syntax: sentence, rules: 2–4]

Definition — Sentence boundary disambiguation

Sentence boundary disambiguation (SBD), also called *sentence breaking* or *sentence boundary detection*, is the problem of deciding where sentences begin and end.

Many NLP tools need their input divided into sentences, but punctuation is ambiguous:

- a **period** may mark an abbreviation (*Mr.*, *etc.*), a decimal point (*3.14*), an ellipsis (. . .) or be part of an e-mail address, rather than the end of a sentence. About **47%** of the periods in the Wall Street Journal corpus mark abbreviations;
- **question and exclamation marks** appear in embedded quotations, emoticons, computer code and slang.

Languages like Japanese and Chinese have unambiguous markers for the end of a sentence.

Key result — The standard rule-based approach

To decide whether a token ends a sentence:
(a) if it is a period, it ends a sentence;
(b) but if the preceding token is in a hand-compiled list of **abbreviations**, it does not;
(c) and if the next token is **capitalised**, then it does end a sentence.
This simple strategy gets about **95%** of sentences right.

Worked example — where the rules break

Mr. Smith paid 3.14 dollars. He left at 5 p.m. Then he slept.

- *Mr.*: abbreviation, rule (b): not a boundary. Correct.
- *3.14*: the period is inside a token, not a token of its own. Correct.
- *p.m. Then*: *p.m.* is an abbreviation (b says "no boundary"), but the next word is capitalised (c says "boundary"). Here the sentence *does* end: the period of an abbreviation can also be the final period.

NLTK's pre-trained sentence splitter returns *["Mr. Smith paid 3.14 dollars.", "He left at 5 p.m. Then he slept."]*: it gets the first two cases right and misses exactly the third. The remaining 5% of errors are cases like this one, which need more context (or a statistical model) to resolve.

7.2 Part-of-Speech Tagging

[Slides · Syntax: sentence, rules: 5–6]

Definition — Part-of-speech tagging

Part-of-speech tagging (POS tagging, POST), also called *grammatical tagging* or *word-category disambiguation*, is the process of marking each word of a text with its part of speech, based on both its **definition** and its **context**, i.e. its relationship with adjacent and related words in a phrase, sentence or paragraph.

(If the categories themselves are fuzzy, see the recap with the Italian equivalents in Section 4.2.) The word "context" is the whole point: many words have several possible tags, and only the sentence tells which one is right.

Worked example — the same word, different tags

- *I book/VB a flight* vs. *I read a book/NN.*
- *Time/NN flies/VBZ like/IN an arrow* (time passes quickly) vs. *Time/VB flies/NNS like/IN an arrow* (measure the speed of flies as you would an arrow) vs. *Time/NN flies/NNS like/VBP an arrow* ("time flies", a kind of fly, are fond of an arrow).

7.2.1 The Penn Treebank tagset

[Slides · Syntax: sentence, rules: 6]

The most widely used set of tags for English is the **Penn Treebank** tagset, with 36 word tags:

	Tag	Description		Tag	Description
1	CC	coordinating conjunction	19	PRP$	possessive pronoun
2	CD	cardinal number	20	RB	adverb
3	DT	determiner	21	RBR	adverb, comparative
4	EX	existential *there*	22	RBS	adverb, superlative
5	FW	foreign word	23	RP	particle
6	IN	preposition / subord. conj.	24	SYM	symbol
7	JJ	adjective	25	TO	*to*
8	JJR	adjective, comparative	26	UH	interjection
9	JJS	adjective, superlative	27	VB	verb, base form
10	LS	list item marker	28	VBD	verb, past tense
11	MD	modal	29	VBG	verb, gerund / present participle
12	NN	noun, singular or mass	30	VBN	verb, past participle
13	NNS	noun, plural	31	VBP	verb, non-3rd person sing. present
14	NNP	proper noun, singular	32	VBZ	verb, 3rd person sing. present
15	NNPS	proper noun, plural	33	WDT	*wh*-determiner
16	PDT	predeterminer	34	WP	*wh*-pronoun
17	POS	possessive ending	35	WP$	possessive *wh*-pronoun
18	PRP	personal pronoun	36	WRB	*wh*-adverb

Table 11: The Penn Treebank POS tagset.

Intuition — reading the tags

The tags encode morphology (Section 4.1): NN/NNS differ by the plural inflection, VB/VBD/VBG/VBN/VBP/VBZ by the verb's inflection, JJ/JJR/JJS by comparison. So a tagger is also, partly, a morphological analyser, and the tag is what a lemmatiser needs to choose the right lemma (Section 6.3).

Worked example — a tagged sentence

The/DT boy/NN 's/POS cars/NNS are/VBP different/JJ colors/NNS ./.

Online taggers to try: `http://parts-of-speech.info/` and Stanford CoreNLP, `https://corenlp.run/`.

There are two families of approaches, both covered in this course:

- **rule-based** (deductive): a grammar and a parser decide the tags as a by-product of the structure of the sentence (Section 7.3);
- **statistical** (inductive): a model learnt from a tagged corpus picks the most probable sequence of tags (Section 7.4).

7.3 Grammars and Parsing

[Slides · Syntax: sentence, rules: 7–17]

Definition — Parsing

In computational linguistics, **parsing** is the formal analysis by a computer of a sentence (or other string of words) into its **constituents**, resulting in a **parse tree** that shows their syntactic relations to each other. The tree may also contain semantic and other information.

Parsing is one of the possible ways to address the POS problem: once the sentence has a tree, every word has a category (the node right above it).

7.3.1 Chomsky and generative grammar

[Slides · Syntax: sentence, rules: 8–9]

In *Syntactic Structures* (1957), Noam Chomsky proposed the theory of **generative grammar**: a finite set of rules that recursively "specify" or "generate" all the well-formed expressions of a natural language (and only them). It is considered one of the most significant contributions to theoretical linguistics of the 20th century.

Recursion is what links generative grammar to the question of Section 1.1. "Animal languages" are forms of animal communication that show similarities to human language: the inventory of signs may be large, the signs relatively arbitrary, and they seem to be produced with some volition. But researchers agree that they are not as complex or expressive as human language: they lack the *creation of new patterns of signs under varied circumstances*, i.e. the productivity that recursive rules provide.

7.3.2 Grammars and the Chomsky hierarchy

[Slides · Syntax: sentence, rules: 10–13]

Definition — Grammar and formal language

A **grammar** specifies the compositional structure of complex messages. A **formal language** is a set of strings of **terminal symbols** (the words). Each string of the language can be analysed (recognised) or generated by the grammar, which is a set of **rewrite rules**, e.g.

$$S \to NP\ VP$$
$$\mathit{Article} \to \textbf{the} \mid \textbf{a} \mid \textbf{an} \mid \ldots$$

Here S is the **sentence** (start) symbol; NP (noun phrase), VP (verb phrase) and *Article* are **nonterminals**; **the**, **a**, **an** are terminals; the bar | separates alternatives.

Restricting the *form* of the rules gives classes of grammars of increasing power, the **Chomsky–Schützenberger hierarchy**:

Grammar	Languages	Automaton	Rules
Type-0	recursively enumerable	Turing machine	no restrictions
Type-1	context-sensitive	linear-bounded automaton	$\alpha A \beta \to \alpha \gamma \beta$
Type-2	context-free	(non-det.) pushdown automaton	$A \to \gamma$
Type-3	regular	finite-state automaton	$A \to aB$, $A \to a$

Table 12: The Chomsky hierarchy. A, B: nonterminals; a: a terminal; α, β, γ: strings of terminals and nonterminals.

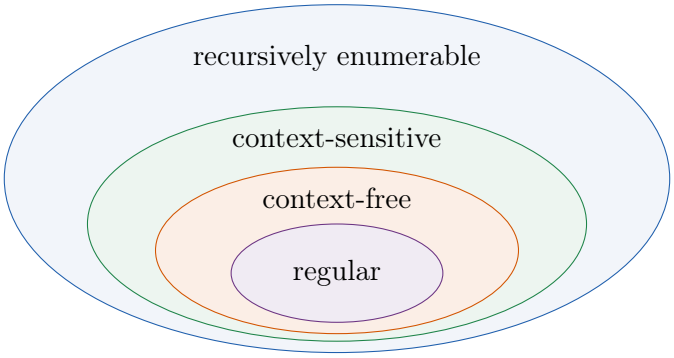


Figure 8: Each class contains the ones below it. Regular expressions (Section 5) describe exactly the regular languages.

In words:

- **Regular**: *nonterminal* $\to$ *terminal* [*nonterminal*], e.g. $S \to \mathbf{a}S$, $S \to \Lambda$ (the empty string): any number of **a**. Recognised with a fixed, finite memory.
- **Context-free**: *nonterminal* $\to$ anything, e.g. $S \to \mathbf{a}S\mathbf{b}$: generates $\mathbf{a}^n\mathbf{b}^n$, which needs to *count* the a's, impossible with finite memory (a stack suffices).
- **Context-sensitive**: more nonterminals on the left-hand side, e.g. $ASB \to AA\mathbf{a}BB$: S can be rewritten only when surrounded by A and B.
- **Recursively enumerable**: no constraints.

Watch out — "natural languages are probably context-free"

The slides say that natural languages are probably context-free, hence parsable in real time. This is a good working assumption: context-free grammars cover most syntactic constructions and are the basis of most parsers. Strictly speaking, however, a few constructions are known to go beyond context-free power, e.g. the cross-serial dependencies of Swiss German (Shieber, 1985). Linguists therefore say that natural languages are *mildly* context-sensitive.

7.3.3 A grammar for the Wumpus world

[Slides · Syntax: sentence, rules: 14–15]

The Wumpus world is a toy game from AI textbooks: an agent explores a cave with pits and a monster, the wumpus, and perceives stenches and breezes. A tiny fragment

of English for talking about it consists of a **lexicon** and a **grammar**.

Lexicon

Noun	→	stench | breeze | glitter | nothing | wumpus | pit | pits | gold | east | ...
Verb	→	is | see | smell | shoot | feel | stinks | go | grab | carry | kill | turn | ...
Adjective	→	right | left | east | south | back | smelly | ...
Adverb	→	here | there | nearby | ahead | right | left | east | ...
Pronoun	→	me | you | I | it | ...
Name	→	John | Mary | Boston | ...
Article	→	the | a | an | ...
Preposition	→	to | in | on | near | ...
RelPronoun	→	that | which | ...

The lexicon is divided into **closed classes** (articles, pronouns, prepositions: a small fixed set, new members are almost never added) and **open classes** (nouns, verbs, adjectives, adverbs: new words appear all the time).

Grammar

S	→	$NP\ VP$	I + feel a breeze
Subject	→	*Pronoun*	I
	|	*Noun*	pits
	|	*Article Noun*	the + wumpus
	|	*Article Adjective Noun*	the + smelly + wumpus
NP	→	*Subject*	the wumpus
	|	*Subject PP*	the wumpus + to the east
	|	*Subject RelClause*	the wumpus + that is smelly
VP	→	$Verb$	stinks
	|	$Verb\ NP$	feel + a breeze
	|	*Verb Adjective*	is + smelly
	|	$Verb\ PP$	turn + to the east
	|	*Verb Adverb*	go + ahead
PP	→	*Preposition* NP	to + the east
RelClause	→	*RelPronoun* VP	that + is smelly

All rules have a single nonterminal on the left: this is a **context-free** grammar. Note the recursion: $NP \rightarrow$ *Subject* PP and $PP \rightarrow$ *Preposition* NP, so noun phrases can nest without limit (*the wumpus to the east of the pit near the gold ...*).

Worked example — parsing "I feel a breeze"

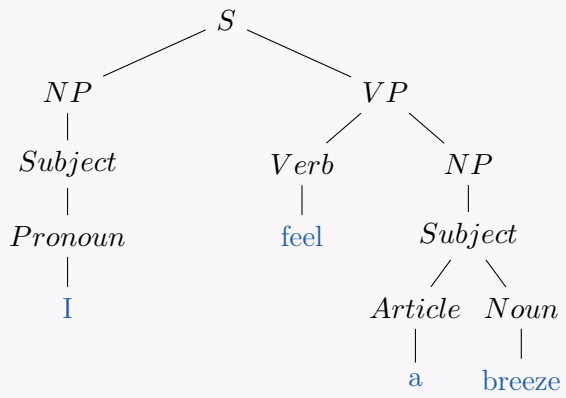


Figure 9: Parse tree. The node right above each word is its part of speech: *I*/Pronoun, *feel*/Verb, *a*/Article, *breeze*/Noun.
Derivation, always rewriting the leftmost nonterminal: $S \Rightarrow NP\ VP \Rightarrow Subject\ VP \Rightarrow Pronoun\ VP \Rightarrow$ I $VP \Rightarrow$ I $Verb\ NP \Rightarrow$ I feel $NP \Rightarrow$ I feel $Subject \Rightarrow$ I feel $Article\ Noun \Rightarrow$ I feel a breeze.

Watch out — this grammar overgenerates

The same rules also generate **I stinks*, **me feel a breeze* and **the wumpus feel a breeze*: nothing checks that subject and verb agree in person and number, or that a pronoun in subject position is *I* and not *me*. Adding a separate nonterminal for every combination (singular/plural, subject/object, ...) would make the grammar explode. Definite Clause Grammars solve this with *arguments*.

7.3.4 Definite Clause Grammars

[Slides · Syntax: sentence, rules: 16–17]

Definition — Definite Clause Grammars (DCG)

DCGs are a notation for grammars natively integrated in the Prolog programming language.

- They represent context-free grammars...
- ...but nonterminals can have **arguments** (compare with *attribute grammars*), which can express context-dependent constraints: this makes the formalism extremely powerful.
- Being Prolog, the same grammar works both as a **recogniser** (is this sentence grammatical?) and as a **generator** (list all grammatical sentences), and arbitrary reasoning can be added on top of it ("but, to your relief, we will not!").

Worked example — agreement with an argument

```
s          --> np(Num), vp(Num).          % subject and verb share Num
np(Num)    --> article, noun(Num).
vp(Num)    --> verb(Num).
article    --> [the].
noun(sing) --> [wumpus].     noun(plur) --> [wumpuses].
verb(sing) --> [stinks].     verb(plur) --> [stink].
```

The variable `Num` is the same in `np(Num)` and `vp(Num)`, so the query `phrase(s, [the, wumpus, stinks])` succeeds while `phrase(s, [the, wumpus, stink])` fails. The query `phrase(s, X)` enumerates all (two) sentences of the language. DCGs can be run online in SWISH, https://swish.swi-prolog.org/.

Intuition — why arguments add power

Without arguments, enforcing agreement needs one copy of each rule per combination of features (NP_{sing}, NP_{plur}, ...). With arguments, one rule says "whatever the number is, it must be the same here and there". Arguments can carry any Prolog term, including counters, so DCGs can describe even languages that are not context-free, such as $\mathbf{a}^n\mathbf{b}^n\mathbf{c}^n$.

7.4 Statistical POS Tagging: Hidden Markov Models

[Slides · Syntax: sentence, HMM: 1–20]

The rule-based approach of Section 7.3 needs a linguist to write the grammar. The statistical approach instead *learns* from a corpus already tagged by hand how likely each tag is after another tag, and how likely each word is for each tag, then picks the most probable tag sequence for a new sentence. The classic model is the Hidden Markov Model. The slides (from a Microsoft course on edX) introduce it with a non-linguistic example, which we follow; then we map it to POS tagging.

7.4.1 The model

[Slides · Syntax: sentence, HMM: 3]

Definition — Hidden Markov Model (HMM)

An **HMM** models a process that moves between **states**:

- the states are **hidden**: we cannot observe them;
- at each step the current state emits an **output** (observation), which we do see.

Our goal: from the sequence of outputs, find the most likely **sequence of states**. Formally an HMM has

- a set of states Q and of possible observations O;
- **start probabilities** $\pi(s)$: probability that the first state is s;

- **transition probabilities** $a(s' \to s) = P(s_t = s \mid s_{t-1} = s')$;
- **emission probabilities** $b_s(o) = P(o_t = o \mid s_t = s)$.

Two assumptions make it "Markov" and keep it tractable: the next state depends only on the current state (not on the whole history), and each output depends only on the state that emits it.

7.4.2 The weather example

[Slides · Syntax: sentence, HMM: 4–5]

We are interested in the weather of a small town during the week of 5 October 1900. No weather records were kept, but we have the diary of a teenager who wrote down the one snack he had each day: hot chocolate, soda or ice cream. For our purposes the weather is *cold*, *normal* or *hot*. We are given:

Start / transition P(today | yesterday)				**Emission** P(snack | weather)			
from ↓ to →	Cold	Normal	Hot		hot choc.	soda	ice cream
Start	.1	.8	.1	Cold	.7	.3	0
Cold	.7	.1	.2	Normal	.1	.7	.2
Normal	.3	.4	.3	Hot	0	.6	.4
Hot	.2	.4	.4				

Each row sums to 1. For example $P(\text{hot} \mid \text{cold}) = .2$ and $P(\text{soda} \mid \text{hot}) = .6$. The diary for the 7 days is

soda, soda, hot chocolate, ice cream, soda, soda, ice cream.

Question: what was the most likely weather sequence for the week?

Intuition — why not try every sequence?

With 3 states and 7 days there are $3^7 = 2187$ possible weather sequences; with the 45 tags of a POS tagset and a 20-word sentence there are 45^{20}, far too many. The Viterbi algorithm finds the best one in time proportional to (days) × (states)^2, using the same trick as the edit distance (Section 6.4): for every state at every step, keep only the best way to get there.

7.4.3 The Viterbi algorithm

[Slides · Syntax: sentence, HMM: 6–16]

Key result — Viterbi

Let $v_t(s)$ be the probability of the *best* state sequence that ends in state s on day t and emits the first t observations.

$$\begin{aligned} \text{day } 1: \quad & v_1(s) = \pi(s) \cdot b_s(o_1) && \triangleleft \text{ p(transition) } \times \text{ p(output)} \\ \text{day } t > 1: \quad & v_t(s) = \max_{s'} \big[\, v_{t-1}(s') \cdot a(s' \to s) \,\big] \cdot b_s(o_t) \end{aligned}$$

For each cell we also store a **backpointer** to the s' that achieved the maximum. At the end, pick the state with the largest $v_T(s)$ and follow the backpointers back to day 1.

Worked example — the first two days

Day 1 (soda):

$$v_1(\text{Cold}) = .1 \cdot .3 = .03, \quad v_1(\text{Normal}) = .8 \cdot .7 = .56, \quad v_1(\text{Hot}) = .1 \cdot .6 = .06.$$

Day 2 (soda), cell Cold: which yesterday is the best way to arrive at Cold?

$$\begin{aligned} v_2(\text{Cold}) &= \max\big(\underbrace{.03 \cdot .7}_{\text{from Cold}}, \ \underbrace{.56 \cdot .3}_{\text{from Normal}}, \ \underbrace{.06 \cdot .2}_{\text{from Hot}} \big) \cdot .3 \\ &= \max(.021,\ .168,\ .012) \cdot .3 = .168 \cdot .3 = .0504 \qquad \triangleleft \text{ backpointer: Normal} \end{aligned}$$

Similarly $v_2(\text{Normal}) = \max(.03 \cdot .1,\ .56 \cdot .4,\ .06 \cdot .4) \cdot .7 = .224 \cdot .7 = .1568$ and $v_2(\text{Hot}) = \max(.03 \cdot .2,\ .56 \cdot .3,\ .06 \cdot .4) \cdot .6 = .168 \cdot .6 = .1008$, both with backpointer Normal.

Continuing the same way for all seven days fills the **trellis**:

	Day 1 soda	Day 2 soda	Day 3 hot choc.	Day 4 ice cream	Day 5 soda	Day 6 soda	Day 7 ice cream
Hot	.06	.1008 ←N	0	**.00263** ←C	.000632 ←H	.000152 ←H	**.0000248** ←N
Normal	**.56**	**.1568** ←N	.00627 ←N	.000659 ←C	**.000738** ←H	**.000207** ←N	.0000165 ←N
Cold	.03	.0504 ←N	**.0329** ←N	0	.000158 ←H	.0000664 ←N	0

Table 13: The Viterbi trellis (values rounded to 3 significant digits). Under each value, the backpointer: the state of the previous day that gave the maximum. Shaded: the best path, recovered by backtracking from the largest value of day 7.

- Some cells are 0 because the output is impossible in that state: no hot chocolate on a hot day, no ice cream on a cold day.
- On day 7 the best value is Hot ($.0000248 > .0000165$), so the path ends in Hot. Its backpointer is Normal (day 6), whose backpointer is Normal (day 5), then Hot (day 4), then Cold (day 3), then Normal, Normal.

Key result — Result

Most likely weather: normal, normal, cold, hot, normal, normal, hot.

Watch out — rounding on the slides

The slides compute with rounded intermediate values, so some cells differ slightly from the exact ones (e.g. day 4 Hot: $.007 \cdot .4 = .0027$ instead of $.0329 \cdot .2 \cdot .4 = .00263$). One cell is a slip: day 2 Normal is written "$.224 \cdot .7 = .1589$", but $.224 \cdot .7 = .1568$. None of this changes the final path.

Intuition — numbers get tiny

Every step multiplies by numbers smaller than 1, so after a few dozen words the values underflow the floating-point range. Real implementations add *logarithms* of the probabilities instead of multiplying them: log is increasing, so the max (and the best path) does not change.

7.4.4 From weather to POS tags

[Slides · Syntax: sentence, HMM: 17–20]

The weather problem is POS tagging in disguise:

	Weather	**POS tagging**
hidden states	cold, normal, hot	tags: DT, NN, VB, ...
observations	snacks in the diary	words of the sentence
transitions	P(hot | cold)	P(NN | DT): a noun after a determiner
emissions	P(soda | hot)	P(flies | VBZ)
answer	the weather of the week	the tag of each word

Transition and emission probabilities are estimated by *counting* in a hand-tagged corpus (e.g. the Brown corpus): P(NN | DT) = (number of times DT is followed by NN) / (number of DTs). The Python code of the course implements Viterbi on the weather example and prints the guessed sequence (`day=1 normal`, `day=2 normal`, `day=3 cold`, ..., `day=7 hot`).

HMM lab. Experiment with the HMM Python code; extend it with other parts of speech (see the Brown corpus tags, https://en.wikipedia.org/wiki/Brown_Corpus#Part-of-speech_tags_used), words and transitions, and use it to tag "*time flies like an arrow*" (Section 7.2); invent other short ambiguous sentences and extend the code so that it tags them properly.

7.4.5 Maximum Entropy Markov Models

[Slides · Syntax: sentence, HMM: 19]

An HMM can reach very high accuracy in POS tagging, but it needs architectural additions to deal with **unknown words**, suffixes and so on: an HMM can only use $P(\text{word} \mid \text{tag})$, so a word never seen in training has no emission probability at all.

Maximum Entropy Markov Models (MEMM) combine the HMM idea (a sequence of hidden states, decoded with Viterbi) with **logistic regression**, a probabilistic classifier trained with supervised machine learning. Instead of $P(\text{word} \mid \text{tag})$, an MEMM directly models $P(\text{tag} \mid \text{word, previous tag, features})$, where the features can be anything: suffix (*-ing*, *-ly*), capitalisation, presence of digits or hyphens. This handles unknown words naturally. An MEMM is trained on a POS-annotated text of the language; this is how the Stanford POS tagger works (https://nlp.stanford.edu/software/tagger.shtml).

7.4.6 Exercises

[Not in the slides: added material]

Exercise 1 — a shorter diary

With the same model, find the most likely weather for a two-day diary: *hot chocolate, ice cream.*

Solution

Day 1 (hot chocolate): $v_1(\text{C}) = .1 \cdot .7 = .07$, $v_1(\text{N}) = .8 \cdot .1 = .08$, $v_1(\text{H}) = .1 \cdot 0 = 0$.
Day 2 (ice cream):

$$
\begin{aligned}
v_2(\text{C}) &= \max(\dots) \cdot 0 = 0 \\
v_2(\text{N}) &= \max(.07 \cdot .1,\ .08 \cdot .4,\ 0) \cdot .2 = .032 \cdot .2 = .0064 && \triangleleft \text{ from N} \\
v_2(\text{H}) &= \max(.07 \cdot .2,\ .08 \cdot .3,\ 0) \cdot .4 = .024 \cdot .4 = .0096 && \triangleleft \text{ from N}
\end{aligned}
$$

Best final state: Hot (.0096), backpointer Normal. Answer: **normal, hot**.
Lesson: on day 1 Cold explains hot chocolate better (.7 vs .1), but the start probabilities favour Normal so much (.8 vs .1) that Normal wins. The best path is a compromise between transitions and emissions.

Cheat sheet — syntax at the sentence level

- Sentence breaking: period = end, unless the previous token is an abbreviation, but yes if the next token is capitalised: about 95% correct.
- POS tagging: tag each word using definition *and* context; Penn Treebank tagset (36 tags: NN, NNS, VB, VBZ, JJ, DT, IN, ...).
- Generative grammar (Chomsky 1957). Chomsky hierarchy: regular $\subset$ context-free $\subset$ context-sensitive $\subset$ recursively enumerable (finite automaton, pushdown automaton, linear-bounded TM, TM).
- CFG = lexicon + rules with one nonterminal on the left; parse tree; plain

CFGs overgenerate (no agreement). DCGs add arguments to nonterminals (Prolog).
- HMM: hidden states, observed outputs; π, transitions a, emissions b. Viterbi: $v_t(s) = \max_{s'} v_{t-1}(s')\, a(s' \to s)\, b_s(o_t)$ + backpointers.
- MEMM = HMM + logistic regression; uses arbitrary features, handles unknown words.

8 Distributional Semantics

[Slides · Distributional semantics: 1–27; Distributional semantics (long version): 28–72; Semantics (long version): 1–6]

With this section we move up from syntax to **semantics**: *what does it mean?* (Section 3.1). The first and today most successful approach does not define meanings at all: it derives them from *how words are used* in large corpora. This is the idea behind word embeddings and, ultimately, behind LLMs. The slides are taken from Part 1 of the *Distributional Semantic Models* tutorial by Stefan Evert, Alessandro Lenci, Marco Baroni and Gabriella Lapesa (`http://wordspace.collocations.de`).

This section covers:

1. the distributional hypothesis (Section 8.1);
2. the co-occurrence matrix and a thought experiment that shows its power (Section 8.2);
3. the geometric view: words as vectors, similarity as angle (Section 8.3);
4. distributional semantic models in practice: how they are built, what they are used for, and three famous models (Section 8.4).

8.1 The Distributional Hypothesis

[Slides · Distributional semantics: 3–16; Semantics (long version): 3–5]

8.1.1 Meaning and distribution

[Slides · Distributional semantics: 4–7]

Four famous quotations, each followed by what it means for us:

- "*Die Bedeutung eines Wortes liegt in seinem Gebrauch*" ("the meaning of a word is its use", Ludwig Wittgenstein).
  ⇒ **meaning = use = distribution in language**.
- "*You shall know a word by the company it keeps!*" (J. R. Firth, 1957).
  ⇒ **distribution = collocations** = habitual word combinations (Section 4.3).
- **Distributional hypothesis**: difference of meaning correlates with difference of distribution (Zellig Harris, 1954).

⇒ we can measure a **semantic distance** between words.

- "*What people know when they say that they know a word is not how to recite its dictionary definition – they know how to use it [. . .] in everyday discourse*" (George Miller, 1986).

Definition — Distributional hypothesis

Words that occur in similar contexts tend to have similar meanings. Conversely, the more the contexts of two words differ, the more their meanings differ.

8.1.2 What is the meaning of "bardiwac"?

[Slides · Distributional semantics: 8–16]

You have never seen the word *bardiwac*. Read these sentences, one at a time:

- He handed her her glass of bardiwac.
- Beef dishes are made to complement the bardiwacs.
- Nigel staggered to his feet, face flushed from too much bardiwac.
- Malbec, one of the lesser-known bardiwac grapes, responds well to Australia's sunshine.
- I dined off bread and cheese and this excellent bardiwac.
- The drinks were delicious: blood-red bardiwac as well as light, sweet Rhenish.

Intuition — what you just did

After a few sentences you know that **bardiwac is a heavy red alcoholic beverage made from grapes**, i.e. a red wine (the word is invented; it stands for *claret*). Nobody gave you a definition: you inferred it from the *company the word keeps*: *glass of*, *too much*, *grapes*, *drinks*, *blood-red*, *beef dishes*. That is the distributional hypothesis in action, and it is how we learn most of our vocabulary.

The examples are hand-picked and edited, of course. But in a corpus like the British National Corpus (BNC) there is at least as much relevant information. The slides show a *word sketch* of the real word (BNC frequency 230): grouped by grammatical relation, its most typical companions are verbs like *uncork*, *gulp*, *sip*, *drink*, *swig*; coordinated nouns like *burgundy*, *Sancerre*, *scampi*, *ruby*; containers like *bottle*, *glass*, *jug*, *goblet*; modifiers like *full-bodied*, *blood-red*, *ready-to-drink*. A computer can collect these statistics automatically.

8.2 The Co-occurrence Matrix

[Slides · Distributional semantics: 17–21]

8.2.1 A thought experiment: deciphering hieroglyphs

[Slides · Distributional semantics: 17–20]

Imagine an unknown language. We know the meaning of some nouns (in brackets) but not of *dog*, and we do not know any of the verbs. For each noun we count how often it appears as the *object* of each verb in a large corpus:

		get	**sij**	**ius**	**hir**	**iit**	**kil**
(knife)	naif	51	20	84	0	3	0
(cat)	ket	52	58	4	4	6	26
???	dog	115	83	10	42	33	17
(boat)	beut	59	39	23	4	0	0
(cup)	kap	98	14	6	2	1	0
(pig)	pigij	12	17	3	2	9	27
(banana)	nana	11	2	2	0	18	0

Which known word is *dog* most similar to? Comparing rows (the similarity measure is explained in Section 8.3):

$$\text{sim}(\text{dog}, \text{naif}) = 0.770, \qquad \text{sim}(\text{dog}, \text{pigij}) = 0.939, \qquad \text{sim}(\text{dog}, \text{ket}) = 0.961.$$

Dog behaves most like *cat*, then *pig*, and much less like *knife*: it is probably an animal, a pet. We concluded this without understanding a single verb.

8.2.2 English as seen by the computer

[Slides · Distributional semantics: 21]

The "unknown language" was English with scrambled spelling: the verbs are **get, see, use, hear, eat, kill**, and the counts are real verb–object co-occurrence counts from the British National Corpus. For a computer, every language looks like hieroglyphs: it only has the counts.

Definition — Co-occurrence matrix and distributional semantic model

A **co-occurrence matrix** M has one row per **target word** and one column per **context** (here: verbs taking the word as object; in general: nearby words, syntactic relations, documents, ...). The cell $M_{w,c}$ counts how often w occurs with context c. The row $\mathbf{x}_w$ is the **distributional vector** of w. A **distributional semantic model** (DSM) represents the meaning of each word by such a vector.

Watch out — how the similarity values were computed

The slides do not say which similarity gives 0.770, 0.939 and 0.961. They are the **cosine similarity** (Section 8.3) of the rows after replacing each count f by $\log(1+f)$. On the raw counts the cosine gives different values:

similarity to *dog*	knife	cat	boat	cup	pig	banana
cosine, raw counts	0.552	**0.923**	0.915	0.832	0.676	0.629
cosine, $\log(1+f)$	0.770	**0.961**	0.828	0.850	0.939	0.785

With raw counts *boat* is almost as close to *dog* as *cat*, and *pig* is far: the large counts of the very frequent verb *get* dominate everything. The logarithm dampens

large counts, so the rarer but more telling contexts (*hear*, *eat*, *kill*) matter too, and the two animals come out on top. Real DSMs always apply such a weighting (log, TF-IDF-like association scores) before comparing vectors.

8.3 Geometric Interpretation

[Slides · Distributional semantics: 22–27; Semantics (long version): 6]

8.3.1 Words as points

[Slides · Distributional semantics: 22–24]

The row vector $\mathbf{x}_{\text{dog}} = (115, 83, 10, 42, 33, 17)$ describes how *dog* is used in the corpus. It can be seen as the coordinates of a **point** in an n-dimensional Euclidean space ($n = 6$ here, one dimension per context). We can draw two dimensions, *get* and *use*: $\mathbf{x}_{\text{dog}} = (115, 10)$, $\mathbf{x}_{\text{cat}} = (52, 4)$, $\mathbf{x}_{\text{knife}} = (51, 84)$, $\mathbf{x}_{\text{boat}} = (59, 23)$.

Two dimensions of the English V–Obj DSM

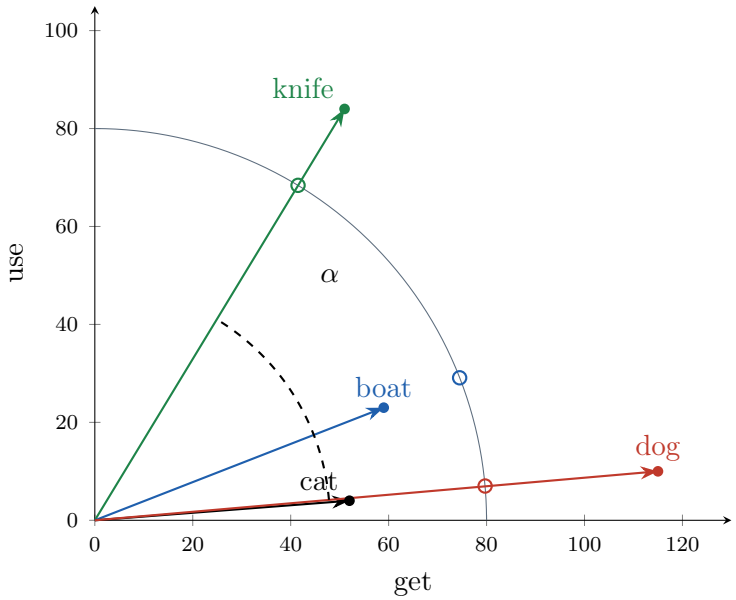


Figure 10: Words as vectors in the (get, use) plane. Circles on the arc: the same vectors normalised to the same length (only the direction is kept). *Dog* and *cat* point in almost the same direction even though they are far apart as points.

A first idea: **similarity = spatial proximity**, measured with the Euclidean distance. But the location of a point depends on the **frequency** of the noun: *dog* is about 2.7 times more frequent than *cat*, so all its counts are larger and its point lies much farther from the origin.

Worked example — Euclidean distance is misleading

$$d(\text{dog}, \text{cat}) = \sqrt{(115-52)^2 + (10-4)^2} = \sqrt{3969+36} \approx 63.3$$
$$d(\text{dog}, \text{boat}) = \sqrt{(115-59)^2 + (10-23)^2} = \sqrt{3136+169} \approx 57.5$$

By Euclidean distance, *boat* is closer to *dog* than *cat* is: a consequence of frequency, not of meaning.

8.3.2 Direction, not location

[Slides · Distributional semantics: 25–27]

A vector can also be seen as an **arrow from the origin**. Its *length* mostly reflects how frequent the word is; its *direction* reflects *the proportions* in which the word occurs with the different contexts, i.e. how it is used. So the direction is more important than the location. Two equivalent solutions:

- use the **angle** α between two vectors as a distance measure;
- or **normalise** the length $\|\mathbf{x}\|$ of every arrow to the same value, and then compare points (the circles on the arc in the figure).

Definition — Cosine similarity

The **cosine similarity** of two vectors is the cosine of the angle between them:

$$\cos\alpha = \frac{\mathbf{x}\cdot\mathbf{y}}{\|\mathbf{x}\|\,\|\mathbf{y}\|} = \frac{\sum_i x_i y_i}{\sqrt{\sum_i x_i^2}\,\sqrt{\sum_i y_i^2}}.$$

It is 1 for vectors pointing in the same direction, 0 for orthogonal vectors (with non-negative counts it is never negative), and it does not change if a vector is multiplied by a positive number: it ignores frequency.

Worked example — dog vs. cat and dog vs. knife in two dimensions

$$\cos(\text{dog}, \text{cat}) = \frac{115\cdot 52 + 10\cdot 4}{\sqrt{115^2+10^2}\,\sqrt{52^2+4^2}} = \frac{6020}{115.43\cdot 52.15} \approx 1.000 \quad (\alpha \approx 0.6^\circ)$$
$$\cos(\text{dog}, \text{knife}) = \frac{115\cdot 51 + 10\cdot 84}{115.43\cdot 98.27} = \frac{6705}{11343} \approx 0.591 \quad (\alpha \approx 53.8^\circ)$$

Now *cat* is (in these two dimensions) practically identical to *dog*, and *knife* is far. The slides report $\alpha = 54.3^\circ$ for knife–dog; computing it from the counts gives 53.8°, the small difference does not matter.

Why this matters for NLP

Every modern word representation, from count-based DSMs to **word embeddings** (*word2vec*, GloVe) to the internal vectors of BERT and GPT, follows this recipe: a word is a vector learnt from its contexts, and similar words have vectors with a small angle between them. Word2vec learns short dense vectors (a few hundred dimensions) directly, instead of counting; implementations in Python and C, with examples, are linked in the course's AulaWeb module.

Cheat sheet — distributional semantics

- Wittgenstein: meaning = use; Firth: "you shall know a word by the company it keeps"; Harris: difference of meaning correlates with difference of distribution.
- *bardiwac*: the meaning of an unknown word can be inferred from its contexts alone.
- Co-occurrence matrix: rows = target words, columns = contexts, cells = counts; each row is the word's vector. Weight the counts (e.g. $\log(1 + f)$) before comparing.
- Euclidean distance is dominated by frequency; use the **angle**, i.e. **cosine similarity** $\mathbf{x} \cdot \mathbf{y}/(\|\mathbf{x}\|\|\mathbf{y}\|)$, or normalise the vectors.
- This is the basis of word embeddings (word2vec) and of the representations in LLMs.

8.4 Distributional Semantic Models in Practice

[Slides · Distributional semantics (long version): 28–72]

The first part of the tutorial (Sections 8.1 to 8.3) gave the idea: a word is a vector of counts, and similar words have vectors pointing in similar directions. The long version of the deck continues with what a real model looks like: how it is defined in general, how it is built step by step, what we can do with it, three famous historical models, and a small hands-on session.

This subsection covers:

1. the general definition of a distributional semantic model (DSM);
2. the pipeline to build one, and the two kinds of co-occurrence matrix;
3. what a DSM gives us: nearest neighbours, semantic maps, clusters, latent dimensions, word embeddings and context vectors;
4. the difference between *distributional* and *distributed* representations;
5. three famous models: LSA, Word Space and HAL;
6. applications, software, and a toy laboratory in R.

8.4.1 General definition of DSMs

[Slides · Distributional semantics (long version): 28–29]

In the hieroglyphs example (Section 8.2) the cells of the matrix were raw counts. Raw counts are rarely used as they are: very frequent contexts (like the verb *get*) dominate, and the numbers are very skewed (a few huge values, many zeros). So a real model first *rescales* or *transforms* the counts.

Definition — Distributional semantic model (DSM)

A **distributional semantic model** is a **scaled and/or transformed co-occurrence matrix M**, such that each row **x** represents the distribution of a **target term** across contexts.
A **term** can be a word, a lemma, a phrase, a morpheme, a word pair, ...: any unit whose meaning we want to represent.

The slide shows the hieroglyphs matrix after such a transformation:

	get	**see**	**use**	**hear**	**eat**	**kill**
knife	0.027	−0.024	0.206	−0.022	−0.044	−0.042
cat	0.031	0.143	−0.243	−0.015	−0.009	0.131
dog	−0.026	0.021	−0.212	0.064	0.013	0.014
boat	−0.022	0.009	−0.044	−0.040	−0.074	−0.042
cup	−0.014	−0.173	−0.249	−0.099	−0.119	−0.042
pig	−0.069	0.094	−0.158	0.000	0.094	0.265
banana	0.047	−0.139	−0.104	−0.022	0.267	−0.042

Table 14: The hieroglyphs matrix after scaling: each row is still the vector of a word, but the values are no longer counts.

Intuition — why negative numbers?

A count can never be negative, so these values cannot be counts. The slide does not say which transformation was applied; negative values appear whenever we measure how much a word occurs with a context *compared to what we would expect by chance*. A positive value says "more often than expected" (*pig–kill*: 0.265, *banana–eat*: 0.267, *knife–use*: 0.206), a negative one "less often than expected" (*cat–use*: −0.243: nobody *uses* a cat). Such "association scores" are much more informative than raw counts, because a high raw count may only mean that both words are frequent.

8.4.2 Building a distributional model

[Slides · Distributional semantics (long version): 30–39]

Building a DSM is a pipeline of choices. Each box in Figure 11 is a decision that changes the resulting model.

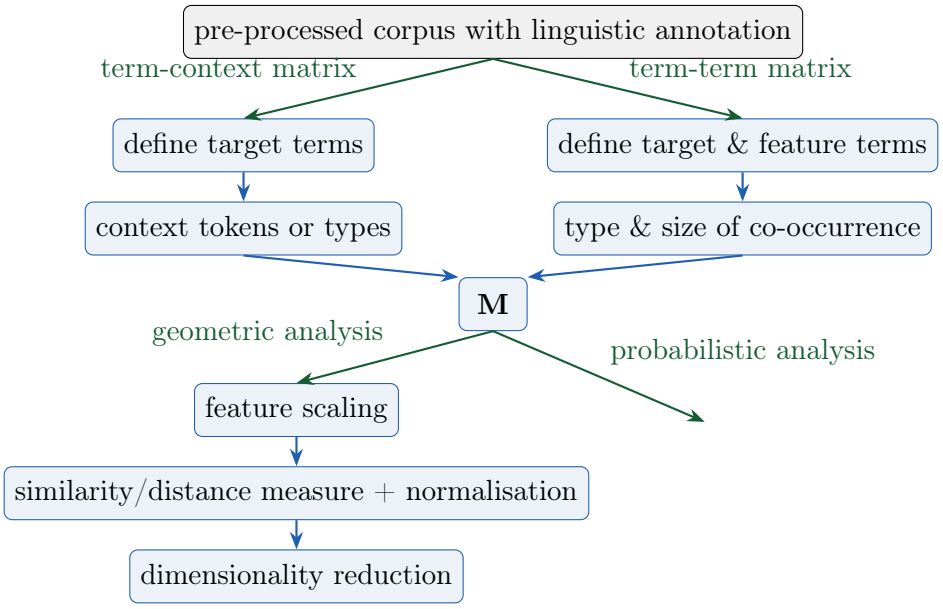


Figure 11: The steps to build a distributional model.

Step by step:

1. **Pre-processed corpus with linguistic annotation.** We start from a large corpus that has been segmented into words (Section 6.1), often lemmatised (Section 6.3), POS-tagged (Section 7.2) and sometimes parsed. The annotation lets us count, e.g., "*dog* as object of *see*" instead of just "*dog* near *see*".

2. **Choose the kind of matrix.** Two families:

   - a **term-term matrix**: rows are target terms, columns are other terms (**feature terms**). We must define both sets and decide the **type and size of co-occurrence**: "within a window of 5 words", "in the same sentence", "as verb and its object", ...
   - a **term-context matrix**: rows are target terms, columns are **contexts** such as sentences, documents, Web pages or encyclopaedia articles. We decide whether the columns are context **tokens** (each single document is a column) or context **types** (similar contexts grouped together).

3. **The matrix M** is filled with counts.

4. **Analysis.** It can be *probabilistic* (the counts are seen as probabilities of co-occurrence) or, as in this course, *geometric* (treat the rows as vectors). The geometric branch has three further choices:

   - **feature scaling**: how to rescale the counts: logarithm, weights similar to TF-IDF, association scores (see the warning in Section 8.2);
   - **similarity/distance measure + normalisation**: e.g. cosine similarity, or vectors normalised to length 1 (Section 8.3);
   - **dimensionality reduction**: compress thousands of columns into a few hundred (e.g. with SVD, see below).

Worked example — a term-term matrix and a term-context matrix

Term-term matrix: how often each target term co-occurs with each feature term (a dash means 0):

	breed	tail	feed	kill	important	explain	likely
cat	83	17	7	37	–	1	–
dog	561	13	30	60	1	2	4
animal	42	10	109	134	13	5	5
time	19	9	29	117	81	34	109
reason	1	–	2	14	68	140	47
cause	–	1	–	4	55	34	55
effect	–	–	1	6	60	35	17

Term-context matrix: how often each term occurs in each context (here, each column is one Wikipedia-like article):

	Felidae	Pet	Feral	Bloat	Philosophy	Kant	Back pain
cat	10	10	7	–	–	–	–
dog	–	10	4	11	–	–	–
animal	2	15	10	2	–	–	–
time	1	–	–	–	2	1	–
reason	–	1	–	–	1	4	1
cause	–	–	–	2	1	2	6
effect	–	–	–	1	–	1	–

In both matrices the words split into two groups. *cat*, *dog*, *animal* co-occur with *breed*, *tail*, *feed* and appear in the articles about pets; *reason*, *cause*, *effect* co-occur with *important*, *explain*, *likely* and appear in the articles about philosophy and medicine. *Time* sits in between: it co-occurs with almost everything, as very general words do.

Watch out — term-term vs. term-context

Do not mix them up. In a **term-term** matrix the columns are *words*, so the matrix tells us which words appear *near* which words. In a **term-context** matrix the columns are *pieces of text* (documents, sentences), so it tells us in *which texts* a word appears. The second is the classic matrix of information retrieval: transposed, its columns are the TF-IDF document vectors of Section 6.2.

8.4.3 What a DSM gives us

[Slides · Distributional semantics (long version): 40–50]

Once every word is a vector, many things become simple geometry.

Nearest neighbours. Take a DSM built from verb–object relations in the British National Corpus (BNC), reduced to 100 dimensions with SVD. For a word, list the other words whose vectors form the smallest **angle** with it (the angle is the "cosine angle" $\alpha = \arccos(\cos\alpha)$ of Section 8.3, in degrees: small angle = very similar):

target	nearest neighbours (angle in degrees)
trousers	shirt (18.5), blouse (21.9), scarf (23.4), jeans (24.7), skirt (25.9), sock (26.2), shorts (26.3), jacket (27.8), glove (28.1), coat (28.8), cloak (28.9), hat (29.1), tunic (29.3), overcoat (29.4), pants (29.8), helmet (30.4), apron (30.5), robe (30.6), mask (30.8), tracksuit (31.0), jersey (31.6), shawl (31.6), …
rage	anger (28.5), fury (32.5), sadness (37.0), disgust (37.4), emotion (39.0), jealousy (40.0), grief (40.4), irritation (40.7), revulsion (40.7), scorn (40.7), panic (40.8), bitterness (41.6), resentment (41.8), indignation (41.9), excitement (42.0), hatred (42.5), envy (42.8), disappointment (42.9), …

All neighbours of *trousers* are clothes, all neighbours of *rage* are emotions, and the closest ones are near-synonyms (*anger*, *fury*). The model was never told what clothes or emotions are: it only saw which verbs take these nouns as objects (*wear*, *put on*, *take off* for clothes; *feel*, *express*, *hide* for emotions).

Worked example — reading an angle

$\alpha = 18.5°$ for *trousers*–*shirt* means $\cos\alpha = \cos 18.5° \approx 0.948$: the two vectors point in almost the same direction. $\alpha = 28.5°$ for *rage*–*anger* gives $\cos\alpha \approx 0.879$. Angles are easier to read than cosines because they spread the values out: 0.948 and 0.879 look close, 18.5° and 28.5° do not.

Similarity graph. The neighbours can be drawn as a graph: one node per word, an edge between two words when they are close (small angle). The graph of the neighbours of *trousers* is a dense web of clothes, with *trousers* and *shirt* at its centre; the graph of *plant* is more interesting, because *plant* is ambiguous: its neighbours form **two groups**, one for the industrial plant (factory, station, reactor, mill, brewery, smelter, laboratory, …) and one for the living plant (crop, species, flower, seed, seedling, shrub, …).

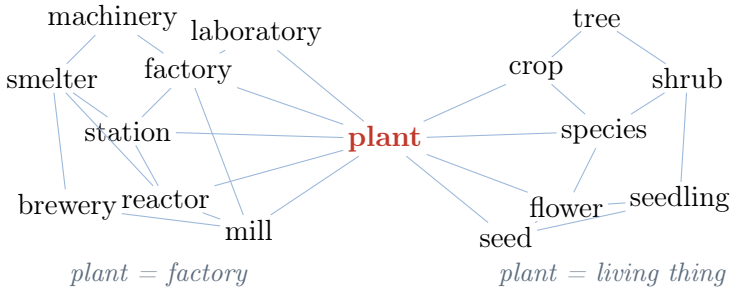


Figure 12: Similarity graph of the neighbours of *plant* (redrawn and simplified; the word *tree* is added for readability). The two senses of the word show up as two clusters joined only through *plant* itself.

Why this matters for NLP — distributional semantics meets word senses

The vector of an ambiguous word is a *mixture* of its senses: *plant* is partly near factories and partly near flowers. Looking at how its neighbours cluster is a way

to discover the senses automatically (*word sense induction*), without a dictionary like WordNet (Section 10.2).

Semantic maps. Vectors have hundreds of dimensions, but they can be projected onto a plane so that close vectors stay close. The slide shows 44 concrete nouns of six categories (birds, ground animals, green vegetables, fruit, tools, vehicles), from the same verb–object DSM:

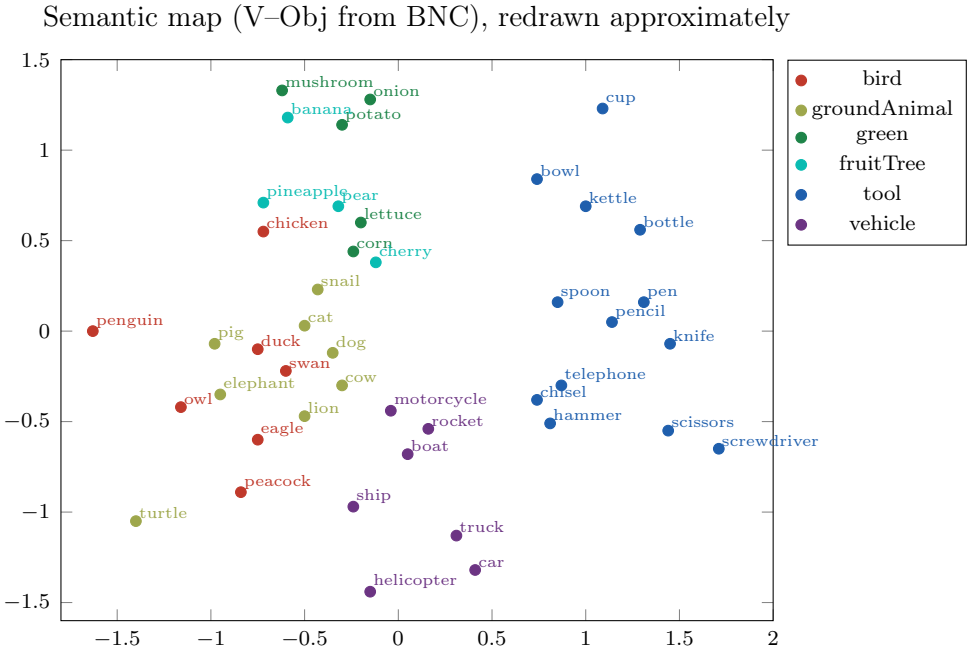


Figure 13: A semantic map: each noun is a point, colours are the true categories (not used by the model). Tools on the right, vehicles at the bottom, food at the top, animals on the left.

The categories come out as regions of the plane, even though the model never saw them. The borders are not perfect: *chicken* sits next to the fruit and vegetables, which makes sense from the point of view of verbs (we *eat* and *cook* chicken).

Clustering. A clustering algorithm groups the vectors automatically. With *hierarchical* clustering we get a **dendrogram**: a tree whose leaves are the words; two branches join at a height that measures how different the two groups are (high joins = very different groups).

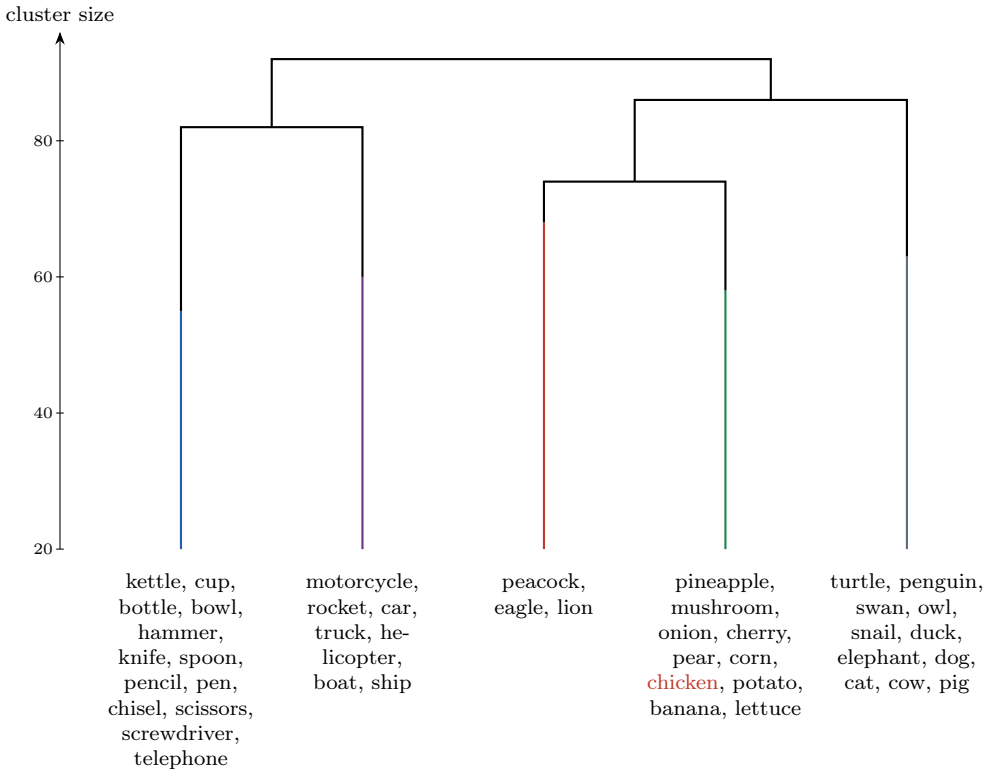


Figure 14: Hierarchical clustering of the 44 concrete nouns (redrawn and simplified: each leaf here is a group of words that the original dendrogram joins low). The vertical axis is the dissimilarity at which groups merge.

Artefacts (tools, vehicles) separate from natural things (food, animals) at the top. The clustering is good but not perfect: *chicken* goes with food, and *peacock*, *eagle*, *lion* form a small group of their own (they are mostly *seen*, rarely *eaten* or *fed*). These "errors" are informative: they show that the model captures how words are *used*, not a biological taxonomy.

Latent dimensions. Plot the nouns using only two columns of the matrix: how often each noun is the object of *buy* (x axis) and of *sell* (y axis), on a log scale.

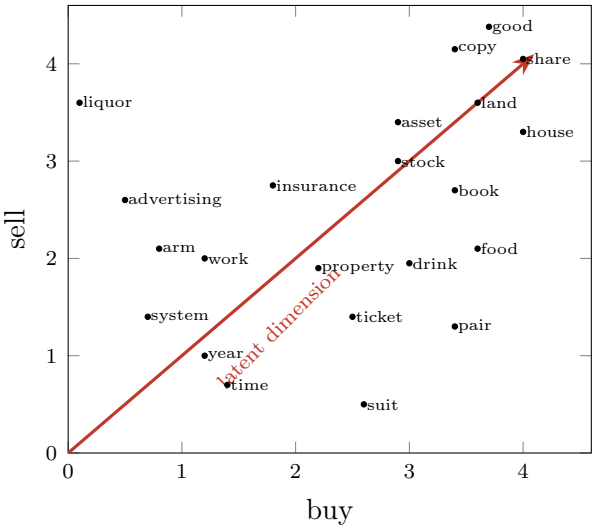


Figure 15: Nouns by how often they are bought and sold (redrawn approximately, a few words only). The red arrow is the direction along which most words lie.

Most words lie along the diagonal: things that are bought are also sold. So the two columns *buy* and *sell* are largely *redundant*; together they really measure one underlying (**latent**) property, "being an object of commercial transactions". A single axis along the red arrow describes these words almost as well as the two original axes. This is exactly what **dimensionality reduction** does on the whole matrix: it finds a few latent dimensions that summarise many correlated columns.

Intuition — SVD in one paragraph

The **singular value decomposition** (SVD) is the standard tool for this. It rotates the coordinate system so that the first new axis follows the direction of largest spread of the points (the red arrow), the second axis the largest remaining spread, and so on. Keeping only the first k new axes (e.g. $k = 100$ or 300) gives short vectors that keep most of the information, remove noise, and make similar words closer (two synonyms that never share an exact context may still share a latent dimension).

Word embeddings and context vectors. A DSM vector is a **sub-symbolic** representation of meaning: no single number means anything to a human (unlike a dictionary definition or a WordNet synset), but the vector as a whole encodes how the word is used. Such vectors are used as

- a **feature vector** for a machine learning algorithm;
- the **input of a neural network**.

They can also represent a *single occurrence* of a word in context. **Context vectors** (Schütze, 1998) use a **bag-of-words** approach: the vector of a sentence is the **centroid** (the average) of the vectors of all the context words in it. This can be applied to word sense disambiguation (WSD, Section 10.3).

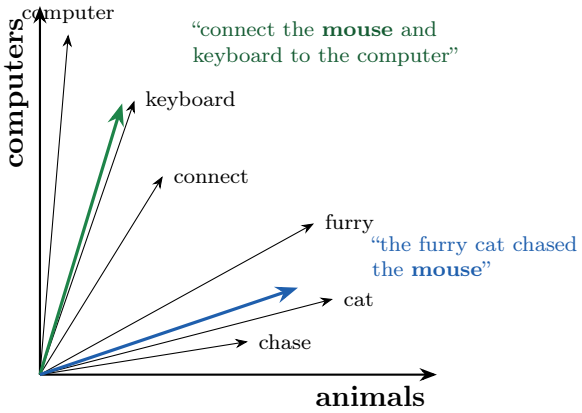


Figure 16: Context vectors in a toy 2-dimensional space. Each thin arrow is a word vector; each thick arrow is the centroid of the context words of a sentence. The two occurrences of *mouse* get very different context vectors.

Worked example — disambiguating *mouse*

With the toy vectors of Figure 16, the first sentence has context words *connect*, *keyboard*, *computer*:

$$\tfrac{1}{3}\big[(1.3, 2.1) + (1.0, 2.9) + (0.3, 3.6)\big] = \tfrac{1}{3}(2.6,\ 8.6) \approx (0.87,\ 2.87),$$

pointing towards "computers". The second has *furry*, *cat*, *chase*:

$$\tfrac{1}{3}\big[(2.9, 1.6) + (3.1, 0.8) + (2.2, 0.35)\big] = \tfrac{1}{3}(8.2,\ 2.75) \approx (2.73,\ 0.92),$$

pointing towards "animals". Comparing each context vector (with the cosine) to prototype vectors of the two senses, we pick the device in the first sentence and the rodent in the second: this is WSD without any dictionary. (The numbers are invented for the picture; real vectors have hundreds of dimensions.)

8.4.4 Distributional vs. distributed

[Slides · Distributional semantics (long version): 51–53]

Two words that sound alike and are often confused:

Distributional model	**Distributed representation**
captures the linguistic *distribution* of each word as a high-dimensional numeric vector typically (but not necessarily) based on co-occurrence counts	a *sub-symbolic* representation of words as high-dimensional numeric vectors similarity of vectors usually (but not necessarily) corresponds to semantic similarity of the words
relies on the distributional hypothesis: distributional similarity ∼ semantic similarity	hot topic: unsupervised neural **word embeddings** (word2vec, GloVe, . . .)

Key result — the relation between the two

Distributional says where the vector comes from (the contexts of the word); *distributed* says what the vector is (meaning spread over many numbers, none of which is meaningful alone). A distributional model **can be used as** a distributed representation, and neural word embeddings are both.

8.4.5 Three famous examples

[Slides · Distributional semantics (long version): 54–61]

Three historical models follow. Each one is a different set of choices in the pipeline of Figure 11.

Latent Semantic Analysis (LSA; Landauer and Dumais, 1997).

- Corpus: 30 473 articles of Grolier's Academic American Encyclopedia (4.6 million words), each article cut to its first 2 000 characters.
- A **word-article** (term-context) frequency matrix for 60 768 words: the row of a word gives its frequency in each article.
- Scaling: logarithmic frequencies, scaled by the **entropy** of the word (a word spread evenly over all articles has high entropy and says little about any of them, so it gets a low weight; the same idea as the idf).
- Reduced to 300 dimensions by **SVD**, a technique borrowed from Latent Semantic Indexing (LSI, Dumais et al., 1988) in information retrieval. Central claim: SVD reveals *latent semantic features*, it is not just a data reduction technique.
- Evaluation: the **TOEFL synonym test** (80 items: given a word, choose its synonym among four options). LSA got **64.4%** correct, about the score of non-native applicants to US universities. The authors also simulated the learning rate of vocabulary based on the TOEFL results.

Word Space (Schütze, 1992, 1993, 1998).

- Corpus: about 60 million words of news messages from the New York Times News Service.
- A **word-word** (term-term) co-occurrence matrix: 20 000 target words and 2 000 context words as features. The row records how often each context word occurs *close to* the target word, in a window of 50 words left and right (1998) or about 1 000 characters (1992).
- Rows weighted by inverse document frequency (tf.idf).
- **Context vector** = centroid of the word vectors of a context (bag-of-words, Figure 16); goal: determine the "meaning" of a context.
- Reduced to 100 SVD dimensions, mainly for efficiency.
- Evaluation: **unsupervised word sense induction** by clustering the context vectors of an ambiguous word; the induced senses improve information retrieval performance.

HAL (Hyperspace Analogue to Language; Lund and Burgess, 1996).

- Corpus: 160 million words from newsgroup postings.
- A **word-word** matrix where the same 70 000 words are both targets and features; co-occurrence window of 1 to 10 words.
- **Separate counts for left and right co-occurrence**: *the* before *dog* and *the* after *dog* are different features, so the context is *structured*. In later work co-occurrences are weighted by (inverse) distance: a word right next to the target counts more than one ten words away (Li et al., 2000).
- No dimensionality reduction.
- Applications include semantic vocabulary maps, obtained by **multidimensional scaling** (MDS) to 2 dimensions (a projection that tries to keep the distances between points).

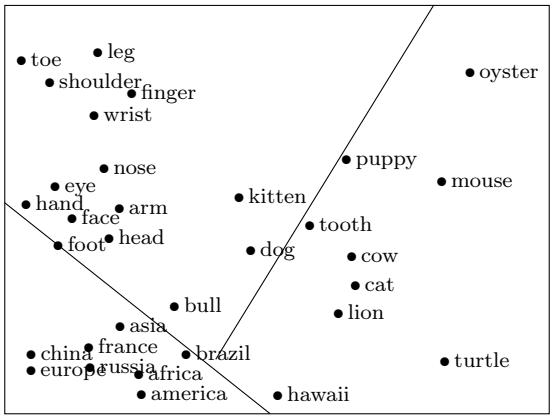


Figure 17: A HAL vocabulary map obtained by multidimensional scaling (redrawn approximately): body parts at the top left, places at the bottom, animals on the right. The two lines separate the three regions.

	LSA	**Word Space**	**HAL**
matrix	term-context (word × article)	term-term, window 50 words	term-term, window 1–10, left/right separate
corpus	encyclopaedia, 4.6M words	news, 60M words	newsgroups, 160M words
scaling	log + entropy	tf.idf	(later) inverse distance
reduction	SVD, 300 dim.	SVD, 100 dim.	none
evaluation	TOEFL synonyms (64.4%)	word sense induction	semantic maps (MDS)

Table 15: The three famous DSMs side by side.

Key result — many parameters

There is an enormous range of DSM parameters and applications: the three examples are three entirely different models, each tuned to its own application. So we need an overview of the parameters and of their effects (the further parts

of the tutorial: the parameters of a DSM, evaluating DSM representations, the mathematics of DSMs, understanding distributional semantics). The lesson: **distributional semantics is an empirical science**: which choices work best is found by experiment, not derived from first principles.

8.4.6 Applications and software

[Slides · Distributional semantics (long version): 62–66]

Some applications of DSMs in computational linguistics:

- unsupervised part-of-speech induction (Schütze, 1995): words with similar distributions tend to have the same POS;
- word sense disambiguation (Schütze, 1998);
- query expansion in information retrieval (Grefenstette, 1994): add to a query the nearest neighbours of its words;
- synonym tasks and other language tests (Landauer and Dumais, 1997; Turney et al., 2003);
- thesaurus compilation (Lin, 1998; Rapp, 2004);
- ontology and wordnet expansion (Pantel et al., 2009);
- attachment disambiguation (Pantel and Lin, 2000): in "eat pizza with a fork" does *with a fork* attach to *eat* or to *pizza*?
- probabilistic language models (Bengio et al., 2003), the ancestor of neural language models;
- sub-symbolic input representation for neural networks;
- many other tasks in computational semantics: entailment detection, noun compound interpretation, identification of non-compositional expressions (idioms such as *kick the bucket*), ...

The field had many dedicated workshops between 2007 and 2014 (CoSMo, GeMS, DiSCo, CVSC, ESSLLI courses, ...). Software packages:

package	**language**	**notes**
HiDEx	C++	re-implementation of the HAL model
SemanticVectors	Java	scalable architecture based on random indexing
S-Space	Java	complex object-oriented framework
JoBimText	Java	UIMA / Hadoop framework
Gensim	Python	complex framework, focus on parallelisation and out-of-core algorithms (it also implements word2vec)
DISSECT	Python	user-friendly, designed for research on compositional semantics
wordspace	R	interactive research laboratory, but scales to real-life data sets

Further material: the wordspace wiki (`http://wordspace.collocations.de/`), the open source tutorial, and the review paper by Turney and Pantel (2010), *From fre-*

quency to meaning: Vector space models of semantics, Journal of Artificial Intelligence Research 37, 141–188.

8.4.7 R as a toy laboratory

[Slides · Distributional semantics (long version): 67–72]

The tutorial uses the statistical environment **R** (with RStudio and the CRAN packages `sparsesvd` and `wordspace`) as a laboratory. The `wordspace` package contains the matrices seen above as example data sets: the hieroglyphs matrix, the term-term matrix and the term-context matrix (each stored under a name such as `DSM_HieroglyphsMatrix`). The basic operations on a DSM matrix:

```
library(wordspace)
# apply log-transformation to de-skew co-occurrence frequencies
M <- log2(DSM_HieroglyphsMatrix + 1)
round(M, 3)
# compute semantic distance (cosine similarity)
pair.distances("dog", "cat", M, convert=FALSE)
#   dog/cat
# 0.9610952
# find nearest neighbours
nearest.neighbours(M, "dog", n=3)
#      cat      pig      cup
# 16.03458 20.08826 31.77784
plot(nearest.neighbours(M, "dog", n=3, dist.matrix=TRUE))
```

What each line does:

- `log2(M + 1)` replaces each count f by $\log_2(1 + f)$: the scaling step of the pipeline. The "+1" avoids $\log 0$ (a zero count stays 0).
- `pair.distances(..., convert=FALSE)` returns the cosine similarity itself; it is 0.961, the value of the hieroglyphs slide (Section 8.2). This confirms how that value was computed.
- `nearest.neighbours` returns the closest words with their *angles* in degrees: $\arccos(0.961) \approx 16.0°$ for *cat*, 20.1° for *pig*, 31.8° for *cup*.

Exercise 8.4.1 — from cosine to angle and back

Using the log-scaled hieroglyphs matrix, $\cos(\text{dog}, \text{pig}) = 0.939$ and $\cos(\text{dog}, \text{cup}) = 0.850$ (Section 8.2). (a) Compute the corresponding angles. (b) Why does *cup* come third among the neighbours of *dog* although it is not an animal?

Solution

(a) $\alpha = \arccos(0.939) \approx 20.1°$ and $\alpha = \arccos(0.850) \approx 31.8°$, the values printed by `nearest.neighbours`.

(b) With only six contexts the model is very coarse. *Cup* and *dog* are both very frequent objects of *get* and *see*, and after the log scaling these two columns weigh a lot. The contexts that would separate them (*hear*, *eat*, *kill*) are only three. With thousands of contexts, as in a real DSM, *cup* would be far from *dog*. Lesson: the quality of a DSM depends on how many and which contexts are used.

Cheat sheet — DSMs in practice

- DSM = scaled and/or transformed co-occurrence matrix; each row represents a target term (word, lemma, phrase, ...).
- Pipeline: annotated corpus → term-term matrix (targets, features, type and size of co-occurrence) or term-context matrix (targets, context tokens/types) → **M** → feature scaling → similarity measure + normalisation → dimensionality reduction.
- Uses: nearest neighbours (angles), similarity graphs (ambiguous words split into clusters), semantic maps, clustering, latent dimensions (SVD), features for ML, context vectors (centroids) for WSD.
- Distributional (where the vector comes from) vs. distributed (sub-symbolic vector); a DSM can be used as a distributed representation.
- LSA: word × article, log + entropy, SVD 300, TOEFL 64.4%. Word Space: word × word, window 50, tf.idf, SVD 100, sense induction. HAL: word × word, left/right counts, no reduction, MDS maps.
- Distributional semantics is an empirical science.

9 Frame and Model-Theoretic Semantics

[Slides · Other semantics: 1–44; Semantics (long version): 7–10]

Distributional semantics (Section 8) tells us that *sold* is similar to *bought*, but not *who* sold *what* to *whom*, nor whether a sentence is *true*. This section presents two other views of meaning, following slides 36–56 of Percy Liang's tutorial *Natural Language Understanding: Foundations and State-of-the-Art* (Simons Institute, 2017, `https://simons.berkeley.edu/talks/percy-liang-01-27-2017-1`). The tutorial covers, in order, properties of language, distributional semantics, frame semantics, model-theoretic semantics, interactive learning and reflections; we use the parts on frame and model-theoretic semantics.

This section covers:

1. **frame semantics** and semantic role labelling (Section 9.1);
2. a comparison of the three types of semantics on one sentence (Section 9.2);
3. **model-theoretic semantics**: executable semantic parsing and compositionality (Section 9.3).

9.1 Frame Semantics

[Slides · Other semantics: 2–18; Semantics (long version): 10]

9.1.1 Word meaning revisited

[Slides · Other semantics: 2–6]

What is the meaning of *sold*?

- **Distributional semantics**: all the contexts in which *sold* occurs (*... was sold by...*, *... sold me that piece of...*). It can find similar words and contexts and generalise (e.g. with dimensionality reduction), but the representation is **monolithic**: a word vector has no internal structure, nothing in it says "there is a seller and a buyer".
- **Frame semantics**: the meaning is given by a **frame**, a stereotypical situation with **roles** (slots) to be filled.

Definition — Frame

A **frame** is a structured representation of a stereotypical situation, made of named **roles** (frame elements). A word *evokes* the frame, and the sentence fills its roles.

Worked example — a commercial transaction

Cynthia sold the bike for $200.

Commercial transaction	
SELLER	Cynthia
BUYER	?
GOODS	the bike
PRICE	$200

The BUYER role stays empty: the sentence does not say who bought the bike, but the frame tells us that *someone* did.

9.1.2 Two properties of frames

[Slides · Other semantics: 7–11]

Following Fillmore (1977) and Langacker (1987):

- **Prototypical**: a frame does not need to handle all the cases. The frame of *widow* is "a woman marries one man, and the man dies". What if a woman had three husbands, two of whom died? The frame does not say, and it does not need to: it describes the typical situation.
- **Profiling**: words evoking the same frame can *highlight* different aspects of it.
  - *sell* is seller-centric, *buy* is buyer-centric: *Cynthia sold the bike (to Bob)* vs. *Bob bought the bike (from Cynthia)*;

– *rob* highlights the person, *steal* the goods: *Cynthia robbed Bob (of the bike)* vs. *Cynthia stole the bike (from Bob)*.

The profiled role becomes the subject or object; the others are optional (in brackets).

9.1.3 From syntax to semantics

[Slides · Other semantics: 12–15]

The same situation, with SELLER Cynthia, BUYER Bob, GOODS the bike, PRICE $200, can be expressed by many **syntactic alternations**, with different verbs and different arguments in different positions:

- Cynthia sold the bike to Bob for $200.
- The bike sold for $200.
- Bob bought the bike from Cynthia.
- The bike was bought by Bob.
- The bike was bought for $200.
- The bike was bought for $200 by Bob.

The subject is the seller in the first sentence, the goods in the second, the buyer in the third. **Goal**: map *syntactic positions* to *semantic roles*.

9.1.4 Semantic role labelling

[Slides · Other semantics: 16–18]

Definition — Semantic role labelling (SRL)

Given a sentence, label each relevant phrase with its semantic role in the frame evoked by the predicate:

Input:	Cynthia	sold	the bike	to Bob	for $200
Output:	SELLER	PREDICATE	GOODS	BUYER	PRICE

Subtasks:

1. **frame identification**: find the PREDICATE and the frame it evokes;
2. **argument identification**: find the phrases filling SELLER, GOODS, etc.

Intuition — SRL needs syntax

Bob is the BUYER in "Cynthia sold the bike *to Bob*" and in "*Bob* bought the bike", but in different syntactic positions. SRL systems therefore use the output of POS tagging and parsing (Section 7): syntax is the input of semantics.

Tools: **FrameNet** (`https://framenet.icsi.berkeley.edu/`) is a lexical database of English frames, with their roles and annotated example sentences; see for example the frame *Commerce_buy*.

9.2 Three Types of Semantics

[Slides · Other semantics: 19–22]

One sentence, three views:

***Every** non-blue block is next to **some** blue block.*

- **Distributional semantics**: *block* is like *brick*, *some* is like *every*. True as far as usage goes (they appear in similar contexts), and fatal for meaning: swapping *every* and *some* changes what the sentence says completely.
- **Frame semantics**: *is next to* has two arguments, *block* and *block*. It identifies the structure, but not the truth.
- **Model-theoretic semantics**: tells the difference between these two situations ("models"):



world A: the sentence is **true**



world B: the sentence is **false**

Figure 18: Two worlds; blocks 1–2 and 3–4 are next to each other.

Worked example — evaluating the sentence in each world

In logic the sentence is

$$\forall x \left(\neg blue(x) \rightarrow \exists y \left(blue(y) \wedge nextTo(x, y)\right)\right).$$

The non-blue blocks are 1 and 3 in world A, 1, 2 and 3 in world B.

- World A: block 1 is next to 2 (blue), block 3 is next to 4 (blue): every non-blue block has a blue neighbour. **True**.
- World B: block 1 is next to 2, which is red, and to nothing else. **False** (one counterexample is enough for $\forall$).

Definition — Model-theoretic semantics

The meaning of a sentence is given by its **truth conditions**: the set of situations (*models* of the world) in which it is true. Understanding a sentence means being able to decide, given a model, whether it is true; more generally, to map the sentence to a formal representation (a logical form, a query, a program) that can be *evaluated* against the world.

Type	Represents a meaning as…	Good at / misses
Distributional	a vector of contexts (Section 8)	similarity, learnt automatically / no structure, no truth
Frame	a frame with roles (Section 9.1)	who did what to whom / no logic (*every*, *not*)
Model-theoretic	truth conditions, a logical form or program	precise reasoning, answers / hard to obtain from free text

Table 16: The three types of semantics compared.

9.3 Model-Theoretic Semantics in Practice: Semantic Parsing

[Slides · Other semantics: 23–44; Semantics (long version): 10]

9.3.1 Executable semantic parsing

[Slides · Other semantics: 23–34]

Definition — Executable semantic parsing

Semantic parsing maps a natural-language sentence to a **program** (logical form) in a formal language. The program is then **executed** in a context (a database, a calendar, a robot's world) to produce a behaviour: an answer, an action.

$$[\text{context}] + [\text{sentence}] \xrightarrow{\text{semantic parsing}} [\text{program}] \xrightarrow{\text{execute}} [\text{behaviour}]$$

Worked example — a question to a database

What is the largest city in Europe by population? The program is built piece by piece:

- *Cities*: the set of all cities;
- $\mathit{ContainedBy}(\mathit{Europe})$: the set of things located in Europe;
- $\mathit{Cities} \cap \mathit{ContainedBy}(\mathit{Europe})$: cities in Europe;
- the one with the largest population:
$$\arg\max\big(\mathit{Cities} \cap \mathit{ContainedBy}(\mathit{Europe}),\ \mathit{Population}\big).$$

Executed on a geographical database it returns **Istanbul**.

Worked example — a command to a calendar

Remind me to buy milk after my last meeting on Monday.

$$\mathit{Add}\big(\mathit{Buy}(\mathit{Milk}),\ \arg\max(\mathit{Meetings} \cap \mathit{HasDate}(\text{2016-07-18}),\ \mathit{EndTime})\big)$$

"My last meeting on Monday" becomes the meeting of that date with the largest end time; executing the program adds the reminder.

Note the context: "Monday" became a precise date (2016-07-18) only because the system knows today's date; "my meetings" are the ones in *this* user's calendar.

Why this matters for NLP

This is exactly what ChatBDI does (Section 2.5): it turns a sentence into a term such as `tire_pressure(front, _)` that an agent can execute. It is also how voice assistants and "text-to-SQL" systems work, and how LLMs call tools (they generate a function call, which is then executed).

9.3.2 A brief history of semantic parsing

[Slides · Other semantics: 35]

GeoQuery	Zelle & Mooney 1996
Inductive logic programming	Tang & Mooney 2001
CCG	Zettlemoyer & Collins 2005
String kernels	Kate & Mooney 2006
Synchronous grammars; relaxed CCG	Wong & Mooney 2007; Zettlemoyer & Collins 2007
Learning from the world	Clarke et al. 2010
Higher-order unification; learning from answers	Kwiatkowski et al. 2011; Liang et al. 2011
Language + vision	Matuszek et al. 2012
Large-scale KBs; regular expressions; instruction following	2013 (Berant et al., Kwiatkowski et al.; Kushman et al.; Artzi & Zettlemoyer)
Reduction to paraphrasing	Berant & Liang 2014
Compositionality on tables; dataset from logical forms	Pasupat & Liang 2015; Wang et al. 2015

The trend: from small hand-annotated datasets (GeoQuery: questions about US geography, each paired with its logical form) to learning from weaker supervision (only the answers, or the effect in the world) and to large knowledge bases.

9.3.3 Compositional semantics

[Slides · Other semantics: 36–41]

How do we get from the sentence to the program? The principle of **compositionality**, formalised by **Richard Montague**: the meaning of a phrase is a function of the meanings of its parts and of the way they are combined syntactically.

Worked example — cities in Europe

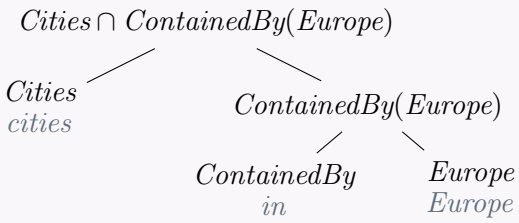


Figure 19: Bottom-up composition: each word gets a meaning from the lexicon (*cities* ↦ *Cities*, *in* ↦ *ContainedBy*, *Europe* ↦ *Europe*), and each node of the tree combines the meanings of its children (function application, then intersection).

The tree is the parse tree of Section 7.3, decorated with meanings: once again, syntax is the scaffolding of semantics.

9.3.4 Language variation

[Slides · Other semantics: 42–44]

The same meaning,

$$Cities \cap ContainedBy(Europe),$$

can be expressed in many ways:

- cities in Europe
- European cities
- cities that are in Europe
- cities located in Europe
- cities on the European continent

Can we write composition rules for all of these by hand? The slides answer: **no!** *European* is a single word that must produce *ContainedBy*(*Europe*); *located in* and *on the . . . continent* are other ways of saying *in*; and natural language has endlessly many such variants. This is why modern semantic parsers *learn* the mapping from data (and today LLMs do it from examples in the prompt), keeping the logical form as the target.

Tools: for model-theoretic semantics the slides report "... not found...": there is no standard off-the-shelf tool comparable to FrameNet.

Cheat sheet — frame and model-theoretic semantics

- Distributional vectors are monolithic. **Frame semantics**: a word evokes a frame (stereotypical situation) with roles (SELLER, BUYER, GOODS, PRICE). Frames are **prototypical** (*widow*) and words **profile** roles (*sell*/*buy*, *rob*/*steal*).
- Many syntactic alternations, one frame: **semantic role labelling** maps syntactic positions to roles (frame identification + argument identification).

Tool: FrameNet.

- "Every non-blue block is next to some blue block": distributional (block ≈ brick), frame (*next to*(block, block)), model-theoretic (true or false in a given world).
- **Model-theoretic semantics**: meaning = truth conditions. Executable semantic parsing: sentence → program → execution (database, calendar).
- **Compositionality** (Montague): meaning of the whole from the meanings of the parts along the parse tree. Language variation makes hand-written rules impossible: the mapping is learnt.

10 Lexical Semantics, WSD and NER

[Slides · Semantics (long version): 11–43]

Watch out — source of this section

This section comes from the *long* version of the semantics slides, which was not shown in class (the lessons used the shortened decks of Sections 8 and 9). It is included because it completes the semantics part of the course map (Section 3.3): lexical semantics, ontologies, word sense disambiguation and named entity recognition. Check with the lecturer how much of it is exam material.

Distributional semantics learns meaning from usage; frame and model-theoretic semantics build structured meanings of sentences. **Lexical semantics** studies the meanings of *words* and the relations among them, and organises them into resources such as WordNet. Two classic tasks use these resources: deciding which meaning of an ambiguous word is meant (WSD), and finding the names of people, places and organisations in a text (NER).

This section covers:

1. the lexicon and the relations among word meanings (Section 10.1);
2. thesauri and WordNet (Section 10.2);
3. word sense disambiguation and the Lesk algorithm (Section 10.3);
4. named entity recognition (Section 10.4).

10.1 The Lexicon and Lexical Relations

[Slides · Semantics (long version): 11–21]

10.1.1 Lexicon and lexical units

[Slides · Semantics (long version): 11–13]

Linguistic theories generally regard a human language as made of two parts:

- a **lexicon**: essentially a catalogue of the language's words (its *wordstock*). It is also thought to include bound morphemes, which cannot stand alone as words (most affixes, Section 4.1), and in some analyses compound words and certain idioms and collocations;
- a **grammar**: a system of rules that combine those words into meaningful sentences (Section 7.3).

Dictionaries try to list the lexicon in alphabetical order, usually without the bound morphemes.

Definition — Lexical semantics

The units of analysis of **lexical semantics** are **lexical units**: not only words but also sub-words (affixes) and larger units (compound words, phrases). Lexical units make up the lexicon. Lexical semantics studies how the meaning of lexical units correlates with the structure of the language (syntax), and how meanings relate to each other through **lexical relations**: hyponymy, hypernymy, synonymy, antonymy, homonymy, ...

10.1.2 Synonymy

[Slides · Semantics (long version): 14]

Definition — Synonymy

Synonymy is similarity of meaning. Leibniz: two expressions are synonymous if substituting one for the other *never* changes the truth value of a sentence in which the substitution is made.

Such *global* synonymy is rare (a language with perfect synonyms would be redundant). A more useful notion is **synonymy relative to a context**: two expressions are synonymous in a linguistic context C if substituting one for the other in C does not alter the truth value. E.g. *board* and *plank* are synonyms in "he nailed a __ to the wall" but not in "the __ of directors". A consequence of defining synonymy by substitution: words of different syntactic categories cannot be synonyms (*quick* and *quickly* cannot replace each other).

10.1.3 Antonymy

[Slides · Semantics (long version): 15]

The antonym of a word x is sometimes *not-x*, but not always: *rich* and *poor* are antonyms, but *not rich* does not imply *poor* (many people consider themselves neither). Antonymy is a lexical relation between *word forms*, not a semantic relation between concepts: *rise*/*fall* and *ascend*/*descend* are antonym pairs, but *rise*/*descend* sounds odd, even though the concepts are the same.

10.1.4 Hyponymy and hypernymy

[Slides · Semantics (long version): 16–17]

Definition — Hyponymy and hypernymy

Hyponymy is a semantic relation between word meanings: {maple} is a **hyponym** of {tree}. The inverse relation is **hypernymy**: {tree} is a **hypernym** of {maple}. Also called subordination/superordination, subset/superset, or the **IS-A** relation. Test: a native speaker must accept sentences of the form "An x is a (kind of) y". For verbs the relation is called **troponymy** (*to lisp* is a manner of *to talk*).

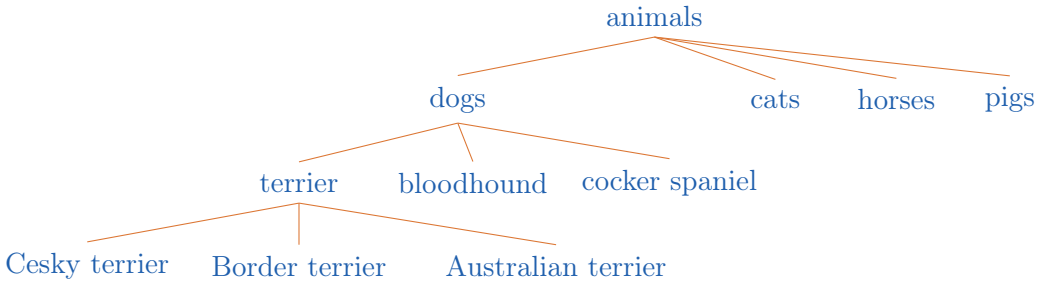


Figure 20: A hyponymy (IS-A) hierarchy: a Border terrier is a terrier, a terrier is a dog, a dog is an animal. Hyponymy is transitive.

10.1.5 Meronymy and holonymy

[Slides · Semantics (long version): 18–20]

Definition — Meronymy and holonymy

A concept C_1 is a **meronym** of a concept C_2 in a language L if native speakers of L accept sentences like "A C_2 has a C_1 (as a part)" or "A C_1 is a part of a C_2". The inverse relation is **holonymy**. It is the **HAS-AS-PART** relation, organised in a part hierarchy (e.g. *car* has *engine*, *door*, *wheel*, *tyres*; *engine* has *cylinder*, *exhaust valve*, ...).

Part-of is asymmetric and, *with caution*, transitive. Transitivity fails when different kinds of part-whole relation are chained:

- A musician has an arm. (component of a body)
- An orchestra has a musician. (member of a group)
- but: ? An orchestra has an arm.

10.1.6 Homonymy

[Slides · Semantics (long version): 21]

Definition — Homonyms, homophones, homographs

- **Homonyms**: the same word form with multiple unrelated meanings.
- **Homophones**: words that *sound* alike but are written differently.
- **Homographs**: words with the same spelling, possibly different pronunciation, and different meanings.

Homonyms	Homophones	Homographs
the *spruce* tree / to *spruce* up	*addition* for maths / *edition* of a book	*desert* = abandon / area of land
suit yourself / wore a *suit*	I want *to* go / I like it *too* / one plus one is *two*	*bass* = fish / instrument
weigh on the *scale* / *scale* the wall	*capitol* building / state *capital*	*close* = nearby / to shut
the price is *fair* / go to the *fair*	pick a *flower* / bake with *flour*	*bow* = to bend down / ribbon

Why this matters for NLP

Homographs and homonyms are a problem for every task that looks at written forms: a search for *bass* returns both fish and guitars (low precision, Section 4.4), and a distributional vector for *bass* mixes the contexts of all its meanings. Choosing the right meaning in context is word sense disambiguation (Section 10.3).

10.2 Thesauri and WordNet

[Slides · Semantics (long version): 22–30]

10.2.1 Structured lexicons and thesauri

[Slides · Semantics (long version): 22–23]

A **thesaurus** (structured lexicon) is an alternative to the alphabetical dictionary: a list of words grouped by *meaning*, with a **hierarchical** organisation, like the taxonomies of the natural sciences (children are "types of" the father and inherit its properties). For ordinary words the hierarchy is given by hyponymy and synonymy (Section 10.1).

- It shows the structure of lexical knowledge, and is much used for technical terminology.
- It can be enriched with other relations: antonyms, part/member relations (*has-as-part*, *has-as-member*), not just *is-a-type-of*.
- It can be explored in any direction (up, down, across), and obvious **distance metrics** (e.g. the number of links between two words) measure their similarity.

10.2.2 WordNet

[Slides · Semantics (long version): 24–30]

WordNet is a large lexical database of English, started in **1985** at Princeton University by a group of psychologists and linguists (led by George Miller, the author of the quotation in Section 8.1). Its theoretical basis is psycholinguistics: what are the properties of the *mental lexicon*? Global organisation: **nouns** are organised as topical hierarchies, **verbs** by entailment relations.

Definition — Synset

In WordNet the basic unit is the **synset** (synonym set). A word meaning is represented simply by listing the word forms that can express it. Synsets are unambiguous designators: the two main senses of *board* are
{board, plank, . . . } (a piece of lumber) vs. {board, committee, . . . } (a group of people assembled for some purpose).

- Members of a synset are rarely true synonyms: WordNet does not try to capture subtle distinctions (connotation, collocation) among them.
- Synsets are often enough to *differentiate* senses. If no suitable synonym exists, a short **gloss** is used, e.g. {board, (a person's meals, provided regularly for money)}. WordNet gives a gloss for every meaning, and often an example sentence.

Worked example — the noun senses of *board* in WordNet (excerpt)

- board (a committee having supervisory powers) "*the board has seven members*";
- board, plank (a stout length of sawn timber; made in a wide variety of sizes and used for many purposes);
- board (a flat piece of material designed for a special purpose) "*he nailed boards across the windows*";
- board, table (food or meals in general) "*she sets a fine table*";
- display panel, display board, board (a vertical surface on which information can be displayed to public view);
- dining table, board (a table at which meals are served);
- control panel, instrument panel, control board, board, panel (electrical device consisting of a flat insulated surface that contains switches and dials).

For the first sense WordNet also lists its relations: its members (*director*, *board member*: meronyms), its hypernym (*committee*), its sister terms.

Size. WordNet 3.0:

	Words	**Synsets**	**Word–sense pairs**	**Monos. words**	**Polys. words**	**Polys. senses**
Noun	117 798	82 115	146 312	101 863	15 935	44 449
Verb	11 529	13 767	25 047	6 277	5 252	18 770
Adjective	21 479	18 156	30 002	16 503	4 976	14 399
Adverb	4 481	3 621	5 580	3 748	733	1 832
Total	155 287	117 659	206 941	128 391	26 896	79 450

Most words have a single sense (128 391 of 155 287), but the polysemous ones are

the frequent ones: verbs in particular have on average $18\,770/5\,252 \approx 3.6$ senses each.

POS	Lexical relations in WordNet
Nouns	synonym ∼ antonym (opposite of); hypernym (is a kind of) ∼ hyponym (for example); coordinate (sister) terms: share the same hypernym; holonym (is part of) ∼ meronym (has as part)
Verbs	synonym ∼ antonym; hypernym ∼ troponym (*lisp* – *talk*); entailment (*snore* – *sleep*: if you snore you sleep); coordinate terms
Adjectives, adverbs	the above, plus related nouns, verb participles, derivational information

WordNet is available in NLTK (Section 6.1) and online ("WordNet Search"); for Italian and many other languages, multilingual resources such as BabelNet extend it.

10.3 Word Sense Disambiguation

[Slides · Semantics (long version): 31–39]

Definition — Word sense disambiguation (WSD)

Word sense disambiguation is the task of identifying which sense (meaning) of a word is used in a sentence, when the word has several meanings. It is an open problem of NLP and ontology engineering.

Consider the senses of *bass* (WordNet lists eight noun senses: the lowest part of the musical range, the lowest part in polyphonic music, an adult male singer with the lowest voice, the lean flesh of a saltwater fish, any of various North American freshwater fish, the member with the lowest range of a family of instruments, ...) and the two sentences

- I went fishing for some sea bass.
- The bass line of the song is too weak.

To a human it is obvious that the first sentence uses *bass* (fish) and the second *bass* (sound/instrument). Replicating this ability with an algorithm is difficult, as shown even by the implicit equivocation between "bass (sound)" and "bass (musical instrument)": the senses themselves are not always clearly separated.

10.3.1 The Lesk algorithm

[Slides · Semantics (long version): 35–37]

The **Lesk algorithm** (Michael Lesk, 1986, "Automatic sense disambiguation using machine readable dictionaries: how to tell a pine cone from an ice cream cone") is the classical WSD algorithm. It assumes that words in a given "neighbourhood" (a section of text) tend to share a common topic.

Definition — Simplified Lesk

For every sense of the word to disambiguate, count the words that occur both in the **neighbourhood** of the word and in the **dictionary definition** (gloss) of that sense. Choose the sense with the largest count.

Worked example — pine cone

In the original version the glosses of *two* ambiguous neighbouring words are compared with each other. Context: "pine cone". Dictionary definitions:

pine 1	kinds of evergreen tree with needle-shaped leaves
pine 2	waste away through sorrow or illness
cone 1	solid body which narrows to a point
cone 2	something of this shape whether solid or hollow
cone 3	fruit of certain evergreen trees

The best intersection is pine 1 ∩ cone 3 = {evergreen, tree(s)}, with 2 words: *pine* is the tree and *cone* is its fruit. All other pairs share no content word.

Watch out — tree vs. trees

The overlap of 2 counts *tree* and *trees* as the same word. A literal string comparison would find only *evergreen*. Lesk needs the normalisation of Section 6.3 (and stop-word removal, Section 6.2, otherwise *of* and *which* would count as overlaps too).

Worked example — simplified Lesk on "I went fishing for some sea bass"

After removing stop words and stemming, the context is {go, fish, sea}. Glosses (with the synset members) of three senses of *bass*:

- *sea bass, bass*: the lean flesh of a saltwater fish of the family Serranidae → overlap {sea, fish} = 2;
- *freshwater bass, bass*: any of various North American freshwater fish with lean flesh → overlap {fish} = 1;
- *bass*: the lowest part of the musical range → overlap 0.

Simplified Lesk chooses the sea bass. Note that it found *sea* only thanks to the synset member *sea bass*: *saltwater* and *sea* do not match as strings.

10.3.2 Extensions and limitations

[Slides · Semantics (long version): 38–39]

- **Adapted Lesk** (Banerjee and Pedersen, 2002): uses WordNet, comparing also the glosses of related synsets (hypernyms, hyponyms, ...), which gives more words to overlap.
- **Limitations**: Lesk is very sensitive to the exact wording of definitions: the absence of a single word can change the result radically. And it only looks at

overlaps among the glosses of the senses considered, which are short and do not provide enough vocabulary for fine-grained sense distinctions.

- Many later variants use other resources (thesauri, synonym dictionaries, morphological and syntactic models), and today distributional/contextual representations (Section 8) are the standard approach.

10.4 Named Entity Recognition

[Slides · Semantics (long version): 40–43]

Definition — Named entity recognition (NER)

Named entity recognition locates and classifies the **named entities** of a text into predefined categories: names of persons, organisations, locations, expressions of times, quantities, monetary values, percentages, etc.

Most NER systems take an unannotated block of text and produce an annotated one:

Jim bought 300 shares of Acme Corp. in 2006.

↓

$[\text{Jim}]_{\text{PERSON}}$ bought 300 shares of $[\text{Acme Corp.}]_{\text{ORGANIZATION}}$ in $[\text{2006}]_{\text{TIME}}$.

Online demos: Stanford NER (`http://nlp.stanford.edu:8080/ner/process`), spaCy NER; for Italian (not only NER): `http://www.italianlp.it/`, `http://parli.di.unito.it/link_it.html`.

Approaches.

- **Grammar-based** (hand-crafted rules by linguists): typically better **precision**, but lower **recall** and months of work by experienced computational linguists.
- **Statistical** (machine learning): need a large amount of manually annotated training data.
- **Semi-supervised** approaches avoid part of the annotation effort.

Intuition — why NER is not a dictionary lookup

A list of names is not enough: *Washington* can be a person, a city, a state or a government ("Washington announced…"); *Apple* can be a fruit or a company; new names appear every day. NER needs context, like POS tagging and WSD: in fact it is often solved with the same sequence models (HMMs, MEMMs and their successors, Section 7.4), with tags such as B-PER, I-PER, O marking the beginning and inside of each entity.

Cheat sheet — lexical semantics, WSD, NER

- Language = lexicon + grammar. Lexical relations: **synonymy** (substitution without changing truth, usually only in context), **antonymy** (between word forms), **hyponymy/hypernymy** (IS-A; troponymy for verbs), **meronymy/holonymy** (HAS-AS-PART; transitive only with caution), homonyms/homophones/homographs.
- Thesauri organise words by meaning in hierarchies. **WordNet**: synsets + glosses + relations; nouns in IS-A hierarchies, verbs by entailment.
- **WSD**: choose the sense in context. **Lesk**: the sense whose gloss overlaps most with the context (or with the glosses of neighbouring words); sensitive to wording, needs normalisation.
- **NER**: find and classify names, times, quantities; rules (precision) vs. statistical (needs annotated data) vs. semi-supervised.

11 The Four Approaches to Semantics Compared

[Slides · Blackboard: semantics B; Blackboard: semantics A]

We have now met four ways of giving meaning to words and sentences: **distributional** semantics (Section 8), **frame** semantics (Section 9.1), **model-theoretic** semantics (Section 9.3) and **lexical** semantics (Section 10). In class the lecturer summarised them in one table on the blackboard, comparing their main features, pros, cons, applications and origins. The table is rewritten below exactly as it was written on the board; the explanations follow.

BLACKBOARD 1 — The four kinds of semantics compared

[Blackboard photo · Blackboard: semantics B (left half); Blackboard: semantics A (right half)]

	Distributional	**Frame**	**Model-theoretic**	**Lexical**
Main features	words represented as vectors; vectors' distance [measures how different the meanings are] • **statistical** •	meaning of words: their role inside frames given by pre-compiled [frame resources] • **human-driven** •	semantics of words & sentences given by their logical meaning, as terms, atoms, formulae, . . . • **translation-based** •	listing relations among words composing the lexicon of a language • **human-driven** •
Pros	learned by software *(a second line was erased)*	explainability; explicit	in very limited application domains it may work; mainly for statement translation	lightweight creation; BabelNet, WordNet exist
Cons	implicit, monolithic learning of the vector space (?); need of large corpus	human annotation; not portable	not "very feasible" in practice	human annotation
Applic-ations	disambiguate meaning (with some intermediate steps); comparison	disambiguate by checking whether a word fits one frame	querying databases; reasoning on the consistency of the sentence	information filtering, retrieval; helping DS
Origins	Wittgenstein	Fillmore	Leibniz, Montague, . . .	*(left empty)*

Words in square brackets are cut off at the edge of the photos and completed from context. "DS" = distributional semantics. The "(?)" is on the board.

Watch out — how the two photos fit together

The board was photographed in two halves. The first photo holds the row headings and the *Distributional* and *Frame* columns; the second holds the *Model-theoretic* and *Lexical* columns. The *Frame* column runs across the gap between two blackboard panels: "human-dri. . . " + ". . . ven", "human annotati. . . " + ". . . on", and "explainability" is followed by "explicit" on the next panel. So there are four columns, not five.

11.1 Reading the table column by column

[Slides · Blackboard: semantics B; Blackboard: semantics A]

Distributional semantics (statistical).

- *Main features*: every word is a vector of numbers learnt from the contexts in which it occurs; the distance (angle) between two vectors measures how different the meanings are (Section 8.3). It is **statistical**: nobody writes the meanings by hand, they come out of counts over a corpus.
- *Pro*: it is **learned by software**, automatically, for any word that appears often enough in the corpus.
- *Cons*: the knowledge is **implicit**: a vector like $(0.031, 0.143, -0.243, \ldots)$ does not say *why* two words are similar. The vector space is learnt **monolithically**, all at once from the whole corpus: one cannot easily correct a single word by hand (the "(?)" on the board signals this is a debatable point). And it needs a **large corpus**: rare words get unreliable vectors.
- *Applications*: **disambiguating meaning**, but only "with some intermediate steps" (context vectors, clustering of senses: Section 8.4); **comparison** of words (similarity, synonyms, nearest neighbours).
- *Origins*: **Wittgenstein**, "the meaning of a word is its use" (Section 8.1).

Frame semantics (human-driven).

- *Main features*: the meaning of a word is the **role** it plays inside a **frame** (a situation with participants, e.g. the commercial-transaction frame with buyer, seller, goods, money; Section 9.1). The frames are **pre-compiled** by people (FrameNet, PropBank): it is **human-driven**.
- *Pros*: **explainability**: a frame states explicitly that *Cynthia* is the seller and *the bike* the goods; the representation is **explicit**, readable by a human.
- *Cons*: frames require **human annotation**, which is slow and costly; and they are **not portable**: frames written for one language or domain do not carry over to another.
- *Applications*: **disambiguation by checking whether a word fits one frame**: *bank* in "I deposited money at the bank" fits the frame of financial transactions, not the frame of rivers.
- *Origins*: **Charles Fillmore** (1977), who introduced frames.

Model-theoretic semantics (translation-based).

- *Main features*: the meaning of words and sentences is their **logical meaning**: words become **terms** and predicates, simple sentences become **atoms**, complex sentences become **formulae**, which can be evaluated as true or false in a model of the world (Section 9.3). It is **translation-based**: the sentence is *translated* into logic.
- *Pros*: in **very limited application domains** it may work (a database of US geography, a calendar, the blocks world), mainly for the translation of **statements** (declarative sentences); questions, commands, irony, metaphors are much harder.

- *Cons*: **not "very feasible" in practice** for open, real language: translating arbitrary text into correct logic is extremely hard.
- *Applications*: **querying databases** (natural-language interfaces: "which states border Texas?" becomes a query); **reasoning on the consistency** of a sentence (is it contradictory? does it follow from what we know?).
- *Origins*: **Leibniz** (the dream of a universal logical language in which disputes are settled by calculation), **Montague** (natural language can be treated like a formal language, compositionally), ...

Lexical semantics (human-driven).

- *Main features*: it **lists the relations** among the words that make up the lexicon of a language: synonymy, antonymy, hyponymy, meronymy, ... (Section 10.1). It is **human-driven**: lexicographers write the relations.
- *Pros*: **lightweight creation** (much simpler than frames or logic: a relation is just a pair of words with a label), and large resources already **exist**: **WordNet** (Section 10.2) and **BabelNet**, its multilingual extension.
- *Cons*: **human annotation**, like frames.
- *Applications*: **information filtering** and **retrieval** (e.g. expanding a query with synonyms and hyponyms); **helping distributional semantics** ("helping DS"): e.g. WordNet senses to label the clusters of a DSM, or lexical relations to evaluate and correct vectors.
- *Origins*: left empty on the board.

Intuition — the big picture

Read the table along the "•" row. One approach is **statistical** (meaning is *learnt* from data), two are **human-driven** (meaning is *written* by people), one is **translation-based** (meaning is a *logical formula*). The pros and cons follow from this one choice:

- statistical ⇒ cheap, automatic, covers everything, but implicit and hungry for data;
- human-driven ⇒ explicit and explainable, but costly to build and hard to port;
- translation-based ⇒ precise, supports reasoning, but works only in small, controlled domains.

This is the same trade-off as between the eras of NLP (Section 2.3): rule-based (human-driven) vs. statistical and neural (learned) methods.

Cheat sheet — the four semantics

- **Distributional**: vectors, distance; statistical; learned automatically; implicit, needs a large corpus; disambiguation (indirectly), comparison; Wittgenstein.
- **Frame**: role of words inside pre-compiled frames; human-driven; explainable, explicit; human annotation, not portable; disambiguation by frame

fitting; Fillmore.
- **Model-theoretic**: logical meaning (terms, atoms, formulae); translation-based; works in limited domains, for statements; not feasible in general; database querying, consistency reasoning; Leibniz, Montague.
- **Lexical**: relations among the words of the lexicon; human-driven; lightweight, WordNet and BabelNet exist; human annotation; information filtering/retrieval, helping DS.

12 Ontologies and the Semantic Web

[Slides · Ontologies: 1–48]

Lexical semantics (Section 10) organises the meanings of *words*; WordNet already looks like a hierarchy of concepts. **Ontologies** go one step further: they describe the *concepts of a domain* (wines, courses, companies, ...), their properties and relations, in a formal language that a machine can reason with. The **Semantic Web** is the project of using ontologies to give the content of the Web a well-defined, machine-readable meaning.

This section is a soft introduction to both. It covers:

1. what an ontology is, why semantic interoperability matters, how ontologies differ from databases, and knowledge graphs (Section 12.1);
2. a methodology for building an ontology, and the Protégé editor (Section 12.2);
3. the ontology languages RDF, RDF Schema and OWL (Section 12.3);
4. the Semantic Web, its layered "stack" and what came after it (Section 12.4).

12.1 What Is an Ontology

[Slides · Ontologies: 3–7, 10–21]

12.1.1 Two definitions

[Slides · Ontologies: 3–7]

The word comes from philosophy, where *ontology* is the study of being. Computer science borrowed it in the early 1990s with a much more practical meaning:

- "The study of that which is common to all things which exist" (Aristotle, *Metaphysics*);
- "**Explicit specification of a conceptualization**" (Tom Gruber, *A Translation Approach to Portable Ontology Specifications*, Knowledge Acquisition 5(2):199–220, 1993).

Intuition — reading Gruber's definition

A *conceptualization* is the way we carve up a domain: which kinds of things exist (wines, producers, foods), how they are related (a wine *is produced by* a producer), which constraints hold. It normally lives only in people's heads. *Explicit specification* means writing it down, in a formal language, so that it can be shared between people and programs and checked by a machine.

12.1.2 Motivation: semantic interoperability

[Slides · Ontologies: 10–12]

Definition — Semantic interoperability

Semantic interoperability is "the ability of computer systems to exchange data with unambiguous, shared meaning".

Two systems can exchange bytes perfectly and still disagree on what they mean: is `price` with or without VAT, is `date` the order date or the shipping date? Semantic interoperability has been one of the most active research fields of the last twenty-five years: the advent of ontologies in the early nineties, Web Services at the beginning of the new millennium, and their combination into Semantic Web Services all aim at making the "semantic interoperability dream" come true.

Why this matters for NLP

The interest is economic as much as scientific. Walker et al. (*The Value of Healthcare Information Exchange and Interoperability*, Health Affairs, 2005) estimated that a fully standardised system for information exchange and interoperability in US health care could yield a net value of **77.8 billion dollars per year** once implemented. More recent work (e.g. on semantic interoperability in health care, 2018, and the European Data Conference on Reference Data and Semantics, 2023) confirms the expected advantages.

12.1.3 Gruber's definition, 2008

[Slides · Ontologies: 13–15]

Fifteen years after his first definition, Gruber gave a more detailed one (*Ontology*, Encyclopedia of Database Systems, Springer, 2008).

Definition — Ontology (Gruber, 2008)

In computer and information science, an **ontology** defines a set of **representational primitives** with which to model a domain of knowledge or discourse. The primitives are typically

- **classes** (or sets),
- **attributes** (or properties),

- **relationships** (relations among class members).

Their definitions include information about their *meaning* and *constraints* on their logically consistent application.

From the point of view of databases:

- an ontology is a level of abstraction of data models, analogous to hierarchical and relational models, but intended for modelling *knowledge* about individuals, their attributes and their relationships to other individuals;
- ontology languages abstract away from data structures and implementation strategies, and are closer in expressive power to **first-order logic** than database languages. Ontologies are therefore said to be at the **"semantic" level**, database schemas at the "logical" or "physical" level;
- being independent of the lower-level data models, ontologies are used to integrate heterogeneous databases, to make disparate systems interoperate and to specify interfaces to knowledge-based services. In the Semantic Web standards they are an explicit layer (Section 12.4), with standard languages and many commercial and open source tools.

12.1.4 An example: a wine ontology

[Slides · Ontologies: 16]

The slides show a small ontology of wines and foods (in Italian). Each node is a class; each child is a subclass ("is a kind of") of its parent, exactly like the hypernym hierarchy of WordNet (Section 10.2).

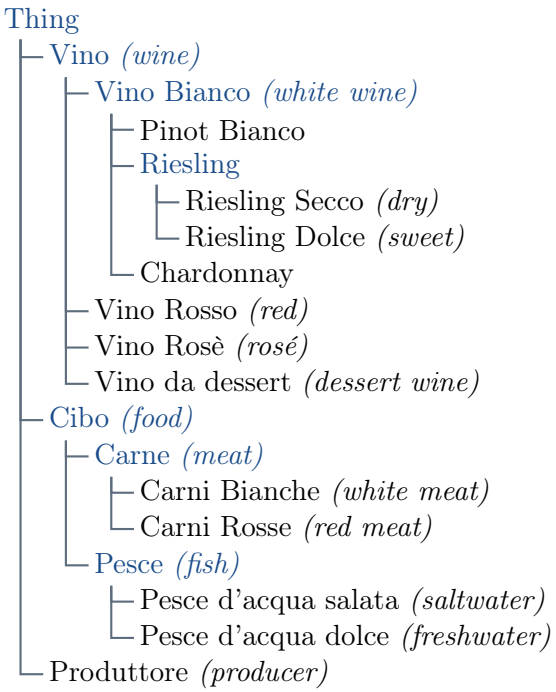


Figure 21: The wine ontology of the slides as a class hierarchy (translations in italics).

The slide also labels the **levels** of the hierarchy: *top level* (the most general classes, such as *Vino*), *middle level* (*Vino Bianco*, *Riesling*, *Chardonnay*, ...) and *bottom level* (the most specific classes, *Riesling Secco* and *Riesling Dolce*). A full ontology would also have *properties* linking the hierarchies, e.g. a wine *is made by* a *Produttore*, a wine *goes well with* a *Cibo*.

12.1.5 Ontologies and databases

[Slides · Ontologies: 17–18]

	Ontologies	**Databases**
Purpose	sharing, reuse, defining semantics	storing large amounts of structured data
Expressive power	close to first-order logic; description logics	not as powerful as first-order logic
Query standards	no consolidated standard, few proposals (e.g. SPARQL)	consolidated standards (e.g. SQL)
Management systems	Protégé, Jena (not commercial)	many solid commercial tools and environments (Oracle, ...)

Table 17: Ontologies and databases compared.

Watch out — SPARQL today

SPARQL has since become a W3C Recommendation (2008; SPARQL 1.1 in 2013), so it is now the standard query language for RDF data, even if it is far less widespread than SQL. The comparison in the slides reflects an earlier state of the field.

12.1.6 Knowledge graphs

[Slides · Ontologies: 19–21]

Definition — Knowledge graph

"The heart of the **knowledge graph** is a knowledge model: a collection of interlinked descriptions of concepts, entities, relationships and events. Knowledge graphs put data in context via linking and semantic metadata and this way provide a framework for data integration, unification, analytics and sharing" (Ontotext).

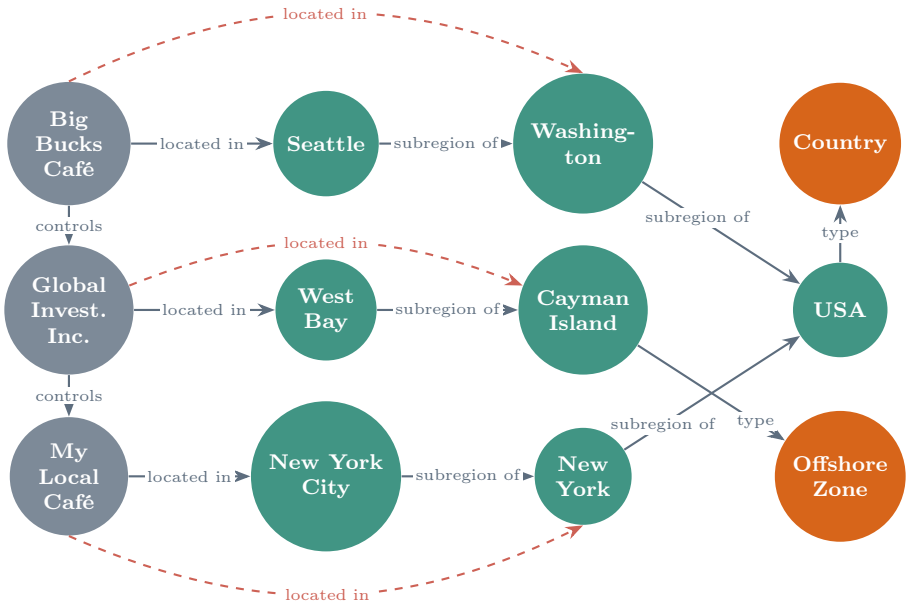


Figure 22: A knowledge graph of companies and places, after the slides. Solid arrows are stated facts, dashed arrows are *inferred* facts.

Worked example — what the graph lets us infer

The stated facts say that *My Local Café* is located in *New York City*, which is a subregion of *New York*: since *located in* propagates along *subregion of*, the café is also located in New York, and in the USA. Following the *controls* links upwards, *My Local Café* is controlled by *Global Investment Inc.*, located in West Bay, a subregion of the Cayman Islands, which are an *Offshore Zone*. The slide's conclusion, "discover suspicious relationships", is a query over the graph: *a company controlled by another company from the same country through an offshore zone*. A news item saying that "My Local Café is a network of independent coffee shops" can be checked against it.

Intuition — ontology vs. knowledge graph

The two terms overlap heavily. A useful way to keep them apart: the *ontology* is the schema (classes such as *Company*, *Region*; properties such as *controls*, *located in*; rules such as "*located in* is transitive through *subregion of*"), while the *knowledge graph* is the large network of concrete entities and facts described with that schema.

12.2 Building Ontologies: Methodology and Tools

[Slides · Ontologies: 22–23, 38]

12.2.1 Ontology Development 101

[Slides · Ontologies: 22–23]

A widely used methodology is the one of Natalya F. Noy and Deborah L. McGuinness, *Ontology Development 101: A Guide to Creating Your First Ontology* (Stanford Knowledge Systems Laboratory, 2001). It has seven steps:

1. determine the **domain and scope** of the ontology;
2. consider **reusing** existing ontologies;
3. enumerate the important **terms** in the ontology;
4. define the **classes** and the **class hierarchy**;
5. define the **properties** of classes (*slots*);
6. define the **facets** of the slots;
7. create **instances**.

Worked example — the seven steps on the wine ontology

The guide itself uses wines and foods as its running example, the same domain as Figure 21.

1. *Domain and scope*: wines and foods, to recommend a wine for a meal. Useful *competency questions*: "which wine goes well with fish?", "is Chardonnay a white wine?".
2. *Reuse*: is there already a food or geography ontology we can import?
3. *Terms*: wine, white wine, Riesling, producer, colour, sugar, region, fish, red meat, ...
4. *Classes and hierarchy*: *Riesling* is a subclass of *White wine*, which is a subclass of *Wine*.
5. *Slots*: a wine has a *colour*, a *sugar* level, a *maker* (a *Producer*), and *goes well with* some *Food*.
6. *Facets* (constraints on slots): *colour* takes one value among {red, white, rosé}; *maker* has exactly one value, of class *Producer*.
7. *Instances*: a specific bottle, e.g. a given producer's Riesling of a given year, as an individual of class *Riesling*.

Intuition — terms first, classes later

Steps 3–4 mirror what we did with words in lexical semantics: first collect the vocabulary of the domain, then decide which terms denote classes, which ones denote properties, and arrange the classes in an IS-A hierarchy (the hyponymy relation of Section 10.1). The process is iterative: the first version of an ontology is almost never the final one.

12.2.2 Tools: Protégé

[Slides · Ontologies: 38]

Protégé (`https://protege.stanford.edu/`) is a free, open source ontology editor and knowledge-base framework developed at Stanford University. It is written in Java, is extensible, and provides a plug-and-play environment that makes it a flexible base for rapid prototyping and application development. It supports two main ways of modelling ontologies:

- the **Protégé-Frames** editor, to build and populate *frame-based* ontologies (classes with slots and facets, as in the methodology above);
- the **Protégé-OWL** editor, to build ontologies for the Semantic Web, in the W3C's OWL language (Section 12.3).

12.3 Ontology Languages: RDF, RDF Schema, OWL

[Slides · Ontologies: 24–37]

12.3.1 Evolution of the languages

[Slides · Ontologies: 25]

Ontology languages for the Web were built on top of each other, starting from the markup languages of the Web itself.

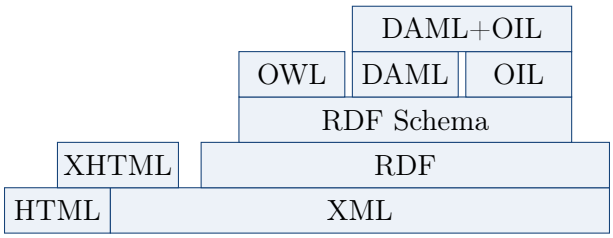


Figure 23: Evolution of the Web and ontology languages: each layer builds on the ones below it.

- **XML** gives a common *syntax* for structured documents, but says nothing about what the tags mean.
- **RDF** adds a simple data model (triples, below) on top of XML.
- **RDF Schema** adds a vocabulary for classes, subclasses and properties.
- **DAML** (US) and **OIL** (Europe) were two richer ontology languages built on RDF Schema, merged into **DAML+OIL**, which in turn was the basis of the W3C standard **OWL**.

12.3.2 RDF: resources, properties, statements

[Slides · Ontologies: 26–29]

RDF (Resource Description Framework) is a W3C Recommendation. It provides a general, flexible method to decompose any knowledge into small pieces, called **triples**, with some rules about the meaning of those pieces.

Definition — RDF data model

The RDF data model has three object types:

- a **resource**: anything that can have a URI (Web identifier). It can be a whole Web page, a part of it, a collection of pages, or an object that is not directly accessible via the Web (a person, a book);

- a **property**: a specific aspect, characteristic, attribute or relation used to describe a resource (*author*, *homepage*). Properties are resources too;
- a **statement**: a triple made of a resource, a property and a property value (which can be a literal or another resource). The three parts are called the **subject**, **predicate** and **object** of the statement.

A triple (O, A, V) is more often written as $A(O, V)$, and means: *object O has attribute A with value V*. A set of triples is a directed, labelled **graph**: subjects and objects are nodes, each predicate is an edge.

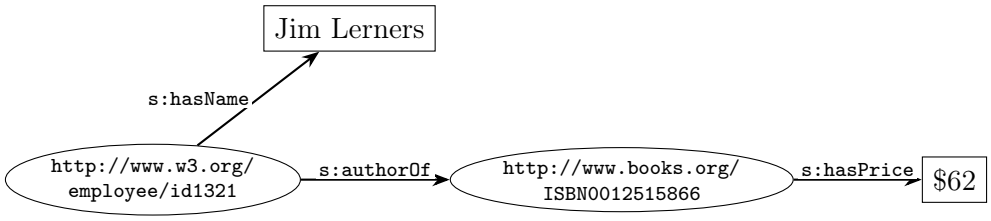


Figure 24: An RDF graph with three statements. Ellipses are resources (identified by URIs), boxes are literal values.

The graph contains three triples:

$$\begin{aligned} &\mathit{hasName}(\mathit{id1321}, \text{“Jim Lerners”}),\\ &\mathit{authorOf}(\mathit{id1321}, \mathit{ISBN0012515866}),\\ &\mathit{hasPrice}(\mathit{ISBN0012515866}, \$62). \end{aligned}$$

Because the book is a resource, it can be the object of one statement and the subject of another: this is what makes triples link into a graph.

Worked example — RDF in XML syntax

```
1 <?xml version="1.0"?>
2 <RDF>
3   <Description about="http://www.w3schools.com/rdf">
4     <author>Jan Egil Refsnes</author>
5     <homepage>http://www.w3schools.com</homepage>
6   </Description>
7 </RDF>
```

The `Description` element describes one resource, and each child element is a property with its value. The document contains two statements:

Subject	Predicate	Object
http://www.w3schools.com/rdf	author	Jan Egil Refsnes
http://www.w3schools.com/rdf	homepage	http://www.w3schools.com

that is, “the author of http://www.w3schools.com/rdf is Jan Egil Refsnes” and “the homepage of http://www.w3schools.com/rdf is http://www.w3schools.com”. In the second statement the value is itself a resource.

Watch out — the example is simplified

In real RDF/XML the RDF vocabulary lives in its own namespace and the properties in another one: the root is `rdf:RDF` with an `xmlns:rdf` declaration, the resource is `rdf:Description rdf:about="..."`, and `author`, `homepage` carry a prefix of a vocabulary of our choice. The unprefixed version above is easier to read but is not valid RDF.

12.3.3 RDF Schema

[Slides · Ontologies: 30–31]

RDF alone lets us state facts but not define a vocabulary. **RDF Schema** (RDFS) is used to define the structure of the metadata that describe Web resources:

- it consists of some basic classes and properties, and can be extended to fit possibly any domain;
- **classes** are arranged hierarchically (`rdfs:subClassOf`). The root of the hierarchy is `rdfs:Resource`; `rdfs:Class` is a subclass of `rdfs:Resource`;
- **properties** are instances of the class `rdf:Property`; they describe resources by assigning values to them, and are resources themselves. Their use can be constrained to members of certain classes: `rdfs:domain` gives the class of the subjects, `rdfs:range` the class of the values.

Worked example — teachers and courses in RDFS

```
<rdfs:Class rdf:ID="Teacher">
  <rdfs:comment>Teacher Class</rdfs:comment>
  <rdfs:subClassOf rdf:resource="#Person"/>
</rdfs:Class>
<rdfs:Class rdf:ID="Course">
  <rdfs:comment>Course Class</rdfs:comment>
  <rdfs:subClassOf
  rdf:resource="http://www.w3.org/1999/02/22-rdf-syntax-ns#Resource"/>
</rdfs:Class>
<rdf:Property rdf:ID="teacher">
  <rdfs:comment>Teacher of a course</rdfs:comment>
  <rdfs:domain rdf:resource="#Course"/>
  <rdfs:range rdf:resource="#Teacher"/>
</rdf:Property>
<rdf:Property rdf:ID="name">
  <rdfs:comment>Name of a Person or Course</rdfs:comment>
  <rdfs:domain rdf:resource="#Person"/>
  <rdfs:domain rdf:resource="#Course"/>
  <rdfs:range
  rdf:resource="http://www.w3.org/1999/02/22-rdf-syntax-ns#Literal"/>
</rdf:Property>
```

Teacher is a subclass of *Person*; *teacher* is a property linking a *Course* (domain) to a *Teacher* (range): "the teacher of this course is …". *name* gives a literal (a string) to persons and courses.

Watch out — two domains mean "and", not "or"

In RDFS semantics, several `rdfs:domain` statements on the same property are *all* applied: whatever has a *name* is inferred to be *both* a *Person* and a *Course*. To express "a Person *or* a Course", the domain must be a single class that is the union of the two (OWL can define it with `owl:unionOf`), or a common superclass.

12.3.4 OWL: the Web Ontology Language

[Slides · Ontologies: 32–37]

OWL extends RDF Schema: it uses the RDF meaning of classes and properties (`rdfs:Class`, `rdfs:subClassOf`, …) and adds language primitives for a richer expressiveness. It comes in three species, which trade expressive power for ease of reasoning:

Species	What it is	Advantage	Disadvantage
OWL Full	all OWL primitives, combinable in arbitrary ways with RDF and RDF Schema	fully upward compatible with RDF	so powerful that it is **undecidable**
OWL DL	a sublanguage of OWL Full that restricts how OWL and RDF constructors can be used (DL = description logic)	permits **efficient reasoning** support	loses full compatibility with RDF
OWL Lite	a further restriction of OWL DL to a subset of the constructors	easier to grasp (users) and to implement (tool builders)	restricted expressivity

Table 18: The three species of OWL, from the most to the least expressive.

Intuition — why give up expressive power?

A reasoner answers questions such as "is class A a subclass of B?" or "is this ontology consistent?". In OWL Full no algorithm can always answer them (undecidability, as for first-order logic); in OWL DL an algorithm always terminates with the right answer. Every legal OWL Lite ontology is a legal OWL DL ontology, and every OWL DL ontology is a legal OWL Full ontology.

The ontology header. An OWL document starts by declaring the namespaces it uses and then describes the ontology itself:

```
<rdf:RDF
   xmlns:owl ="http://www.w3.org/2002/07/owl#"
   xmlns:rdf ="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
   xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
   xmlns:xsd ="http://www.w3.org/2001/XMLSchema#">
<owl:Ontology rdf:about="">
  <rdfs:comment>An example OWL ontology</rdfs:comment>
  <owl:priorVersion rdf:resource="http://www.mydomain.org/uni-ns-old"/>
  <owl:imports rdf:resource="http://www.mydomain.org/persons"/>
  <rdfs:label>University Ontology</rdfs:label>
</owl:Ontology>
```

`owl:imports` includes other ontologies, whose content is assumed to be part of the current document. It is **transitive**: if ontology A imports B and B imports C, then A also imports C.

Classes. Classes are defined with `owl:Class`; further information can be added to a class later, e.g. that it is **disjoint** from another one (no individual belongs to both):

```
<owl:Class rdf:ID="Mammalia">
  <rdfs:subClassOf rdf:resource="#Animal"/>
</owl:Class>
<owl:Class rdf:about="#Mammalia">
  <owl:disjointWith rdf:resource="#Insecta"/>
</owl:Class>
```

There are two predefined classes: `owl:Thing` (the class of everything, the top of every hierarchy) and `owl:Nothing` (the empty class).

Properties. OWL distinguishes two types of property:

- **object properties** link individuals to individuals (*married* links a person to a person);
- **datatype properties** link individuals to data values (*age* links a person to a non-negative integer).

```
<owl:DatatypeProperty rdf:ID="age">
  <rdfs:domain rdf:resource="#Person"/>
  <rdfs:range
  rdf:resource="http://www.w3.org/2001/XMLSchema#nonNegativeInteger"/>
</owl:DatatypeProperty>
<owl:ObjectProperty rdf:ID="married">
  <rdfs:domain rdf:resource="#Person"/>
  <rdfs:range rdf:resource="#Person"/>
```

```
9 </owl:ObjectProperty>
```

Watch out — corrections to the slides

The listings above fix three small slips in the slides: the `xsd` namespace is `XMLSchema` (not `XLMSchema`); domain and range of OWL properties are still given with `rdfs:domain` and `rdfs:range` (OWL has no `owl:domain`/`owl:range`); and `rdf:about` on a class defined elsewhere refers to it with a `#`.

References. The specifications are on the W3C site: RDF (http://www.w3.org/RDF/), RDF Schema (http://www.w3.org/TR/rdf-schema/) and OWL (http://www.w3.org/2004/OWL/).

12.4 The Semantic Web

[Slides · Ontologies: 8, 39–48]

12.4.1 The vision

[Slides · Ontologies: 8, 40–41]

Definition — Semantic Web

"The **Semantic Web** is not a separate Web but an extension of the current one, in which information is given a well-defined meaning, better enabling computers and people to work in cooperation." (Tim Berners-Lee, James Hendler and Ora Lassila, *Scientific American*, May 2001.)

The same article continues: the Semantic Web will bring structure to the meaningful content of Web pages, creating an environment where **software agents** roaming from page to page can readily carry out sophisticated tasks for users. The goals are

- automatic **discovery** of Web resources and Web services;
- automatic **composition** of Web resources and Web services;
- automatic **execution and monitoring** of Web services.

Intuition — why the Web needs ontologies

A page saying "Dr. Rossi, Tuesday 10–12, room 506" is perfectly clear to a human and opaque to a program: is it an office hour, a lecture, an appointment? If the page also states, in RDF, that this is an instance of *OfficeHour* with properties *teacher*, *day* and *room*, defined in a shared ontology, an agent can find all office hours of a department, compare them with your calendar and book one. Ontologies give the shared vocabulary; RDF gives the format of the facts.

12.4.2 The Semantic Web stack

[Slides · Ontologies: 42–45]

The Semantic Web technologies are usually drawn as a **stack of layers**, each built on the ones below, like the language evolution of Figure 23. The stack itself evolved over the years.

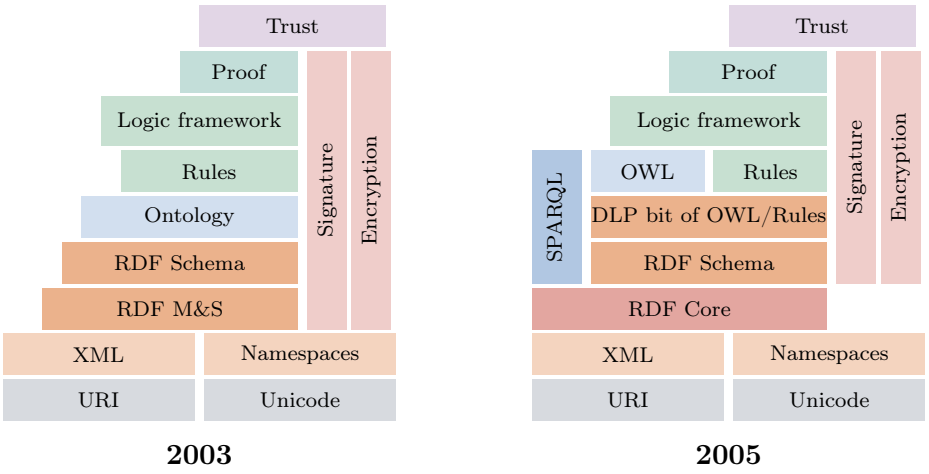


Figure 25: The Semantic Web stack according to Tim Berners-Lee, in 2003 (*WWW past & future*) and in 2005 (*Web for real people*).

Reading the 2003 stack from the bottom:

- **URI** and **Unicode**: how to name any resource, how to write any character;
- **XML** and **namespaces**: a common syntax for structured documents, and a way to mix vocabularies without name clashes;
- **RDF** (model & syntax) and **RDF Schema**: facts as triples, and a basic vocabulary of classes and properties (Section 12.3);
- **Ontology** (OWL): rich descriptions of the concepts of a domain;
- **Rules** and **logic framework**: inference beyond what the ontology language can express;
- **Proof** and **Trust**: explaining why a conclusion holds, and deciding whether to believe it (and its sources);
- **Signature** and **encryption** run alongside all the upper layers: who said what, and who can read it.

In the 2005 version the ontology layer is split into **OWL** and **Rules** side by side, sharing a common fragment (*DLP*, the part of OWL that can also be written as rules), and the query language **SPARQL** appears next to them.

Key result — Stack or two towers?

Horrocks, Parsia, Patel-Schneider and Hendler (*Semantic Web Architecture: Stack or Two Towers?*, 2005) argued that a single stack does not really work: rules with *negation as failure* (NAF, as in Prolog and Datalog) and the open-world logic of OWL do not combine smoothly. They proposed **two towers** sharing the bottom layers (URI, Unicode, XML, namespaces): one with *DLP/Datalog*, rules and NAF, the other with RDF, RDF Schema, OWL and first-order logic

(FOL/SCL), possibly joined at the top by a common logic framework. As an alternative they also drew a single stack in which the rule layer (SWRL) sits *above* OWL and below full FOL. The slides illustrate the debate with Genoa landmarks: the two towers of Porta Soprana for the two-towers view, the Lanterna (the lighthouse) for the single stack.

12.4.3 What is the Semantic Web really about?

[Slides · Ontologies: 46]

James Hendler (2009) distinguishes three meanings in which the term is used:

- **Linked Data** (or Web of Data): datasets rendered into RDF, using very simple ontologies (really more like thesauri or vocabularies than AI languages) to provide terms for linking and describing the data;
- **Web 3.0**: sometimes used as a synonym for Semantic Web; it grew up as a way of explaining how semantics (especially the simple ontologies above) can be integrated with Web 2.0 technologies;
- **Semantic Web**: some people use it to mean all of the above (Hendler does).

Why this matters for NLP

Linked Data is where the Semantic Web had its largest practical impact: resources such as DBpedia and Wikidata are huge RDF knowledge graphs (Section 12.1), and they are used in NLP for entity linking (connecting a name found by NER, Section 10.4, to the entity it denotes), question answering and as background knowledge for language models.

12.4.4 Web 4.0 and Web 5.0

[Slides · Ontologies: 47–48]

- **Web 4.0**: its services will be autonomous, proactive, self-learning, content-exploring, collaborative and content-generating **agents**, based on fully matured semantic and reasoning technologies as well as AI. They will support adaptive content presentation through an intelligent agent; examples are services interacting with sensors and implants, natural-language services and virtual reality services.
- **Web 5.0**: centred on users' **control over identity** (credentials for all platforms stored on a decentralised platform controlled only by the user, with no identity information collected by the Internet) and **data ownership** (complete decentralisation: users choose what to share with companies, and store their data with their own blockchain participation).

As the slides conclude, people no longer seem very interested in numbering the Web versions.

Cheat sheet — ontologies and the Semantic Web

- **Ontology**: an explicit specification of a conceptualization (Gruber); classes, properties and relations with their meaning and constraints, at the "semantic" level, closer to first-order logic than database schemas. Main motivation: **semantic interoperability**.
- **Knowledge graph**: interlinked entities and facts described with an ontology; new facts can be inferred.
- **Methodology** (Noy & McGuinness): scope, reuse, terms, classes and hierarchy, slots, facets, instances. Tool: **Protégé**.
- **RDF**: triples subject–predicate–object, forming a graph. **RDFS**: classes, `subClassOf`, properties with domain and range. **OWL**: richer primitives (disjointness, object vs. datatype properties, imports); Full (undecidable), DL (efficient reasoning), Lite (simplest).
- **Semantic Web**: the Web with machine-understandable meaning, organised as a layered stack (URI/XML → RDF → ontologies/rules → logic, proof, trust); in practice, mostly Linked Data.

13 Ontology Engineering, Learning and Matching

[Slides · Ontology development: 1–104; Ontology learning: 1–25; Ontology matching: 1–14; Ontology matching (long version): 1–112]

Section 12 explained what an ontology is and in which languages it is written. This section is about the *life* of an ontology: how people design it, how part of it can be learnt automatically from text, how two ontologies that describe the same domain differently can be put in correspondence, and where ontologies are used. The deck *Ontology Development and Evolution* gives the outline: ontology engineering, ontology learning and population, ontology matching, applications of ontologies.

This section covers:

1. **ontology engineering**: methodologies, and the design process step by step on an animals-and-plants example (Section 13.1);
2. how to represent relations among more than two things, the **n-ary relations** patterns (Section 13.2);
3. **ontology learning**: extracting terms, synonyms, concepts, taxonomies, relations and axioms from text (Section 13.3);
4. **ontology matching**: finding correspondences between two ontologies (Section 13.4);
5. **applications** of ontologies and some research from the course's group (Section 13.5).

13.1 Ontology Engineering: the Design Process

[Slides · Ontology development: 2–60]

Ontology engineering is the discipline of building ontologies in a systematic way, as software engineering is for programs. The slides include material by Valentina Tamma, A. Gómez-Pérez, N. Noy, D. McGuinness, E. Kendal, A. Rector and O. Corcho.

13.1.1 Many methodologies, one winner

[Slides · Ontology development: 4–15]

Many methodologies have been proposed. The slides show the first page of several of them, just to give an idea of how many there are:

- *Ontology Engineering Methodologies* (Sure, Tempich, Vrandečić) and the *Ontology Engineering Methodology* of Sure, Staab and Studer (On-To-Knowledge), focused on knowledge management in companies;
- *HCOME*, Human-Centered Ontology Engineering (Kotis, Vouros, 2006): the ontology is developed collaboratively by communities of knowledge workers;
- *DILIGENT* (Pinto, Staab, Tempich): for distributed, loosely controlled and evolving ontologies;
- the *NeOn* methodology (Suárez-Figueroa, Gómez-Pérez, Fernández-López): no rigid workflow, nine scenarios (pathways) for building networks of ontologies;
- *DOGMA* (Spyns, Tang, Meersman, STAR Lab, 2008);
- *RapidOWL* (Auer): an *agile* methodology, inspired by eXtreme Programming and wikis;
- the *SWO project* (Copeland et al.): agile ontology engineering in community-driven ontologies;
- a literature review comparing them (Iqbal et al., 2013), which concludes that no methodology is completely mature.

"... but for me the winner is" (the lecturer's choice): Natalya F. Noy and Deborah L. McGuinness, **Ontology Development 101: A Guide to Creating Your First Ontology** (Stanford, 2001), already introduced in Section 12.2. It is one of the oldest and simplest methodologies, but still a good, flexible one (cited by more than 9 500 papers).

13.1.2 The ontology design process

[Slides · Ontology development: 16–18]

The design process has eight steps. They can be drawn as a straight pipeline, but in reality the process is a **cycle**: after adding instances we discover problems and go back to the requirements.

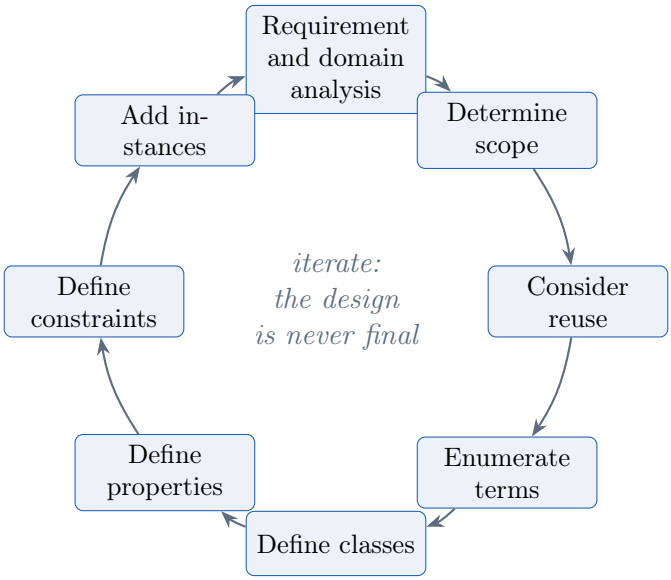


Figure 26: The ontology design process: eight steps, repeated in a cycle.

Compared with the seven steps of Section 12.2, there is an extra first step (requirement analysis) and the "facets" step is called *define constraints*. We now go through the steps on a running example.

13.1.3 Steps 1–2: requirements and scope

[Slides · Ontology development: 19–24]

Requirement analysis. As in any software engineering design, analysing the requirements, the domain and the use cases is a critical stage: it grounds the work and lets the engineers set priorities. It must make explicit:

- the nature of the knowledge and the **questions** (*competency questions*) that the ontology, through a reasoner, must answer. This drives the scope, the design and the architecture;
- architectural issues.

Application requirements are found by:

- identifying any **controlled vocabulary** already used by the application;
- identifying **hierarchical (taxonomic) structures** intrinsic to the domain, which can be used for *query expansion*: a query for *vegetarian pizza* should also return margherita, funghi, grilled vegetables pizza;
- analysing **structured queries** and the knowledge they require;
- deciding the **expressive power** needed: efficient inference requires limited expressivity, more expressivity requires expensive (or resource-bounded) computation;
- ad-hoc reasoning for particular needs: temporal relations, geospatial reasoning, processes, conditional operations;

- computational tractability;
- need for explanations, traces, provenance (where a piece of knowledge comes from).

Domain requirements to take into account:

- heterogeneity, distribution and autonomy (e.g. applications made of software agents);
- **open vs. closed world**: does lack of information imply negative information? (see below);
- static vs. dynamic ontology processes (evolution, alignment);
- limited or incomplete knowledge; knowledge evolving over time;
- analysis and consistency checking of the instance data.

Use case analysis should clarify which **information** is likely to be available, which **questions** are likely to be asked, and the **types and roles of users**.

Determine the scope. Straightforward questions: what is the ontology going to be used for? How will the software ultimately use it? What should the ontology be aware of, i.e. what is the scope of the knowledge it must contain?

Definition — Competency questions

Competency questions are questions, in natural language, that the ontology must be able to answer. They define the scope (anything not needed to answer them can be left out) and later serve as a test: if the ontology cannot answer them, it is not finished.

Worked example — competency questions from the slides

- Which investigations were done with a high-fat-diet study?
- Which study employs microarray in combination with metabolomics technologies?
- List those studies in which the fasting phase had as duration one day.
- What is a vegetarian pizza?
- What type of wine can accompany seafood?

The first three come from a biology ontology, the last two from the classic pizza and wine examples. Each one names classes (study, technology, pizza, wine), properties (duration, accompanies) and the kind of answer expected (a list of instances, a definition, a class).

13.1.4 Step 3: consider reuse

[Slides · Ontology development: 25–28]

We rarely have to start from scratch: there is almost always an ontology from a third party that is at least a useful starting point. Reuse allows us

- to save effort;

- to interact with the tools that use other ontologies;
- to use ontologies already *validated* through use in applications.

Standard vocabularies exist for most domains, and many overlap: we identify the set most relevant to our problem. A **component-based approach based on modules** helps with overlapping domains: we reuse an ontology module as we reuse a software module. Initial brainstorming with domain experts can be very productive; refinement and iteration then bring the ontology to the level required by the application.

What to reuse:

kind	**examples**
ontology libraries	DAML ontology library, W3C "Good Ontologies" list, ontologies listed by Wikipedia, Protégé ontology library
upper ontologies	IEEE Standard Upper Ontology, Cyc
general ontologies	DMOZ (a Web directory), WordNet
domain-specific ontologies	UMLS Semantic Net (medicine), GO, the Gene Ontology (biology)

An **upper ontology** describes very general concepts (object, event, process, time, space) shared by all domains; a **domain ontology** describes one domain.

13.1.5 Step 4: enumerate terms

[Slides · Ontology development: 29–31]

Write down, in an unstructured list, all the relevant terms expected to appear in the ontology:

- **nouns** form the basis for **class** names;
- **verbs** (or verb phrases) form the basis for **property** names.

Card sorting is often the best way: write each concept on a card, organise the cards into piles, link the piles together, and do it again, and again. It works best in a small group.

Worked example — animals & plants: the raw list of terms

Dog, Cat, Cow, Person, Tree, Grass, Herbivore, Male, Female, Dangerous, Pet, Domestic animal, Farm animal, Draft animal, Food animal, Fish, Carp, Goldfish, Carnivore, Plant, Animal, Fur, Child, Parent, Mother, Father.
At this stage the list is a mess on purpose: it mixes kinds of things (Dog, Plant), properties (Dangerous, Male), roles (Pet, Parent) and parts (Fur). Sorting them out is the job of the next step.

13.1.6 Step 5: define classes and their taxonomy

[Slides · Ontology development: 32–43]

Definition — Class and instance

A **class** is a concept in the domain (Animal; or a class of roles, like Father, Mother): a collection of elements with similar properties. A class can state *necessary conditions for membership* (type of food, dwelling). An **instance** of a class is a particular element: a particular farm animal, a particular person, *Tweety the penguin*.

Organise the concepts. Take a group of things and ask what they have in common, then which other "siblings" there might be:

- Plant, Animal → *Living thing* (we might add Bacteria and Fungi, but not now);
- Cat, Dog, Cow, Person → *Mammal* (others: Goat, Sheep, Horse, Rabbit, . . .);
- Cow, Goat, Sheep, Horse → *Hoofed animal* ("Ungulate"). What others are there? Do they divide among themselves?
- Wild, Domestic → *Domestication*. What other states are there? "Feral" (a domestic animal returned to the wild).

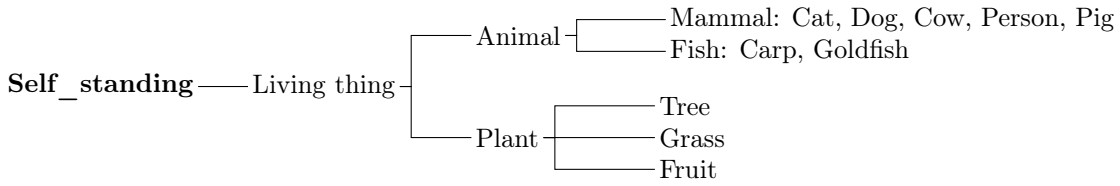
Choose some main axes. Add abstractions where needed (e.g. "Living thing"); identify **relations** (*eats*, *owns*, *parent of*), which feed the next step; identify **definable** things (child, parent, mother, father): things for which you can say clearly what they mean in terms of other things. Make names explicit.

Self-standing entities and modifiers.

- **Self-standing entities**: things that can exist on their own: people, animals, houses, actions, processes, . . . Roughly the *nouns*.
- **Modifiers**: things that modify ("inhere in") other things. Roughly the *adjectives* and *adverbs*.

Key result — the method of the primitives

Reorganise everything except the *definable* things into **pure trees** (each class has exactly one parent): these are the **primitives**. The definable things are not placed in the tree by hand: they are *defined* in terms of the primitives (step 7), and the reasoner computes where they belong.



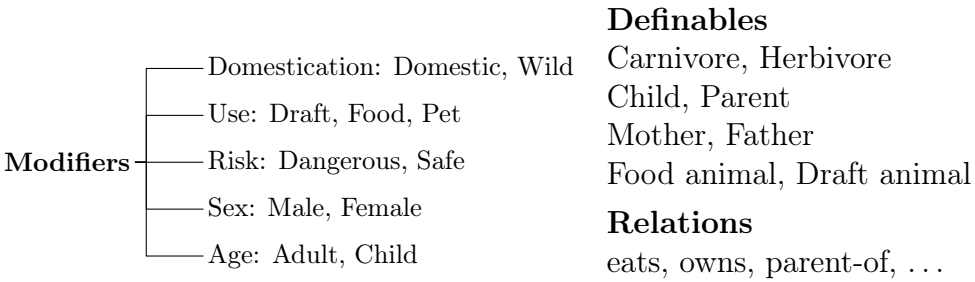


Figure 27: The animals & plants terms reorganised: self-standing entities and modifiers as pure trees (the primitives), plus the definables and the relations, kept aside.

Intuition — why pure trees?

Pet looks like a subclass of *Animal*, *Dog* too; but a dog can be a pet or not, and a pet can be a dog, a cat or a goldfish. If we put *Pet* in the same tree as *Dog* we get multiple inheritance everywhere and the tree becomes a tangle. Keeping "what a thing is" (Dog) separate from "how it is used" (Pet) and "what state it is in" (Domestic) keeps each tree simple. A class like *Pet dog* can then be *defined* as Dog $\sqcap$ having use Pet, and the reasoner works out that it is both a Dog and a Pet.

Comments help to clarify: when a class is used in a special way, add a comment, e.g. on *Living thing*: "abstract ancestor concept including all living things; restrict to plants and animals for now".

Class inheritance. Classes are organised into **subclass–superclass** hierarchies (also called generalisation–specialisation hierarchies). Two classes are in an "is-a" relation if every instance of the subclass is an instance of the superclass. Classes can be seen as **sets**: a subclass is a subset of its superclass. Examples: Mammal is a subclass of Animal; every penguin is a bird (Tweety, an instance of Penguin, is also an instance of Bird); Draft animal is a subclass of Animal.

Levels in the class hierarchy. Three modes of development:

- **top-down**: define the most general concepts first and then specialise them;
- **bottom-up**: define the most specific concepts first, then group them into more general classes;
- a **combination** (the typical choice): breadth at the top level and depth along a few branches to test the design.

Class inheritance is transitive: if A is a subclass of B and B a subclass of C, then A is a subclass of C.

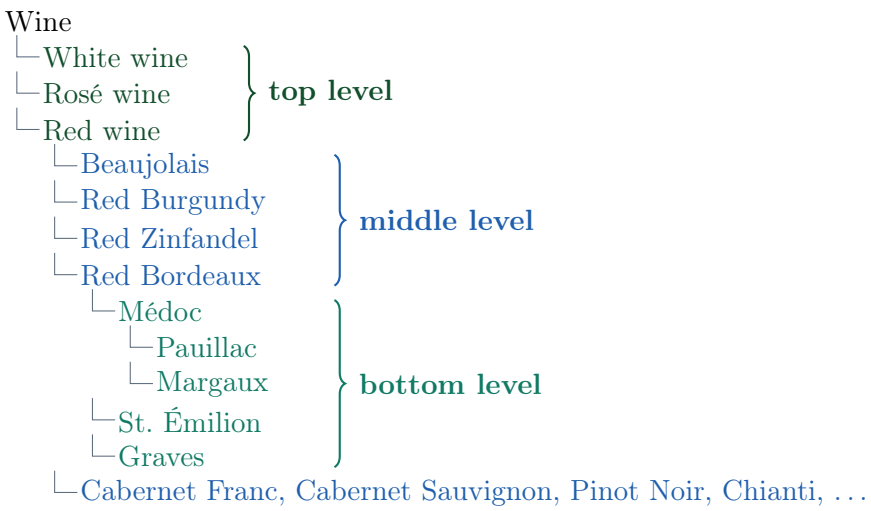


Figure 28: Levels in the class hierarchy of the wine ontology (redrawn and simplified from a Protégé screenshot): general classes at the top, specific ones at the bottom.

13.1.7 Step 6: define properties

[Slides · Ontology development: 44–47]

This step is often interleaved with the previous one. **Properties** (called *roles* in description logics, *slots* in frames) describe the attributes of the members of a class.

Key result — properties are inherited

The semantics of *subClassOf* demands that whenever A is a subclass of B, every property statement that holds for instances of B also applies to instances of A. So it makes sense to attach a property to the **highest** class in the hierarchy to which it applies (attach *eats* to Animal, not separately to Cat, Dog, Cow, ...).

Types of properties, on the wine example:

- **intrinsic** properties: flavour and colour of a wine (they belong to the thing itself);
- **extrinsic** properties: name and price of a wine (assigned from outside);
- **parts**: the ingredients of a dish;
- **relations to other objects**: the producer of a wine (a winery).

They are represented by two kinds of properties (Section 12.3):

- **datatype properties**: simple values (strings, numbers): name, price;
- **object properties**: complex values, other objects: the producer is an instance of Winery.

In the animals example, the modifiers of Figure 27 (domestication, use, risk, sex, age) and the relations (eats, owns, parent-of) become properties.

13.1.8 Step 7: define constraints

[Slides · Ontology development: 48–56]

Domain and range. For each property, state the **domain** (the class of the things that can have it) and the **range** (the class of its values):

statement	property	domain	range
Animal eats Living_thing	eats	Animal	Living_thing
Person owns Living_thing except person	owns	Person	Living_thing ⊓¬Person
Living_thing parent_of Living_thing	parent_of	Living_thing	Living_thing

If anything is used in a special way, add a text comment. E.g. on *eats*: "ignore the difference between parts of living things and living things; also things derived from living things" (strictly, a cow eats *leaves*, not whole plants, and we eat cheese, which is not a living thing).

Definable things. Paraphrase the definitions and formalise them in terms of the primitives, the relations and other definables. Note any assumptions (as comments when implementing).

Worked example — three definitions

The slides write them in the Manchester syntax used by Protégé (*and*, *some*, *only*); next to it the same in description-logic notation, where ⊓ is "and", $\exists r.C$ is "has some r-value in C" and $\forall r.C$ is "all r-values are in C":

in words	formal
A *Parent* is an animal that is the parent of some other animal (ignore plants for now)	Parent ≡ Animal ⊓ ∃parent_of.Animal
A *Herbivore* is an animal that eats only plants	Herbivore ≡ Animal ⊓ ∀eats.Plant
An *Omnivore* is an animal that eats both plants and animals	Omnivore ≡ Animal ⊓ ∃eats.Animal ⊓ ∃eats.Plant

Watch out — "only" does not mean "some"

∀eats.Plant ("eats *only* plants") is also true of something that eats *nothing*: all the (zero) things it eats are plants. That is why the slides add "NB: all animals eat some living thing": with that axiom, an animal that eats only plants really eats some plant. Mixing up *some* (∃) and *only* (∀) is the most common modelling error in OWL.

Fill in properties at the class level. What can we say about *all* members of a class? All cows eat some plants; all cats eat some animals; all pigs eat some animals and eat some plants. In Protégé this can be done with a "property matrix" (one row per class, one column per property: Cat eats Animal, Dog eats Animal, Cow eats Grass and Leafy_plant, Person eats Animal and Plant).

Check with the classifier: the closure axiom. A **classifier** (reasoner) computes the hierarchy of the defined classes. We expect *Cow* to be classified as a *Herbivore*. It is not. Why?

Worked example — why cows are not herbivores (yet)

What we have said: cows are animals and, among other things, eat some grass and eat some leafy plants:

$$\text{Cow} \sqsubseteq \text{Mammal} \sqcap \exists \text{eats}.\text{Grass} \sqcap \exists \text{eats}.\text{Leafy_plant}.$$

Nothing here says that cows eat *only* plants: a cow could also eat some animal, and the reasoner must allow that. What we need to say: cows eat some plants **and eat only plants**. We add the **closure axiom**

$$\text{Cow} \sqsubseteq \forall \text{eats}.(\text{Grass} \sqcup \text{Leafy_plant}),$$

which "closes" the list of things cows eat. Now every cow is an Animal that eats only plants, so the reasoner classifies Cow under Herbivore.

Definition — Open vs. closed world reasoning

- **Open world** reasoning: *negation as contradiction*. Anything might be true unless it can be proven false. We reason about *any* world consistent with what we know.
- **Closed world** reasoning: *negation as failure*. Anything that cannot be found is false. We reason about *this* world, the one in the database.

OWL ontologies use the open world assumption: **ontologies are not databases** (Section 12.1).

Intuition — the closure axiom and the open world

In a database, if the table of what cows eat lists only grass and leafy plants, then cows eat nothing else: missing rows are false (closed world). In an ontology, missing information is just *unknown* (open world): "cows eat grass" leaves open that they also eat fish. The closure axiom is how we tell an open-world reasoner "and that is all".

13.1.9 Step 8: create instances

[Slides · Ontology development: 57–60]

To create an instance of a class: the class becomes the **direct type** of the instance, and every superclass of the direct type is also a type of the instance (Tweety is a Penguin, hence a Bird, hence an Animal). Then assign the slot (property) values; they must conform to the facet constraints, and knowledge-acquisition tools often check this.

Filling the ontology with instances is a **separate step**, and the number of instances is much larger than the number of classes. So **populating** an ontology is usually not done by hand: instances are retrieved from legacy data sources (databases) or **extracted automatically from a text corpus**: this is *ontology population*, part of ontology learning (Section 13.3).

Cheat sheet — the design process

- Many methodologies (On-To-Knowledge, HCOME, DILIGENT, NeOn, DOGMA, RapidOWL, ...); the lecturer's choice: Noy & McGuinness, *Ontology Development 101.*
- Cycle of 8 steps: requirement and domain analysis → scope → reuse → enumerate terms → classes → properties → constraints → instances, and again.
- Competency questions define the scope and test the ontology.
- Reuse: libraries, upper ontologies (SUO, Cyc), general (WordNet), domain (GO, UMLS).
- Terms: nouns → classes, verbs → properties; card sorting.
- Classes: self-standing entities and modifiers as pure trees (primitives); definables defined by axioms; inheritance is transitive; top-down, bottom-up or combined.
- Properties: intrinsic, extrinsic, parts, relations; datatype vs. object; attach to the highest class.
- Constraints: domain and range; definitions with *some* (∃) and *only* (∀); closure axioms; open world (ontology) vs. closed world (database).
- Instances: many, populated automatically from databases or text.

13.2 N-ary Relations

[Slides · Ontology development: 61–73]

13.2.1 The problem

[Slides · Ontology development: 61–62]

In OWL (and RDF) a property is a **binary** relation: an instance of a property links two individuals (*Christine* –has_diagnosis→ *Breast_Tumor*), or an individual and a value. Sometimes, though, the most natural way to describe a situation links an individual to *more than one* other individual or value. Such relations are called **n-ary relations**. Three issues arise:

1. how do we describe *an instance of a relation* (e.g. attach a probability to a diagnosis)?
2. how do we represent relations among more than two individuals? ⇒ **Pattern 1**;
3. how do we represent a relation in which one participant is an *ordered list* of individuals? ⇒ **Pattern 2**.

The solutions come from the W3C Working Group Note *Defining N-ary Relations on the Semantic Web* (W3C Semantic Web Best Practices and Deployment Working Group).

Worked example — four situations that binary properties cannot express

1. *Christine has breast tumor with high probability*: a relation first thought to be binary needs a further argument (the probability).
2. *Steve has temperature, which is high, but falling*: two binary properties always go together and should be represented as one n-ary relation.
3. *John buys a "Lenny the Lion" book from books.example.com for $15 as a birthday gift*: from the beginning, the relation is among several things.
4. *United Airlines flight 3177 visits the following airports: LAX, DFW, and JFK*: one argument is fundamentally a *sequence*, not a single individual.

13.2.2 Pattern 1: the relation becomes a class

[Slides · Ontology development: 63–70]

Key result — Pattern 1 (reification)

Represent the relation as a **class** rather than a property. Each individual instance of this class corresponds to one instance of the relation; additional properties provide binary links to each argument of the relation. The classes created in this way are often called **reified relations** (to *reify* = to make a thing out of something abstract).

Case 1: additional attributes describing a relation. *Christine has breast tumor with high probability.* We create an individual that represents the relation instance itself, with a link from the subject (Christine) to it, and links from it to everything else (the diagnosis and its probability).

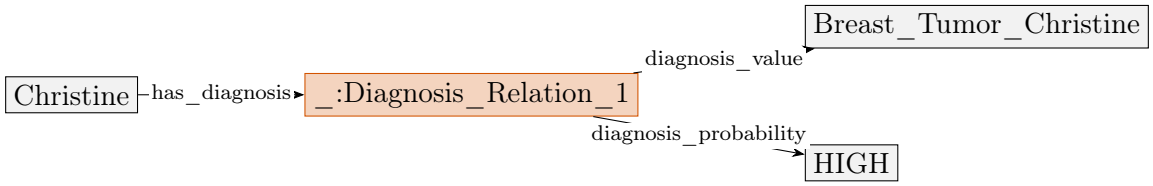


Figure 29: Pattern 1, case 1: the diagnosis relation becomes an individual.

The individual `_:Diagnosis_Relation_1` is a single object that encapsulates both

the diagnosis and its probability: it contains all the information of the three original arguments (who is diagnosed, what the diagnosis is, how probable it is). In RDF such instances are **blank nodes**: an `rdf:Description` element without an `rdf:about` attribute, i.e. a node without a global name (hence the "`_:`" prefix). Class definitions: Person –has_diagnosis→ Diagnosis_Relation (all values from); Diagnosis_Relation –diagnosis_value→ Disease and –diagnosis_probability→ Probability (both functional: exactly one value).

Case 2: different aspects of the same relation. *Steve has temperature, which is high, but falling.* This is not a binary relation with an extra attribute: it relates the individual Steve to a *complex object* representing different facts about his temperature.

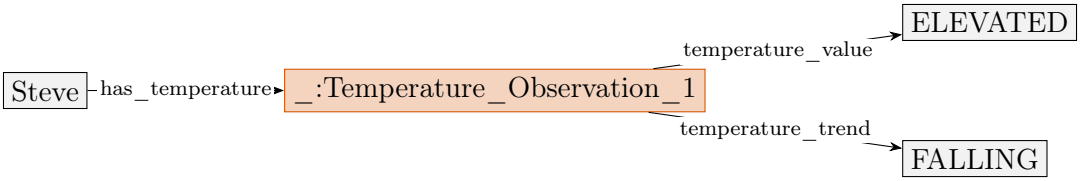


Figure 30: Pattern 1, case 2: an observation object groups two aspects of Steve's temperature.

Such cases often appear while an ontology **evolves**, when one realises that two relations must be collapsed into one: initially one might have had two properties, *has_temperature_level* and *has_temperature_trend*, both relating to people; then one realises they are inextricably intertwined, because one needs to talk about "temperatures that are elevated but falling".

Case 3: n-ary relation with no distinguished participant. *John buys a "Lenny the Lion" book from books.example.com for $15 as a birthday gift.* The relation links individuals that play different roles, and none of them stands out as the "owner" of the relation. We create an individual for the relation instance with links to *all* participants.

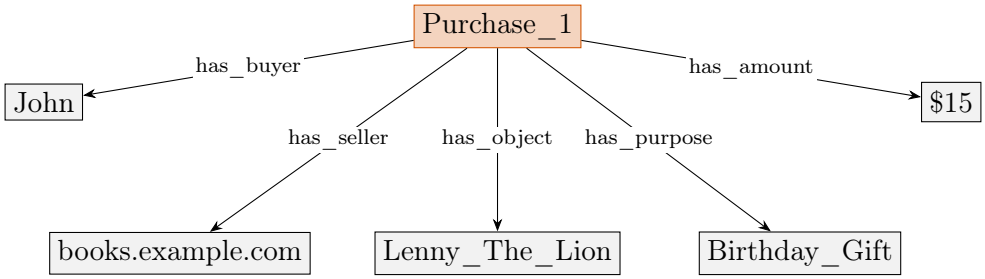


Figure 31: Pattern 1, case 3: a Purchase individual linked to all participants. Class level: Purchase –has_buyer→ Person, –has_seller→ Company, –has_object→ Object, –has_purpose→ Purpose, –has_amount→ Quantity.

Watch out — considerations in introducing a new class

- The individuals that represent relation instances (and often their classes) are not given meaningful names, just labels: they do not stand on their own, they only group other objects together, so a distinguishing name serves no purpose (the same is done when reifying statements in RDF).
- Creating a class to represent an n-ary relation **limits the use of many OWL constructs** and creates a **maintenance problem**, especially with inverse relations (the inverse of a 5-ary relation is not a single property).

13.2.3 Pattern 2: lists for arguments in a relation

[Slides · Ontology development: 71–73]

Some n-ary relations do not fall into the cases above: they are more like a list or sequence of arguments. *United Airlines flight 3177 visits the following airports: LAX, DFW, and JFK*: the relation holds between the flight and the airports, *in the order* in which the aircraft arrives at each one. It might hold between any number of arguments, there is no natural way to break it into distinct properties (first_stop, second_stop, ...), and the order is highly meaningful.

Key result — Pattern 2

When all participants but one have no specific role and form an ordered list, connect them into a **sequence** with an ordering relation, and relate the one remaining participant to the sequence (or to its first element).

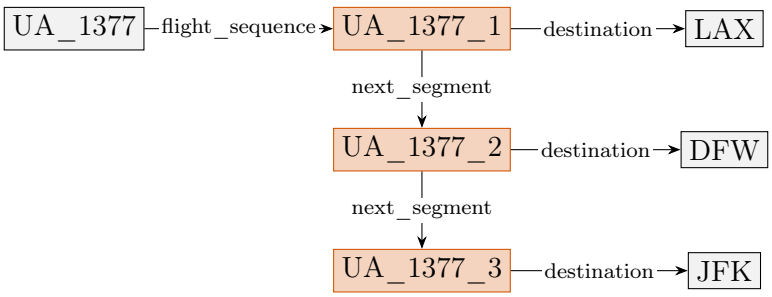


Figure 32: Pattern 2: the flight points to its first segment; segments are chained by *next_segment*; each segment has a destination.

nextSegment is an ordering relation between instances of the class *FlightSegment*; each segment has a property for its destination. A special subclass, *FinalFlightSegment*, has a **maximum cardinality of 0** on *nextSegment*: the last segment has no next one, which marks the end of the sequence. (This is how linked lists work in programming: each node points to the next, the last points to nothing.)

Cheat sheet — n-ary relations

- OWL properties are binary; n-ary relations need a pattern.
- Pattern 1 (reified relation): a new class whose instances are the relation instances, with one binary property per argument. Case 1: extra attributes of a relation (diagnosis + probability); case 2: different aspects of one relation (temperature value + trend); case 3: no distinguished participant (purchase). Instances are often blank nodes; drawbacks: fewer OWL constructs usable, maintenance problems with inverses.
- Pattern 2: ordered arguments as a linked sequence (flight segments with *nextSegment*; final segment with max cardinality 0).

13.3 Ontology Learning

[Slides · Ontology learning: 1–25; Ontology development: 74]

Building an ontology by hand (Section 13.1) is slow and expensive. **Ontology learning** uses NLP and machine learning to do part of the work automatically. The deck is a tutorial by Ícaro Medeiros (CIn–UFPE, 2008).

13.3.1 What, why and from where

[Slides · Ontology learning: 4–9]

The field has **too many names for the same subject**: ontology extraction, emergence, generation, acquisition, discovery, population, enrichment… The common name, after Philipp Cimiano's book *Ontology Learning and Population from Text* (2006), is **ontology learning** (OL).

Definition — Ontology learning

Ontology learning is the set of methods and techniques for

- building an ontology from scratch;
- enriching or adapting an existing ontology;

by extracting concepts and relations (from text or other sources) to form an ontology. OL is a **semi-automatic** task of information extraction: a human remains in the loop to validate and correct.

Why? Ontology engineering has three problems (Maedche and Staab, 2001):

- **time**: can you develop an ontology fast?
- **difficulty**: is it difficult to build an ontology?
- **confidence**: how do you know that you have got the ontology right?

OL can help with all three, especially with the **knowledge acquisition bottleneck**: experts are few and busy, and writing their knowledge down by hand is the slowest part of building any knowledge-based system.

Information sources. Relevant text (mainly Web documents); Web document schemata (XML, DTD, RDF); databases on the Web; dictionaries; semi-structured documents; personal wikis, e-mail and file folders; existing Web ontologies.

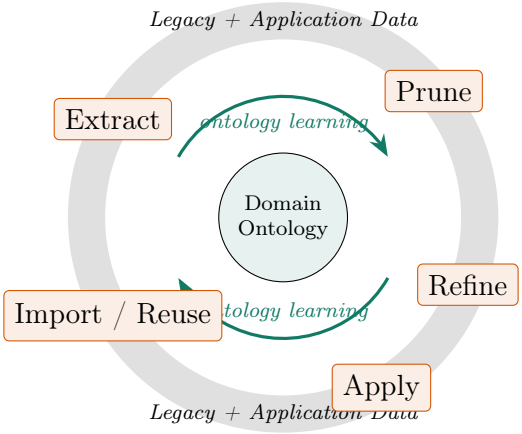


Figure 33: The ontology engineering cycle of Maedche and Staab (2001), redrawn: import/reuse existing ontologies, extract, prune, refine, apply, and start again. Ontology learning is not only the task of extraction.

The cycle shows that OL is **not only extraction**: one also *imports* and reuses existing ontologies, *prunes* what is irrelevant for the application, *refines* the result, and *applies* it, which produces new data for the next round.

13.3.2 How to learn ontologies

[Slides · Ontology learning: 10–15]

The main families of techniques: natural language processing; dictionary parsing; statistical analysis; machine learning; hierarchical concept clustering; Formal Concept Analysis (lattices).

Why text? Text is massively available on the Web; relevant texts contain relevant knowledge about a domain; and when an ontology is learnt from text, the *linguistic* knowledge (how concepts are named and talked about) remains associated with the ontology (Sintek et al., 2004).

Intuition — OL as reverse engineering (Buitelaar et al., 2005)

People share a model of the world (a "shared world model") and use it to *write* texts: newspapers, manuals, Web pages. Ontology learning goes the opposite way: from the texts back to the model. It is **reverse engineering**: like recovering the design of a program from its compiled code, we recover the conceptual model from the documents it produced. And like reverse engineering, it is only partial: much of the shared knowledge is never written down because everybody already knows it.

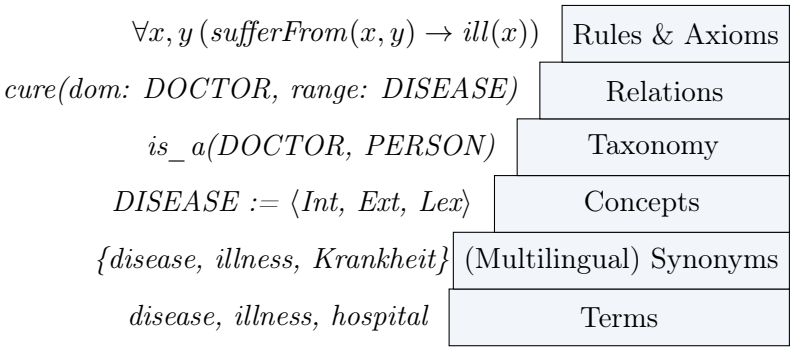


Figure 34: The ontology learning layer cake (Buitelaar et al., 2005), redrawn. Each layer is a learning task that builds on the layers below it; on the left, an example of the output of each task.

Key result — the layer cake

Ontology learning from text is organised in six layers, from the bottom:

1. **terms**: the relevant words and phrases of the domain (*disease*, *illness*, *hospital*);
2. **(multilingual) synonyms**: groups of terms with the same meaning, also across languages (*disease*, *illness*, German *Krankheit*);
3. **concepts**: a concept is defined by its **int**ension, its **ext**ension and its **lex**ical realisations (see below);
4. **taxonomy**: the is-a hierarchy between concepts (a doctor is a person);
5. **relations**: other relations between concepts, with domain and range (doctors cure diseases);
6. **rules and axioms**: general logical statements (whoever suffers from something is ill).

Lower layers are easier to learn automatically and more reliable; upper layers are harder and need more human validation.

13.3.3 Term extraction

[Slides · Ontology learning: 16–18]

Linguistic methods reuse the syntactic tools of the course:

- **part-of-speech tagging** (Section 7.2) to identify the syntactic class: a noun suggests a class, a verb a relation;
- **stemming** (Section 6.3): *formalize*, *formalization*, *formalized*, *formalizing* are one term;
- **head-modifier analysis**: in *fast car* the head is *car* and *fast* a modifier; in *the hood of the car*, *hood* is a part of *car*;
- **grammatical function analysis**: "John played football in the garden" → *play(John, football)*: the subject and the object of a verb become the arguments of a relation.

Statistical methods:

- **term weighting** with TF-IDF (Section 6.2);
- **co-occurrence analysis** (a common method in text mining);
- **comparison of frequencies between a domain corpus and a general corpus**: a word much more frequent in medical texts than in newspapers (*tumour*) is likely a medical term; a word equally frequent in both (*table*) is not.

Hybrid methods: linguistic rules extract term candidates (e.g. the POS pattern "adjective* noun+" finds *heart disease*, *chronic kidney disease*), then statistics filter them (before or after).

13.3.4 Synonym extraction

[Slides · Ontology learning: 19–20]

Two approaches: **extending WordNet** (*term classification*: attach a new term to the right synset of WordNet, Section 10.2) and **co-occurrence between terms** (*term clustering*: terms with similar contexts are grouped together, the distributional hypothesis of Section 8.1).

13.3.5 Concept extraction

[Slides · Ontology learning: 21–25]

A term may indicate a concept, if we define its:

- **intension**: an (in)formal definition of the objects the concept describes. E.g. "a disease is an impairment of health or a condition of abnormal functioning";
- **extension**: the set of objects described by the concept. E.g. cancer, heart disease;
- **lexical realisations**: the term itself and its (multilingual) synonyms.

Intuition — intension vs. extension

The *intension* is the *recipe* ("what it takes to be a disease"); the *extension* is the *list* ("all the diseases"). The concept "even number between 1 and 9" has intension "divisible by 2, between 1 and 9" and extension $\{2, 4, 6, 8\}$. In an ontology, the class definition is the intension, and the instances are the extension.

Learning the intension. An *informal* definition is a shallow definition as used in WordNet (a gloss): find the appropriate WordNet concept for a term and the appropriate conceptual relations (Navigli and Velardi, 2004). A *formal* definition gives formal constraints for class membership, e.g. with Formal Concept Analysis.

Learning the extension: ontology population. Extracting the instances of a concept from text is **ontology population**. It is related to knowledge markup and tag suggestion (semantic metadata), and it uses **named entity recognition** (Section 10.4): "John is a football player" → *John* (a Person) is an instance of *Football Player*. Instances can be

- names for objects: persons, organisations, countries, cities;
- event instances: a football match, with its teams, players, officials, ...

Watch out — what the shortened deck leaves out

The outline of the tutorial continues with taxonomy, relation and axiom learning, ontology learning from folksonomies (collaborative tags), tools and conclusions. These parts are not in the shortened deck shown in class; the layer cake of Figure 34 gives what these tasks produce.

Cheat sheet — ontology learning

- OL (a.k.a. extraction, generation, acquisition, population, enrichment): semi-automatic building or enrichment of ontologies; fights the knowledge acquisition bottleneck (time, difficulty, confidence).
- Sources: Web text, schemata, databases, dictionaries, semi-structured documents, existing ontologies. OE cycle: import/reuse, extract, prune, refine, apply.
- Methods: NLP, dictionary parsing, statistics, ML, clustering, Formal Concept Analysis. OL from text = reverse engineering of the shared world model.
- Layer cake: terms → synonyms → concepts → taxonomy → relations → rules & axioms.
- Terms: POS, stemming, head-modifier, grammatical functions; TF-IDF, co-occurrence, domain vs. general corpus; hybrid.
- Concept = intension + extension + lexical realisations; extension learning = ontology population, with NER.

13.4 Ontology Matching

[Slides · Ontology matching: 1–14; Ontology matching (long version): 1–112; Ontology development: 75–76]

Different people build different ontologies of the same domain: they use different names (*PC* vs. *Personal Computers*), different structures, different languages. To make two systems that use two ontologies work together we must find which entity of one corresponds to which entity of the other. This is **ontology matching**. The source is the tutorial by Pavel Shvaiko and Jérôme Euzenat (SWAP 2006); the slides shown in class are the definitions of correspondence and alignment, the matching process and the application domains; the rest comes from the long version.

13.4.1 The matching problem

[Slides · Ontology matching (long version): 2–25]

The goals of the tutorial: illustrate the role of ontology matching, give an overview of the basic matching techniques, show how state-of-the-art systems use them, and

motivate future research.

Definition — Matching operation

The **matching** operation takes as input ontologies, each consisting of a set of discrete entities (tables, XML elements, classes, properties, ...), and determines as output the **relationships** (equivalence, subsumption, ...) holding between these entities.

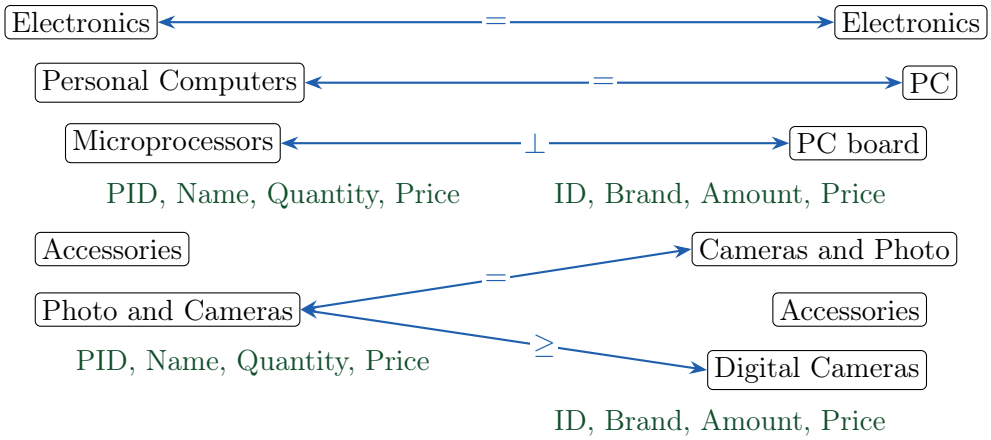


Figure 35: Motivation: two XML schemas of an electronics catalogue (redrawn, simplified). Blue arrows are correspondences: equal (=), more general (≥), disjoint (⊥). Among the attributes (green): Name ≥ Brand, Quantity = Amount, Price = Price.

Worked example — reading Figure 35

Personal Computers on the left is the same as *PC* on the right (equivalence, found e.g. by expanding the abbreviation). *Microprocessors* and *PC board* are disjoint: nothing is both. *Photo and Cameras* is more general than *Digital Cameras*. Among the attributes, *Quantity* = *Amount* (synonyms), and *Name* is more general than *Brand* (a brand is a kind of name). The second motivating example in the slides does the same with two *ontologies* of products and books (*Product* vs. *Monograph*, *creator* vs. *author*, *Person* vs. *Human*), including instances such as "Bertrand Russell: My life" (an Autobiography) and "Albert Camus: La chute" (Literature).

Schema matching vs. ontology matching. **Differences**: schemas often do not give explicit semantics for their data; relational schemas provide no generalisation (no is-a); ontologies are logical systems that constrain the meaning, defined as sets of logical axioms. **Commonalities**: both provide a vocabulary of terms that describes a domain of interest, and both constrain the meaning of these terms. Techniques developed for both problems are of mutual benefit.

Scope. Heterogeneity between ontologies can occur when different *languages* are used (OWL vs. XML schema), different *terminologies* (*car* vs. *automobile*), different

modelling (colour as a property vs. as a class), ... Reducing heterogeneity takes two steps: **match**, which determines the alignment, and **process the alignment** (merging, transforming, ...). Matching can happen at **design time** (by the developers, before the system runs) or at **run time** (on the fly, e.g. when two agents meet for the first time).

13.4.2 Correspondence, alignment, matching process

[Slides · Ontology matching: 1–13; Ontology matching (long version): 26–38]

Definition — Correspondence

Given two ontologies O and O', a **correspondence** M between O and O' is a 5-tuple $\langle id, e, e', R, n\rangle$ such that:

- id is a unique **identifier** of the correspondence;
- e and e' are **entities** of O and O' (e.g. XML elements, classes);
- R is a **relation**, e.g. equivalence ($=$), more general ($\sqsupseteq$), disjointness ($\perp$);
- n is a **confidence measure** in some mathematical structure (typically in the $[0, 1]$ range).

Definition — Alignment

Given two ontologies O and O', an **alignment** A between O and O'

- is a **set of correspondences** on O and O';
- with some **cardinality**: 1-1, 1-*, etc.;
- with some additional **metadata** (method, date, properties, etc.).

Worked example — correspondences from Figure 35

$$\langle c_1,\ \text{Personal Computers},\ \text{PC},\ =,\ 0.95\rangle$$
$$\langle c_2,\ \text{Microprocessors},\ \text{PC board},\ \perp,\ 0.80\rangle$$
$$\langle c_3,\ \text{Photo and Cameras},\ \text{Digital Cameras},\ \sqsupseteq,\ 0.70\rangle$$

The set $\{c_1, c_2, c_3, \ldots\}$ is an alignment. A **1-1** alignment matches each entity to at most one entity of the other ontology; a **1-*** alignment lets one entity of O match several entities of O' (*Photo and Cameras* matches both *Cameras and Photo* and *Digital Cameras*). The confidence values here are invented for the example.

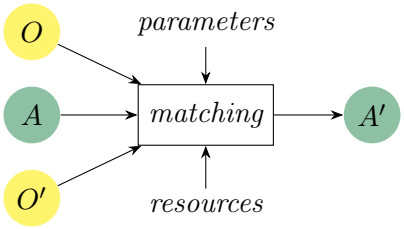


Figure 36: The matching process.

Key result — the matching process

The matching process takes as input two ontologies O and O', optionally an *input alignment* A (correspondences already known, to be completed), some **parameters** (thresholds, weights) and external **resources** (a thesaurus like WordNet, an upper ontology), and returns an alignment A'.

13.4.3 Application domains

[Slides · Ontology matching: 14; Ontology matching (long version): 39–52]

- **Traditional**: ontology evolution (match the old and the new version of an ontology), schema integration, catalog integration, data integration.
- **Emergent** (long version): P2P information sharing, agent communication, Web service composition, query answering on the Web.

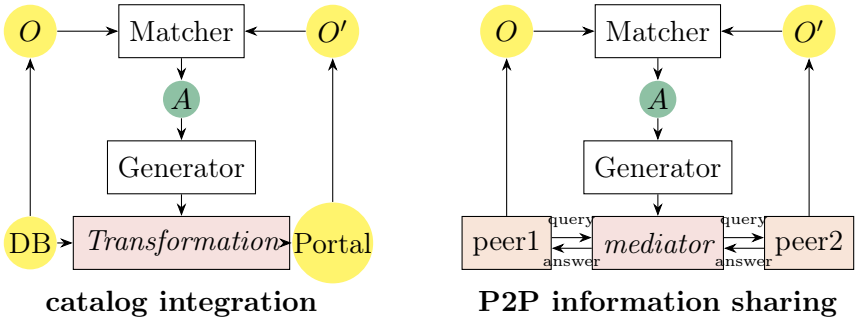


Figure 37: Two applications (simplified). Left: the ontologies of a database and of a portal are matched; from the alignment a generator produces the transformation that moves data from one to the other. Right: two peers with different ontologies; the alignment produces a mediator that translates queries and answers.

application	instances	run time	automatic	correct	complete	operation
Ontology evolution	✓			✓	✓	transformation
Schema integration	✓			✓	✓	merging
Catalog integration	✓			✓	✓	data translation
Data integration	✓			✓	✓	query answering
P2P information sharing		✓				query answering
Web service composition		✓	✓	✓		data mediation
Multi agent communication		✓	✓	✓	✓	data translation
Query answering	✓	✓				query reformulation

Table 19: Requirements of each application (long version): does it need instances, run-time matching, full automation, a correct alignment, a complete one; and what is done with the alignment.

Intuition — reading the table

Traditional applications (top four) match at *design time*: a human can check the result, and the alignment must be *correct and complete* because it is used once and for all to transform or merge. Emergent applications match at *run time*, often *automatically* (no human available when two agents meet): there, a partial alignment is acceptable as long as what is found is correct.

13.4.4 Classification of matching techniques

[Slides · Ontology matching (long version): 53–65]

Matching dimensions:

- *input*: the underlying models (XML, OWL, ...); **schema-level** (only classes and properties) vs. **instance-level** (also the data);
- *process*: approximate vs. exact; how the input is interpreted;
- *output*: cardinality (1-1, 1-*); only equivalence vs. diverse relations (subsumption, ...); graded (a confidence in $[0, 1]$) vs. absolute confidence.

The classification of schema-based techniques has **three layers**: the *upper* layer is about the granularity of the match and how the input is interpreted; the *middle* layer lists the classes of elementary (basic) techniques; the *lower* layer classifies them by the kind of input they use.

granularity	interpretation	basic technique	what it uses	kind of input
Element-level	syntactic	string-based	name and description similarity	terminological
	syntactic	language-based	tokenisation, elimination	terminological (linguistic)
	syntactic	constraint-based	type similarity, key properties	structural (internal)
	external	linguistic resources	lexicons, thesauri (WordNet)	terminological (linguistic)
	external	upper / domain-specific formal ontologies	DOLCE, FMA	semantic
Structure-level	syntactic	graph-based	graph homomorphism, children, leaves	structural (relational)
	syntactic	taxonomy-based	taxonomy structure	structural (relational)
	external	repository of structures	structure metadata (previous matches)	structural
	semantic	model-based	SAT solvers, DL reasoners	semantic

Table 20: Classification of schema-based matching techniques (simplified), rewritten as a table from the slide's diagram. Read each row from left to right: the upper layer (granularity and interpretation), the middle layer (the basic technique), the lower layer (the kind of input it works on). Upper/domain ontologies are used at both granularities.

- **Element-level** techniques look at each entity in isolation (its name, its type); **structure-level** techniques look at how entities are related to the others (its children, its position in the taxonomy).
- **Syntactic** techniques use only the input itself; **external** ones use outside resources (WordNet, an upper ontology, a repository of previous matches); **semantic** ones reason on the formal meaning (logic).

13.4.5 Basic techniques

[Slides · Ontology matching (long version): 66–88]

Element-level, string-based. (Used e.g. by COMA, SF, S-Match, OLA.)

- **Prefix**: checks whether the first string starts with the second: *net* = *network*; but also, wrongly, *hot* = *hotel*.
- **Suffix**: checks whether the first string ends with the second: *ID* = *PID*; but also, wrongly, *word* = *sword*.
- **Edit distance** (Section 6.4): the number of edit operations (insertions, deletions, substitutions of characters) needed to transform one string into the other, **normalised by the length of the longer string**.

Worked example — EditDistance(NKN, Nikon) = 0.4

Ignoring case, *nkn* becomes *nikon* with two insertions: n i k o n. So the edit distance is 2, and normalised by the length of the longer string: $2/5 = 0.4$. A distance of 0.4 (similarity $1 - 0.4 = 0.6$) suggests that *NKN* may be an abbreviation of *Nikon*. (Used e.g. by S-Match, OLA, Anchor-Prompt.)

Element-level, language-based. (COMA, Cupid, S-Match, OLA.)

- **Tokenisation**: parse names into tokens by recognising punctuation and case: *Hands-Free Kits* → ⟨hands, free, kits⟩.
- **Lemmatisation**: morphological analysis of the tokens to find their basic forms: *Kits* → *Kit*.
- **Elimination**: discard "empty" tokens: articles, prepositions, conjunctions, . . . (*a, the, by, type of, their, from*), i.e. stop words (Section 6.2).

Element-level, linguistic resources. Sense-based, with WordNet (Artemis, Ctx-Match, S-Match):

- $A \sqsubseteq B$ if A is a hyponym or meronym of B: *Brand* $\sqsubseteq$ *Name*;

- $A \sqsupseteq B$ if A is a hypernym or holonym of B: *Europe* $\sqsupseteq$ *Greece*;
- $A = B$ if they are synonyms: *Quantity* $=$ *Amount*;
- $A \perp B$ if they are antonyms or siblings in the part-of hierarchy: *Microprocessors* $\perp$ *PC Board*.

Gloss-based (S-Match): compare the WordNet glosses of the two terms; the more words they share, the higher the similarity, and equivalence is returned above a threshold. E.g. *Maltese dog*: "a breed of toy dogs having a long straight silky white coat"; *Afghan hound*: "a tall graceful breed of hound with a long silky coat" share *breed, long, silky, coat*. This is the idea of the Lesk algorithm (Section 10.3).

Structure-level, taxonomy-based. Ontologies are seen as graphs of terms and their relations (Anchor-Prompt, NOM, QOM):

- **bounded path matching**: take two paths of hierarchical links, compare the terms and their positions along the paths, and identify similar terms;
- **super(sub)-concept rules**: if the super-concepts are the same, the concepts themselves are similar.

Structure-level, tree-based. (Cupid, COMA.)

- **children**: two non-leaf elements are similar if their sets of immediate children are highly similar;
- **leaves**: two non-leaf elements are similar if their sets of leaves are highly similar, even if their immediate children are not.

E.g. *Photos and cameras* (left) and *Digital cameras* (right) both have the leaves PID/ID, Name/Brand, Quantity/Amount, Price/Price: their leaves match, so the two parents are probably related.

Structure-level, model-based. Propositional satisfiability (SAT) (CtxMatch, S-Match): the question "are the two nodes equivalent?" is turned into a logical formula Axioms $\rightarrow rel(context_1, context_2)$ and a SAT solver checks whether it is valid. The *context* of a node is the conjunction of the labels on its path from the root.

Worked example — SAT-based matching of *Personal Computers* and *PC*

Axioms from the lexicon: $\text{Electronics}_1 \leftrightarrow \text{Electronics}_2$ and $\text{Personal Computers}_1 \leftrightarrow \text{PC}_2$. Contexts:

$context_1 = \text{Electronics}_1 \wedge \text{Personal Computers}_1, \qquad context_2 = \text{Electronics}_2 \wedge \text{PC}_2.$

To prove equivalence we check that

$$(\text{Electronics}_1 \leftrightarrow \text{Electronics}_2) \wedge (\text{Personal Computers}_1 \leftrightarrow \text{PC}_2) \rightarrow \big((\text{Electronics}_1 \wedge \text{Personal Computers}_1) \leftrightarrow (\text{Electronics}_2 \wedge \text{PC}_2)\big)$$

is valid, i.e. that its negation is unsatisfiable. It is: if the two pairs of labels are equivalent, the two conjunctions are equivalent. So the two nodes are equivalent.

Description logics (DL)-based: use a DL reasoner on the definitions.

Worked example — micro-company $\sqsubseteq$ SME

$$\text{micro-company} = \text{company} \sqcap \leq_5 \text{employee}$$
$$\text{SME} = \text{firm} \sqcap \leq_{10} \text{associate}$$

Background axioms: company = firm; associate $\sqsubseteq$ employee. A micro-company has at most 5 employees; every associate is an employee, so it has at most 5 associates, hence at most 10; and a company is a firm. Therefore every micro-company is an SME:

$$\text{micro-company} \sqsubseteq \text{SME}.$$

No string or structure technique could find this: the names share nothing. Only reasoning on the meaning does.

13.4.6 Composing matchers and selecting the alignment

[Slides · Ontology matching (long version): 89–95]

Real systems combine several basic matchers.

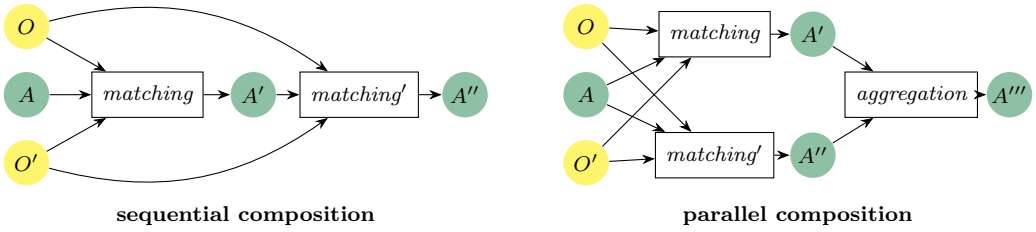


Figure 38: Composing matchers (parameters and resources of each matcher omitted). Sequential: the output of the first matcher is the input alignment of the second. Parallel: independent matchers whose results are aggregated.

Selecting the final alignment from the candidate correspondences:

- **ranking strategies**: *thresholds* (keep correspondences with confidence above t), *MaxDelta* (keep those within δ of the best one);
- **cardinalities**: 1-1, 1-*, *-*;
- **optimisation**: *stable marriage* (no two entities would both prefer each other to their assigned partners), *maximal weight match* (maximise the total confidence);
- **directionality**: $O \to O'$ or $O' \to O$ (SmallLarge, LargeSmall), or both directions (Both).

13.4.7 State-of-the-art systems

[Slides · Ontology matching (long version): 96–104]

About 50 matching systems existed in 2006; four examples:

system	what it does	architecture
Cupid (U. Washington, Microsoft, U. Leipzig)	schema-based; similarity coefficients in $[0, 1]$; linguistic and structure matching	sequential: linguistic matching (with thesauri) $\rightarrow$ structure matching $\rightarrow$ weighting $\rightarrow$ alignment
Falcon-AO (Southeast U., Nanjing; the slide says "Southwest")	linguistic and structure matching	linguistic matching $\rightarrow$ structure matching $\rightarrow$ alignment
OLA (INRIA Rhône-Alpes, U. de Montréal)	schema- and instance-based; computes dissimilarities, then extracts alignments (equivalences in $[0, 1]$); terminological (incl. linguistic) and structural (internal and relational) distances	neither sequential nor parallel: the similarity computation is iterated until it stabilises
S-Match (U. Trento)	schema-based; computes $=$, $\sqsupseteq$, $\sqsubseteq$, $\perp$; analyses the meaning (concepts, not labels) codified in the elements and structures	sequential with composition at the element level: translator $\rightarrow$ pre-processing (with oracles) $\rightarrow$ PTrees $\rightarrow$ match manager (SAT solvers, semantic matchers)

13.4.8 Conclusions, tools and benchmarks

[Slides · Ontology matching (long version): 105–112; Ontology development: 75–76]

Uses of the classification of Table 20: it gives a common basis to compare existing systems; it helps design a new system or elementary matcher from the state of the art; it helps design systematic benchmarks (e.g. discard features from ontologies one at a time and see which class of technique deals with which feature).

Challenges: missing background knowledge; performance of the systems; interactive approaches; explanations of matching; social aspects of ontology matching; large-scale evaluation; infrastructures.

Tools and benchmarks: `http://www.ontologymatching.org` (the portal of the field), a list of 50 ontology mapping and alignment tools, and the **OAEI** (Ontology Alignment Evaluation Initiative, `http://oaei.ontologymatching.org/`), a yearly campaign that evaluates matching systems on common benchmarks. The reference book is Euzenat and Shvaiko, *Ontology Matching* (Springer).

Cheat sheet — ontology matching

- Matching: from ontologies (sets of entities) to relationships between their entities. Heterogeneity of languages, terminologies, modelling. Match, then process the alignment; at design time or run time.
- Correspondence $\langle id, e, e', R, n \rangle$: identifier, two entities, relation $(=, \sqsupseteq, \perp)$,

confidence in $[0, 1]$. Alignment: set of correspondences + cardinality + metadata.
- Matching process: O, O', input alignment A, parameters, resources $\to A'$.
- Applications: ontology evolution, schema/catalog/data integration; P2P, agents, Web services, query answering.
- Techniques: element-level (string: prefix, suffix, edit distance; language: tokenisation, lemmatisation, elimination; resources: WordNet relations, glosses) and structure-level (taxonomy, tree: children/leaves, model: SAT, DL).
- Composition: sequential or parallel + aggregation; selection by thresholds, cardinality, optimisation, directionality. Systems: Cupid, Falcon-AO, OLA, S-Match. Benchmark: OAEI.

13.5 Applications of Ontologies

[Slides · Ontology development: 77–104]

13.5.1 Some research from the course's group

[Slides · Ontology development: 77–83]

The slides list research papers from the University of Genoa group (with Viviana Mascardi), showing that ontologies are an active research topic, often combined with software agents:

year	paper (topic)
2010	*Automatic Ontology Matching via Upper Ontologies: A Systematic Evaluation*: using upper ontologies (SUMO-OWL, OpenCyc, DOLCE) as a semantic bridge in ontology matching (Section 13.4); a "mixed" method that combines structural matching with upper ontologies improves the recall
2012	*Semantic Web and Declarative Agent Languages and Technologies: Current and Future Trends* (position paper)
2014	*CooL-AgentSpeak*: agents that exchange plans and use ontology services (ontology matching to find relevant plans of other agents)
2017	*Agent-oriented and ontology-driven digital libraries: the IndianaMAS experience*: a multi-agent system that classifies documents of a cultural heritage domain formalised by an ontology
2020	*OntoScene, a Logic-based Scene Interpreter*: ontologies and Prolog to understand visual scenes, applied to rock art
2023	*Integrating Ontologies and Cognitive Conversational Agents in On2Conv*
2024	*Integrating Procedural Ontologies in VEsNA*: procedural ontologies for BDI agents

13.5.2 Application areas

[Slides · Ontology development: 84–104]

The application part integrates material from a tutorial by Ian Horrocks.

- **e-Science, geoscience**: mediating geological maps globally, integrating multi-source geodata, common schemas for water resources. The challenges of an "ontology spectrum" for geological data interoperability: modelling vs. encoding, multilinguality, flexibility and usefulness, mediation and evolution. Resources: geoscience-semantics ontologies, the W3C Geo vocabulary, the GeoNames ontology; also semantic enhancement of volunteered geographic information (user-generated tags on maps, a PhD project in Genoa).
- **e-Science, bioinformatics**: used for "in silico" investigations, which relate theory and data, e.g. data on plant structure and functionalities (the **Plant Ontology**, PO). Resources: the OBO Foundry (`http://www.obofoundry.org/`), books on ontologies for bioinformatics and anatomy ontologies.
- **Medicine**: building and maintaining terminologies such as **SNOMED**, **NCI** and **GALEN**. Examples: the brain anatomy in GALEN, where a reasoner checks definitions such as which gyri are bounded by which sulci; the **Disease Ontology** (DOID), e.g. *viral infectious disease* defined as "a disease by infectious agent that results in infection, has material basis in some Viruses"; the ICPC-2 ontology for primary care; an ontological framework for a healthcare research cluster.
- **Organising complex and semi-structured information**: UN-FAO (the *geopolitical ontology*, a master reference of country names and codes in several languages), NASA (**SWEET**, Semantic Web for Earth and Environmental Terminology), Ordnance Survey, General Motors, Lockheed Martin, ...; Genoa projects: ontology-driven management of digital rock art objects (IndianaMAS), *MUSE: MUltilinguality and SEmantics for the citizens of the world*, *Ontologica*: ontologies and natural language for railway management.
- **Military, government, legal domains**: DARPA, NSA, NIST, SAIC, the UK Ministry of Defence, the US Department of Homeland Security, ...; e.g. ontologies for e-government and **LKIF-Core**, a core ontology of legal concepts.
- **The Semantic Web** (Section 12.4).

Why this matters for NLP

In all these areas the problem is the same as in Section 12.1: many sources and many people describe the same things differently. An ontology gives them a shared, machine-readable vocabulary; ontology learning helps build and populate it from text; ontology matching connects it with the ontologies of others.

Cheat sheet — applications

- e-Science: geoscience (data interoperability), bioinformatics (Plant Ontology, OBO Foundry).

- Medicine: SNOMED, NCI, GALEN; Disease Ontology.
- Organising complex information: UN-FAO geopolitical ontology, NASA SWEET, cultural heritage, railways.
- Military, government, legal (LKIF-Core); the Semantic Web.

14 Text Features

[Slides · Text features: 1–23; Blackboard: features]

Most NLP tasks that classify texts (by topic, by author, by sentiment, human or bot, ...) need to turn a text into numbers first. Each number is a **feature**: a measurable property of the text, such as the percentage of pronouns, the TF-IDF weight of a word, the number of emotion words. The course map (Section 3.3) listed the most common ones; this section explains them and, above all, *when* to use which.

Definition — Feature

A **feature** of a text is a number (or a symbol) computed from the text that describes one of its properties. A text is represented by a **feature vector** $(f_1, f_2, \ldots, f_k)$, which is the input of a classifier or of another algorithm. Some features belong to a *single word* (being a function word), some to a *word in its sentence* (its POS), some to the *whole document* (the percentage of pronouns), and some need a whole *corpus* (TF-IDF).

This section covers:

1. the families of features: stylistic, content-based, content and style together (LIWC), vocabulary and idiosyncrasies, graph-based (Section 14.1);
2. which features to use for which goal, with the summary table written on the blackboard (Section 14.2).

14.1 Families of Features

[Slides · Text features: 2–13]

14.1.1 Stylistic features

[Slides · Text features: 2–8]

Stylistic features describe *how* something is written, not *what* it is about: sentence length, punctuation, the proportion of pronouns or adjectives. Two people writing about the same topic use different styles, and the same person writing about different topics keeps much of their style. This is why stylistic features are the basis of **author profiling** (guessing the age, gender, native language, personality of the author) and **authorship attribution** (who wrote this text?).

The slides show the 29 stylistic features used by an author profiling system for four languages:

No.	feature	all 4	No.	feature	all 4
1	% question sentences	✓	16	% words with 3+ syllables	
2	% short sentences	✓	17	average syllables per word	
3	% long sentences	✓	18	% adjectives	
4	average sentence length	✓	19	% adverbs	
5	average word length	✓	20	% capitals	✓
6	% words with 6+ letters	✓	21	% colons	✓
7	% words with 2–3 letters	✓	22	% determiners	
8	% semicolons	✓	23	% digits	✓
9	% punctuation	✓	24	% full stops	✓
10	% pronouns		25	% interjections	
11	% prepositions		26	% modals	
12	% coordinating conjunctions		27	% nouns	
13	% commas	✓	28	% personal pronouns	
14	% articles		29	% verbs	
15	% words with one syllable				

Table 21: The stylistic features of an author profiling system (from a paper cited in the slides). All 29 are used for English; "all 4" marks those used also for Dutch, Spanish and Italian. The others need a POS tagger or a syllable counter, which the system had only for English.

Intuition — why only some features in every language?

Features 1–9, 13, 20, 21, 23, 24 can be computed by simply counting characters, words and sentences: they work in any language. Features like "% adjectives" need a POS tagger (Section 7.2) and "syllables per word" needs a syllable counter, which are language-specific tools. A recurring lesson: the cheapest features are the most portable.

N-grams and collocations. The co-occurrence of the same n characters or words: character n-grams (*ing_*, *_th*) capture spelling and morphology habits, word n-grams and collocations (*strong tea*, Section 4.3) capture habitual phrasing.

POS (part of speech). Playing a particular role ("part") in a sentence ("speech") is a feature of *one word inside a sentence*: no need for the full document or corpus. The *number* of words with a given POS (number of pronouns, of adjectives, . . .), possibly normalised by the length of the document, is a feature of the *document*.

FW (function words).

Definition — Function word

In linguistics, **function words** (also called *functors*) are words that have little lexical meaning, or an ambiguous meaning, and express grammatical relationships among other words within a sentence, or specify the attitude or mood of the

speaker.

Being a function word is a feature of the *word itself*: if the word is not ambiguous, we do not even need the sentence it belongs to. The *number* of function words, possibly normalised by the document length, is a feature of the document. Kinds of function words:

- **articles**: *the*, *a* (in some inflected languages the article takes the case of the following noun);
- **pronouns**: inflected in English (*he–him*, *she–her*);
- **conjunctions**: *and* (uninflected in English);
- **subordinating conjunctions**: *if*, *then*, *well*, *however*, *thus*;
- **auxiliary verbs**: part of the conjugation of main verbs, always inflected (*has* gone, *will* go);
- **particles**: *up*, *on*, *down* (give *up*);
- **interjections**: sometimes called "filled pauses" (*uh*, *um*);
- **expletives**: words that take the place of a sentence, among other functions;
- **pro-sentences**: *yes*, *okay*.

There is a **huge overlap with stop words** (Section 6.2): the word cloud on the slide, of a novel, is dominated by *the*, *and*, *to*, *of*, *a*, *was*, *her*, *he*.

Watch out — stop words are not always noise

For topic classification we *remove* stop words because they say nothing about the topic. For author profiling we *count* them, because *how often* an author uses *the*, *of*, *I*, *she* is one of the best clues to who the author is. The same words are noise for one task and signal for another.

Readability. **Readability** measures how easy a piece of text is to read. It can include elements of complexity, familiarity, legibility and typography. Readability formulas usually look at sentence length, syllable density and word familiarity; an example is the Flesch–Kincaid grade level of Section 3.3. Online tools (e.g. `https://readable.com`) compute many such scores.

14.1.2 Content-based features

[Slides · Text features: 9]

Content-based features describe *what* the text is about:

- the frequency of words belonging to some given, known **semantic category** (dictionary-based, topic-based): e.g. how many words are about food, sport, money;
- **emotion words** (based on Ekman's or Plutchik's basic emotions, Section 3.3);

- **TF-IDF** (Section 6.2). Recall:

$$\text{tf-idf}(t, d, C) = \text{tf}(t, d) \cdot \text{idf}(t, C),$$
$$\text{tf}(t, d) = \frac{\#\, t \text{ in } d}{\text{length}(d)}, \qquad \text{idf}(t, C) = \log \frac{|C|}{|\{d \in C : t \in d\}|};$$

- **LSA** (Latent Semantic Analysis, Section 8.4): dense vectors of the words or of the document.

Watch out — LSA and word2vec

The slide writes "LSA, implemented by word2vec". Strictly, they are two different methods that produce the same kind of object, a dense vector per word: LSA reduces a word–document matrix with SVD (Section 8.4), word2vec trains a small neural network to predict words from their contexts. Both are distributional, and both can be used as content features.

14.1.3 Content- and style-based features: LIWC

[Slides · Text features: 10–11]

LIWC (Linguistic Inquiry and Word Count) is a tool with hand-built dictionaries of psychologically meaningful word categories. It counts, for a text, the percentage of words in each category: some categories are about style (I-words, function words), others about content (social words, emotions, cognitive processes). It then combines them into summary variables.

Worked example — LIWC on a technical paragraph

The slides feed LIWC a paragraph from a computer science paper (about the execution time of some reasoners analysing log files). The result, compared with the average for professional or scientific writing:

LIWC dimension	this text	average (scientific writing)
I-words (I, me, my)	0.0	0.63
social words	0.0	7.62
positive emotions	0.0	2.32
negative emotions	1.1	1.45
cognitive processes	17.0	7.52
summary variables		
analytic	99.0	92.57
clout	37.5	68.17
authenticity	65.6	24.84
emotional tone	11.9	43.61

Reading it: the paragraph is extremely *analytic* (99), full of cognitive-process words (*because*, *depends*, *except*: 17% vs. 7.5%), impersonal (no I-words, no social words) and emotionally flat (tone 11.9). This is what we expect from a technical text, and LIWC finds it by counting words alone.

14.1.4 Vocabulary and idiosyncrasies

[Slides · Text features: 12]

Features that capture personal or informal writing habits:

- **unique words**: the number of distinct words in the sample (vocabulary richness);
- **capital words**: the number of words (longer than 2) written only in capital letters (SHOUTING);
- **spelling errors**: the number of words not found in the dictionaries of the language;
- **character flooding**: *sooooo saaaad...*;
- **neologisms**: newly invented words;
- **slang**: *tanta roba!* (Italian slang, "great stuff!");
- **emoticons**: `>(`.

14.1.5 Graph-based features

[Slides · Text features: 13]

A text can also be turned into a **graph**, and the properties of the graph used as features. The slide shows an example of **EmoGraph** for a Spanish sentence (*He estado tomando cursos en línea sobre temas valiosos que disfruto estudiando y que podrían ayudarme a hablar en público*, "I have been taking online courses about valuable subjects that I enjoy studying and that might help me to speak in public"):

- the nodes are the **POS tags** of the words (detailed tags such as `NCMP000`, common noun masculine plural, or `VMIP1S0`, main verb indicative present first person singular) and **semantic and emotional labels** of the words (*education*, *language*, *joy*, *positive*, ...);
- the edges follow the order of the words in the text: an edge from a tag to the next one, and from a word's tag to its semantic/emotional labels;
- graph measures (which nodes are most central, how connected the graph is, ...) become the features. They capture *how* a person combines grammar and emotions, a stylistic fingerprint.

Cheat sheet — families of features

- Stylistic: n-grams/collocations, POS counts, function words (huge overlap with stop words), readability, sentence/word length, punctuation.
- Content-based: semantic categories, emotion words, TF-IDF, LSA.
- Content + style: LIWC (psychological word categories + summary variables).
- Vocabulary and idiosyncrasies: unique words, capitals, spelling errors, flooding, neologisms, slang, emoticons.
- Graph-based: e.g. EmoGraph (POS + emotion labels as a graph).
- Scope of a feature: the word (FW), the word in its sentence (POS), the

document (counts), the corpus (TF-IDF).

14.2 Which Features for Which Goal

[Slides · Text features: 14–23; Blackboard: features]

14.2.1 Content, FW, POS: when, why?

[Slides · Text features: 14–22]

"Well… it depends on what your goal is."

Topic classification: content matters. If you need to classify a document into categories such as "music", "biology", "food" (three examples of BabelNet categories), you probably do not care about function words and POS: what matters are the content words. Two texts from the slides:

- *"Baked sea bass with lemon caper dressing. To make the dressing, mix the oil with the lemon zest and juice, capers, mustard, …"* (a recipe: food);
- *"These Russian basses sing so low you'll think the world is ending. … the sound of a Russian oktavist, singing a full octave below a normal bass…"* (music).

Both contain *bass*, which in BabelNet has several senses: *bass, basso* (an adult male singer with the lowest voice), *sea bass* (the lean flesh of a saltwater fish), *freshwater bass* (a fish), *bass voice* (the lowest adult male singing voice). The content words around it (*baked*, *lemon*, *capers* vs. *sing*, *octave*) both pick the right sense (word sense disambiguation, Section 10.3) and the right topic. Counting pronouns or articles would not help at all.

Author profiling: function words and POS matter. But if you are profiling the *author* of a document…

Key result — Argamon, Koppel, Pennebaker and Schler (2009)

"However, the preponderance of evidence suggests that the most consistently effective features over a wide variety of authorship problems are **function words** and individual **parts-of-speech**."

S. Argamon, M. Koppel, J. W. Pennebaker, J. Schler, *Automatically profiling the author of an anonymous text*, Communications of the ACM 52(2), 2009.

Worked example — female vs. male

The slides show two abstracts of linguistics papers, one by a female and one by a male author, about similar topics (stylistic analysis, rhetorical devices). First the **personal pronouns** are highlighted (*my*, *I*, *he*, *their*), then the preposition **of**. The point: the topic of the two texts is the same, so content features cannot tell

the authors apart, but the *rates* of such small words can. In the large studies by Argamon, Koppel and colleagues, texts by women use more personal pronouns, texts by men more determiners, numbers and prepositions such as *of* (words that specify nouns).

14.2.2 So, what?

[Slides · Text features: 23]

- There is a possibly endless amount of features. Many are standard and well studied, and it is also clear which NLP tasks they suit best; others (EmoGraph, for example) are less standard.
- Anyone can use existing features, already implemented by existing tools.
- The real challenge is to **invent new features**, or well-calibrated combinations of existing ones, and to **demonstrate** that they are useful for some given NLP task, on some existing benchmark.

14.2.3 The blackboard summary

[Slides · Blackboard: features]

In class the lecturer summarised which features are useful for which tasks in a table on the blackboard. On the board the features are the *columns*; here the table is transposed (one feature per row) so that it fits the page. Nothing else is changed.

BLACKBOARD 2 — Which feature is useful for which task

[Blackboard photo · Blackboard: features]

feature	useful for	useless or not applicable for
Function words	author profiling: • gender of [the author] / pron[ouns]; • one specific author	topic classification or filtering
TF-IDF	topic classification	not applicable without a corpus; WSD
POS	author profiling; genre profiling; readability	*(empty)*
Period / stylistic features	Bob or bot?; human author profiling; document genre profiling	*(empty)*
Emotions	sentiment analysis; affective computing; legal cases; fake news	*(empty)*
Length of words	*(empty)*	*(empty)*
Variability of words	readability	*(empty)*
Slang, flooding, jargon	*(empty)*	*(empty)*

On the board a double arrow links the headings "Period / stylistic features" and "Length of words" (they belong together). In the first cell the text after "gender of" is partly unreadable; brackets mark what is completed from context.

Reading the blackboard, row by row.

- **Function words**. Useful for **author profiling**: to guess the *gender* of the author (through the use of pronouns, as in the female vs. male example) or to recognise *one specific author* (authorship attribution: everybody has a personal rate of *the*, *of*, *however*). Useless for **topic classification or filtering**: function words are the same in every topic, which is why they are removed as stop words.
- **TF-IDF**. Useful for **topic classification**: the words with high TF-IDF in a document are the ones that characterise its topic. **Not applicable without a corpus**: the idf needs a collection of documents C; with a single text we cannot compute it. Useless for **WSD**: TF-IDF weighs a word for a whole document, it says nothing about which sense one occurrence has.
- **POS**. Useful for **author profiling** (Argamon et al.), **genre profiling** (a scientific paper has more nouns, a novel more verbs and pronouns, a recipe many imperatives) and **readability** (many nouns and long noun phrases make a text harder).

- **Period / stylistic features**. "Period" here means the sentence (Italian *periodo*): sentence length, punctuation, question sentences, the stylistic features of Table 21. Useful to tell **a human (Bob) from a bot**, for **human author profiling**, and for **document genre profiling**.
- **Emotions**. Emotion words are useful for **sentiment analysis** (positive or negative opinion), **affective computing** (systems that recognise and react to human emotions), **legal cases** (e.g. the emotional load of testimonies or of threatening messages) and **fake news** (fake news often uses a strongly emotional, alarming language).
- **Length of words**. No task written on the board; it is linked by the arrow to the stylistic features (word length is one of them, features 5–7 of the stylistic table) and enters readability formulas.
- **Variability of words** (vocabulary richness, the number of unique words). Useful for **readability**: a text that uses many different and rare words is harder to read.
- **Slang, flooding, jargon**: no task written on the board; from the previous subsection they are idiosyncrasies of informal writers (useful for author profiling, e.g. age, and for bot detection).

Intuition — the rule behind the table

Ask: *does the task depend on what is said, or on who says it and how?*

- **What** is said (topic, filtering): use *content* features (TF-IDF, semantic categories); remove function words.
- **Who** says it and **how** (author, gender, human vs. bot, genre, readability): use *style* features (function words, POS, sentence and word length, vocabulary richness, idiosyncrasies).
- **How the author feels** (sentiment, fake news, affective computing): use *emotion* features.

Cheat sheet — choosing features

- Topic classification: content words (TF-IDF); function words and POS useless.
- Author profiling: function words and POS are the most consistently effective features (Argamon et al., 2009); female vs. male: pronouns vs. noun specifiers.
- TF-IDF needs a corpus; useless for WSD.
- Stylistic (sentence-level) features: bot detection, author and genre profiling.
- Emotions: sentiment analysis, affective computing, legal cases, fake news.
- Variability of words, POS: readability.
- The challenge: invent new features and prove they help on a benchmark.

15 Pragmatics

[Slides · Pragmatics: 1–66]

We reach the third level of linguistic analysis (Section 3.1): after *what is grammatical?* (syntax) and *what does it mean?* (semantics), **pragmatics** asks *what does it do?* What does the speaker want to achieve by saying it, and how does the context change what is understood? The first slide sets the tone with a comic in which a character believes he has "decoded" what *yes*, *maybe* and *no* really mean: the joke works because what is *said* and what is *meant* can differ, which is exactly the subject of pragmatics.

This section covers:

1. what pragmatics is (Section 15.1);
2. communication as action: Wittgenstein, Austin, Searle, speech acts and the stages of communication (Section 15.2);
3. whether language shapes thought: Newspeak, the Sapir–Whorf hypothesis and the experiments presented by Lera Boroditsky (Section 15.3);
4. the language of numbers and the power of names (Section 15.4).

15.1 What Is Pragmatics

[Slides · Pragmatics: 1–3]

Definition — Pragmatics

Pragmatics is a subfield of linguistics and semiotics that studies the ways in which **context contributes to meaning**. It encompasses speech act theory, conversational implicature, talk in interaction and other approaches to language behaviour in philosophy, sociology, linguistics and anthropology.
Pragmatics studies how the transmission of meaning depends not only on the structural and linguistic knowledge (grammar, lexicon, . . .) of speaker and listener, but also on

- the **context of the utterance**;
- any **pre-existing knowledge** about those involved;
- the **inferred intent** of the speaker;
- and other factors.

Semiotics is the study of meaning-making: the study of sign processes (*semiosis*) and of meaningful communication. Words are one kind of sign; gestures, pictures, traffic lights are others.

Worked example — same words, different meanings

"Can you pass the salt?" Syntactically it is a question; semantically it asks about an ability (are you able to pass the salt?). Pragmatically, at a dinner table, it is a *request*: answering "yes" without moving would be absurd. "It's cold in here"

can be a statement, a complaint, or a request to close the window, depending on who says it to whom, and where. Only the context tells.

Why this matters for NLP

A chatbot that understands every word but not the intention behind it answers "Yes, I can" to "Can you tell me the time?". Recognising intentions (*intent detection*) is a central task of dialogue systems, and the reason why the agent communication languages of Section 2.5 label every message with the act it performs.

15.2 Communication as Action

[Slides · Pragmatics: 4–10]

These slides come from another course (on agents) and give a very quick introduction to "communication as an action".

15.2.1 From true/false sentences to actions

[Slides · Pragmatics: 5–8]

Definition — Communication

Communication, in general, is the *intentional* exchange of information brought about by the production and perception of *signs* drawn from a *shared system of conventional signs*.

Two views of language:

- the **"classical" view** (before 1953): language consists of sentences that are **true or false** (as in logic; this is the view behind model-theoretic semantics, Section 9.3);
- the **"modern" view** (after 1953): **language is a form of action**.

The turning point is marked by three philosophers.

Wittgenstein (1953), *Philosophical Investigations*. According to Wittgenstein:

- the structure of reality determines the structure of language;
- **meaning is determined by use**, in the context of a **"language-game"**;
- words are not defined by reference to the objects or things they designate in the external world, nor by the thoughts, ideas or mental representations one might associate with them, but by **how they are used** in effective, ordinary communication.

Intuition — language-games

Think of the word *"Slab!"* shouted by a builder to an assistant: it does not describe a slab, it means "bring me a slab", and it means that only inside the "game" of building together. Every use of language is a move in some game with its own rules: giving orders, asking, telling a joke, praying. This is the same Wittgenstein of "meaning is use" at the root of distributional semantics (Section 8.1).

Watch out — two Wittgensteins

The first bullet ("the structure of reality determines the structure of language") is the idea of the *early* Wittgenstein (*Tractatus*, 1921), in which a sentence is a picture of a fact. The *Philosophical Investigations* (1953) largely abandon it in favour of meaning as use. The slide lists both; remember that the 1953 work is the one of language-games.

Austin (1962), *How to Do Things with Words*. In saying something, we do something: when a minister joins two people in marriage saying "I now pronounce you husband and wife", the sentence does not describe a marriage, it *makes* it (Section 3.3). Such sentences are **performatives**.

Searle (1969), *Speech Act Theory*.

- Each **illocution** (the act performed *in* saying something) can be described in terms of what it is attempting to do. An *assertion* counts as an undertaking that something really is the case; a *question* counts as an attempt to elicit some information; *thanking* someone counts as an expression of gratitude.
- What matters is the assumed **intent of the speaker**, the **intentionality** of the sentence.

[Not in the slides: added material]

Definition — Speech act: three levels (Austin)

- **Locution**: the act of saying something, the words with their literal meaning ("It's cold in here").
- **Illocution**: the act performed *in* saying it, the speaker's intention (a request to close the window).
- **Perlocution**: the effect produced *by* saying it on the hearer (the hearer gets up and closes the window).

15.2.2 Speech acts and the stages of communication

[Slides · Pragmatics: 9–10]



Figure 39: A speech act: a speaker produces an utterance for a hearer, inside a situation.

Speech acts achieve the speaker's goals. The examples come from the Wumpus world, the cave game used for the grammar of Section 7.3:

act	example
Inform	"There's a pit in front of you"
Query	"Can you see the gold?"
Command	"Pick it up"
Promise	"I'll share the gold with you"
Acknowledge	"OK"

Speech act planning (deciding what to say to reach a goal) requires knowledge of the *situation*, of the *semantic and syntactic conventions*, and of the *hearer's goals, knowledge base and rationality*.

	stage	what happens
speaker S	Intention	S wants to inform H that P
	Generation	S selects words W to express P in context C
	Synthesis	S utters words W
hearer H	Perception	H perceives W' in context C'
	Analysis	H infers possible meanings $P_1, \ldots, P_n$
	Disambiguation	H infers the intended meaning P_i
	Incorporation	H incorporates P_i into its knowledge base

Table 22: The stages in communication.

Intuition — where things can go wrong

Notice the primes: the hearer perceives W' in context C', not necessarily the same W and C of the speaker (noise, mishearing, a different idea of the situation). The analysis gives *several* possible meanings, and disambiguation picks one, maybe the wrong one. Every stage maps onto a part of NLP: generation and synthesis are natural language generation and speech synthesis; perception is speech recognition or reading; analysis is syntax and semantics (parsing, Sections 7 to 10); disambiguation is WSD and intent recognition; incorporation is updating a knowledge base, e.g. an ontology (Section 12).

15.3 Does Language Shape Thought?

[Slides · Pragmatics: 11–49]

"Thoughts shape language... but... does language shape thought?" If pragmatics studies how context and intentions shape what we mean, the reverse question is natural: does the language we speak shape how we think?

15.3.1 Newspeak and the Sapir–Whorf hypothesis

[Slides · Pragmatics: 13–16]

In George Orwell's novel *Nineteen Eighty-Four* (1949), the regime designs a new language, called **Newspeak**, to control thought:

> "Don't you see that the whole aim of Newspeak is to narrow the range of thought? In the end we shall make thoughtcrime literally impossible, because there will be no words in which to express it. Every concept that can ever be needed will be expressed by exactly one word, with its meaning rigidly defined and all its subsidiary meanings rubbed out and forgotten. [...] Every year fewer and fewer words, and the range of consciousness always a little smaller. [...] The Revolution will be complete when the language is perfect."

Definition — Linguistic relativity (Sapir–Whorf hypothesis)

The hypothesis of **linguistic relativity** (also **Sapir–Whorf hypothesis**, or Whorfianism, after Edward Sapir, 1884–1939, and Benjamin Lee Whorf, 1897–1941) holds that the structure of a language affects its speakers' world view or cognition. It has two versions:

- the **strong** version, **linguistic determinism**: language *determines* thought, linguistic categories limit and determine cognitive categories; language entirely determines the range of cognitive processes;
- the **weak** version, **linguistic influence**: linguistic categories and usage only *influence* thought and decisions; language provides constraints in some areas of cognition, but is by no means determinative.

Key result — what we know today

Linguistic **determinism** (the strong version) is now generally agreed to be **false**: we can think things for which our language has no word (otherwise new words could never be invented). Research on the **weaker** forms has produced **positive empirical evidence**: language does influence some areas of cognition. Newspeak is the strong version, and that is why it is fiction.

15.3.2 How languages and cultures shape the ways we think

[Slides · Pragmatics: 17–30]

The rest of this part follows a talk by **Lera Boroditsky** (Associate Professor of Cognitive Science at UC San Diego, previously at MIT and Stanford, who studies the relationships between mind, world and language), *How languages and cultures shape the ways we think*.

Languages force us to say different things. Take the English sentence *"Trump read Ta-Nehisi's latest book"*. Depending on the language, to say it you *must* decide:

- did this happen in the past? (tense: Mandarin does not require it, English does);
- Donald or Melania? (in Russian the past-tense verb changes with the gender of the subject);
- was the action completed? (aspect: Russian forces a choice);
- did you witness this with your own eyes? (evidentiality: in Turkish, for example, the verb marks whether you saw it or heard about it).

Similarly, *"the 7th blue house to the left of the big tree"*: seventh? blue, light or dark (Russian has no single word for "blue")? left of, or north-northwest of, or "volcano-ward" of (some languages use absolute directions)? and which tree: banyan, red cypress, eucalyptus? Each language makes its speakers pay attention to different aspects of the world, all the time.

How many languages in the world? According to Ethnologue: 7 168 languages in use (October 2023), 7 164 (October 2024), 7 159 (November 2025). The number is constantly in flux, because we keep learning about languages and because languages themselves change. Roughly **44%** of all languages are **endangered**, often with fewer than 1 000 users left, while the 20 largest languages are the native tongue of more than 3.6–3.7 billion people: about 0.3% of the languages account for nearly half of the world's population. Another source is the *World Atlas of Language Structures* (WALS), which describes the structural features of a sample of languages.

Navajo verbs of eating. Navajo has no single verb "to eat": the verb stem changes with the *kind* of food and the *manner* of eating.

	stem	**manner / food of consumption**	**example edibles**
i.	-yíih	consume/chew UNSPECIFIED	anything
ii.	-'aał	consume/chew HARD, COMPACT	corn, peyote, candy
iii.	-chozh	consume/chew LEAFY	lettuce, hay
iv.	-ghał	consume/chew MEAT	meat
v.	-kééd	consume/chew ROUND	bun, melon
vi.	-ts'ééh	consume/chew MUSHY MATTER	mush, jello
vii.	-deeł	consume/chew PLURAL OBJECT	berries, eggs
viii.	-wol	consume/chew MARROW	marrow
ix.	-jooł	consume NON-COMPACT MATTER	cotton candy
x.	-tłeeh	consume MUSHY MATTER	ice cream
xi.	-máá'	consume/devour by GULPING/BOLTING	single piece of food
xii.	-'ááh	consume SOLID, ROUND OBJECT dunked in liquid	moistened bread, cake
xiii.	-dlą	consume/drink UNSPECIFIED	soup, thin gruel
xiv.	-kaah	consume/drink from OPEN CONTAINER	milk from glass
xv.	-t'aah	consume/drink from CLOSED CONTAINER	milk from bottle

Table 23: Navajo verb stems of consumption (some diacritics simplified).

Learning languages. Quotes on the slides: "To have a second language is to have a second soul" (Charlemagne); "A man who knows four languages is worth four men" (Charles V); "I speak English to my accountants, French to my ambassadors, Italian to my mistress, Latin to my God, and German to my horse" (Frederick the Great). And the opposite camp: "I hate relativism more than I hate anything else, excepting, maybe, fiberglass powerboats" (**Jerry Fodor**, 1935–2017, one of the principal philosophers of mind of the late twentieth century, very influential on cognitive science, and a strong critic of linguistic relativity).

Why start from time? The most common nouns in English (lists exist also for Italian and French) are, in order: **time**, year, people, way, day. Time is the most talked-about thing, so it is a good place to look for differences between languages.

15.3.3 Space and time

[Slides · Pragmatics: 31–38]

We cannot see or touch time, so all languages talk about it using **space**; but they use space differently.

- **Writing direction**. English speakers arrange a temporal sequence (e.g. pictures of a growing bear, as in the old Nestlé logo) from *left to right*; speakers of Hebrew or Arabic, who write right to left, arrange it *right to left* (Tversky et al., 1991; Fuhrman & Boroditsky, 2010).
- **Front and back**. For English speakers the future is *ahead* and the past *behind*. In **Aymara** (Andes) the past is *in front* (it is known, you can "see" it) and the future *behind*: speakers gesture forward when talking about the past (Núñez & Sweetser, 2006).
- **Up and down**. In Mandarin, last week is the "up" week and next week the "down" week: time is also *vertical*. So time can be laid on three axes: **transverse** (left–right), **sagittal** (front–back), **vertical** (up–down) (Boroditsky et al., 2010; Fuhrman et al., 2011).
- **Absolute directions**. In some Aboriginal Australian communities (Kuuk Thaayorre, Pormpuraaw) people do not use "left" and "right" but cardinal directions ("there is an ant on your south-west leg"). Given cards showing a man at different ages, they arrange them **from east to west**, whichever way they are facing: left to right when facing south, towards their body when facing east (Boroditsky & Gaby, 2010).
- **Uphill and downhill**. The Yupno of Papua New Guinea place the past downhill and the future uphill, following the slope of their valley (Núñez et al., 2012).

Key result — is language the cause?

How do we know that language *causes* these differences, and does not only correlate with them (with culture, landscape, . . .)? **Teach people new metaphors for time** (e.g. English speakers trained to talk about time vertically) and see if this changes the way they think. Answer: **yes**.

15.3.4 Colour

[Slides · Pragmatics: 39–40]

The colour spectrum is continuous, but languages cut it into categories differently. English has one basic word, *blue*; Russian has two: **goluboy** (light blue) and **siniy** (dark blue). In an experiment (Winawer, Witthoft, Frank, Wade, Boroditsky, PNAS) Russian speakers were **faster at distinguishing two blues when they fell on different sides** of the goluboy/siniy boundary; English speakers showed no such advantage. The words we have change, measurably, how we perceive.

15.3.5 Grammatical gender

[Slides · Pragmatics: 41–44]

In German the sun is feminine (*die Sonne*) and the moon masculine (*der Mond*); in Spanish it is the opposite (*el sol*, *la luna*). Gender is not a small detail of a few words: it is **pervasive**. Russian examples with *okno* (window, neuter), *taburetka* (stool, feminine) and *stul* (chair, masculine):

where gender shows up	**example**
different phonology	okn**o** (-o, -e = neuter), taburetk**a** (-a = feminine), stu**l** (consonant = masculine)
different number words	**odna** taburetka vs. **odin** stul (one stool, one chair)
different adjective endings	bel**oye** okno, bel**aya** taburetka, bel**y** stul (white window/stool/chair)
different pronouns and possessives	**ono** beloye, **ona** belaya, **on** bely; **moyo** okno beloye, **moya** taburetka belaya, **moy** stul bely
different verb endings	moyo okno **bilo** beloye, moya taburetka **bila** belaya, moy stul **bil** bely (my . . . was white)

To say "my chair was white" a Russian speaker must mark the masculine gender of *chair* on the possessive, the verb and the adjective.

Do people really take these genders as meaningful? Yes: speakers use grammatical gender when they give voices to animated characters, when they describe objects, they underestimate hurricanes with female names, and artists personify abstract entities according to the gender of the word. Michelangelo's statues on the tomb of Giuliano de' Medici (1526–1531) and of Lorenzo de' Medici represent *Dawn*

(*l'aurora*) and *Night* (*la notte*) as women, *Day* (*il giorno*) and *Dusk* (*il crepuscolo*) as men: exactly the genders of the Italian words.

15.3.6 Agency: who did it?

[Slides · Pragmatics: 45–49]

"Pat broke the vase." English describes even accidents with an agent. Spanish distinguishes: *Pat rompió el florero* (Pat broke the vase, on purpose) vs. *Se rompió el florero* (the vase broke itself, i.e. by accident). Language primes the same contrast in English: *"The match blew out"* vs. *"She blew out the match"*; *"The toast burned"* vs. *"She burned the toast"*.

In the experiment, people watched videos of someone breaking things, intentionally or by accident, and were later asked **"who did it the first time?"**. For intentional events everybody remembers the agent equally well; for **accidental** events, English speakers remember the agent better than Spanish (or Japanese) speakers, whose language would normally describe the event without an agent. What a language makes you say influences what you notice and remember, and, as studies on agentive vs. non-agentive descriptions show, also how much blame you assign.

Cheat sheet — language and thought

- Newspeak (Orwell): language designed to make thoughts impossible.
- Sapir–Whorf: strong (determinism, false) vs. weak (influence, supported by evidence).
- Languages force different information: tense, gender, aspect, evidentiality; Navajo verbs of eating. About 7 160 languages, 44% endangered.
- Time in space: left–right by writing direction; Aymara past in front; Mandarin vertical; Kuuk Thaayorre east-to-west; Yupno uphill future. Teaching new metaphors changes thinking ⇒ causal.
- Colour: Russian goluboy/siniy, faster discrimination across the boundary.
- Gender: pervasive (Russian), influences descriptions, voices, art (Michelangelo).
- Agency: English agentive descriptions; better memory of agents of accidents.

15.4 The Language of Numbers and the Power of Names

[Slides · Pragmatics: 50–66]

15.4.1 A few words on maths

[Slides · Pragmatics: 50–59]

Galileo Galilei (1564–1642), in *Il Saggiatore*, called mathematics a language:

> "Philosophy is written in this grand book, which stands continually open before our eyes (I say the 'Universe'), but cannot be understood without first learning

> to comprehend the language and know the characters in which it is written. It is written in mathematical language, and its characters are triangles, circles and other geometric figures, without which it is impossible to humanly understand a word; without these one is wandering in a dark labyrinth."

If mathematics is a language, the way numbers are *written and named* should also matter. The slides follow its history:

- **Greek numerals**. The ancient Greeks first had a system like the Romans; from the 4th century BC they used different symbols for 1–9 (as we do), but *another* set for 10, 20, 30, … and yet another for 100, 200, 300: like many ancient systems, they eventually ran out of symbols. They had **no symbol for zero** (*mēdén*) and no digit placeholder: they were unsure whether zero was a number at all ("how can nothing be something?"). Yet with these numerals **Eratosthenes** of Cyrene (c. 276–194 BC), chief librarian of Alexandria, computed the circumference of the Earth with remarkable accuracy, invented geography as a discipline, and gave the *sieve of Eratosthenes* for finding prime numbers.
- **Roman numerals**. Seven symbols: I = 1, V = 5, X = 10, L = 50, C = 100, D = 500, M = 1000. They remained the usual way of writing numbers in Europe well into the Late Middle Ages and were replaced from the 14th century by the more convenient Hindu-Arabic numerals; they survive in minor uses today. The Romans nevertheless built aqueducts and roads that still stand.
- **Hindu-Arabic numerals**: ten symbols, 1, 2, …, 9, 0, for the decimal positional system. They originated in India in the 6th or 7th century and reached Europe through Middle Eastern mathematicians (especially al-Khwarizmi and al-Kindi) around the 12th century. They were a profound break with earlier methods of counting (such as the abacus) and paved the way for algebra.
- **Counting with the body**. The Oksapmin of Papua New Guinea count with a system of 27 body parts: from the thumb of one hand, up the arm, across the head and down the other arm; the number is the name of the body part reached.
- **Languages with few number words**. Some Amazonian languages have no words for exact numbers beyond very small ones. In matching tasks (put on the table as many objects as you see in a row), speakers succeed with small quantities but become approximate with larger ones: without number words, exact large quantities are hard to keep in mind.

Intuition — the message

Notation and number words are a language, and some languages make some thoughts easier: long multiplication is easy in positional decimal notation and painful with Roman numerals; exact counting is hard without number words. But the Greeks and Romans did great mathematics and engineering with clumsy numerals. Again: language *influences* thought (weak Whorf), it does not *deter-*

mine it.

15.4.2 The importance of what a thing is called

[Slides · Pragmatics: 60–66]

The same thing can be named in ways that push the hearer to judge it differently:

- *patriots*, *activists*, or *terrorists*?
- *torture* or *enhanced interrogation techniques*?

But sometimes language seems ineffective. Why? In 2003 the cafeterias of the US House of Representatives renamed French fries **"freedom fries"** and French toast "freedom toast", a symbolic protest against France's opposition to the war in Iraq (BBC News, 12 March 2003). It had happened before: during World War I sauerkraut became "liberty cabbage", liverwurst "liberty sausage", hamburger "Salisbury steak", dachshunds "liberty pups". In 2003 the joke list went on: freedom poodles, freedom kissing, freedom manicures, and "France" itself would become "freedom land", "French" "the language of freedom". These renamings did not change much: everybody knew what the new names stood for, so the old meaning (and the old attitude) remained.

And sometimes it works. Prunes had an image problem (old people, laxatives). Producers renamed them **dried plums**, and sales went up: "Folks favor dried plums over prunes", with sales up 5.5% over the previous year, according to the state marketing board (Business Journal, 2001). Many other examples from all around the world concern the names of products, and the classic marketing mistakes when a name has an unwanted meaning in another language.

Why this matters for NLP

Naming is pragmatics in action: choosing a word is choosing what the hearer will notice (an agent or not, an act of violence or a technique, an old person's remedy or a fruit). NLP systems meet this every day: sentiment and stance detection must recognise that *terrorist* and *freedom fighter* may refer to the same person with opposite attitudes, and text generation systems choose words with a pragmatic effect, wanted or not.

The deck ends with Nelson Mandela:

> "If you talk to a man in a language he understands, that goes to his head.
> If you talk to him in his language, that goes to his heart."

Cheat sheet — pragmatics

- Pragmatics: how context (situation, pre-existing knowledge, inferred intent) contributes to meaning. Semiotics: the study of signs and meaning-making.
- Communication: intentional exchange of information through a shared sys-

tem of signs. Classical view (sentences are true/false) vs. modern view (language is action).
- Wittgenstein (1953): meaning is use in a language-game. Austin (1962): in saying something we do something (performatives). Searle (1969): speech acts, illocutions, intentionality.
- Speech acts: inform, query, command, promise, acknowledge. Stages: intention, generation, synthesis (speaker); perception, analysis, disambiguation, incorporation (hearer).
- Language and thought: Sapir–Whorf strong (false) vs. weak (supported): time/space, colour, gender, agency, numbers.
- Names matter: freedom fries (ineffective), dried plums (effective).

16 Language Models and RNNs

[Slides · Language models & RNNs: 1–101]

Until now we have studied language mostly by *analysing* it: splitting it into words, tagging them, parsing sentences, representing meanings. This section takes a different point of view, the one behind every modern NLP system and every chatbot: we build a **model that can predict language**. Given the beginning of a sentence, which word comes next? Given a whole sentence, how likely is it?

A model that answers these questions is called a **language model** (LM). GPT and the other Large Language Models are language models; they are just very large and built with a more advanced architecture (the transformer). To understand them we start from the beginning: what a language model is, the simplest way to build one by counting (n-grams), how neural networks improve on counting, and how **recurrent neural networks** (RNNs) read text of any length, together with the problems that eventually led to their replacement.

No previous knowledge of neural networks is assumed: every tool (probability chain rule, softmax, cross-entropy, gradient descent, backpropagation) is introduced when it is first needed.

This section covers:

1. what a language model is and why it is useful (Section 16.1);
2. classical language models built by counting: n-grams, their sparsity and storage problems, and text generation (Section 16.2);
3. neural language models with a fixed window, word embeddings, the softmax and the cross-entropy loss (Section 16.3);
4. recurrent neural networks: architecture, training, backpropagation through time, text generation and evaluation with perplexity (Section 16.4);
5. the problems of RNNs: vanishing and exploding gradients, gradient clipping and the ideas that solve them (Section 16.5).

16.1 What Is a Language Model?

[Slides · Language models & RNNs: 2–11]

16.1.1 Predicting and evaluating language

[Slides · Language models & RNNs: 3–6]

A **language model** is a mathematical model that describes human language. It should let us do two things:

- **predict** language: continue a text that has been started;
- **evaluate** language: say how "natural" or likely a piece of text is.

How can we build such a model? Two facts make it hard to write it down as a fixed set of rules:

- **language is ambiguous**: the same beginning can be continued in many reasonable ways ("she went to . . . " the shop? work? bed?);
- **language is evolving**: new words and new ways of saying things appear all the time, so any fixed list of rules goes out of date.

Both facts push in the same direction: instead of saying *which* continuation is correct, we say *how likely* each continuation is. We therefore use **probabilistic models of language**.

16.1.2 Next-word prediction

[Slides · Language models & RNNs: 7–9]

The fundamental goal of a language model is **prediction**, and the simplest form of prediction is guessing the **next word**. We number the words of a sentence by their position: $\mathbf{x}^{(1)}$ is the first word, $\mathbf{x}^{(2)}$ the second, and so on (the superscript (t) is a position, "time step t", not a power).

Anna	was	tired	so	she	went	to	?
$\mathbf{x}^{(1)}$	$\mathbf{x}^{(2)}$	$\mathbf{x}^{(3)}$	$\mathbf{x}^{(4)}$	$\mathbf{x}^{(5)}$	$\mathbf{x}^{(6)}$	$\mathbf{x}^{(7)}$	$\mathbf{x}^{(8)}$

Figure 40: Next-word prediction: having read the first seven words, the model must say which word is likely to come in position 8.

Given the words $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots, \mathbf{x}^{(t)}$ read so far, the model computes the **probability distribution of the next word** $\mathbf{x}^{(t+1)}$:

$$p\big(\mathbf{x}^{(t+1)} \mid \mathbf{x}^{(t)}, \ldots, \mathbf{x}^{(1)}\big).$$

[Not in the slides: added material]

A few words on this notation, since it is used everywhere in this section.

- The **vocabulary** V is the set of all words the model knows, and $|V|$ is its size (tens of thousands of words for a real model).
- A **probability distribution over the next word** is a list of $|V|$ numbers, one per word of the vocabulary, each between 0 and 1, which add up to 1. For example, after "she went to" a model could give *the*: 0.5, *work*: 0.25, *school*: 0.2, *bed*: 0.05, and 0 to all other words.
- The vertical bar "|" reads "given": $p(A \mid B)$ is the probability of A *knowing that* B happened. What is written after the bar is called the **context** or the **conditioning**.

Language models are part of daily life: the slide shows a search engine completing a query while it is being typed (each suggestion is a likely continuation of the characters typed so far) and a chat assistant writing an answer word by word.

16.1.3 Evaluating a text: the chain rule

[Slides · Language models & RNNs: 10]

An equivalent goal is to **evaluate** a text, i.e. assign it a probability. Given the words $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}$ (T is the length of the text), we want $p(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)})$: the probability that a text is exactly this sequence of words. Why is this "equivalent" to next-word prediction? Because of the **chain rule of probability**.

The starting point is the definition of conditional probability:

$$p(A, B) = p(A \mid B)\, p(B),$$

"the probability that both A and B happen is the probability of B, times the probability of A once we know B happened". Applying it again and again we can peel a sentence one word at a time, from the end. For three words:

$$\begin{aligned} p(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}) &= p(\mathbf{x}^{(3)} \mid \mathbf{x}^{(2)}, \mathbf{x}^{(1)})\; p(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}) && \triangleleft\ A = \mathbf{x}^{(3)},\ B = (\mathbf{x}^{(1)}, \mathbf{x}^{(2)}) \\ &= p(\mathbf{x}^{(3)} \mid \mathbf{x}^{(2)}, \mathbf{x}^{(1)})\; p(\mathbf{x}^{(2)} \mid \mathbf{x}^{(1)})\; p(\mathbf{x}^{(1)}) && \triangleleft\ \text{again, on } p(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}) \end{aligned}$$

Reading the last line from right to left: probability of the first word, times the probability of the second word given the first, times the probability of the third given the first two. In general:

Key result — Chain rule for a text

$$\begin{aligned} p(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}) &= p(\mathbf{x}^{(1)}) \times p(\mathbf{x}^{(2)} \mid \mathbf{x}^{(1)}) \times \cdots \times p(\mathbf{x}^{(T)} \mid \mathbf{x}^{(T-1)}, \dots, \mathbf{x}^{(1)}) \\ &= \prod_{t=1}^{T} \underbrace{p(\mathbf{x}^{(t)} \mid \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(1)})}_{\text{next-word prediction!}} \end{aligned}$$

($\prod$ is the product symbol, the multiplicative analogue of $\sum$; for $t = 1$ the context is empty and the factor is just $p(\mathbf{x}^{(1)})$.)

So the two goals are really one: a model that can predict the next word can evaluate any text (multiply the next-word probabilities of all its words), and a model that can evaluate texts can predict the next word (compare the probabilities of the possible continuations). This identity is exact: no approximation has been made yet.

Worked example — probability of a short sentence

[Not in the slides: added material]

Suppose a model says $p(\textit{the}) = 0.06$ for the first word, $p(\textit{cat} \mid \textit{the}) = 0.01$ and $p(\textit{sleeps} \mid \textit{the cat}) = 0.05$. Then

$$p(\textit{the cat sleeps}) = 0.06 \times 0.01 \times 0.05 = 0.00003.$$

The number is tiny, as is the probability of any particular sentence (there are astronomically many possible sentences), but it can be *compared*: a good model gives *the cat sleeps* a much higher probability than *sleeps cat the*.

16.1.4 Why care about language models?

[Slides · Language models & RNNs: 11]

Definition — Language model

A **language model** is a probabilistic model of natural language: it assigns a probability to every sequence of words, or equivalently gives, for every context, the probability distribution of the next word.

- Language modelling is a good **benchmark** for general NLP models: to predict the next word well a model must know grammar, meaning and facts about the world.
- Every task that needs to **estimate the probability of a text** can be built on top of an LM: speech recognition, handwriting recognition, spell checking, machine translation, summarisation, dialogue, ...
- Today **everything else in NLP is built on top of language modelling**: GPT is a language model.

Intuition — how a probability of text helps other tasks

[Not in the slides: added material]

A speech recogniser hears a sound that could be "recognise speech" or "wreck a nice beach". Both match the audio; the language model says the first is a far more likely English sentence, so it wins. A spell checker that sees "I went to the sea *shor*" proposes *shore* rather than *short* because $p(\textit{shore} \mid \textit{the sea})$ is much higher. In both cases the LM provides the knowledge of "what people usually say".

16.2 Classical Language Models: n-grams

[Slides · Language models & RNNs: 12–34]

Before neural networks, language models were built by **counting**. This section shows how, why it works surprisingly well, and why it eventually hits a wall.

16.2.1 n-grams and the Markov assumption

[Slides · Language models & RNNs: 13–16]

Definition — n-gram

An **n-gram** is a sequence of n consecutive words of a text.

For the sentence "The quick brown fox jumps over the lazy dog":

n	n-grams
1 (**unigrams**)	*the*, *quick*, *brown*, *fox*, ...
2 (**bigrams**)	*the quick*, *quick brown*, *brown fox*, *fox jumps*, ...
3 (**trigrams**)	*the quick brown*, *quick brown fox*, *brown fox jumps*, ...
4 (**four-grams**)	*the quick brown fox*, *quick brown fox jumps*, ...

An **n-gram language model**:

1. collects the **frequencies** (counts) of all n-grams in a large text corpus;
2. uses these frequencies to predict the next word.

The Markov assumption. The chain rule (Section 16.1) needs the probability of the next word given the *whole* history. That is impossible to estimate by counting: most long histories appear only once in any corpus, or never. An n-gram model makes a simplifying assumption, named after the mathematician Markov (we met the same idea in Hidden Markov Models, Section 7.4).

Definition — Markov assumption (n-gram model)

The next word depends only on the $n-1$ words just before it:

$$p\big(\mathbf{x}^{(t+1)} \mid \mathbf{x}^{(t)}, \ldots, \mathbf{x}^{(1)}\big) \approx p\big(\mathbf{x}^{(t+1)} \mid \underbrace{\mathbf{x}^{(t)}, \ldots, \mathbf{x}^{(t-n+2)}}_{n-1 \text{ words}}\big).$$

(Counting the context: from position $t-n+2$ to position t there are $t-(t-n+2)+1 = n-1$ words. Together with the predicted word they form one n-gram.)

For example, a **bigram** model ($n = 2$) looks only at the previous word. The probability of "the quick brown fox" is decomposed as

$$p(\texttt{<s>})\, p(\textit{the} \mid \texttt{<s>})\, p(\textit{quick} \mid \textit{the})\, p(\textit{brown} \mid \textit{quick})\, p(\textit{fox} \mid \textit{brown})\, p(\texttt{</s>} \mid \textit{fox}).$$

`<s>` and `</s>` are two special tokens marking the **beginning** and the **end of the sentence**. They let the model learn which words tend to *start* a sentence ($p(\textit{the} \mid \texttt{<s>})$ is high) and which tend to *end* it. Every sentence starts with `<s>`, so $p(\texttt{<s>}) = 1$.

Estimating the probabilities by counting. By the definition of conditional probability, the probability we need is a ratio:

$$p\big(\mathbf{x}^{(t+1)} \mid \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}\big) = \frac{p\big(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}\big)}{p\big(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}\big)}.$$

We do not know these probabilities, but we have a corpus, a finite sample of language. The **finite-sample approximation** replaces each probability with how often the sequence occurs in the corpus:

$$p\big(\mathbf{x}^{(t+1)} \mid \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}\big) \approx \frac{\overbrace{\mathrm{count}\big(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}\big)}^{\text{an n-gram}}}{\underbrace{\mathrm{count}\big(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}\big)}_{\text{an } (n-1)\text{-gram}}}.$$

In words: *among all the times the context appeared, in what fraction was it followed by this word?* The quality of the approximation depends on the **size of the corpus**: the more text, the closer the fractions get to the true probabilities.

Worked example — a four-gram model

[Slides · Language models & RNNs: 17–19]

We want to continue "Anna was tired so she went to __" with a **four-gram** model ($n = 4$). It looks only at the last $n - 1 = 3$ words, *she went to*, and discards the rest:

$$p(x \mid \textit{she went to}) \approx \frac{\mathrm{count}(\textit{she went to } x)}{\mathrm{count}(\textit{she went to})}.$$

Suppose the corpus contains:

sequence	count	estimated probability of x
she went to	1000	(the context)
she went to the	500	$500/1000 = 0.50$
she went to work	250	$250/1000 = 0.25$
she went to school	200	$200/1000 = 0.20$
she went to bed	50	$50/1000 = 0.05$

The four probabilities add up to 1 (here every occurrence of the context is followed by one of these four words). The model predicts *the* as most likely and *bed* as least likely.

Was it a good idea to discard the longer context? No: "Anna was *tired*" is exactly the clue that makes *bed* the most sensible continuation, but the model never saw it, because it fell outside the window of three words.

Worked example — a bigram model from a toy corpus

[Not in the slides: added material]

The corpus has three sentences:

`<s>` I like cats `</s>` `<s>` I like dogs `</s>` `<s>` you like cats `</s>`

Counts we need: <s> appears 3 times, followed twice by I and once by *you*; I appears 2 times, always followed by *like*; *like* appears 3 times, followed twice by *cats* and once by *dogs*; *dogs* appears once, followed by </s>. So

$$p(I \mid \texttt{<s>}) = \tfrac{2}{3}, \quad p(like \mid I) = \tfrac{2}{2} = 1, \quad p(dogs \mid like) = \tfrac{1}{3}, \quad p(\texttt{</s>} \mid dogs) = \tfrac{1}{1} = 1.$$

The probability of the sentence "I like dogs" is

$$\begin{aligned} p(I\ like\ dogs) &= p(I \mid \texttt{<s>})\, p(like \mid I)\, p(dogs \mid like)\, p(\texttt{</s>} \mid dogs) \\ &= \tfrac{2}{3} \cdot 1 \cdot \tfrac{1}{3} \cdot 1 = \tfrac{2}{9} \approx 0.22. \end{aligned}$$

Two observations that lead to the next topic:

- "you like dogs" *never* appears in the corpus, yet the model gives it $\frac{1}{3} \cdot 1 \cdot \frac{1}{3} \cdot 1 = \frac{1}{9}$: by cutting sentences into short pieces, the model **generalises** to new sentences;
- "I hate cats" gets probability **0**, because the bigram *I hate* was never seen. One unseen bigram is enough to kill the whole sentence.

16.2.2 Sparsity problems

[Slides · Language models & RNNs: 20–26]

Sparsity problem 1: unseen continuations. For many words x, the sequence "she went to x" **never occurs** in the corpus ("she went to Reykjavik", "she went to dinner" may simply be absent). Their count is 0, so their probability is 0: the model says they are *impossible*. Many words with probability 0 make the LM **inflexible**.

Mitigation: smoothing. Add a small value ϵ to the count of every word $x \in V$, so that nothing is ever exactly 0:

$$p(x \mid she\ went\ to) = \frac{\text{count}(she\ went\ to\ x) + \epsilon}{\text{count}(she\ went\ to) + \epsilon|V|}.$$

Watch out — the denominator must change too

The slide adds ϵ only to the numerator. Then the probabilities no longer add up to 1: summing the numerators over all $|V|$ words gives $\text{count}(she\ went\ to) + \epsilon|V|$, not $\text{count}(she\ went\ to)$. Adding $\epsilon|V|$ to the denominator, as above, fixes this. With $\epsilon = 1$ the method is called **add-one** or **Laplace smoothing**.

Worked example — add-one smoothing on the toy corpus

[Not in the slides: added material]

In the toy corpus above, a word can be followed by one of the 6 tokens I, *you*,

like, *cats*, *dogs*, </s>, so $|V| = 6$; and count(I) $= 2$. With $\epsilon = 1$:

$$p(like \mid I) = \frac{2+1}{2+6} = \frac{3}{8}, \qquad p(cats \mid I) = \frac{0+1}{2+6} = \frac{1}{8},$$

and the same $\frac{1}{8}$ for each of the other 4 unseen tokens. Check: $\frac{3}{8} + 5 \cdot \frac{1}{8} = 1$. *like* is still by far the most likely, but no word is impossible any more. The price: probability mass has been taken from the words that were actually seen ($1 \to \frac{3}{8}$) and spread over unseen ones.

Sparsity problem 2: unseen contexts. What if "she went to" **never appeared** in the corpus? Then the denominator is 0, and we cannot compute the probability at all.

Mitigation: backoff. Condition on a shorter context, just "went to" (or "she went", as the slide writes), and if that is also missing, shorter still, down to the unigram probability of x. Shorter contexts are more frequent, so they are more likely to have been observed.

Key result — Increasing n makes sparsity worse

A longer n-gram is a more specific sequence, so it is seen more rarely. With a larger n the model uses more context, but more of its counts are 0 and smoothing/backoff are needed more often. In practice $n \leq 5$.

An example of sparsity. The **Berkeley Restaurant Project** dataset contains 9332 sentences of people asking about restaurants, with a vocabulary of $|V| = 1446$ words. The table shows the bigram counts for 8 of these words: the entry in row *want*, column *to* is how many times *want* was immediately followed by *to*.

	i	*want*	*to*	*eat*	*chinese*	*food*	*lunch*	*spend*
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Table 24: Bigram counts in the Berkeley Restaurant Project (row = first word, column = second word). Half of the 64 entries are 0, even among frequent words.

32 of the 64 cells are zero. Some zeros are real impossibilities (*want want*), but others are perfectly good English that simply did not occur (*lunch food*? *spend lunch*?). And this is with **bigrams**: there are $1446^2 \approx 2.1$ million possible bigrams, far more than a corpus of 9332 sentences can show. With $n > 2$ it is much worse.

16.2.3 Storage problems

[Slides · Language models & RNNs: 27]

An n-gram model must **store the counts** of all n-grams seen in the corpus. As n grows, or as the corpus grows, this can become huge. With $|V| = 1446$ there are

$$1446^4 \approx 4.37 \times 10^{12}$$

possible four-grams: naively storing a count for each of them, even at just one byte per count, requires about **4 TB** of memory, for a toy vocabulary. Real vocabularies have tens of thousands of words. (Storing only the n-grams actually seen saves a lot, but on a large corpus that is still billions of entries.)

16.2.4 Generating text with an n-gram model

[Slides · Language models & RNNs: 28–34]

A language model can also **generate** text: start from a context, compute the distribution of the next word, **sample** a word from it, append it to the context and repeat.

Definition — Sampling from a distribution

[Not in the slides: added material]

Sampling a word means drawing it at random *with the probabilities given by the model*: a word with probability 0.3 is picked, on average, 3 times out of 10. It is like rolling a die whose faces are the words of the vocabulary and whose faces have different sizes. Sampling (instead of always taking the most likely word) gives varied text rather than the same sentence every time.

The slides use a **trigram** model built on the **Reuters corpus** (1.7 million words of financial news). Starting from "today the":

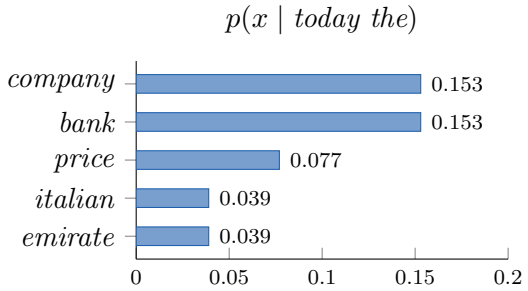


Figure 41: The five most likely words after "today the" in the trigram model (the rest of the probability is spread over many other words).

1. Context *today the* → distribution: *company* 0.153, *bank* 0.153, *price* 0.077, *italian* 0.039, *emirate* 0.039, … Sample: *price*.
2. Text *today the price*; context (last 2 words) *the price* → *of* 0.308, *for* 0.050, *it* 0.046, *to* 0.046, *is* 0.031, … Sample: *of*.

3. Text *today the price of*; context *price of* → *the* 0.072, *18* 0.043, *oil* 0.043, *its* 0.036, *gold* 0.018, . . . and so on.

Note that at each step only the last $n-1 = 2$ words are used as context: the beginning *today* is forgotten as soon as it leaves the window.

The effect of n. The slide shows text generated by n-gram models trained on a novel (Mary Shelley's *Frankenstein*), for increasing n:

n	generated text
1	"care I of . its destined , in , from . . for the Felix an not which measure excited"
2	"I compassionated him , how heavily ; but I had arrived , but they averred , my unhallowed damps and"
3	"I continued their single offspring . At length I gathered from a man who , born in freedom , spurned"
4	"I continued walking in this manner , during which I enjoyed the feeling of happiness . Still thou canst listen"
5	"I continued walking in this manner for some time , and I feared the effects of the dæmon's disappointment ."

- Increasing n, the model is **more coherent**: unigrams give a word salad, bigrams give locally plausible pairs, 4- and 5-grams read like real sentences.
- Increasing n **increases the sparsity problem**.
- With a high n the model **repeats the training set**: most 5-word contexts were seen only once, so they have only one possible continuation, and the model copies whole passages of the novel instead of creating new text.

Cheat sheet — n-gram language models

- Markov assumption: the next word depends only on the previous $n-1$ words.
- Estimate: $p(x \mid \text{context}) \approx \text{count}(\text{context}, x)/\text{count}(\text{context})$.
- Sparsity 1 (unseen n-gram, probability 0) → smoothing; sparsity 2 (unseen context, division by 0) → backoff. Both get worse as n grows; $n \leq 5$.
- Storage grows with n and with the corpus.
- Small n: incoherent text; large n: coherent but copied from the corpus.
- The context is cut off: anything older than $n-1$ words is forgotten.

Exercise 1 — bigram probabilities

[Not in the slides: added material]

Using the toy corpus of this section (`<s>` I like cats `</s>`, `<s>` I like dogs `</s>`, `<s>` you like cats `</s>`) and a bigram model without smoothing, compute $p(\textit{you like cats})$. Which sentence of the form "*subject* like *animal*" has the highest probability?

Solution

Counts: $p(\textit{you} \mid \texttt{<s>}) = \frac{1}{3}$, $p(\textit{like} \mid \textit{you}) = \frac{1}{1} = 1$, $p(\textit{cats} \mid \textit{like}) = \frac{2}{3}$, $p(\texttt{</s>} \mid \textit{cats}) = \frac{2}{2} = 1$. So

$$p(\textit{you like cats}) = \tfrac{1}{3} \cdot 1 \cdot \tfrac{2}{3} \cdot 1 = \tfrac{2}{9}.$$

The four candidates are $p(\textit{I like cats}) = \frac{2}{3} \cdot \frac{2}{3} = \frac{4}{9}$, $p(\textit{I like dogs}) = \frac{2}{3} \cdot \frac{1}{3} = \frac{2}{9}$, $p(\textit{you like cats}) = \frac{1}{3} \cdot \frac{2}{3} = \frac{2}{9}$, $p(\textit{you like dogs}) = \frac{1}{3} \cdot \frac{1}{3} = \frac{1}{9}$ (the factors equal to 1 are omitted). Check: they add up to $\frac{4+2+2+1}{9} = 1$, since these are the only sentences the model can produce. *I like cats* wins: it combines the most frequent start (I) and the most frequent animal (*cats*).

16.3 Neural Language Models

[Slides · Language models & RNNs: 35–50]

n-gram models fail because they treat every word sequence as a separate, unrelated item: seeing "she went to school" a thousand times teaches them nothing about "he walked to college". Neural language models fix this by representing words as **vectors**, so that similar words get similar representations and what is learnt about one word transfers to its neighbours.

16.3.1 Aside: word embeddings via word2vec

[Slides · Language models & RNNs: 36]

One-hot vectors. [Not in the slides: added material]

A computer needs numbers, not words. The simplest way to turn a word into numbers is a **one-hot vector**: number the words of the vocabulary from 1 to $|V|$; word number i becomes a vector of length $|V|$ with a 1 in position i and 0 everywhere else. With the vocabulary (*cat*, *dog*, *the*, *sleeps*):

$$\textit{cat} = \begin{pmatrix}1\\0\\0\\0\end{pmatrix}, \quad \textit{dog} = \begin{pmatrix}0\\1\\0\\0\end{pmatrix}, \quad \textit{the} = \begin{pmatrix}0\\0\\1\\0\end{pmatrix}, \quad \textit{sleeps} = \begin{pmatrix}0\\0\\0\\1\end{pmatrix}.$$

One-hot vectors identify words but say nothing about their meaning: *cat* is exactly as different from *dog* as from *sleeps* (any two of them have dot product 0).

Word embeddings. The goal of a **word embedding** is to represent each word with a short, dense vector (a few hundred real numbers) such that:

- the vector of a word is **similar** to the vectors of words that occur in *similar* contexts;
- it is **dissimilar** from the vectors of words that are unlikely to appear in the same contexts.

This is the distributional hypothesis (Section 8.1) turned into vectors (Section 8.3). **Word2vec** is a popular method that learns such vectors by training a small neural network to predict words from their neighbours. Similarity between two vectors is measured by their **dot product**: multiply the entries position by position and add.

Worked example — similar and dissimilar words

[Slides · Language models & RNNs: 36]

Eight-dimensional embeddings from the slide:

$$\textit{baking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}, \quad \textit{flour} = \begin{pmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{pmatrix}, \quad \textit{dollar} = \begin{pmatrix} -0.154 \\ 0.024 \\ 0.512 \\ 0.782 \\ 0.359 \\ 0.141 \\ -0.581 \\ -0.027 \end{pmatrix}.$$

$$\begin{aligned} \textit{baking}^\top \textit{flour} &= 0.286 \cdot 0.413 + 0.792 \cdot 0.582 + (-0.177)(-0.007) + \cdots \\ &\quad + 0.271 \cdot 0.051 \\ &= 0.118 + 0.461 + 0.001 - 0.026 + 0.024 + 0.389 + 0.051 + 0.014 \\ &\approx 1.03 \end{aligned}$$

The large positive entries are in the same places (positions 2 and 6 contribute most), so the dot product is large: *baking* and *flour* appear in similar contexts (recipes). For *baking* and *dollar* the signs disagree in most positions and the dot product is negative: they are used in unrelated contexts.

Watch out — the number on the slide

The slide gives $\textit{baking}^\top \textit{dollar} = -0.35$. With the vectors as printed the dot product is -0.45; -0.34 is their *cosine similarity* (the dot product divided by the lengths of the two vectors, Section 8.3). Either way the conclusion is the same: negative, hence dissimilar.

16.3.2 A fixed-window neural language model

[Slides · Language models & RNNs: 37–41]

Recall the language-modelling task: given a sequence of words $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(t)}$, output the probability distribution of the next word, $p(\mathbf{x}^{(t+1)} \mid \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(t)})$. The first neural approach keeps the n-gram idea of a **fixed window** of previous words, but replaces counting with a **neural network** that *learns* to predict.

For "Anna was tired so she went to her __", a window of 4 words takes *she went to her* and predicts the missing word. The network is built in four layers, from bottom to top:

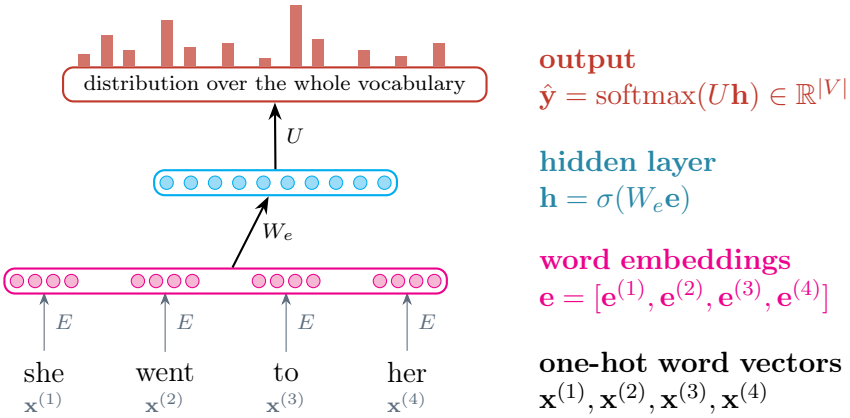


Figure 42: The fixed-window neural language model. Information flows upwards: four words → their embeddings, glued together → a hidden layer → a probability for every word of the vocabulary (tall bars = likely next words).

1. **One-hot word vectors** $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(4)} \in \mathbb{R}^{|V|}$: the four words of the window.
2. **Word embeddings.** Each word is turned into its embedding $\mathbf{e}^{(i)} = E\mathbf{x}^{(i)}$. E is a matrix with one column per word of the vocabulary ($E \in \mathbb{R}^{d \times |V|}$, d = embedding size). Multiplying a matrix by a one-hot vector simply *picks one column*: the column of that word. So E is just a table of word vectors, stored as a matrix. The four embeddings are then **concatenated** (written one after the other) into a single vector $\mathbf{e}$ of length $4d$.
3. **Hidden layer** $\mathbf{h} = \sigma(W_e\mathbf{e})$. The matrix W_e mixes all the numbers of the four embeddings; σ is a non-linear function applied to each entry (see below). $\mathbf{h}$ is a summary of the window useful for prediction.
4. **Output** $\hat{\mathbf{y}} = \text{softmax}(U\mathbf{h}) \in \mathbb{R}^{|V|}$: one number per word of the vocabulary, turned into a probability distribution by the softmax (see below). $\hat{\mathbf{y}}_w$ is the predicted probability that the next word is w. The hat in $\hat{\mathbf{y}}$ means "predicted".

[Not in the slides: added material]

Two building blocks deserve a closer look.

- **The non-linearity** σ**.** Typically the *sigmoid* $\sigma(z) = \frac{1}{1+e^{-z}}$, which squashes any number into $(0, 1)$, or tanh, which squashes into $(-1, 1)$. Without it, the whole network would be a product of matrices, i.e. just one big linear map, and could only learn very simple relations. (Real networks also add a *bias* vector inside σ, $\sigma(W_e\mathbf{e} + \mathbf{b})$; the slides omit it for simplicity.)
- **The weights** E**,** W_e**,** U are the **parameters** of the model, written together as θ. They start as random numbers; *training* (below) adjusts them so that the

predictions become good.

16.3.3 How to train a neural language model

[Slides · Language models & RNNs: 42–49]

Definition — The four ingredients of training a neural network

1. **A corpus of data**: here a collection of documents, i.e. a long sequence of words $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(T)}$.
2. **A model**: here the fixed-window neural LM.
3. **A loss function** J: a number measuring how wrong the model's predictions are; it compares the output distribution with the true one.
4. **An optimisation algorithm** that changes the parameters to make the loss smaller: **stochastic gradient descent**.

The softmax. The vector $U\mathbf{h} \in \mathbb{R}^{|V|}$ contains one *score* per word, but scores can be negative and do not add up to 1: it is an **un-normalised** probability distribution. The **softmax** normalises it:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \qquad i = 1, \ldots, K.$$

The exponential makes every entry positive (and enlarges the gaps between scores), and dividing by the sum makes the entries add up to 1.

Worked example — softmax

[Slides · Language models & RNNs: 47]

Scores $U\mathbf{h} = (1.3,\ 5.1,\ 2.2,\ 0.7,\ 1.1)$ for a vocabulary of $K = 5$ words.

$$\begin{aligned}(e^{1.3}, e^{5.1}, e^{2.2}, e^{0.7}, e^{1.1}) &= (3.67,\ 164.02,\ 9.03,\ 2.01,\ 3.00) && \triangleleft \text{ exponentiate}\\ \text{sum} &= 3.67 + 164.02 + 9.03 + 2.01 + 3.00 = 181.73\\ \hat{\mathbf{y}} &= (0.02,\ 0.90,\ 0.05,\ 0.01,\ 0.02) && \triangleleft \text{ divide by the sum}\end{aligned}$$

Check: $0.02 + 0.90 + 0.05 + 0.01 + 0.02 = 1$. The highest score (5.1) gets most of the probability, but no word gets exactly 0.

The loss: cross-entropy. We compare

- the **computed** probability $\hat{y}_i = q(V_i \mid \textit{she went to her})$, what the model says about word V_i of the vocabulary;
- the **true** probability $y_i^* = p(V_i \mid \text{full context})$, what the word really is.

The standard way to compare two distributions is the **cross-entropy**:

$$\text{CE}(\mathbf{y}^*, \hat{\mathbf{y}}) = -\sum_{i=1}^{|V|} y_i^* \log \hat{y}_i.$$

In training data we do not know the "true distribution": we only know which word *actually came next* in the corpus. So $\mathbf{y}^*$ is a **one-hot vector**: 1 for the true next word, 0 for every other word. All the terms of the sum vanish except one. If the true word in the data is the fifth word of the sentence, $\mathbf{x}^{(5)}$, then

$$J^{(t)} = \mathrm{CE}(\mathbf{y}^*, \hat{\mathbf{y}}) = -\log \hat{y}_{\mathbf{x}^{(5)}} \qquad \text{(the negative log-probability of the true word).}$$

(The slide writes $\hat{y}_5$, meaning "the entry of $\hat{\mathbf{y}}$ belonging to the true word $\mathbf{x}^{(5)}$".)

Intuition — why $-\log$ of the probability?

[Not in the slides: added material]

$-\log p$ is 0 when $p = 1$ (the model was certain and right: no penalty) and grows without limit as $p \to 0$ (the model said the true word was nearly impossible: huge penalty). With the softmax output above (natural logarithm):

- if the true word is the 2nd one ($\hat{y} = 0.90$): loss $= -\ln 0.90 \approx 0.10$, small;
- if the true word is the 4th one ($\hat{y} = 0.011$): loss $= -\ln 0.011 \approx 4.50$, large.

Minimising the loss means: *give high probability to the words that really come next.*

Gradient descent. [Not in the slides: added material]

The loss $J(\theta)$ depends on all the parameters θ (the entries of E, W_e, U). The **gradient** $\nabla_\theta J$ is the vector of the derivatives of J with respect to each parameter: it points in the direction in which the loss *increases* fastest. **Gradient descent** takes a small step in the opposite direction, again and again:

$$\theta \leftarrow \theta - \alpha \, \nabla_\theta J(\theta),$$

where $\alpha > 0$ (the **learning rate**) controls the step size. It is like walking down a hill in the fog: you cannot see the valley, but you can feel the slope under your feet and step downhill. The gradients are computed automatically by **backpropagation**, i.e. the chain rule of calculus applied layer by layer from the output back to the input. *Stochastic* gradient descent computes the gradient on a small random part of the data at each step instead of on the whole corpus (Section 16.4).

16.3.4 Pros and cons of the fixed-window model

[Slides · Language models & RNNs: 50]

This model was introduced by Y. Bengio et al., *A Neural Probabilistic Language Model*, 2003.

Improvements over n-gram LMs:

- **no sparsity problem**: the network computes a probability for every word from the embeddings, so unseen sequences get sensible, non-zero probabilities; if *school* and *college* have similar embeddings, what is learnt for one transfers to the other;

- **no need to store n-grams**: only the weights are stored.

Remaining problems:

- the **fixed window is too restrictive**: like an n-gram model it cannot see anything older than its window ("Anna was *tired*" is still invisible);
- **enlarging the window increases the size of W_e**: W_e has one column per entry of $\mathbf{e}$, and $\mathbf{e}$ grows with every extra word; a window big enough for any text is impossible;
- **no symmetry**: $\mathbf{x}^{(1)}$ and $\mathbf{x}^{(2)}$ are multiplied by *different* columns of W_e. What the model learns about the word *school* in position 4 does not transfer to *school* in position 2: the same knowledge has to be learnt separately for every position.

Why this matters for NLP

The question left open by this slide, *how can we process inputs of arbitrary length?*, is answered by recurrent neural networks (next section), and later by transformers. Both keep the two good ideas introduced here: words as learnt embeddings, and a softmax over the vocabulary trained with cross-entropy. Every modern LLM, GPT included, still ends with exactly this softmax layer.

Cheat sheet — neural language models

- Embedding: $\mathbf{e} = E\mathbf{x}$ (picks the column of word $\mathbf{x}$); similar words have a large dot product.
- Fixed window: $\hat{\mathbf{y}} = \text{softmax}(U\,\sigma(W_e[\mathbf{e}^{(1)}, \ldots, \mathbf{e}^{(n)}]))$.
- $\text{softmax}(\mathbf{z})_i = e^{z_i} / \sum_j e^{z_j}$: positive, sums to 1.
- Loss: cross-entropy with a one-hot target $= -\log \hat{y}_{\text{true word}}$.
- Training: corpus + model + loss + (stochastic) gradient descent $\theta \leftarrow \theta - \alpha \nabla_\theta J$.
- Solves sparsity and storage; still a fixed window, growing W_e, no weight sharing across positions.

16.4 Recurrent Neural Networks

[Slides · Language models & RNNs: 51–84]

16.4.1 The idea: reuse the same weights at every step

[Slides · Language models & RNNs: 52–56]

A **recurrent neural network** (RNN) reads a text *one word at a time*, the way we do, and keeps a **hidden state $\mathbf{h}^{(t)}$**: a vector that summarises everything read so far. At each step it combines the previous summary with the new word to produce the new summary. Crucially, **the same weights are applied repeatedly** at every step, so the network has the same size whether the text has 4 words or 4000: inputs can be of **any length**.

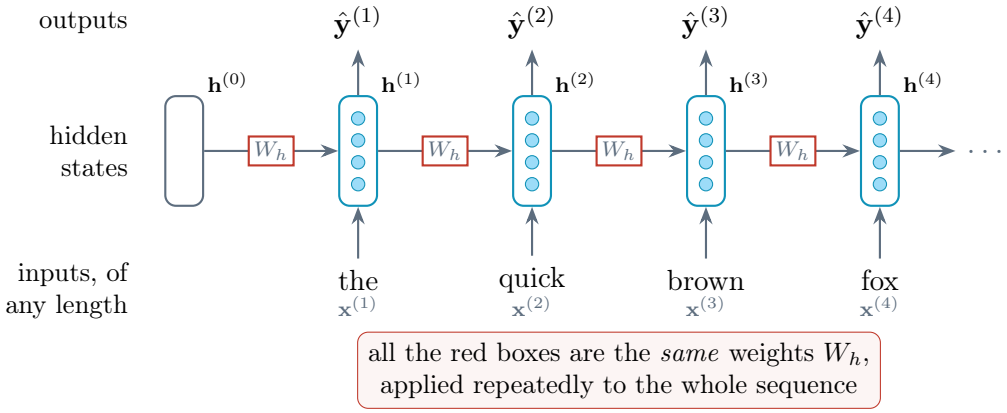


Figure 43: An RNN "unrolled" over four time steps. There is only one matrix W_h (and one W_e, one U): the copies are the same weights used again at every step.

Definition — RNN language model

At each time step $t = 1, 2, \ldots$

$\mathbf{e}^{(t)} = E\mathbf{x}^{(t)}$	word embedding of the one-hot word $\mathbf{x}^{(t)} \in \mathbb{R}^{\lvert V\rvert}$
$\mathbf{h}^{(t)} = \sigma\big(W_h\mathbf{h}^{(t-1)} + W_e\mathbf{e}^{(t)}\big)$	new hidden state
$\hat{\mathbf{y}}^{(t)} = \text{softmax}\big(U\mathbf{h}^{(t)}\big) \in \mathbb{R}^{\lvert V\rvert}$	distribution of the next word $\mathbf{x}^{(t+1)}$

$\mathbf{h}^{(0)}$ is an initial hidden state (often a vector of zeros). The parameters are E, W_h, W_e, U, shared by all time steps.

Read the hidden-state equation slowly: $W_h\mathbf{h}^{(t-1)}$ is "what I remember", $W_e\mathbf{e}^{(t)}$ is "what I just read"; they are added and squashed by σ into the new memory $\mathbf{h}^{(t)}$. Since $\mathbf{h}^{(t-1)}$ itself depended on $\mathbf{h}^{(t-2)}$ and on $\mathbf{x}^{(t-1)}$, and so on, $\mathbf{h}^{(t)}$ depends, in principle, on *all* the words $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(t)}$. There is no window any more.

Intuition — an RNN is a reader taking notes

[Not in the slides: added material]

Imagine reading a novel with a notebook of fixed size. After each word you update your notes (same rules every time), and from the notes alone you guess the next word. The notebook never grows, whatever the length of the book, so you must decide what to keep and what to overwrite. This is both the strength of RNNs (any length, constant size) and, as we will see, their weakness (old information gets overwritten).

16.4.2 Pros and cons

[Slides · Language models & RNNs: 57]

Pros	Cons
not restricted to fixed-length inputs	computation is slow: step t needs $\mathbf{h}^{(t-1)}$, so the steps cannot be computed in parallel
can use information from many time steps ago (in theory)	in practice long-range information is hard to capture (Section 16.5)
input words are processed symmetrically: the same weights at every position (**weight sharing**), so what is learnt about a word in one position applies everywhere	

Compare with the fixed-window model (Section 16.3): the first and third pros solve exactly its "fixed window" and "no symmetry" problems, and the model size no longer depends on the length of the context.

16.4.3 Training an RNN language model

[Slides · Language models & RNNs: 58–67]

1. Gather a large corpus of text $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(T)}$.
2. Feed it to the RNN one word at a time, computing $\hat{\mathbf{y}}^{(t)}$ for every $t = 1, \ldots, T$.
3. At each step compute the loss: the cross-entropy between the predicted distribution $\hat{\mathbf{y}}^{(t)}$ and the true next word $\mathbf{y}^{(t)} = \mathbf{x}^{(t+1)}$ (one-hot):

$$J^{(t)} = \mathrm{CE}(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = -\sum_{w \in V} \underbrace{y_w^{(t)}}_{\text{one-hot}} \log \hat{y}_w^{(t)} = -\log \hat{y}^{(t)}_{\mathbf{x}^{(t+1)}},$$

   the negative log-probability of the correct next word.

4. Average the loss over the whole dataset (the last word has no next word to predict, hence $T-1$ terms):

$$J = \frac{1}{T-1} \sum_{t=1}^{T-1} J^{(t)} = \frac{1}{T-1} \sum_{t=1}^{T-1} -\log \hat{y}^{(t)}_{\mathbf{x}^{(t+1)}}.$$

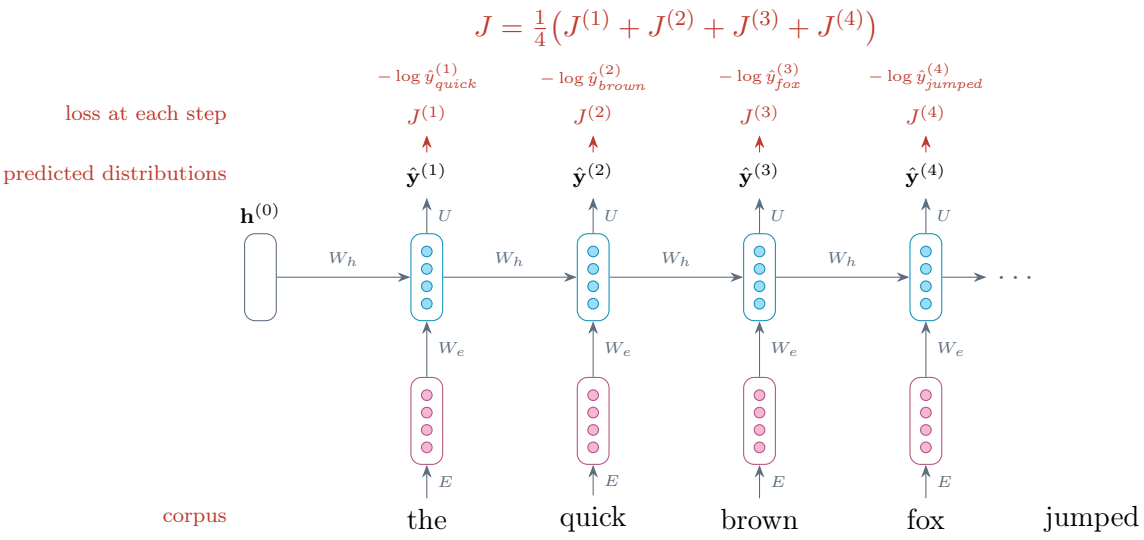


Figure 44: Training on "the quick brown fox jumped". At step 1 the network has read *the* and is penalised by $-\log$ of the probability it gave to *quick*; at step 2 by the probability it gave to *brown*; and so on. The total loss is the average.

Note that the input at every step is the *true* word from the corpus, not the word the model predicted: during training the model is always shown the correct text and asked to predict one step ahead.

Training in practice: stochastic gradient descent. A typical corpus has millions of words. Computing the loss $J(\theta) = \frac{1}{T-1}\sum_t J^{(t)}(\theta)$ and its gradient over all of it at once requires too much memory and time. So:

- divide the corpus into smaller **chunks** (sentences, documents);
- **Stochastic Gradient Descent** (SGD) approximates the full gradient with the average gradient over a small **batch** of S chunks:

$$\nabla_\theta J(\theta) \approx \frac{1}{S}\sum_{j=1}^{S} \nabla_\theta J^{(j)}(\theta);$$

- loss and gradients are computed for one batch of sentences at a time, then the weights are updated ($\theta \leftarrow \theta - \alpha\nabla_\theta J$) and the next batch is taken.

The approximation is noisy (a different batch gives a slightly different gradient), but on average it points in the right direction, and each step is cheap. That noise is the "stochastic" in the name.

16.4.4 Backpropagation through time

[Slides · Language models & RNNs: 68–76]

To run gradient descent we need the gradient of the loss with respect to every parameter. The tricky one is W_h, because it is used at *every* step. To see the structure

clearly, the slides use an even simpler model with no input and no non-linearity:

$$\mathbf{h}^{(t)} = W_h \mathbf{h}^{(t-1)}, \qquad J^{(t)}(\theta) = \mathrm{CE}(\mathbf{h}^{(t)}, \mathbf{y}^{*(t)}).$$

Key result — Backpropagation through time (BPTT)

The gradient with respect to a repeated weight is the **sum of the gradients for each time it appears**:

$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^{t} \left. \frac{\partial J^{(t)}}{\partial W_h} \right|_{(i)},$$

where $\left. \frac{\partial J^{(t)}}{\partial W_h} \right|_{(i)}$ is the gradient obtained by treating only the copy of W_h used at step i as variable.

Intuition — why a sum?

[Not in the slides: added material]

Think of W_h as a knob that turns t dials at once (one per step). Turning it a little changes the output through each dial; for small changes the effects simply add up. So the total sensitivity is the sum of the sensitivities through each use.

The derivation. Start with the **chain rule**: the loss depends on W_h only through $\mathbf{h}^{(t)}$, so

$$\frac{\partial J^{(t)}}{\partial W_h} = \frac{\partial J^{(t)}}{\partial \mathbf{h}^{(t)}} \frac{\partial \mathbf{h}^{(t)}}{\partial W_h}.$$

The first factor is easy (it only involves the output). The second is the interesting one: $\mathbf{h}^{(t)} = W_h \mathbf{h}^{(t-1)}$ depends on W_h in two ways, *directly* (it is multiplied by W_h) and *indirectly* (through $\mathbf{h}^{(t-1)}$, which was itself computed with W_h). Write it as $\mathbf{h}^{(t)}(W_h, \mathbf{h}^{(t-1)})$.

To keep the algebra readable we first do it with **numbers instead of matrices** (a hidden state of size 1, so $h^{(t)} = w\, h^{(t-1)}$). The derivative of a product is "derivative of the first times the second plus the first times the derivative of the second" (product rule), so:

$$\begin{aligned}
\frac{\mathrm{d}h^{(t)}}{\mathrm{d}w} &= \underbrace{h^{(t-1)}}_{\text{direct}} + \underbrace{w \frac{\mathrm{d}h^{(t-1)}}{\mathrm{d}w}}_{\text{indirect}} && \triangleleft \text{ product rule on } w\, h^{(t-1)} \\
&= h^{(t-1)} + w\left(h^{(t-2)} + w \frac{\mathrm{d}h^{(t-2)}}{\mathrm{d}w}\right) && \triangleleft \text{ same rule for } h^{(t-1)} \\
&= h^{(t-1)} + w\, h^{(t-2)} + w^2 \frac{\mathrm{d}h^{(t-2)}}{\mathrm{d}w} && \triangleleft \text{ expand} \\
&= \cdots = h^{(t-1)} + w\, h^{(t-2)} + w^2 h^{(t-3)} + \cdots + w^{t-1} h^{(0)} && \triangleleft\ \mathrm{d}h^{(0)}/\mathrm{d}w = 0 \\
&= \sum_{i=0}^{t-1} w^{t-1-i}\, h^{(i)}.
\end{aligned}$$

Each term is one "use" of w: the term with $h^{(i)}$ comes from the copy of w that multiplies $h^{(i)}$ at step $i+1$, and the factor w^{t-1-i} carries its effect through the remaining $t-1-i$ steps up to $h^{(t)}$. This is the BPTT sum.

Worked example — checking the formula

[Not in the slides: added material]

Take $w = 0.7$, $h^{(0)} = 2$, $t = 5$. Since $h^{(t)} = w^t h^{(0)}$, directly $\frac{\mathrm{d}h^{(5)}}{\mathrm{d}w} = 5w^4 h^{(0)} = 5 \cdot 0.2401 \cdot 2 = 2.401$. With the formula, $h^{(i)} = 2 \cdot 0.7^i$ and every term equals $w^{4-i} \cdot 2w^i = 2w^4$:

$$\sum_{i=0}^{4} w^{4-i} h^{(i)} = 5 \cdot 2 \cdot 0.7^4 = 2.401. \quad \checkmark$$

With matrices the computation is the same, and the slides write the result as

$$\frac{\partial \mathbf{h}^{(t)}}{\partial W_h} = \sum_{i=0}^{t-1} \Big(\prod_{j=i+1}^{t-1} W_h \Big) \mathbf{h}^{(i)} = \sum_{i=0}^{t-1} W_h^{t-1-i} \, \mathbf{h}^{(i)}.$$

Watch out — what the matrix formula means

The derivative of a vector with respect to a matrix is a three-index object, so the slide's formula (and its intermediate step "$\mathbf{h}^{(t-1)} I$") is a shorthand. The precise statement is: if W_h is changed by a small matrix ΔW, then $\mathbf{h}^{(t)}$ changes by

$$\Delta \mathbf{h}^{(t)} \approx \sum_{i=0}^{t-1} W_h^{t-1-i} \, \Delta W \, \mathbf{h}^{(i)}.$$

The structure is exactly the scalar one: one term per use of W_h, each carried to step t by a power of W_h.

- BPTT is **equivalent to ordinary backpropagation through the whole unrolled network**: once the RNN is drawn as a long chain of copies, it is just a (very deep) feed-forward network whose copies share the same weights.
- In practice the backward pass is **truncated** at about 20 steps, for efficiency: terms older than that are ignored (**truncated BPTT**).
- For more complex RNNs (with inputs and non-linearities) the formula is very similar.

16.4.5 Generating text with an RNN

[Slides · Language models & RNNs: 77]

To generate text at test time, **repeatedly sample** from the network, **feeding each output back as the next input**:

1. feed the start token `<s>`; the network outputs $\hat{\mathbf{y}}^{(1)}$;
2. sample a word from $\hat{\mathbf{y}}^{(1)}$, say *the*;

3. feed *the* as the next input; the network updates its hidden state and outputs $\hat{\mathbf{y}}^{(2)}$; sample *quick*;
4. feed *quick*, sample *brown*, and so on, until `</s>` is sampled or a length limit is reached.

Unlike an n-gram model, the hidden state carries information from *all* the words generated so far, not only the last $n-1$. This is also, in essence, how chatbots write their answers, one token at a time.

16.4.6 Evaluating language models: perplexity

[Slides · Language models & RNNs: 78–84]

How do we measure whether a language model is good? We take a **test sentence** $X_{\text{test}} = \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(T)}$ that was **not seen during training** (otherwise a model that memorised the corpus would look perfect) and check how much probability the model gives it.

Definition — Perplexity

The **perplexity** of a sentence is its inverse probability, normalised by its length T:

$$\text{perplexity}(X_{\text{test}}) = p(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(T)})^{-1/T} = \sqrt[T]{\frac{1}{p(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(T)})}}.$$

To compute it:

1. compute the probability of the sentence with the tested model and the chain rule, $p(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(T)}) = \prod_{t=1}^{T} p(\mathbf{x}^{(t)} \mid \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(t-1)})$;
2. raise it to the power $-1/T$.

Why both pieces?

- **Inverse**: a good model gives the test sentence a *high* probability, hence a *low* perplexity. Lower is better.
- **T-th root**: the probability of a sentence shrinks with every word, so long sentences would always look worse. The T-th root is the *geometric mean* per word, which makes sentences of different lengths comparable.

Intuition — perplexity as a number of choices

[Not in the slides: added material]

A model that spreads its probability uniformly over k words at every step gives the sentence probability $(1/k)^T$, so its perplexity is $\left((1/k)^T\right)^{-1/T} = k$. Perplexity is therefore the *effective number of equally likely words* the model is hesitating between at each step. A perplexity of 50 means "as confused as if choosing among 50 words". A model that knows nothing (uniform over the whole vocabulary) has perplexity $|V|$.

Perplexity in practice and its link with the loss.

- Computing $\prod_{t=1}^{T} p(\mathbf{x}^{(t)} \mid \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(t-1)})$ for long sequences is **numerically unstable**: the product of thousands of numbers smaller than 1 becomes too small for the computer to represent (it rounds to 0). In practice we include begin and end markers `<s>`, `</s>` and **truncate the conditioning** at sentence or document boundaries, and we work with logarithms.
- **Connection to the cross-entropy loss.** For an RNN, the probability of each word is read from the output of the previous step: $p(\mathbf{x}^{(t+1)} \mid \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(t)}) = \hat{y}^{(t)}_{\mathbf{x}^{(t+1)}}$. Using $a = e^{\log a}$ and $\log \prod = \sum \log$:

$$\text{perplexity} = \Big[\prod_{t=1}^{T} \hat{y}^{(t)}_{\mathbf{x}^{(t+1)}}\Big]^{-1/T} = \exp\Big(-\frac{1}{T}\log \prod_{t=1}^{T} \hat{y}^{(t)}_{\mathbf{x}^{(t+1)}}\Big)$$

$$= \exp\Big(\frac{1}{T}\sum_{t=1}^{T} \underbrace{-\log \hat{y}^{(t)}_{\mathbf{x}^{(t+1)}}}_{J^{(t)}}\Big) = \exp(\text{average cross-entropy loss}).$$

  Training an LM by minimising the cross-entropy loss is the same as minimising perplexity.
- **Lower perplexity** means the model is **less "surprised"** by the text it is given.

Watch out — indices on the slide

The slide writes $p(\mathbf{x}^{(t)} \mid \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(t-1)}) = \hat{y}^{(t)}_{\mathbf{x}^{(t+1)}}$, mixing two conventions, and sums over i while the terms depend on t. The consistent version is the one above: $\hat{\mathbf{y}}^{(t)}$ is computed after reading $\mathbf{x}^{(t)}$ and predicts $\mathbf{x}^{(t+1)}$, so the t-th factor is $\hat{y}^{(t)}_{\mathbf{x}^{(t+1)}}$. Here the sequence starts with $\mathbf{x}^{(1)} =$ `<s>` (which is given, not predicted) and the T predicted tokens are $\mathbf{x}^{(2)}, \dots, \mathbf{x}^{(T+1)}$, the last one being `</s>`.

Worked example — perplexity of two models

[Slides · Language models & RNNs: 82–84]

The vocabulary is the digits 0 to 9, $|V| = 10$.

- LM1 outputs any digit with probability $\frac{1}{10}$;
- LM2 outputs 0 with probability $\frac{91}{100}$ and each other digit with probability $\frac{1}{100}$ (check: $0.91 + 9 \cdot 0.01 = 1$).

Which one has the lowest perplexity on the sequence "0 0 0 3 0" ($T = 5$)?

$$\text{LM1:}\quad \text{perplexity} = \Big[\big(\tfrac{1}{10}\big)^{5}\Big]^{-1/5} = \big(\tfrac{1}{10}\big)^{-1} = 10$$

$$\text{LM2:}\quad \text{perplexity} = \Big[\big(\tfrac{91}{100}\big)^{4} \cdot \tfrac{1}{100}\Big]^{-1/5} = (0.006857)^{-1/5} \approx 2.709$$

LM2 wins: it expects zeros, the sequence is mostly zeros, so it is much less surprised. LM1 is uniform, so its perplexity is $|V| = 10$, as predicted by the intuition box. Note that perplexity depends on the test text: on "3 3 3 3 3" LM2 would get $\big((1/100)^5\big)^{-1/5} = 100$, ten times worse than LM1.

Exercise 2 — perplexity of the toy bigram model

[Not in the slides: added material]

Compute the perplexity of the bigram model of Section 16.2 (toy corpus, no smoothing) on the sentence "I like dogs", counting the end marker as a predicted word.

Solution

The model predicts $T = 4$ tokens: *I*, *like*, *dogs*, `</s>`, with probabilities $\frac{2}{3}$, 1, $\frac{1}{3}$, 1. The sentence probability is $\frac{2}{9}$, so

$$\text{perplexity} = \left(\tfrac{2}{9}\right)^{-1/4} = 4.5^{1/4} \approx 1.46.$$

Very low: the toy model has almost no choices to make (after *I* it is certain of *like*). Lesson: perplexity is only meaningful when comparing models on the same, realistic, unseen test text; a tiny model evaluated on its own training sentences looks deceptively good.

Cheat sheet — RNN language models

- $\mathbf{h}^{(t)} = \sigma(W_h \mathbf{h}^{(t-1)} + W_e E \mathbf{x}^{(t)})$, $\hat{\mathbf{y}}^{(t)} = \text{softmax}(U\mathbf{h}^{(t)})$; same weights at every step.
- Loss: $J = \frac{1}{T-1} \sum_t -\log \hat{y}^{(t)}_{\mathbf{x}^{(t+1)}}$, minimised with SGD on batches.
- BPTT: gradient w.r.t. a shared weight = sum over its uses, $\frac{\partial \mathbf{h}^{(t)}}{\partial W_h} \sim \sum_i W_h^{t-1-i} \mathbf{h}^{(i)}$; truncated at ∼20 steps.
- Generation: sample, feed the sample back as input, repeat.
- Perplexity $= p(X)^{-1/T} = \exp(\text{average CE loss})$; lower is better; uniform model $\Rightarrow |V|$.

16.5 Problems with RNNs: Vanishing and Exploding Gradients

[Slides · Language models & RNNs: 85–101]

In practice, training the RNN presented so far (often called a **vanilla RNN**, i.e. the plain basic version) is hard, because of two related problems:

1. **vanishing gradients**;
2. **exploding gradients**.

Both come from the same source: in BPTT the gradient travels backwards through many steps, and at each step it is *multiplied* by the same matrix.

16.5.1 Vanishing gradients: intuition

[Slides · Language models & RNNs: 87–91]

How much does the loss at step 4 depend on the hidden state at step 1? By the chain rule, the influence of $\mathbf{h}^{(1)}$ on $J^{(4)}$ passes through $\mathbf{h}^{(2)}$, $\mathbf{h}^{(3)}$ and $\mathbf{h}^{(4)}$, and each passage contributes one factor:

$$\frac{\partial J^{(4)}}{\partial \mathbf{h}^{(1)}} = \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \times \frac{\partial \mathbf{h}^{(3)}}{\partial \mathbf{h}^{(2)}} \times \frac{\partial \mathbf{h}^{(4)}}{\partial \mathbf{h}^{(3)}} \times \frac{\partial J^{(4)}}{\partial \mathbf{h}^{(4)}}.$$

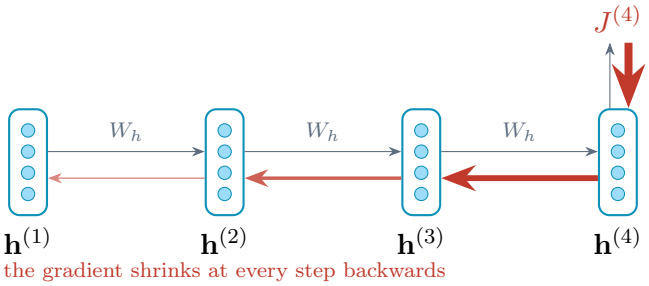


Figure 45: The gradient of $J^{(4)}$ flows backwards from $\mathbf{h}^{(4)}$ to $\mathbf{h}^{(1)}$ (red arrows). If each factor is "small", the signal reaching the first tokens is tiny.

Intuition — multiplying small numbers

[Not in the slides: added material]

If each factor were 0.5, after 3 steps the signal would be $0.5^3 = 0.125$, after 10 steps 0.001, after 20 steps one millionth. A product of many numbers smaller than 1 *vanishes*; a product of many numbers larger than 1 *explodes*. It is the same as the children's game "telephone": a message whispered along a long chain of people arrives at the end faint or distorted.

Definition — Vanishing gradients

When the factors $\partial \mathbf{h}^{(t)} / \partial \mathbf{h}^{(t-1)}$ are small, the gradients with respect to the **first tokens become tiny**: the loss at a late step hardly influences how the network processes early words.

16.5.2 Vanishing gradients: proof sketch

[Slides · Language models & RNNs: 92–95]

Use again the simplified model $\mathbf{h}^{(t)} = W_h \mathbf{h}^{(t-1)}$, $J^{(t)}(\theta) = \mathrm{CE}(\mathbf{h}^{(t)}, \mathbf{y}^{*(t)})$. Consider the loss at step i and its gradient with respect to the hidden state of an earlier step

$j < i$. Call $l = i - j$ the **distance** between the two steps.

$$\frac{\partial J^{(i)}(\theta)}{\partial \mathbf{h}^{(j)}} = \frac{\partial J^{(i)}(\theta)}{\partial \mathbf{h}^{(i)}} \prod_{j<t\leq i} \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}} \qquad \triangleleft \text{ chain rule, one factor per step}$$

$$= \frac{\partial J^{(i)}(\theta)}{\partial \mathbf{h}^{(i)}} \prod_{j<t\leq i} W_h \qquad \triangleleft\ \mathbf{h}^{(t)} = W_h \mathbf{h}^{(t-1)}$$

$$= \frac{\partial J^{(i)}(\theta)}{\partial \mathbf{h}^{(i)}} W_h^l \qquad \triangleleft\ l \text{ equal factors}$$

If W_h is "small", the power W_h^l becomes minuscule as l grows. What "small" means for a matrix is made precise by its **eigenvalues**.

[Not in the slides: added material]

Recall that an eigenvector $\mathbf{q}$ of a matrix is a direction that the matrix only stretches: multiplying by the matrix gives $\lambda\mathbf{q}$, where the number λ is the eigenvalue. Multiplying l times stretches it by λ^l.

Suppose W_h has eigenvalues $\lambda_1, \ldots, \lambda_n$ with $|\lambda_k| < 1$ and (left) eigenvectors $\mathbf{q}_1, \ldots, \mathbf{q}_n$ that form a basis. Write the gradient row vector in this basis, $\frac{\partial J^{(i)}}{\partial \mathbf{h}^{(i)}} = \sum_k c_k \mathbf{q}_k^\top$. Each multiplication by W_h multiplies the k-th component by λ_k, so

$$\frac{\partial J^{(i)}(\theta)}{\partial \mathbf{h}^{(i)}} W_h^l = \sum_{k=1}^{n} c_k \lambda_k^l \mathbf{q}_k^\top \approx 0 \quad \text{for large } l,$$

because every $\lambda_k^l \to 0$: the gradient **vanishes**. How fast?

λ	$l=1$	$l=5$	$l=10$	$l=20$	$l=50$
0.5	0.5	0.031	0.00098	9.5×10^{-7}	≈ 0
0.9	0.9	0.59	0.35	0.12	0.0052
1.1	1.1	1.61	2.59	6.73	117

Table 25: λ^l: the factor by which a component of the gradient is multiplied after travelling back l steps. Below 1 it decays exponentially, above 1 it grows exponentially.

The same phenomenon occurs with more realistic RNN models (reference: Pascanu et al., *On the difficulty of training recurrent neural networks*, 2013).

Watch out — details the slide leaves out

- The condition is on the *absolute value*: $|\lambda_k| < 1$. The slide writes $\lambda_k < 1$, but an eigenvalue -2 is smaller than 1 and still makes the gradient explode. (Eigenvalues can even be complex numbers; then $|\lambda|$ is their modulus.)
- The argument assumes W_h has a basis of eigenvectors (it is *diagonalisable*); the general case is similar, using the largest singular value instead.
- The slide uses i both for the time step and for the eigenvalue index; above the eigenvalue index is k.

- In a real RNN, $\mathbf{h}^{(t)} = \sigma(W_h \mathbf{h}^{(t-1)} + \dots)$, so each factor is $\text{diag}\big(\sigma'(\cdot)\big) W_h$. The derivative of the sigmoid is at most $\frac{1}{4}$ (of tanh at most 1), which makes the factors even smaller and vanishing even more likely.

16.5.3 Why it matters: long-term dependencies

[Slides · Language models & RNNs: 96–98]

The gradient that updates the weights is the sum of contributions from all steps. A contribution from a **nearby token** arrives almost intact; one from a **far-away token** arrives multiplied by W_h^l, i.e. almost 0. So the weights are updated almost only to exploit short-range patterns.

Key result — Long-term dependencies

With vanishing gradients, **long-term dependencies are (practically) impossible to learn**, because the gradient from far-away tokens is much smaller than from nearby tokens.

Worked example — a dependency 20+ tokens back

Predict the next word:

> After leaving **work**, he went to pick up his daughter from school. They had planned to go to the pet shop to buy a hamster and drove through town looking for one she liked. Eventually they found it, but before heading back home he had to rush back to his ____

The right answer is *work*. To predict it the model must connect the blank with the word *work* at the very beginning, more than 20 tokens earlier. During training, the gradient that would teach this connection is tiny compared with the gradients of short-range patterns ("his" → a noun, "rush back to his" → *car*, *house*, ...), so it provides little signal and learning long-term dependencies becomes very slow. A vanilla RNN tends to guess from the last few words only, i.e. it behaves much like an n-gram model.

16.5.4 Exploding gradients and gradient clipping

[Slides · Language models & RNNs: 99–100]

Symmetrically, if the eigenvalues of W_h are larger than 1 (in absolute value), the gradients become **too big**:

- an update $\theta \leftarrow \theta - \alpha \nabla_\theta J$ with a huge gradient is a huge step, which can jump over the minimum and **increase** the loss instead of decreasing it;
- in the worst case the numbers overflow and the weights become `inf` or `NaN` ("not a number"): training is ruined.

Definition — Gradient clipping

If the gradient is too large, scale it down. With a threshold $\tau > 0$:

1. set $\hat{\mathbf{g}} \leftarrow \nabla_\theta J(\theta)$;
2. **if** $\|\hat{\mathbf{g}}\| \geq \tau$:
3. set $\hat{\mathbf{g}} \leftarrow \dfrac{\tau}{\|\hat{\mathbf{g}}\|}\,\hat{\mathbf{g}}$;
4. **end if**

then update the weights with $\hat{\mathbf{g}}$.

Multiplying by the positive number $\tau/\|\hat{\mathbf{g}}\|$ does not change the direction of the vector, only its length, which becomes exactly τ. So **the update direction remains the same, but the step size is reduced**.

Worked example — clipping a gradient

[Not in the slides: added material]

Gradient $\hat{\mathbf{g}} = (30, 40)$, threshold $\tau = 5$. Its length is $\|\hat{\mathbf{g}}\| = \sqrt{30^2 + 40^2} = 50 \geq 5$, so

$$\hat{\mathbf{g}} \leftarrow \frac{5}{50}\,(30, 40) = (3, 4),$$

which has length $\sqrt{9 + 16} = 5 = \tau$ and points in the same direction (the ratio between the entries is still $3 : 4$).

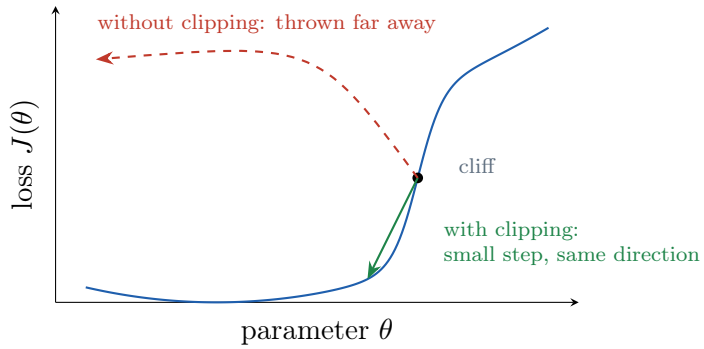


Figure 46: A "cliff" in the loss, typical of RNNs. On the steep wall the gradient is enormous: a plain gradient step throws the parameters far away, undoing training; the clipped step goes in the same direction but stays local.

The slide illustrates this with the 3D loss surface from Pascanu et al.: a flat region with a steep wall, where unclipped updates bounce off the wall, while clipped ones proceed smoothly. In practice **clipping essentially solves the exploding gradient problem**. It does nothing, however, for vanishing gradients: one cannot enlarge a gradient that carries no information.

16.5.5 Solutions for vanishing gradients

[Slides · Language models & RNNs: 101]

Vanilla RNNs cannot learn long-range dependencies because **they do not have a memory**: in

$$\mathbf{h}^{(t)} = \sigma\big(W_h \mathbf{h}^{(t-1)} + W_e \mathbf{e}^{(t)}\big)$$

the hidden state is **constantly rewritten**: at every step all of it is multiplied by W_h and squashed again, so old information fades. The solutions follow two ideas:

- **Separate memory mechanisms**: **LSTMs** (Long Short-Term Memory) and **GRUs** (Gated Recurrent Units) add a memory cell and *gates*, small learnt switches that decide at each step what to keep, what to forget and what to write. Information the gates decide to keep passes from step to step almost unchanged, so its gradient does not vanish. These were the **state-of-the-art NLP models in the 2010s**.
- **Direct connections between far-away tokens**: **attention** lets each position look directly at any earlier position, and **residual connections** add a shortcut that skips over layers. The gradient can then travel along a short path instead of through dozens of multiplications. Attention is the core of the transformer, the architecture of today's LLMs.

Cheat sheet — vanishing and exploding gradients

- Gradient across l steps: $\frac{\partial J^{(i)}}{\partial \mathbf{h}^{(j)}} = \frac{\partial J^{(i)}}{\partial \mathbf{h}^{(i)}} W_h^l$ (simplified model).
- All $|\lambda_k| < 1 \Rightarrow \lambda_k^l \to 0$: **vanishing** gradients, long-term dependencies are not learnt.
- Some $|\lambda_k| > 1 \Rightarrow$ **exploding** gradients: loss jumps up, `inf`/`NaN`.
- Exploding $\to$ **gradient clipping**: if $\|\mathbf{g}\| \geq \tau$, use $\frac{\tau}{\|\mathbf{g}\|}\mathbf{g}$ (same direction, length τ).
- Vanishing $\to$ memory with gates (LSTM, GRU) or direct connections (attention, residual connections).

Exercise 3 — vanishing or exploding?

[Not in the slides: added material]

In the simplified model $h^{(t)} = w\, h^{(t-1)}$ with a scalar hidden state, the gradient reaching a token l steps back is multiplied by w^l. For $w = 0.8$ and $w = 1.25$, after how many steps does the factor fall below $\frac{1}{100}$ or rise above 100? What happens with $w = -1.25$?

Solution

For $w = 0.8$ we need $0.8^l < 0.01$, i.e. $l > \frac{\ln 0.01}{\ln 0.8} = \frac{-4.605}{-0.223} \approx 20.6$: from $l = 21$ on (check: $0.8^{21} \approx 0.0092$).
For $w = 1.25$ we need $1.25^l > 100$, i.e. $l > \frac{\ln 100}{\ln 1.25} = \frac{4.605}{0.223} \approx 20.6$: again from $l = 21$ on ($1.25^{21} \approx 108$). (The two answers coincide because $1.25 = 1/0.8$.)
For $w = -1.25$: $|w^l| = 1.25^l$, the same growth, with a sign that alternates at every step. The gradient explodes even though $w < 1$, which is why the condition

must be stated on $|w|$ (on $|\lambda|$ for matrices).
Lesson: about 20 steps are enough to lose or blow up the signal by a factor 100, even with a factor only 20% away from 1. This is the same order of magnitude as the truncation length used in BPTT and as the distance in the "rush back to his work" example.